



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia de Software

Unidade Curricular de Laboratórios de Informática IV

Ano Letivo de 2025/2026

Ciclo de Vida de Desenvolvi- mento de Software Assistido por Inteligência Artificial

Francisco Martins Nuno Rebelo Marco Sèvegrand Lucas Robertson
Marco Ferreira

27 de maio de 2026

LI4

Data de Receção	
Responsável	
Avaliação	
Observações	

Ciclo de Vida de Desenvolvimento de Software Assistido por Inteligência Artificial

Francisco Martins Nuno Rebelo Marco Sèvegrand Lucas Robertson Marco Ferreira

27 de maio de 2026

Resumo

O presente relatório documenta e avalia criticamente o desenvolvimento integral de um sistema de gestão para uma oficina de reparação de trotinetes elétricas, desde a conceção e engenharia de requisitos até à verificação, validação e avaliação da qualidade do produto, conduzido com recurso sistemático a modelos de linguagem de grande dimensão (*Large Language Models*, doravante LLM). O caso de estudo é o sistema das *Oficinas Gengis Khan*, uma pequena e média empresa sediada em Braga que pretende substituir processos manuais assentes em papel e folhas de cálculo por uma plataforma *web* integrada, capaz de suportar o crescimento operacional projetado para os próximos anos.

A engenharia de requisitos consolidou a modelação do domínio, identificou os *stakeholders* relevantes, formalizou as *user stories* e os requisitos funcionais e não funcionais do sistema, e produziu a Especificação de Requisitos de *Software* bem como o catálogo de casos de uso que orientam todo o restante desenvolvimento.

A arquitetura e desenho fixou as opções estruturais do sistema, designadamente o padrão arquitetural, a *stack* tecnológica, o modelo de persistência e os mecanismos de partilha de contratos entre camadas, complementadas por diagramas UML, pela especificação da API REST e por uma proposta de *design* de interface adaptada ao ambiente de operação previsto.

A implementação materializou os módulos do *backend* e o respetivo *frontend*, dando forma ao desenho fixado nas etapas anteriores num produto funcional pronto para verificação.

A verificação e validação avaliou o produto resultante segundo um modelo de qualidade reconhecido na indústria, articulando verificação automática em *pipeline* de integração contínua, validação manual contra os casos de uso e avaliação dos atributos não funcionais relevantes.

A metodologia adotada manteve-se transversal a todas as fases. Cada artefacto foi produzido em diálogo com o LLM e revisto pela equipa, com registo integral dos *prompts* utilizados e das decisões de aceitação, modificação ou rejeição tomadas. O relatório inclui ainda uma reflexão crítica sobre o valor acrescentado e as limitações observadas na utilização de LLMs ao longo do ciclo de vida, identificando as fases em que a assistência foi mais produtiva e aquelas em que a intervenção humana se revelou indispensável para assegurar rigor técnico e fidelidade ao domínio de negócio.

Área de Aplicação: Engenharia de *Software* Assistida por Inteligência Artificial. Ciclo de Vida de Desenvolvimento de *Software*. Arquitetura, Implementação e Verificação de *Software*.

Palavras-Chave: Inteligência Artificial Generativa, Modelos de Linguagem, Ciclo de Vida de *Software*, Engenharia de Requisitos, Arquitetura de *Software*, Implementação Assistida por LLM, Verificação e Validação, Gestão de Oficina, Trotinetes Elétricas.

Índice

Introdução	1
Contextualização	1
Apresentação do Caso de Estudo	2
Motivação e Objetivos	2
1 Conceção e Engenharia de Requisitos Assistida por LLM	4
1.1 Definição do Sistema	4
1.1.1 Contextualização	4
1.1.2 Fundamentação	6
1.1.3 Objetivos	8
1.1.4 Viabilidade	10
1.1.5 Recursos	14
1.1.6 Plano de Execução	17
1.2 Definição do Domínio	19
1.2.1 Introdução e Contexto do Domínio	19
1.2.1.1 Caracterização do Setor	20
1.2.1.2 Necessidades Identificadas	21
1.2.2 Âmbito do Sistema e Componentes Funcionais	22
1.2.2.1 Componente de Gestão de Clientes e Equipamentos	23
1.2.2.2 Componente de Diagnóstico e Reparação	23
1.2.2.3 Componente de Gestão de <i>Stock</i> de Peças	23
1.2.2.4 Componente de Faturação e Relatórios	24
1.2.2.5 Componente de Comunicação com Fornecedores	24
1.2.2.6 Componente de Notificações ao Cliente	24
1.2.3 Intervenientes e Papéis	25
1.2.3.1 Gerente da Oficina	25
1.2.3.2 Mecânicos	26
1.2.3.3 Clientes	26
1.2.3.4 Fornecedores	26
1.2.4 Modelo de Domínio	27
1.2.5 <i>Stakeholders</i> e Partes Interessadas	30
1.2.5.1 <i>Stakeholders</i> Diretos	31
1.2.5.2 <i>Stakeholders</i> Indiretos	33
1.2.5.3 <i>Stakeholders</i> Técnicos	34
1.2.5.4 Entidades Regulamentares	35
1.2.5.5 Matriz de <i>Stakeholders</i>	36

1.2.5.6	Considerações de Gestão de Partes Interessadas	37
1.2.6	Contexto e Restrições	37
1.2.6.1	Contexto de Negócio	38
1.2.6.2	Restrições Legais e Regulamentares	39
1.2.6.3	Restrições Temporais e Académicas	39
1.2.6.4	Restrições Tecnológicas	40
1.2.7	Síntese	40
1.3	Engenharia de Requisitos	41
1.3.1	<i>User Stories</i>	41
1.3.1.1	Excertos Ilustrativos	43
1.3.1.2	Síntese e Transição para a Especificação Formal	43
1.3.2	Elicitação e Especificação de Requisitos	44
1.3.2.1	Processo de Elicitação e Descoberta	45
1.3.2.2	Análise e Especificação de Requisitos	47
1.3.2.3	Síntese e Transição para a Modelação	51
1.3.3	Especificação de Requisitos de Software (SRS)	51
1.3.3.1	Introdução e Propósito do SRS	53
1.3.3.2	Âmbito do Sistema	54
1.3.3.3	Descrição Geral	55
1.3.3.4	Requisitos Funcionais Consolidados	57
1.3.3.5	Requisitos Não Funcionais Consolidados	57
1.3.3.6	Critérios de Aceitação por Módulo	58
1.3.3.7	Nota de Transição para os <i>Use Cases</i>	59
1.4	Modelação Comportamental	59
1.4.1	Atores do Sistema	61
1.4.2	Especificação dos Casos de Uso	61
1.4.2.1	Excertos Ilustrativos	61
1.4.3	Síntese e Transição para a Fase Seguinte	63
2	Arquitetura e Design do <i>Software</i> Assistidos por LLM	64
2.1	Decisões Arquiteturais	64
2.1.1	Padrão Arquitetural	65
2.1.2	Stack Tecnológica e Fundamentação	68
2.1.3	Organização do Código em <i>Monorepo</i>	70
2.1.4	Partilha de Contratos entre Camadas	71
2.1.5	Modelo de Persistência e Autoridade do <i>Schema</i>	73
2.1.6	Modelo de Implantação e Ambientes	75
2.1.7	Registo de Decisões Arquiteturais	77
2.1.8	Síntese das Decisões Arquiteturais	78
2.2	Modelo Estrutural do Sistema	78
2.2.1	Estratégia de Apresentação e Notação	81
2.2.2	Vista Geral: Diagrama de Pacotes	82
2.2.3	Subsistema Operacional: Clientes, Trotinetes e Ordens de Serviço	84
2.2.4	Subsistema de Aprovisionamento e Inventário	86

2.2.5	Subsistema de Faturação, Configuração e Notificações	88
2.2.6	Decisões Estruturais Transversais de Modelação	89
2.3	Modelo Comportamental: Fluxos Críticos do Sistema	91
2.3.1	Participantes Recorrentes nos Diagramas	93
2.3.2	Autenticação e Estabelecimento de Sessão	94
2.3.3	Receção de Trotinete e Criação de Ordem de Serviço	95
2.3.4	Transição de Estado da Ordem de Serviço	96
2.3.5	Emissão de Fatura sobre Ordem de Serviço	97
2.3.6	Síntese e Articulação com as Vistas Subsequentes	98
2.4	Modelo Comportamental: Máquinas de Estado	98
2.4.1	Máquina de Estados da Ordem de Serviço	100
2.4.2	Máquina de Estados do Cronómetro de Intervenção	101
2.4.3	Máquina de Estados do Pagamento da Fatura	102
2.4.4	Síntese	102
2.5	Especificação da API	103
2.5.1	Convenções Transversais	104
2.5.2	Catálogo de <i>Endpoints</i>	105
2.5.3	Exemplos de Pedidos e Respostas	106
2.5.4	Síntese	108
2.6	Modelo de Componentes	108
2.6.1	Vista Geral dos Componentes em Tempo de Execução	110
2.6.2	Decomposição Interna do <i>Backend</i>	111
2.6.3	Síntese	112
2.7	Design de Interface	112
2.7.1	Princípios e Linguagem Visual	114
2.7.2	Estrutura de Navegação	114
2.7.3	Painel Operacional (<i>Dashboard</i>)	115
2.7.4	Gestão de Ordens de Serviço	116
2.7.5	Registo de Intervensões	119
2.7.6	Catálogo de Peças e <i>Stock</i>	120
2.7.7	Faturação	121
2.7.8	Relatórios Executivos	121
2.7.9	Notificações e Estado de Envio	123
2.7.10	Adaptação a <i>Tablets</i>	123
2.7.11	Síntese	124
3	Implementação e Desenvolvimento Assistido por LLM	125
3.1	Metodologia de Desenvolvimento	126
3.2	Configuração do Ambiente de Trabalho	127
3.3	Implementação Incremental por Módulos	129
3.4	Geração de Código Assistida por LLM	130
3.4.1	Exemplo Concreto de <i>Prompt</i> de Geração	132
3.5	Revisão de Código	133
3.6	Deteção de <i>Code Smells</i> e <i>Refactoring</i>	135

3.7	Métricas de Desenvolvimento	137
3.8	Síntese	138
4	Verificação, Validação e Qualidade do <i>Software</i> Assistidas por LLM	140
4.1	Estratégia de Verificação e Validação	140
4.2	Verificação	142
4.3	Validação	144
4.4	Avaliação da Qualidade dos Testes	146
4.5	V&V Assistida por LLM	147
4.6	Síntese e Prioridades de Evolução	149
5	Conclusão e Trabalho Futuro	150
5.1	Síntese das Capacidades Observadas	150
5.2	Limitações e Cuidados a Manter	151
5.3	Reflexão sobre o Ecossistema Emergente	152
5.4	Trabalho Futuro	153
	Glossário	156
	Anexos	163
	Anexo A: Dossiê Completo de Prompts	163
	Anexo B: Entrevista-001, Definição do Domínio (<i>Prompt</i> e Transcrição)	207
	Anexo C: Entrevista-002, Elicitação de Requisitos (<i>Prompt</i> e Transcrição)	215
	Anexo D: Entrevista-003, Validação de Requisitos (<i>Prompt</i> e Transcrição)	223
	Anexo E: Transcrição de Documentos Físicos da Oficina	230
	Anexo F: Registo de Decisões Arquiteturais (ADRs)	239
	Anexo G: Catálogo Completo de <i>Endpoints</i> do Subsistema Operacional	243
	Anexo H: Catálogo Completo de Requisitos Levantados e Consolidados	247
	Anexo I: Catálogo Completo de <i>User Stories</i>	258
	Anexo J: Catálogo Completo dos Casos de Uso	262
	Anexo K: Composição Detalhada da <i>Stack</i> Tecnológica	279

Lista de Figuras

1.1	Diagrama de Gantt do plano de execução do projeto.	19
1.2	Modelo de domínio do sistema de gestão das <i>Oficinas Gengis Khan</i>	28
2.1	Arquitetura em três camadas do sistema, com o pacote de contratos partilhados a atravessar transversalmente a fronteira entre <i>frontend</i> e <i>backend</i>	67
2.2	Vista geral do modelo estrutural: diagrama de pacotes UML.	82
2.3	Diagrama de classes do subsistema operacional.	84
2.4	Diagrama de classes do subsistema de aprovisionamento e inventário.	86
2.5	Diagrama de classes do subsistema de faturação, configuração financeira e notificações.	88
2.6	Diagrama de sequência da autenticação (POST /auth/login)	94
2.7	Diagrama de sequência da criação de ordem de serviço (POST /service-orders)	95
2.8	Diagrama de sequência da transição de estado (PATCH /service-orders/:id/status)	96
2.9	Diagrama de sequência da emissão de fatura (POST /billing/invoices)	97
2.10	Máquina de estados da entidade <i>ServiceOrder</i>	100
2.11	Máquina de estados do cronómetro de intervenção	101
2.12	Máquina de estados de <i>Invoice.paymentStatus</i>	102
2.13	Vista geral dos componentes do sistema em tempo de execução	110
2.14	Decomposição interna do <i>backend Fastify</i>	111
2.15	Painel operacional (<i>Dashboard</i>)	115
2.16	Listagem de ordens de serviço	116
2.17	Detalhe de ordem de serviço com cronómetro de intervenção	117
2.18	Diálogo modal de criação de ordem de serviço	118
2.19	Registo de intervenção: lista de ordens com trabalho em curso	119
2.20	Catálogo de peças e movimentos de <i>stock</i>	120
2.21	Faturação: lista de faturas com estados de pagamento	121
2.22	Relatórios executivos: vista agregada de desempenho	122
2.23	Painel de notificações com estado de entrega por mensagem	123
4.1	Pirâmide de testes automáticos adotada pela equipa, complementada por uma camada transversal de qualidade que avalia atributos não funcionais segundo o modelo ISO/IEC 25010. Cada etiqueta indica o nome da camada, a contagem efectiva de testes e a função desempenhada.	143

Lista de Tabelas

1.1	Objetivos primários	9
1.2	Objetivos secundários	10
1.3	Objetivo transversal	10
1.4	Avaliação da viabilidade técnica	12
1.5	Avaliação de riscos temporais	14
1.6	Distribuição dos recursos humanos	16
1.7	Distribuição dos recursos materiais	17
1.8	Plano de execução	18
1.9	Matriz de <i>stakeholders</i>	36
1.10	Tecnologias selecionadas	40
1.11	Excerto da listagem bruta dos requisitos levantados	47
1.12	Excerto dos requisitos funcionais consolidados	48
1.13	Excerto dos requisitos não funcionais consolidados	49
1.14	Resolução de ambiguidades e conflitos	50
1.15	Definição do Âmbito do Sistema	54
1.16	Definição dos perfis de utilizador	56
1.17	Atores identificados na modelação comportamental	61
2.1	Excerto da composição da <i>stack</i> tecnológica	69
2.2	Comparação entre ambientes de execução	76
2.3	Síntese das decisões arquiteturais adotadas	78
2.4	Enumerações de domínio definidas no sistema	81
2.5	Rastreabilidade entre entidades e módulos do <i>backend</i>	91
2.6	Participantes recorrentes nos diagramas de sequência	93
2.7	Códigos de erro simbólicos da API	104
2.8	Excerto do catálogo de <i>endpoints</i> do subsistema operacional	105
2.9	Excerto do catálogo de <i>endpoints</i> do subsistema de aprovisionamento e inventário	105
2.10	Excerto do catálogo de <i>endpoints</i> do subsistema de faturação, notificações e configuração	105
2.11	Excerto do catálogo de <i>endpoints</i> de relatório transversal	106
3.1	Métricas de volume da base de código (final da etapa)	138
4.1	Inventário da <i>suite</i> de testes automáticos	144
4.2	Avaliação da qualidade da <i>suite</i> de testes	147
5.1	Análise comparativa de padrões arquiteturais candidatos	242

5.2	Catálogo completo de <i>endpoints</i> do subsistema operacional	243
5.3	Catálogo completo de <i>endpoints</i> do subsistema de aprovisionamento e inventário	245
5.4	Catálogo completo de <i>endpoints</i> do subsistema de faturação, notificações e configuração	246
5.5	Catálogo completo de <i>endpoints</i> de relatório transversal	247
5.6	Requisitos levantados não estruturados	247
5.7	Requisitos funcionais consolidados	253
5.8	Requisitos não funcionais consolidados	257
5.38	Composição detalhada da <i>stack</i> tecnológica adotada	279

Lista de *Prompts*

1	Contextualização do Negócio	4
2	Fundamentação do Projeto	6
3	Definição de Objetivos	8
4	Análise de Viabilidade	10
5	Identificação de Recursos	14
6	Plano de Execução	17
7	Introdução ao Domínio	19
8	Âmbito do Sistema	22
9	Intervenientes e Papéis	25
10	Modelo de Domínio	27
11	Identificação de <i>Stakeholders</i>	30
12	Contexto e Restrições	37
13	Geração de <i>User Stories</i>	41
14	Análise e Categorização de Requisitos	44
15	Estruturação do SRS	51
16	Modelação de Casos de Uso UML	59
17	Escolha do Padrão Arquitetural	65
18	Composição da <i>Stack</i> Tecnológica	68
19	Organização em <i>Monorepo</i>	70
20	Mecanismo de Partilha de Contratos	71
21	Modelo de Persistência	73

22	Modelo de Implantação	75
23	Diagrama de Classes UML	79
24	Modelo Comportamental: Diagramas de Sequência	91
25	Modelação de Máquinas de Estado	98
26	Especificação da API REST	103
27	Modelo de Componentes	108
28	<i>Design</i> de Interface	112
29	Metodologia de Desenvolvimento	126
30	Configuração do Ambiente de Trabalho	127
31	Estratégia de Implementação por Módulos	129
32	Tarefas de Geração de Código por LLM	130
33	Geração do Módulo <i>Service-Orders</i>	132
34	Modelo de Revisão de Código	133
35	Detecção de <i>Code Smells</i> e <i>Refactoring</i>	135
36	Métricas de Desenvolvimento	137
37	Estratégia de Verificação e Validação	140
38	Arquitetura de Testes de Verificação	142
39	Procedimento de Validação contra o SRS	144
40	Avaliação da Qualidade da <i>Suite</i> de Testes	146
41	Geração da <i>Suite</i> de Testes por LLM	147

Introdução

Contextualização

A integração de agentes de inteligência artificial generativa no processo de desenvolvimento de *software* constitui um dos fenómenos mais transformadores da engenharia informática contemporânea. Os modelos de linguagem de grande dimensão (*Large Language Models*, doravante LLM) transitaram, em poucos anos, de ferramentas experimentais de autocompletar código para assistentes capazes de produzir artefactos completos de engenharia de *software*, desde transcrições de entrevistas simuladas e documentos de domínio até especificações de requisitos, diagramas UML e código funcional. Esta evolução coloca uma questão central à formação em engenharia de *software*: em que medida podem estes agentes inteligentes participar no ciclo de vida de desenvolvimento de *software* e qual o papel que resta ao engenheiro humano quando a máquina consegue gerar, em segundos, artefactos que tradicionalmente exigiam dias de trabalho analítico?

O presente relatório insere-se neste contexto. Desenvolvido no âmbito da unidade curricular de Laboratórios de Informática IV (LI4) da Licenciatura em Engenharia de Software da Universidade do Minho, o trabalho tem como eixo central a execução e avaliação crítica de um ciclo de vida de desenvolvimento de *software* conduzido com recurso sistemático a LLMs. Não se trata, portanto, de um relatório convencional de projeto de *software*, mas de um estudo sobre o próprio processo de engenharia quando este é mediado por agentes de IA generativa, constituindo um meta-relatório que documenta simultaneamente os artefactos produzidos e a reflexão sobre como, porquê e com que qualidade foram produzidos.

Os modelos efetivamente utilizados ao longo do projeto foram o *Claude*, da *Anthropic*, nas variantes *Opus* (versões 4.6 e, após a sua disponibilização durante o decurso do trabalho, 4.7) e *Sonnet* 4.5, como assistente principal na produção e revisão dos artefactos textuais e de modelação que constituem a quase totalidade do presente relatório. A esta utilização principal somou-se o *GitHub Copilot*, com o *ChatGPT* 5.3 da *OpenAI* como modelo subjacente, como complemento durante a fase de implementação de código, e ainda o *Stitch AI*, da *Google*, no apoio à fase de *design* de interface. Estas ferramentas acompanharam a totalidade do trabalho, designadamente a redação académica do próprio relatório, a especificação de requisitos, a modelação do domínio, a geração de código e o desenho da interface do sistema, configurando assim uma utilização transversal do auxílio de IA generativa ao longo de todas as fases do ciclo de vida.

Apresentação do Caso de Estudo

O caso de estudo selecionado para suporte a esta avaliação é o desenvolvimento de um sistema de gestão integrado para as *Oficinas Gengis Khan*, uma empresa de pequena e média dimensão sediada em Braga, especializada no diagnóstico, reparação e manutenção de trotinetes elétricas de múltiplas marcas. A oficina serve simultaneamente clientes particulares e frotas corporativas de operadores de mobilidade partilhada, gerindo um volume operacional já significativo e um inventário de aproximadamente 300 referências de peças de substituição.

O domínio de reparação de trotinetes elétricas foi atribuído à equipa no contexto da unidade curricular, tendo sido posteriormente desenvolvido e aprofundado ao longo do projeto. A riqueza funcional deste domínio e a diversidade de desafios de engenharia que coloca justificam a sua adequação como caso de estudo, ao envolver gestão de entidades heterogéneas (clientes, equipamentos, fornecedores), um ciclo de vida operacional com múltiplos estados e transições (ordem de serviço), regras de negócio não triviais (faturação sequencial obrigatória, conformidade com o RGPD, controlo de crédito a clientes empresariais) e requisitos de comunicação automática (notificações por *email*). Esta complexidade oferece um terreno de teste exigente para avaliar a capacidade dos LLMs em compreender, modelar e especificar um domínio real de negócio.

Motivação e Objetivos

A motivação principal deste trabalho reside na necessidade de avaliar, de forma fundamentada e transparente, o impacto da utilização de agentes de IA generativa nas diversas fases do ciclo de vida de desenvolvimento de *software*. A proliferação destas ferramentas no contexto académico e profissional exige uma reflexão estruturada que vá além da mera adoção acrítica ou da rejeição dogmática.

Neste enquadramento, os objetivos do presente relatório são:

1. **Executar as fases do ciclo de vida de desenvolvimento de *software***, nomeadamente a definição do sistema, a modelação do domínio, a engenharia de requisitos, a modelação comportamental, a arquitetura, a implementação e os testes, utilizando LLMs como instrumento de apoio em todas as etapas.
2. **Documentar de forma integral o processo de interação com os LLMs**, registando os *prompts* formulados, as respostas obtidas e as decisões de aceitação, modificação ou rejeição tomadas pela equipa para cada artefacto.
3. **Avaliar criticamente a qualidade dos artefactos gerados**, aferindo a sua correção técnica, coerência interna, completude e adequação ao domínio do problema.

4. **Refletir sobre o valor acrescentado e as limitações dos LLMs** no contexto da engenharia de *software*, identificando as fases em que a assistência foi mais produtiva e aquelas em que a intervenção humana se revelou indispensável.

1 Conceção e Engenharia de Requisitos Assistida por LLM

O presente capítulo documenta a primeira etapa do ciclo de vida de desenvolvimento de *software* do projeto, dedicada à conceção e engenharia de requisitos e realizada com recurso sistemático a modelos de linguagem de grande dimensão (LLM) como instrumento de apoio. Esta etapa articula-se em quatro momentos sequenciais, designadamente a definição do sistema e do seu âmbito (Secção 1.1), a modelação do domínio de aplicação (Secção 1.2), a engenharia de requisitos propriamente dita (Secção 1.3) e a modelação comportamental através de casos de uso e diagramas UML (Secção 1.4). Cada secção é precedida pela análise crítica da interação com os LLMs, incluindo os *prompts* utilizados, a avaliação dos resultados obtidos e as decisões tomadas pela equipa quanto à aceitação, modificação ou rejeição dos artefactos gerados.

1.1 Definição do Sistema

A definição do sistema desenvolve-se em seis dimensões complementares, designadamente a contextualização do negócio, a fundamentação do projeto, a especificação dos objetivos, a análise de viabilidade, a identificação dos recursos e o plano de execução. Cada subsecção é precedida pelo *prompt* utilizado e por uma análise crítica do processo de geração.

1.1.1 Contextualização

Análise Crítica do Processo de Geração com IA

Prompt 1. Contextualização do Negócio

Contexto: Projeto da UC de LI4, sistema de gestão das "Oficinas Gengis Khan". Fase de definição do domínio, anterior ao levantamento formal de requisitos.

Objetivo: Gerar a transcrição de uma entrevista técnica entre um Analista de Sistemas e o Engenheiro da oficina, a utilizar como base documental para a "Definição do Domínio" e o "SRS".

Instruções de Escrita:

- Cobrir quatro tópicos: Registo e Identificação, Ciclo de Vida da Reparação, Gestão de Peças e Custos, Output e Relatórios
- Manter registo natural de diálogo entre dois interlocutores técnicos
- Privilegiar exemplos concretos sobre afirmações genéricas

Restrições: Forneça apenas a transcrição da entrevista, sem introduções meta-textuais nem comentários fora do diálogo.

[Numa segunda iteração foi solicitada a reescrita do texto de contextualização com base no domínio extraído da entrevista gerada.]

Análise Crítica

O processo de produção da contextualização envolveu duas interações distintas com o LLM. Na primeira, solicitámos a geração de uma entrevista simulada entre analistas de sistemas e os *stakeholders* da oficina, opção metodológica destinada a reproduzir o tipo de recolha de informação que seria conduzida num contexto profissional com interlocutores reais. O registo daí resultante é tratado, ao longo de toda a etapa, como base documental autêntica do domínio, sobre a qual assentam todas as decisões subsequentes de modelação e especificação. Na segunda iteração, pedimos ao modelo que sintetizasse essa informação num texto de contextualização formal.

O principal desafio residiu na calibração do nível de detalhe. A primeira versão gerada pelo LLM era demasiado genérica, descrevendo o setor de micromobilidade em termos que poderiam aplicar-se a qualquer oficina. Optámos por solicitar uma versão mais ancorada nos dados concretos da oficina (volume operacional real, número de referências de peças, segmentos de clientes), o que resultou num texto significativamente mais útil. O parágrafo sobre o crescimento do mercado europeu foi aceite na íntegra por estar factualmente alinhado com dados públicos do setor. A caracterização técnica das trotinetes (eletrónica de potência, mecânica de precisão) foi validada pela equipa e considerada adequada.

Um segundo desafio, transversal a toda a redação do relatório e não exclusivo desta secção, residiu na depuração linguística do *output*. A escrita técnica em português gerada pelo LLM revelou um vício recorrente, designadamente a tendência para traduzir termos do inglês de forma literal, particularmente acentuada nas passagens mais próximas do vocabulário do desenvolvimento de *software*. Termos como “comitar” (de *commit*) ou “merjar” (de *merge*), inexistentes em português, surgiram com frequência no *output* bruto e exigiram um esforço considerável de revisão para os substituir pelas formas corretas. Ainda assim, o tempo gasto nesta revisão manteve-se substancialmente inferior ao que a redação integral pela equipa teria exigido. O processo, embora imperfeito, preserva um ganho de produtividade significativo na produção do relatório, ganho esse que justifica o recurso ao modelo apesar dos desvios linguísticos pontuais que produz.

A mobilidade urbana europeia atravessa uma transformação estrutural. A adoção massiva de trotinetes elétricas como alternativa ao automóvel particular, motivada por preocupações ambientais, eficiência de deslocação e políticas municipais de descarbonização, gerou um ecossistema de serviços de pós-venda que, até há poucos anos, era praticamente inexistente. O mercado europeu de micromobilidade partilhada e privada registou um crescimento anual composto superior a 10 % entre 2020 e 2025, arrastando consigo a procura por manutenção especializada.

As *Oficinas Gengis Khan* posicionam-se neste segmento emergente. Sediada em Braga, Portugal, a empresa presta serviços de diagnóstico, reparação e manutenção de trotinetes elétricas de múltiplas marcas, servindo tanto clientes particulares como frotas corporativas de operadores de mobilidade partilhada. A natureza do negócio implica lidar simultaneamente com eletrónica de potência (baterias de iões de lítio, controladores, *firmware*), mecânica de precisão (travões regenerativos, suspensões, direção) e uma cadeia de aprovisionamento fragmentada, com cerca de 300 referências de peças de origens e prazos de entrega heterogéneos.

Com um volume atual de aproximadamente 150 ordens de serviço mensais e uma tendência de crescimento sustentado, a oficina atingiu o limite operacional dos seus processos de gestão, ainda assentes em cadernos físicos, folhas avulsas e ficheiros Excel sem integração. Este desfasamento entre a sofisticação técnica das reparações e a maturidade dos processos administrativos constitui o problema central que o presente projeto visa resolver.

1.1.2 Fundamentação

Análise Crítica do Processo de Geração com IA

Prompt 2. Fundamentação do Projeto

Contexto: Documento tecnico/academico, ciclo de vida de desenvolvimento de software, projeto da UC de LI4.

Objetivo: escrever o paragrafo introdutorio da seccao de "Fundamentacao".

Instrucoes de Escrita:

- Utilize terminologia precisa
- Evite informacao redundante ou repeticoes desnecessarias
- Tenha em conta o papel da Fundamentacao no ciclo de vida de desenvolvimento de software
- Mantenha a coesao e coerencia textual exigidas em contextos universitarios

Restricoes: Forneca apenas o novo paragrafo. Nao inclua introducoes, explicacoes ou notas de rodape. Conteudo base em anexo.

Análise Crítica

A fundamentação exigia a identificação das vulnerabilidades operacionais da oficina e a sua organização em dimensões estruturadas. O LLM produziu uma versão inicial que identificava corretamente quatro dimensões problemáticas (fragmentação de informação, opacidade operacional, gestão reativa de *stock* e falta de rigor financeiro), mas apresentava-as de forma excessivamente descritiva. Optámos por manter a estrutura proposta pelo modelo, por considerarmos que a divisão em quatro dimensões era lógica e abrangente, mas reformulámos diversos parágrafos para incluir dados quantitativos concretos extraídos da entrevista simulada (estimativa de 15 % de atrasos evitáveis, taxa de 25 €/h). Este percurso de revisão e reformulação ilustra o princípio metodológico subjacente a toda a etapa, em que o ganho de produtividade conferido pelo modelo se mantém condicionado a verificação humana atenta.

A proposta de desenvolvimento de um sistema de gestão integrado fundamenta-se num diagnóstico estruturado aos processos de negócio, executado mediante uma sessão formal de levantamento e documentação dos processos do negócio com o gerente e o mecânico-chefe da oficina (Entrevista-001). A análise do domínio resultante desta interação permitiu mapear as vulnerabilidades vigentes, agrupando os problemas operacionais identificados em quatro dimensões interdependentes que justificam a intervenção tecnológica e orientam o ciclo de engenharia de *software* subsequente.

Fragmentação e perda de informação A oficina não dispõe de um repositório centralizado de dados. Os diagnósticos são anotados em folhas soltas, frequentemente extraviadas antes de serem transcritas. O histórico de intervenções sobre um mesmo equipamento (identificado pelo número de série) não é recuperável de forma fiável, impedindo a deteção de padrões de avaria recorrente e comprometendo a resposta a reclamações. A relação entre cliente, equipamento e reparação existe apenas na memória dos colaboradores.

Opacidade operacional Não existe qualquer mecanismo que permita conhecer, em tempo real, o estado de uma ordem de serviço ou a carga de trabalho da oficina. As transições entre estados (receção, diagnóstico, aprovação, espera de peças, reparação, entrega) não são registadas com *timestamps*, o que torna impossível calcular tempos de ciclo, identificar gargalos ou fundamentar compromissos de prazo perante o cliente. O gerente estima que cerca de 15 % das ordens sofrem atrasos evitáveis por falta de visibilidade interna.

Gestão reativa de aprovisionamento O controlo de *stock* é inteiramente mental. O gerente e o mecânico apercebem-se da rutura de uma peça apenas quando esta é necessária para uma reparação já em curso. Não existem níveis mínimos formalizados, alertas de reposição nem histórico consolidado de encomendas. Este comportamento reativo provoca tempos mortos nas reparações e obriga a contactos urgentes com fornecedores, frequentemente a preços menos competitivos.

Falta de rigor financeiro e de comunicação A faturação é calculada manualmente, somando preços de peças e horas de mão de obra à taxa de 25 €/h, num processo lento e sujeito a erros aritméticos. O apuramento da margem de lucro exige cruzamento *ad hoc* de múltiplas fontes. Para clientes corporativos com pagamento a 30 dias, não existe controlo sistemático de faturas pendentes ou atrasadas, criando risco de tesouraria. Do lado da comunicação, o cliente só recebe informação sobre a reparação quando contacta a oficina por iniciativa própria, gerando insatisfação e sobrecarga telefónica.

Síntese Estas quatro dimensões partilham uma causa comum, a ausência de um sistema de informação que integre dados, processos e comunicações. O presente projeto propõe colmatar esta lacuna com uma plataforma única e coerente, desenhada à medida dos fluxos de trabalho já praticados na oficina, mas dotada de automação, rastreabilidade e capacidade analítica que os métodos atuais não oferecem.

1.1.3 Objetivos

Análise Crítica do Processo de Geração com IA

Prompt 3. Definição de Objetivos

Contexto: Documento tecnico/academico, modelo de desenvolvimento em cascata, fase de definicao do sistema (anterior ao levantamento de requisitos).

Objetivo: Escrever paragrafo curto e introdutorio da seccao de "Objetivos primarios", "Objetivos secundarios" e "Objetivo transversal", explicando a importancia dos mesmos e o motivo de estarem a ser apresentados.

Instrucoes de Escrita:

- Tenha em conta o papel dos Objetivos no ciclo de vida de desenvolvimento de software
- Utilize terminologia precisa
- Mantenha coesao e coerencia universitaria

Restricoes: Forneca apenas os 3 novos paragrafos separadamente. Conteudo base em anexo.

Análise Crítica

Para esta secção, a interação com o LLM decorreu em dois momentos. Primeiro, solicitámos os parágrafos introdutórios de cada categoria de objetivos. O modelo produziu textos que, embora corretos do ponto de vista formal, tendiam para uma linguagem ex-

cessivamente abstrata e desligada do domínio concreto, pelo que foi pedida nova iteração que ancorasse os parágrafos no contexto específico das *Oficinas Gengis Khan*. A estrutura tripartida (primários, secundários, transversal) foi aceite sem reservas, por se alinhar com boas práticas de engenharia de requisitos. O conteúdo das tabelas de objetivos foi igualmente gerado pelo modelo a partir da informação recolhida na Entrevista-001, cabendo à equipa exclusivamente a validação de cada entrada, com aceitação imediata ou pedido de reformulação consoante a fidelidade ao levantamento original.

A especificação rigorosa dos objetivos assume um papel essencial no ciclo de vida de desenvolvimento de *software*, atuando como o fio condutor que alinha os requisitos de engenharia com as necessidades operacionais do negócio e fornecendo os indicadores métricos essenciais para a posterior validação e verificação do sistema. Neste enquadramento, os propósitos do presente projeto foram delineados em estreita colaboração com os *stakeholders* através da realização de uma entrevista inicial (Entrevista-001).

Objetivos primários A definição dos objetivos primários estabelece o núcleo funcional crítico da solução tecnológica, sendo a sua satisfação indispensável para a viabilidade operacional do sistema de gestão. A fixação rigorosa destas metas fundacionais orienta o desenho arquitetural e garante que as etapas subsequentes de implementação se alicerçam numa base sólida, direcionada para a resolução das lacunas operacionais identificadas no domínio.

Tabela 1.1: Objetivos primários

#	Objetivo	Indicador de sucesso
O1	Centralizar clientes, equipamentos, ordens de serviço, intervenções e peças numa base de dados relacional única	Eliminação total de registos em papel e Excel para operações correntes
O2	Implementar o ciclo de vida completo da ordem de serviço com sete estados e <i>timestamps</i> automáticos	Qualquer utilizador consegue consultar o estado atual e o histórico de uma OS em menos de 15 segundos
O3	Registar cada intervenção com descrição, mecânico, peças consumidas e tempo de execução (cronómetro integrado)	100 % das intervenções documentadas com dados completos
O4	Atualizar automaticamente o <i>stock</i> ao associar peças a intervenções e gerar alertas quando o nível mínimo é atingido	Zero ruturas de <i>stock</i> não sinalizadas
O5	Gerar faturas automaticamente a partir da OS concluída, com numeração sequencial contínua e cálculo de IVA	Conformidade legal verificável; tempo de emissão de fatura inferior a 1 minuto

Objetivos secundários Os objetivos secundários agregam um conjunto de requisitos de valor acrescentado que visam a otimização avançada dos processos de negócio, englobando

componentes analíticas e automação comunicacional. A sua especificação exaustiva nesta fase inicial assegura que o desenho detalhado do *software* contempla a integração destas funcionalidades complementares desde a sua génese, mitigando o risco de reestruturações profundas.

Tabela 1.2: Objetivos secundários

#	Objetivo	Indicador de sucesso
O6	Automatizar notificações por <i>email</i> ao cliente (re- ceção, orçamento, conclusão, atraso superior a 10 dias)	Redução de pelo menos 60 % dos contactos telefónicos ini- ciados pelo cliente
O7	Disponibilizar <i>dashboard</i> analítico (faturação, pe- ças mais consumidas, produtividade, distribuição por estado)	Gerente capaz de responder a qualquer questão de gestão corrente sem cálculo manual
O8	Suportar gestão diferenciada de clientes corpora- tivos (crédito, prazos, faturas consolidadas)	Controlo de faturas penden- tes com alerta automático de atraso
O9	Gerir fornecedores e encomendas (criação, envio por <i>email</i> , rastreio de estado, histórico)	Prazo médio de reposição de <i>stock</i> reduzido em pelo me- nos 25 %
O10	Garantir conformidade RGPD (consentimento, acesso, retificação, eliminação de dados pessoais)	Auditoria de conformidade positiva

Objetivo transversal A formulação de um objetivo transversal tem como propósito elucidar os atributos de qualidade sistémica e as restrições globais que deverão integrar toda a solução de *software*.

Tabela 1.3: Objetivo transversal

#	Objetivo	Indicador de sucesso
O11	Garantir escalabilidade ao crescimento operacional projetado, sem degradação perceptível de desem- penho	Testes de carga representati- vos do volume-alvo

1.1.4 Viabilidade

Análise Crítica do Processo de Geração com IA

Prompt 4. Análise de Viabilidade

Contexto: Sistema de gestão das "Oficinas Gengis Khan", projeto da UC de LI4 (Universidade do Minho). Fase de definição do sistema, anterior ao desenho da arquitetura.

Objetivo: Escrever a secção de Viabilidade do relatório com rigor académico médio-alto em PT-PT, articulando argumentos em quatro dimensões (técnica, organizacional, económica e temporal).

Instruções de Escrita:

- Introdução curta sobre o papel da viabilidade no ciclo de vida de desenvolvimento
- Viabilidade Económica fundamentada em ROI e ganhos operacionais
- Viabilidade Temporal alinhada com o cronograma do semestre LI4
- Ancorar argumentos em dados reais da oficina (volume 150-500 OS/mes)

Restrições: Manter a análise agnóstica face a tecnologias específicas ainda não decididas. Sem estimativas quantitativas sem base documental.

Análise Crítica

A secção de viabilidade foi a que mais beneficiou da assistência do LLM, por exigir a articulação de argumentos em quatro dimensões distintas (técnica, organizacional, económica e temporal). O modelo produziu uma estrutura bem organizada e textos introdutórios de qualidade aceitável. Foram, contudo, identificados dois problemas relevantes. Em primeiro lugar, a viabilidade técnica mencionava tecnologias específicas que ainda não tinham sido formalmente decididas pela equipa, obrigando a uma reformulação para manter a análise agnóstica. Em segundo lugar, a viabilidade económica apresentava estimativas quantitativas sem base documental, que foram removidas e substituídas por argumentos qualitativos ancorados nos dados reais da oficina. A tabela de riscos temporais foi aceite com modificações pontuais na coluna de mitigação.

A análise de viabilidade constitui uma etapa fundamental no ciclo de vida de desenvolvimento de *software*, funcionando como um mecanismo de validação antecipada que minimiza o risco de investimento em soluções tecnologicamente inadequadas, economicamente inviáveis ou temporalmente irrealistas. Ao examinar sistematicamente as dimensões técnica, organizacional, económica e temporal antes do início do desenvolvimento propriamente dito, esta análise assegura o alinhamento entre as expectativas dos *stakeholders* e as restrições reais do projeto. No presente projeto, a viabilidade foi avaliada em quatro dimensões complementares, cujos resultados se apresentam nas subsecções seguintes.

Viabilidade Técnica A análise da viabilidade técnica tem como propósito confirmar que o sistema proposto pode ser desenvolvido com segurança e eficiência, recorrendo apenas a tecnologias consolidadas e acessíveis. Esta avaliação considera aspetos como a arquitetura da aplicação, o volume esperado de dados, os mecanismos de notificações e a sincronização em tempo real entre cliente e servidor.

Tabela 1.4: Avaliação da viabilidade técnica

Dimensão	Avaliação
Arquitetura	Aplicação <i>web</i> em três camadas (apresentação, lógica de negócio, dados), padrão maduro, com vasto suporte de <i>frameworks</i> e documentação.
Volume de dados	Projeção a 10 anos na ordem das dezenas de milhares de ordens de serviço acumuladas, cada uma com cerca de 5 intervenções e 10 linhas de peças associadas.
Notificações	Serviço SMTP externo (ex.: servidor de <i>email</i> do domínio da oficina); sem dependência de infraestrutura adicional.
Cronómetro	Implementável no <i>browser</i> com sincronização ao servidor via REST ou, opcionalmente, <i>WebSockets</i> para atualização em tempo real.

Nenhuma das funcionalidades identificadas requer tecnologia experimental ou infraestrutura além do habitual para um projeto *web* profissional.

Viabilidade Organizacional A predisposição dos *stakeholders* é o fator crítico de sucesso em projetos de digitalização de PME. Neste caso, ambos os intervenientes entrevistados, gerente e mecânico-chefe, manifestaram iniciativa própria na identificação de problemas e na sugestão de melhorias, o que sinaliza elevada motivação para a adoção do sistema.

O risco de resistência à mudança é mitigado por três fatores de *design*:

1. **Continuidade de processos:** o sistema digitaliza fluxos já existentes (criação de OS, registo de intervenções, verificação de *stock*) em vez de impor novos.
2. **Ausência de interação externa:** os clientes recebem apenas notificações passivas por *email*, não precisando de criar conta nem de aprender a usar qualquer interface.
3. **Introdução faseada:** o MVP cobre as operações diárias essenciais, sendo as funcionalidades avançadas adicionadas após estabilização do uso básico.

Viabilidade Económica A análise de viabilidade económica tem como propósito avaliar se os benefícios gerados pela implementação do sistema justificam os custos inerentes ao seu desenvolvimento e manutenção, constituindo a base racional para a decisão de investimento. No presente contexto, a quantificação do retorno sobre o investimento (ROI) assenta essencialmente em ganhos de eficiência operacional, isto é, na redução de custos indiretos associados a processos manuais ineficientes em vez de receitas adicionais diretas imediatamente mensuráveis.

Os principais vetores de benefício económico identificados são os seguintes:

- **Redução de erros de faturação:** o processo atual de cálculo manual de faturas, envolvendo a soma de preços de peças e horas de mão de obra à taxa de 25 €/hora com aplicação posterior de IVA a 23 %, é suscetível a erros aritméticos que podem resultar em sub-cobrança ou em litígios com clientes. A automatização deste processo elimina esta classe de erros, reduzindo o risco financeiro e o tempo despendido em correções.
- **Otimização da gestão de *stock*:** o comportamento reativo atual, em que a rutura de uma peça é detetada apenas no momento em que esta é necessária para uma reparação em curso, implica custos acrescidos decorrentes de encomendas urgentes a preços menos competitivos e de tempos mortos de reparação. A implementação de alertas automáticos de nível mínimo de *stock*, aliada ao histórico consolidado de consumo, permitirá uma política de aprovisionamento proativa, reduzindo tanto os custos unitários de aquisição como o impacto operacional das rupturas.
- **Aumento da capacidade de faturação:** o crescimento operacional projetado implicaria, mantidos os processos atuais, um aumento proporcional do esforço administrativo. O sistema proposto absorve esse crescimento sem incremento equivalente de recursos humanos, conferindo escalabilidade económica à operação.
- **Controlo de crédito a clientes corporativos:** a ausência de controlo sistemático das faturas pendentes de clientes com pagamento a 30 dias representa um risco de tesouraria não quantificado. O sistema introduz alertas automáticos de atraso de pagamento, mitigando este risco e melhorando o fluxo de caixa da oficina.

Dado o enquadramento académico do projeto, em que os custos de desenvolvimento são nulos para o cliente, por serem suportados no âmbito da unidade curricular LI4, o rácio custo-benefício é estruturalmente favorável. Os custos remanescentes limitam-se à infraestrutura de alojamento e manutenção futura, que se enquadram nas despesas operacionais correntes de qualquer PME com presença digital.

Conclui-se que a viabilidade económica é confirmada, com um ROI positivo esperado no curto prazo, impulsionado principalmente pela redução de ineficiências operacionais e pelo suporte ao crescimento do volume de negócio.

Viabilidade Temporal A viabilidade temporal avalia a exequibilidade do projeto dentro das restrições de calendário impostas pelo contexto em que se insere, assegurando que o âmbito definido é realizável no tempo disponível sem comprometer a qualidade dos artefactos produzidos. No presente caso, o projeto está alinhado com o semestre da unidade curricular LI4 da Universidade do Minho, o que estabelece um horizonte temporal fixo e não negociável, tornando a gestão disciplinada de prazos um fator crítico de sucesso.

A abordagem em cascata adotada, estruturada nas quatro fases descritas no Plano de Execução, é precisamente a estratégia que melhor se adapta a este tipo de restrição temporal. Os principais riscos temporais identificados e as respetivas estratégias de mitigação são:

Tabela 1.5: Avaliação de riscos temporais

Risco	Probabilidade	Impacto	Mitigação
Indisponibilidade dos <i>stakeholders</i> para validações intermédias	Média	Médio	Agendamento antecipado das sessões de validação; comunicação assíncrona por <i>email</i> como alternativa
Subestimação do esforço de implementação de funcionalidades específicas	Média	Alto	Priorização rigorosa do <i>backlog</i>
Curva de aprendizagem nas tecnologias selecionadas	Baixa	Médio	Seleção de tecnologias com que a equipa já possui familiaridade prévia

Conclui-se que o projeto é temporalmente viável, desde que a equipa mantenha aderência à metodologia definida. O calendário do semestre de LI4 é suficiente para a entrega de um sistema funcional, testado e documentado, cobrindo a totalidade dos objetivos primários e a maioria dos objetivos secundários definidos.

1.1.5 Recursos

Análise Crítica do Processo de Geração com IA

Prompt 5. Identificação de Recursos

Contexto: Sistema de gestão das "Oficinas Gengis Khan" (projeto da UC de LI4). Fase de definição do sistema, secção de Recursos.

Objetivo: Escrever a introdução da secção de Recursos, com 1 a 2 parágrafos densos e bem fundamentados.

Instruções de Escrita:

- Enquadramento teórico da gestão de recursos como pilar da engenharia de software
- Contextualização no projeto específico, justificando a transição do modelo manual (papel/Excel) para plataforma web
- Divisão explícita entre capital humano e recursos materiais/tecnológicos

Restrições: Usar nomes reais dos intervenientes. Sem nomes fictícios nem humorísticos. Fornecer apenas a introdução, sem tabelas.

Análise Crítica

Esta interação ilustrou de forma particularmente nítida uma vantagem do recurso ao LLM, transversal às secções anteriores e merecedora de destaque a esta altura do relatório, designadamente a capacidade de associar rapidamente informação dispersa pelas entrevistas e demais artefactos de levantamento à estrutura textual destinada a integrá-la. Onde a equipa teria, por método tradicional, de percorrer integralmente a transcrição da Entrevista-001, sinalizar as passagens relevantes e organizar manualmente o seu mapeamento aos atributos das tabelas, o modelo procedeu à extração, classificação e correlação dessa informação de forma quase instantânea, dispensando a leitura iterativa habitualmente exigida por esta operação. O esforço de validação pela equipa cingiu-se, em consequência, à verificação por amostragem do conteúdo produzido contra o levantamento original, tendo a única correção significativa incidido sobre os nomes fictícios e humorísticos inicialmente atribuídos aos *stakeholders*, substituídos pelos nomes reais dos intervenientes.

A concretização de um sistema de informação com a complexidade exigida pelo presente projeto pressupõe uma gestão rigorosa dos recursos disponíveis, entendida como um pilar estruturante do ciclo de vida de desenvolvimento de *software*. A identificação, alocação e monitorização dos ativos necessários, sejam eles humanos, materiais ou tecnológicos, constitui uma condição prévia indispensável para assegurar que as competências técnicas e as ferramentas adequadas se encontram disponíveis no momento certo, minimizando o risco de bloqueios operacionais e garantindo o cumprimento dos objetivos definidos dentro das restrições temporais impostas pelo calendário da unidade curricular LI4.

Recursos humanos A alocação de recursos humanos reflete a colaboração estreita entre a equipa operacional da oficina, os estudantes de Engenharia de Software responsáveis pelo desenvolvimento e o docente da unidade curricular, que assegura o enquadramento académico do projeto.

Tabela 1.6: Distribuição dos recursos humanos

Papel	Responsável	Contribuição principal
Gerente da Oficina	António Silva	Validação de requisitos de negócio; definição de parâmetros (taxas, margens, IVA); aprovação de entregas
Mecânico Chefe	Ricardo Costa	Conhecimento operacional dos processos de diagnóstico e reparação; validação de usabilidade
Mecânico Auxiliar	Miguel Abrantes	Conhecimento operacional dos processos de diagnóstico e reparação; validação de usabilidade
5 Estudantes de Software (LI4)	Francisco Martins, Nuno Rebelo, Marco Sèvegrand, Lucas Robertson, Marco Ferreira	Ciclo completo de desenvolvimento, designadamente requisitos, modelação, arquitetura, implementação, testes e documentação, com recurso sistemático a LLMs como instrumento de apoio sob supervisão e validação crítica da equipa
Orientação Académica	Docentes da unidade curricular	Orientação académica do trabalho, validação do alinhamento com os objetivos da unidade curricular e enquadramento metodológico

Recursos materiais Os recursos materiais reúnem as ferramentas, plataformas e serviços que sustentam o desenvolvimento e a operação do sistema, agrupados por categoria funcional e descritos na Tabela 1.7.

Tabela 1.7: Distribuição dos recursos materiais

Categoria	Recurso
Hardware	Servidor de desenvolvimento/produção (local); estação de trabalho para desenvolvimento
Base de dados	PostgreSQL 16 (Prisma ORM como única autoridade de <i>schema</i> e migrações)
<i>Backend</i>	TypeScript sobre Node.js, <i>framework</i> Fastify organizada como monólito modular
<i>Frontend</i>	TypeScript com <i>framework</i> React 18, SPA empacotada com Vite e estilizada com Tailwind CSS
Contratos partilhados	Pacote de esquemas Zod reutilizados entre <i>backend</i> e <i>frontend</i>
Gestão do <i>monorepo</i>	pnpm com Turborepo para orquestração de <i>builds</i> e tarefas
Notificações	Serviço SMTP (servidor de <i>email</i> existente da oficina ou fornecedor gratuito)
Modelação	Ferramenta UML (PlantUML, Visual Paradigm)
Apoio IA	Modelos de linguagem (GitHub Copilot, Claude)

1.1.6 Plano de Execução

Análise Crítica do Processo de Geração com IA

Prompt 6. Plano de Execução

Contexto: Sistema de gestão das "Oficinas Gengis Khan" (projeto da UC de LI4), calendário do semestre académico fixado pela unidade curricular.

Objetivo: Estruturar o plano de execução do projeto em quatro fases alinhadas com o calendário da UC LI4, identificando as atividades principais e os entregáveis de cada fase.

Instruções de Escrita:

- Cada fase deve ter início, fim e entregáveis claros
- Justificar o alinhamento de cada fase com etapas reconhecidas do ciclo de vida (requisitos, design, implementação, V&V)
- Indicar dependências sequenciais entre fases

Restrições: Forneça a tabela do plano e um parágrafo introdutório. Sem detalhe de atribuições individuais.

[Gerado no mesmo fluxo de interacao que as secções de Viabilidade e Recursos.]

Análise Crítica

O LLM propôs uma estrutura em quatro fases alinhada com a abordagem em cascata adotada para o ciclo de vida do projeto, que a equipa aceitou na íntegra nesta etapa. Posteriormente, no decurso da implementação, viria a perceber-se que a Fase 3 exigiria, em específico, uma execução incremental, em consequência do modo de operação dos LLM enquanto agentes de geração de código. A distinção entre o ciclo de vida do projeto e o ciclo de produção de código, bem como as suas implicações sobre o método de trabalho da equipa, será desenvolvida em maior detalhe no Capítulo 3.

O plano de execução organiza o projeto numa abordagem em cascata, dividida em quatro fases claras que acompanham o calendário académico da UC LI4. Esta estrutura permite entregar valor progressivo, começando pelo essencial e evoluindo para funcionalidades avançadas, enquanto se mantém controlo sobre prazos, qualidade e *feedback* contínuo dos *stakeholders*. Cada fase tem objetivos específicos e entregas tangíveis, garantindo que o sistema evolui de forma controlada e alinhada com as necessidades reais da oficina.

Tabela 1.8: Plano de execução

Fase	Designação	Atividades principais
1	Requisitos e Modelação	Sessão de elicitação com <i>stakeholders</i> ; síntese de ata; definição de casos de uso, <i>user stories</i> e requisitos (funcionais e não funcionais); modelação de domínio (diagrama de classes UML)
2	Arquitetura e Desenho	Definição da arquitetura em camadas; desenho do esquema relacional; especificação de <i>endpoints</i> REST/API; definição do modelo de estados da OS
3	Implementação	Codificação integral do sistema segundo o desenho aprovado nas fases anteriores, abrangendo os módulos de clientes, equipamentos, ordens de serviço, intervenções, <i>stock</i> , faturação, notificações, fornecedores e relatórios
4	Testes e Validação	Testes unitários, de integração e de aceitação; validação com <i>stakeholders</i> ; verificação de conformidade (faturação sequencial, RGPD); documentação final

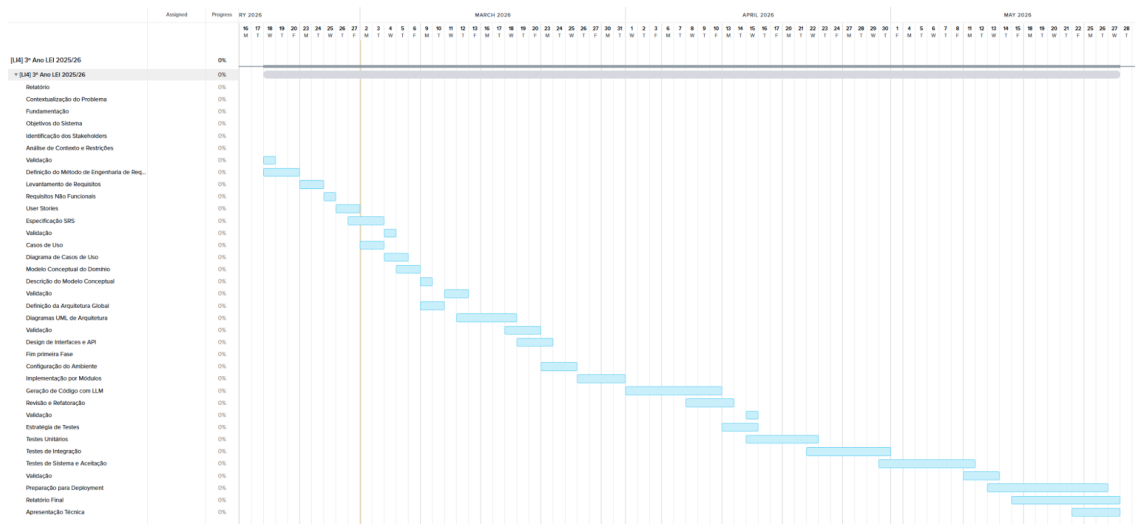


Figura 1.1: Diagrama de Gantt do plano de execução do projeto.

1.2 Definição do Domínio

O presente capítulo documenta a fase de definição do domínio do sistema, na qual se procede à caracterização do setor, à identificação das necessidades operacionais, à delimitação do âmbito funcional e à modelação conceptual das entidades e relações que estruturam o negócio das *Oficinas Gengis Khan*.

1.2.1 Introdução e Contexto do Domínio

Análise Crítica do Processo de Geração com IA

Prompt 7. Introdução ao Domínio

Contexto: Projeto da UC de LI4 (UMinho), sistema de gestão para oficina de reparação de trotinetes. Etapa 1 do ciclo de vida em cascata, fase de Conceção e Engenharia de Requisitos.

Objetivo: Elaborar a análise de contexto e a definição do domínio do sistema, identificando as entidades principais (Cliente, Trotinete, Peça, etc.) e as relações entre elas.

Instruções de Escrita:

- Caracterizar o ecossistema da oficina e o posicionamento no setor
- Identificar entidades de domínio com nomes claros e responsabilidades distintas
- Explicitar as relações estruturais entre entidades

Restrições: Postura de consultor de Engenharia de Software. Ser específico e identificar edge cases. Não escrever código.

Análise Crítica

O LLM produziu uma análise de contexto abrangente que identificava corretamente o posicionamento do setor de reparação de trotinetes no ecossistema de mobilidade urbana sustentável. A caracterização técnica dos equipamentos (componentes eletrónicos, mecânicos, sistemas de travagem regenerativa) foi considerada precisa e útil para fundamentar a complexidade do domínio. O modelo identificou seis necessidades críticas que a equipa validou integralmente por estarem alinhadas com os problemas reportados na Entrevista-001. A principal intervenção da equipa consistiu em reorganizar o texto para separar claramente a introdução conceptual, a caracterização do setor e as necessidades identificadas, que o LLM tinha apresentado de forma menos estruturada (P-07).

Nota. As análises críticas dos cinco *prompts* que se seguem nesta secção replicam o mesmo padrão sistemático, designadamente avaliação do conteúdo gerado, identificação de eventuais desvios ou correções necessárias e aceitação final. Para evitar a repetição desse esquema ao longo da secção, regista-se em cada subsecção apenas o resultado essencial da interação, remetendo-se a discussão detalhada, em conjunto com o respetivo enunciado, para a entrada correspondente no Anexo A (*prompts* P-08 a P-12).

1.2.1.1 Caracterização do Setor

O presente documento tem como objetivo definir e caracterizar o domínio de aplicação do sistema de *software* a desenvolver no âmbito da unidade curricular de Laboratórios de Informática IV (LI4), designadamente um sistema de gestão integrado para oficinas de reparação de trotinetes elétricas.

A crescente popularização das trotinetes elétricas como meio de transporte urbano, aliada à necessidade de manutenção periódica e reparação destes equipamentos, justifica o desenvolvimento de soluções tecnológicas que apoiem a gestão eficiente das atividades inerentes a este tipo de estabelecimentos comerciais. Neste sentido, o sistema a desenvolver visa colmatar lacunas identificadas nos processos operacionais de oficinas de pequena e média dimensão, caracterizadas pela utilização de métodos manuais de registo e controlo, nomeadamente através de cadernos físicos e folhas de cálculo dispersas. Tal abordagem resulta em ineficiências operacionais, perda de informação crítica, dificuldades na rastreabilidade de intervenções e na tomada de decisões baseadas em dados consolidados.

O setor de reparação de trotinetes elétricas insere-se no contexto mais amplo da mobilidade urbana sustentável e apresenta características específicas que o distinguem de outras atividades de manutenção de equipamentos. As trotinetes elétricas são compostas por diversos componentes eletrônicos e mecânicos de elevada especialização, incluindo sistemas de propulsão elétrica, baterias de íons de lítio, controladores eletrônicos, sistemas de travagem regenerativa e componentes estruturais sujeitos a desgaste acelerado devido às condições de utilização urbana.

As oficinas especializadas neste segmento enfrentam desafios particulares relacionados com a gestão de inventário de peças de substituição, que inclui componentes de diferentes fabricantes, versões de *firmware* e compatibilidades específicas entre modelos. Adicionalmente, a natureza técnica das intervenções requer registo detalhado de diagnósticos, procedimentos executados e peças substituídas, não apenas para efeitos de faturação, mas também para garantir a rastreabilidade de intervenções e a conformidade com eventuais obrigações de garantia.

1.2.1.2 Necessidades Identificadas

A análise do contexto operacional de oficinas de reparação de trotinetes permitiu identificar as seguintes necessidades críticas:

- **Centralização de informação:** necessidade de consolidar dados de clientes, equipamentos, histórico de intervenções, inventário de peças e transações financeiras numa plataforma única e coerente, substituindo métodos de registo fragmentados e propensos a perdas de informação.
- **Rastreabilidade operacional:** capacidade de documentar de forma sistemática e rigorosa todas as fases do ciclo de vida de uma ordem de serviço, desde a receção do equipamento até à sua entrega ao cliente, incluindo diagnósticos técnicos, intervenções realizadas, peças aplicadas e custos associados.
- **Gestão de inventário:** controlo eficaz do *stock* de peças de substituição, com alertas automáticos de reposição, gestão de fornecedores e histórico de encomendas, permitindo otimizar custos de inventário e minimizar ruturas de *stock* que comprometem prazos de reparação.
- **Automação de comunicações:** necessidade de automatizar processos de comunicação com clientes e fornecedores, nomeadamente através do envio de notificações de estado de reparações, orçamentos para aprovação e requisições de peças, reduzindo a carga de trabalho administrativo e melhorando a experiência do cliente.
- **Suporte à decisão:** produção de relatórios analíticos que permitam identificar padrões de falhas, peças mais consumidas, rentabilidade por tipo de intervenção, tempos médios de reparação e indicadores de desempenho operacional, fundamentando decisões estratégicas de gestão.

- **Conformidade regulamentar:** garantir a conformidade com requisitos legais aplicáveis, nomeadamente no que concerne à faturação sequencial obrigatória, proteção de dados pessoais segundo o Regulamento Geral sobre a Proteção de Dados (RGPD) e eventual rastreabilidade de intervenções para efeitos de garantia.

1.2.2 Âmbito do Sistema e Componentes Funcionais

Análise Crítica do Processo de Geração com IA

Prompt 8. Âmbito do Sistema

Contexto: Sistema de gestão das "Oficinas Gengis Khan" (projeto da UC de LI4). Fase de definição do domínio, sequência da Subseção anterior.

Objetivo: Identificar as entidades principais e as relações entre elas, organizando-as em componentes funcionais coerentes que sirvam de base a engenharia de requisitos subsequente.

Instruções de Escrita:

- Cada componente funcional descrito com responsabilidades distintas e fronteiras claras
- Cobertura abrangente do domínio operacional, administrativo e de gestão
- Nível de detalhe suficiente para sustentar o levantamento de requisitos sem precisar de retrabalho

Restrições: Forneça apenas a estrutura em componentes e a descrição de cada um. Sem requisitos específicos nem detalhes de implementação.

[Gerado no mesmo fluxo de interação que a Subseção anterior.]

Análise Crítica

Conteúdo aceite na íntegra após validação cruzada da divisão em seis componentes funcionais contra a informação recolhida na Entrevista-001, sem lacunas significativas identificadas. Bloco completo no Anexo A (P-08).

O sistema a desenvolver visa cobrir de forma integrada e abrangente todas as atividades operacionais, administrativas e de gestão de uma oficina de reparação de trotinetes elétricas. O âmbito funcional do sistema estrutura-se em seis componentes funcionais principais, cada um responsável por um conjunto coerente de funcionalidades e processos de negócio.

1.2.2.1 Componente de Gestão de Clientes e Equipamentos

Este componente funcional é responsável pela gestão centralizada de toda a informação relativa aos clientes da oficina e aos equipamentos que estes submetem para reparação. Engloba o registo completo de dados de identificação dos clientes, distinguindo entre clientes individuais (particulares) e clientes corporativos (empresas), incluindo nome ou denominação social, Número de Identificação Fiscal, contactos telefónicos e eletrónicos, morada e condições comerciais específicas (prazos de pagamento, limites de crédito). A gestão de clientes corporativos requer funcionalidades adicionais de controlo de crédito e acompanhamento de faturas pendentes, dado que estes clientes usufruem tipicamente de prazos de pagamento alargados. O componente engloba também a caracterização técnica das trotinetes elétricas (marca, modelo, número de série único e observações quanto ao estado geral do equipamento no momento da receção), permitindo consultar o histórico completo de interações de cada cliente com a oficina, bem como o histórico de todas as intervenções realizadas sobre um equipamento específico, identificado univocamente pelo seu número de série.

1.2.2.2 Componente de Diagnóstico e Reparação

Constitui o núcleo operacional do sistema, gerindo todo o ciclo de vida das ordens de serviço desde a sua criação até à conclusão das reparações. Cada ordem de serviço é identificada por um número sequencial único e documenta o problema reportado pelo cliente, o diagnóstico técnico realizado pelos mecânicos, as múltiplas intervenções executadas (cada uma com descrição detalhada, tempo de execução registado através de cronómetro integrado e identificação do mecânico responsável) e as peças utilizadas em cada intervenção com respetivas quantidades. O componente implementa um modelo de estados que reflete o progresso da reparação (Recebida, Em Diagnóstico, Aguarda Aprovação do Cliente, Aguarda Peças, Em Reparação, Concluída e Entregue ao Cliente), registando automaticamente *timestamps* de todas as transições de estado para permitir análises de tempos de ciclo e identificação de gargalos operacionais.

1.2.2.3 Componente de Gestão de *Stock* de Peças

Responsável pelo controlo integral do inventário de peças de substituição, este componente funcional mantém para cada referência de peça a sua descrição técnica, o fornecedor associado, o preço de custo de aquisição, o preço de venda ao cliente (com margem configurável), a quantidade atualmente em *stock* e o nível de *stock* mínimo que desencadeia alertas de reposição. O sistema atualiza automaticamente as quantidades em *stock* quando peças são associadas a intervenções e gera notificações visuais quando os níveis de inventário se aproximam ou ultrapassam os limiares mínimos definidos, permitindo uma gestão proativa de aprovisionamento e minimizando ruturas de *stock* que possam atrasar reparações.

1.2.2.4 Componente de Faturação e Relatórios

Este componente funcional assegura a geração automática de documentos de faturação com base nas ordens de serviço concluídas, calculando o valor total através da soma dos preços de venda das peças utilizadas e do custo de mão de obra (calculado multiplicando o tempo total de intervenções por uma taxa horária configurável), acrescido do Imposto sobre o Valor Acrescentado (IVA) aplicável. As faturas são numeradas sequencialmente sem quebras em conformidade com requisitos legais e podem apresentar diferentes estados de pagamento (Pendente, Paga, Atrasada). Para além da vertente de faturação, o componente disponibiliza um conjunto abrangente de relatórios analíticos, incluindo análises de faturação por período temporal, identificação de peças mais utilizadas, custos de aquisição de peças, margens de lucro por reparação, problemas técnicos mais frequentes, tempos médios de reparação, distribuição de ordens de serviço por estado e métricas de produtividade por mecânico, fornecendo informação estruturada para suporte a decisões de gestão estratégica e operacional.

1.2.2.5 Componente de Comunicação com Fornecedores

Componente funcional dedicado à gestão das relações comerciais com fornecedores de peças de substituição. Mantém um registo completo de fornecedores (incluindo identificação, contactos e condições comerciais praticadas), permite a criação de encomendas de reposição de *stock* (manual ou gerada automaticamente a partir de alertas de *stock* crítico) e gere o envio dessas encomendas por correio eletrónico. O sistema rastreia o estado de cada encomenda (“Solicitada” e “Recebida”), regista datas de encomenda, datas efetivas de receção e mantém um histórico completo de transações com cada fornecedor, facilitando a avaliação de desempenho de fornecedores e a negociação de condições comerciais.

1.2.2.6 Componente de Notificações ao Cliente

Componente funcional responsável pela automação de comunicações com clientes ao longo do ciclo de vida das reparações. Implementa um mecanismo de notificações automáticas por correio eletrónico (com possibilidade futura de integração de SMS), acionadas em eventos específicos do processo de reparação, designadamente a confirmação de receção do equipamento com número de ordem de serviço atribuído, o pedido de aprovação de orçamento quando o estado transita para “Aguarda Aprovação do Cliente”, a notificação de conclusão da reparação quando o estado atinge “Concluída” e alertas proativos em caso de atrasos significativos (por exemplo, equipamento há mais de 10 dias em “Aguarda Peças”). O sistema utiliza *templates* de mensagem configuráveis e personalizáveis para cada tipo de evento e mantém um registo completo de todas as notificações enviadas para efeitos de auditoria e controlo de qualidade do serviço ao cliente.

1.2.3 Intervenientes e Papéis

Análise Crítica do Processo de Geração com IA

Prompt 9. Intervenientes e Papéis

Contexto: Sistema de gestão das "Oficinas Gengis Khan" (projeto da UC de LI4). Definição do domínio, sequência da identificação dos componentes funcionais.

Objetivo: Identificar e caracterizar todos os intervenientes do sistema, os seus papéis, responsabilidades e níveis de acesso.

Instruções de Escrita:

- Distinguir utilizadores diretos (operacionais) de intervenientes indiretos (clientes, fornecedores)
- Para cada papel, indicar responsabilidades funcionais e fronteiras de autoridade
- Cobrir interlocutores externos relevantes ao fluxo do negócio

Restrições: Forneça a caracterização em prosa por papel. Sem detalhe de implementação de RBAC ou autenticação.

[Gerado no mesmo fluxo que a definição do domínio.]

Análise Crítica

Conteúdo aceite na íntegra, com ajustes menores de pontuação, após confirmação de que a distinção entre utilizadores diretos (gerente, mecânicos) e indiretos (clientes, fornecedores) era adequada à fase de requisitos. Bloco completo no Anexo A (P-09).

O sistema de gestão para oficina de reparação de trotinetes envolve diferentes categorias de intervenientes, cada um com necessidades, responsabilidades e níveis de acesso distintos:

1.2.3.1 Gerente da Oficina

O gerente ou proprietário da oficina assume um papel de gestão estratégica e operacional, necessitando de acesso integral a todas as funcionalidades do sistema. As suas responsabilidades incluem a definição de parâmetros de configuração do sistema (taxas horárias de mão de obra, taxas de IVA, margens de comercialização de peças), a gestão de fornecedores e aprovação de encomendas, a consulta de relatórios financeiros e de desempenho para suporte à tomada de decisões e a supervisão geral das operações da oficina. O gerente é também responsável pela gestão de utilizadores do sistema e atribuição de perfis de acesso.

1.2.3.2 Mecânicos

Os técnicos mecânicos constituem os utilizadores operacionais primários do sistema, responsáveis pela execução de diagnósticos e reparações. As suas interações com o sistema centram-se na receção de equipamentos de clientes, criação de ordens de serviço, registo de diagnósticos técnicos, documentação de intervenções realizadas (incluindo descrições detalhadas, tempos de execução e peças utilizadas), atualização de estados das ordens de serviço à medida que as reparações progridem e consulta do histórico de intervenções anteriores em equipamentos recorrentes. Os mecânicos têm acesso restrito a funcionalidades operacionais, não dispondo de permissões para aceder a relatórios financeiros, definir preços ou gerir fornecedores.

1.2.3.3 Clientes

Os clientes da oficina são os proprietários das trotinetes elétricas submetidas à reparação e podem ser categorizados em dois tipos distintos, designadamente clientes individuais (particulares) e clientes corporativos (empresas). Os clientes individuais representam a maioria das interações e procedem habitualmente ao pagamento no momento da recolha do equipamento reparado. Já os clientes corporativos, tipicamente empresas de serviços de mobilidade partilhada ou frotas empresariais, podem beneficiar de condições comerciais diferenciadas, nomeadamente prazos de pagamento alargados (por exemplo, pagamento a 30 dias), o que implica funcionalidades específicas de controlo de crédito e gestão de faturas pendentes.

Embora não interajam diretamente com o sistema através de uma interface dedicada, ambos os tipos de clientes são beneficiários indiretos do mesmo através da receção de notificações automáticas que os mantêm informados sobre o progresso das reparações, pedidos de aprovação de orçamentos e avisos de conclusão.

1.2.3.4 Fornecedores

Os fornecedores de peças de substituição são entidades externas à oficina que recebem encomendas geradas através do sistema. Embora não tenham acesso direto à plataforma, beneficiam da comunicação estruturada e automática de requisições de peças, que chegam via correio eletrónico com especificações detalhadas de referências e quantidades necessárias, facilitando o processamento de encomendas e reduzindo erros de comunicação.

1.2.4 Modelo de Domínio

Análise Crítica do Processo de Geração com IA

Prompt 10. Modelo de Domínio

Contexto: Sistema de gestão das "Oficinas Gengis Khan" (LI4-UMinho). Modelação de Domínio com diagrama de classes já definido. Escreve-se a introdução e fundamentação do modelo.

Objetivo: Produzir um texto académico (PT-PT) que explique o que é o Diagrama de Domínio, por que é necessário, e como foi derivado a partir das fontes de conhecimento do domínio.

Instruções de Escrita:

- Definir Diagrama de Domínio como representação conceptual das entidades e relações do negócio
- Justificar enquanto blueprint para a base de dados e lógica de negócio, eliminando ambiguidade dos registos manuais
- Descrever o processo de derivação (extração de substantivos como entidades, verbos como associações, regras de negócio como cardinalidades)
- Mencionar transição do sistema informal para modelo normalizado, em coerência com o crescimento de 150 para 500 OS/mes

Restrições: Forneça apenas o texto introdutório e a fundamentação. Sem repetir conteúdos do diagrama em si.

Análise Crítica

Conteúdo aceite após uso do LLM em duas fases (definição das entidades e relações e, em seguida, fundamentação do modelo), com a explicação metodológica da derivação por análise linguística das notas de entrevista a constituir o contributo mais valioso. Bloco completo no Anexo A (P-10).

A modelação do domínio constitui a ponte entre a realidade operacional das *Oficinas Gengis Khan* e a arquitetura técnica do sistema a desenvolver. Ao contrário de um diagrama de implementação, o modelo de domínio abstrai deliberadamente os detalhes tecnológicos, focando-se na captura fiel do conhecimento do negócio tal como foi descrito junto dos *stakeholders* da oficina.

A justificação para a construção deste modelo incide na necessidade imperativa de eliminar a ambiguidade e a entropia informacional que caracterizam o sistema de gestão vigente. Na situação atual, as relações entre clientes, equipamentos, intervenções e transações financeiras existem apenas de forma implícita, distribuídas por cadernos físicos, folhas avulsas e ficheiros Excel sem integração, tornando a rastreabilidade de qualquer operação um processo manual

e falível. A formalização destas relações num modelo de domínio explícito e normalizado é, portanto, uma pré-condição técnica incontornável para suportar o crescimento operacional projetado.

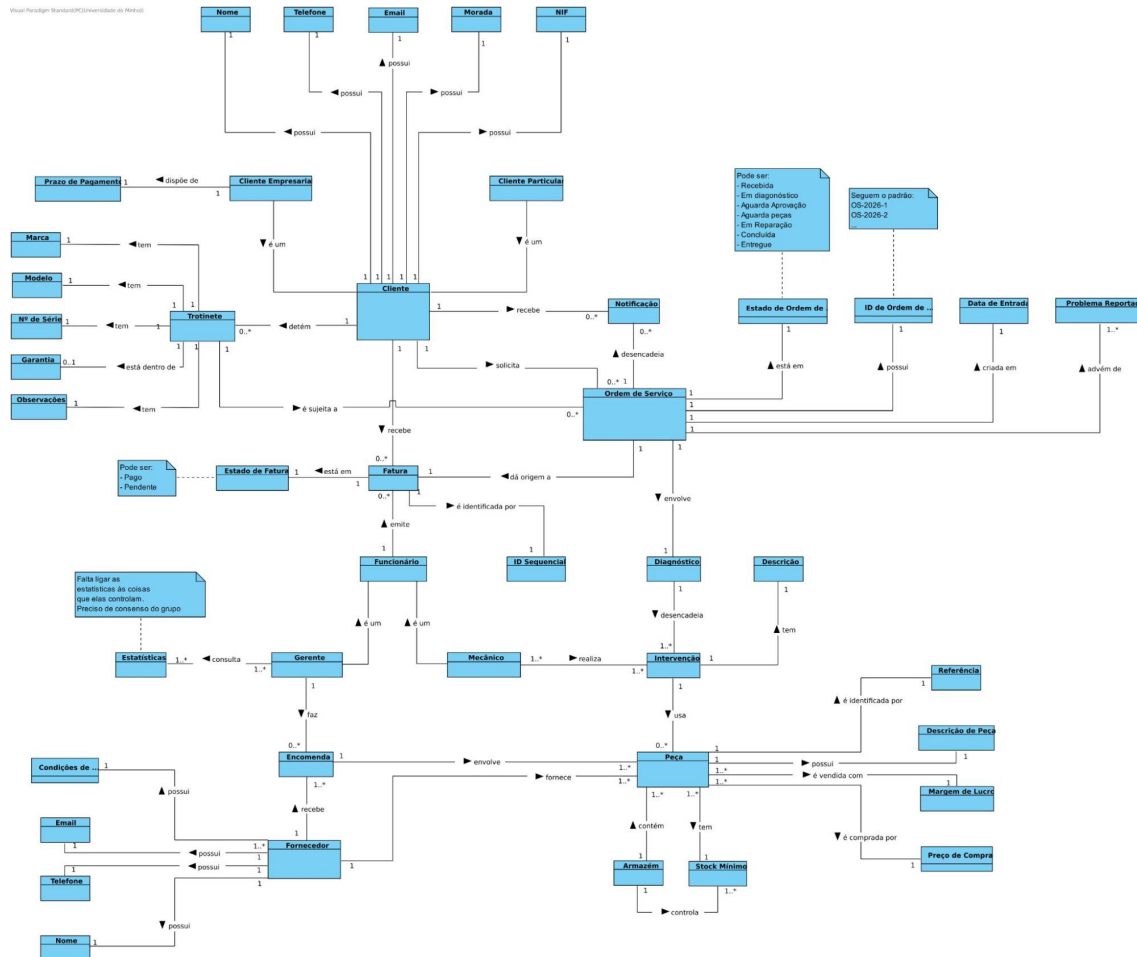


Figura 1.2: Modelo de domínio do sistema de gestão das *Oficinas Gengis Khan*.

O diagrama não resulta de uma construção arbitrária, mas antes de um processo sistemático de derivação a partir das fontes de conhecimento do domínio recolhidas durante a fase de definição do domínio, designadamente a transcrição integral da Entrevista-001. Metodologicamente, o processo de derivação assentou na técnica de análise linguística aplicada às notas de entrevista. Os substantivos recorrentes com relevância operacional, tais como cliente, troinete, ordem de serviço, intervenção, peça, diagnóstico, fatura, fornecedor, encomenda e funcionário, foram identificados como candidatos a entidades do modelo. Os verbos e expressões relacionais como “possui”, “sujeita a”, “baseia-se em”, “gera”, “fornece”, “origina” e “é responsável por” foram mapeados como associações entre essas entidades. As cardinalidades de cada relação foram determinadas pela análise das regras de negócio explicitadas na entrevista, por exemplo o facto de um cliente poder possuir múltiplas trotinetes mas cada troinete pertencer a um

único cliente, ou o facto de uma ordem de serviço poder englobar múltiplas intervenções, cada uma da responsabilidade de um mecânico específico e potencialmente consumidora de várias peças de *stock*.

O resultado deste processo é um modelo de domínio que assegura rastreabilidade total ao longo de todo o ciclo de vida operacional da oficina, desde a receção da trotinete e o registo dos dados do cliente, passando pela abertura da ordem de serviço, execução do diagnóstico técnico, realização das intervenções e gestão das peças consumidas, até à emissão da fatura final e ao controlo do seu estado de pagamento.

1.2.5 *Stakeholders* e Partes Interessadas

Análise Crítica do Processo de Geração com IA

Prompt 11. Identificação de *Stakeholders*

Contexto: Sistema de gestão das "Oficinas Gengis Khan" (LI4).
Extensão natural da definição do domínio, na qual se consolidou o quadro operacional.

Objetivo: Identificar e caracterizar de forma exaustiva todos os *stakeholders* do sistema, distinguindo diretos, indiretos, técnicos e regulamentares.

Instruções de Escrita:

- Para cada *stakeholder*, indicar papel, responsabilidades, interesses e nível de influência
- Incluir entidades regulamentares (Autoridade Tributária, CNPD) e a equipa de desenvolvimento como *stakeholders* técnicos
- Apresentar uma matriz final com estratégia de gestão diferenciada por *stakeholder*

Restrições: Não incluir estimativas percentuais sem base documental. Limitar-se a informação verificável na Entrevista-001.

[Produzida como extensão natural da definição do domínio.]

Análise Crítica

Conteúdo aceite na sua quase totalidade, com correção da única falha relevante, designadamente a remoção de estimativas percentuais de distribuição de clientes sem base documental na Entrevista-001. A inclusão de entidades regulamentares (Autoridade Tributária, CNPD) e da equipa de desenvolvimento como *stakeholders* técnicos foi considerada um contributo valioso. Bloco completo no Anexo A (P-11).

A identificação e caracterização rigorosa dos *stakeholders* constitui um elemento crítico no processo de engenharia de requisitos, assegurando que o sistema a desenvolver contempla as necessidades, expectativas e restrições de todos os intervenientes relevantes no projeto. No contexto da engenharia de *software*, os *stakeholders* são todas as entidades, sejam elas pessoas, organizações ou entidades regulamentares, que de forma direta ou indireta influenciam ou são influenciadas pelo desenho, implementação e operação do sistema. A sua identificação exaustiva é determinante para garantir que nenhuma perspetiva relevante é omitida durante a especificação de requisitos. O sistema de gestão para oficina de reparação de trotinetes elétricas envolve múltiplos tipos de *stakeholders*, cada um com papéis, responsabilidades, interesses e níveis de influência distintos no ciclo de vida do sistema.

1.2.5.1 *Stakeholders* Diretos

Gerente da Oficina (António Silva) Papel e Responsabilidades: O gerente assume a posição de utilizador primário e decisor estratégico do sistema, acumulando responsabilidades de gestão operacional, financeira e administrativa da oficina. Cabe-lhe a configuração global do sistema (definição de taxas horárias de mão de obra fixadas em 25 €/hora, taxa de IVA de 23 %, margens de comercialização de peças e níveis de *stock* mínimo para cada referência), gestão do registo de fornecedores e aprovação de encomendas de reposição de *stock*, supervisão do ciclo de faturação sequencial e acompanhamento de pagamentos pendentes (particularmente relevante para clientes corporativos com prazos de 30 dias) e análise de indicadores de desempenho através de relatórios analíticos.

Interesses: Maximização da eficiência operacional da oficina e capacidade de absorver o crescimento projetado sem aumento proporcional de recursos administrativos; obtenção de visibilidade completa sobre métricas financeiras (faturação mensal, margens de lucro por reparação, custos de aquisição de peças); capacidade de tomada de decisões baseada em dados consolidados (identificação de problemas técnicos recorrentes, avaliação de desempenho de fornecedores, análise de produtividade por mecânico); e conformidade legal rigorosa com obrigações fiscais (faturação sequencial sem quebras) e regulamentares (RGPD).

Nível de Influência: Elevado. O gerente constitui o *stakeholder* com maior poder decisório e influência sobre o projeto, sendo simultaneamente o patrocinador do investimento, o validador final de requisitos e o responsável pela aceitação do sistema. Possui capacidade de definir prioridades funcionais, aprovar ou rejeitar propostas de *design* e determinar critérios de sucesso do projeto. A satisfação deste *stakeholder* é crítica para a viabilidade e adoção do sistema.

Frequência de Interação: Acesso diário durante aproximadamente 2 horas, concentrado em atividades de consulta de relatórios (início do dia), aprovação de orçamentos pendentes, emissão de faturas para clientes corporativos e monitorização de alertas críticos de *stock*.

Mecânicos (Ricardo Costa e Miguel Abrantes) Papel e Responsabilidades: Os mecânicos constituem os utilizadores operacionais intensivos do sistema, responsáveis pela execução de todas as atividades técnicas quotidianas. Ricardo Costa, na qualidade de chefe de oficina, possui responsabilidades adicionais de coordenação técnica e supervisão de intervenções complexas. As responsabilidades operacionais incluem receção de equipamentos de clientes com registo de dados técnicos completos (marca, modelo, número de série único da trotinete), criação de ordens de serviço sequenciais, documentação detalhada de problemas reportados pelos clientes e diagnósticos técnicos efetuados, gestão do ciclo de vida completo das ordens de serviço através de transições de estado (desde “Recebida” até “Entregue ao Cliente”), execução e registo de múltiplas intervenções por ordem de serviço com controlo rigoroso de tempos através de cronómetro integrado, associação de peças consumidas em cada intervenção com atualização automática de inventário e comunicação com clientes para aprovação de orçamentos quando reparações excedem limiares predefinidos.

Interesses: Interface intuitiva e responsiva que minimize tempo de registo administrativo, permitindo maximizar tempo dedicado a reparações técnicas; acesso rápido ao histórico completo de intervenções anteriores em equipamentos recorrentes, facilitando diagnósticos e identificação de problemas crónicos; funcionalidade de cronómetro automático para registo rigoroso de tempos de mão de obra sem necessidade de estimativas manuais; e sistema de alertas de disponibilidade de peças em *stock* antes de iniciar intervenções, evitando reparações incompletas por rutura de *stock*.

Nível de Influência: Médio-Alto. Embora não possuam poder decisório sobre requisitos de alto nível ou prioridades estratégicas, os mecânicos exercem influência significativa sobre a usabilidade e eficácia operacional do sistema. A sua aceitação e adoção efetiva do sistema determinam o sucesso da implementação e o seu *feedback* sobre fluxos de trabalho e ergonomia de interface é crítico para a qualidade do produto final. Ricardo Costa, como chefe de oficina, possui canal direto de comunicação com o gerente para escalar questões operacionais relevantes.

Frequência de Interação: Acesso intensivo durante 6 horas diárias (horário de funcionamento da oficina), com interações frequentes de curta duração para registo de diagnósticos, início/pausa/conclusão de intervenções e consulta de *stocks*.

1.2.5.2 *Stakeholders* Indiretos

Clientes Individuais (Particulares) Papel e Responsabilidades: Proprietários de trotinetes elétricas que submetem equipamentos para reparação em caráter individual. Representam a maioria do volume de transações da oficina. São responsáveis por fornecer informação precisa sobre problemas reportados, autorizar orçamentos de reparação quando solicitados e efetuar pagamento no momento da recolha do equipamento reparado.

Interesses: Transparência completa sobre o estado de reparação do seu equipamento através de notificações automáticas proativas; clareza na discriminação de custos (peças, mão de obra, impostos) antes da aprovação de orçamentos; prazos de reparação previsíveis com alertas antecipados em caso de atrasos (particularmente quando aguardam peças de fornecedores); e confiança na rastreabilidade de intervenções realizadas com registo de peças genuínas aplicadas.

Nível de Influência: Baixo-Médio. Não possuem influência direta sobre requisitos funcionais ou decisões de *design* do sistema. Contudo, a experiência destes *stakeholders* com comunicações automáticas (qualidade, frequência, clareza das notificações) impacta diretamente a reputação da oficina e a retenção de clientes, constituindo um indicador indireto de qualidade do sistema.

Frequência de Interação: Indireto. Recebem 3 a 5 notificações por *email* ao longo do ciclo de vida de cada reparação (confirmação de receção, pedido de aprovação de orçamento se aplicável, notificação de conclusão, eventuais alertas de atraso), sem acesso direto à interface do sistema.

Clientes Corporativos (Empresas de Mobilidade Partilhada e Frotas) Papel e Responsabilidades: Entidades empresariais que gerem frotas de trotinetes elétricas (operadores de mobilidade partilhada, empresas com frotas corporativas para colaboradores) e submetem volumes significativos de equipamentos para manutenção preventiva e reparação. Contribuem desproporcionalmente para a faturação devido à recorrência e escala. São responsáveis por nomear um gestor de frotas como ponto de contacto único, autorizar orçamentos consolidados para múltiplas ordens de serviço e honrar prazos de pagamento acordados (tipicamente 30 dias).

Interesses: Relatórios analíticos agregados sobre padrões de falhas por modelo de trotinete, permitindo identificar equipamentos problemáticos e fundamentar decisões de substituição de frotas; faturação consolidada mensal em vez de faturas individuais por reparação, simplificando contabilidade e aprovações internas; notificações agrupadas para múltiplas ordens de serviço da mesma frota, evitando sobrecarga de comunicações; e condições comerciais diferenciadas (prazos de pagamento alargados) com controlo rigoroso de limites de crédito.

Nível de Influência: Médio. Embora não participem diretamente na definição de requisitos do sistema, estes clientes exercem influência significativa através do seu poder económico e volume de transações. Requisitos específicos para faturação consolidada, relatórios analíticos

personalizados e gestão de crédito foram incorporados no âmbito funcional do sistema em resposta direta às necessidades deste segmento de clientes.

Frequência de Interação: Indireto. Recebem comunicações agregadas semanais ou mensais (conforme volume), relatórios analíticos trimestrais e faturas consolidadas no final de cada período de faturação.

Fornecedores de Peças de Substituição Papel e Responsabilidades: Entidades externas especializadas no fornecimento de componentes de substituição para trotinetes elétricas (sistemas de travagem, baterias de íões de lítio, controladores eletrónicos, componentes estruturais). A oficina mantém relações comerciais com 2 fornecedores principais e recorre ocasionalmente a fornecedores secundários para peças específicas. São responsáveis por processar encomendas recebidas via *email*, confirmar disponibilidade e prazos de entrega e assegurar conformidade das peças entregues com especificações técnicas encomendadas.

Interesses: Receção de encomendas estruturadas e não ambíguas via *email*, com especificação completa de referências técnicas, quantidades e preços unitários acordados; minimização de comunicações manuais de *follow-up* através de informação completa na encomenda inicial; e visibilidade sobre histórico de transações para análise de padrões de procura e planeamento de *stocks*.

Nível de Influência: Baixo. Os fornecedores não possuem influência sobre requisitos funcionais do sistema, atuando como recetores passivos de encomendas geradas automaticamente. Contudo, a qualidade da estruturação das encomendas (clareza, completude) impacta diretamente a eficiência do processo de aprovisionamento e os prazos de entrega, afetando indiretamente os tempos de ciclo de reparações da oficina.

Frequência de Interação: Indireto. Recebem encomendas automáticas via *email* quando peças atingem níveis de *stock* mínimo (frequência variável conforme consumo) e são rastreados pelo sistema quanto a tempos de entrega e conformidade de encomendas para avaliação de desempenho.

1.2.5.3 Stakeholders Técnicos

Equipa de Desenvolvimento (Estudantes LI4) Papel e Responsabilidades: Equipa académica composta por 5 estudantes de Laboratórios de Informática IV, responsável pela análise de requisitos, *design* de arquitetura, implementação, testes e *deployment* do sistema. Com conhecimento académico das tecnologias especificadas (TypeScript em *frontend* e *backend*, React, Node.js, PostgreSQL, Git).

Interesses: Requisitos funcionais claramente definidos e estáveis, minimizando alterações de âmbito durante a implementação; restrições técnicas e tecnológicas explícitas que orientem decisões de arquitetura; disponibilidade do *stakeholder* gerente para esclarecimentos e validações iterativas; e prazo académico realista compatível com a complexidade funcional do sistema.

Nível de Influência: Médio. Capacidade de propor soluções técnicas alternativas, questionar viabilidade de requisitos dentro do prazo académico e negociar priorização de funcionalidades. Contudo, decisões finais sobre âmbito funcional e critérios de aceitação permanecem com o gerente/cliente.

Docente Orientador (LI4) Papel e Responsabilidades: Supervisão académica do projeto, validação de conformidade com objetivos pedagógicos da unidade curricular, avaliação da qualidade técnica da solução e garantia de cumprimento de prazos académicos.

Interesses: Aplicação rigorosa de metodologias de engenharia de *software* (elicitação de requisitos, modelação UML, testes sistemáticos); qualidade da documentação técnica produzida; e evidência de aprendizagem das tecnologias e conceitos lecionados.

Nível de Influência: Alto (contexto académico). Autoridade para requerer alterações metodológicas, impor restrições técnicas pedagógicas (tecnologias obrigatórias) e determinar critérios de avaliação que podem diferir de critérios de aceitação do cliente.

1.2.5.4 Entidades Regulamentares

Autoridade Tributária Portuguesa Papel e Responsabilidades: Entidade reguladora responsável pela fiscalização do cumprimento de obrigações fiscais, nomeadamente conformidade com requisitos de faturação sequencial sem quebras (Decreto-Lei n.º 28/2019) e comunicação de documentos fiscais ao sistema *e-Fatura*.

Interesses: Garantia de numeração sequencial rigorosa de faturas (FAT-2026-001, FAT-2026-002, ...) sem duplicações ou omissões; conformidade com taxas de IVA aplicáveis (23 % regime geral); e rastreabilidade completa de transações comerciais para efeitos de auditoria fiscal.

Nível de Influência: Alto. Requisitos de faturação sequencial constituem restrições legais não negociáveis que condicionam de forma determinante o *design* do módulo de faturação. Não conformidade implica sanções legais e inviabiliza a operação legal da oficina.

Comissão Nacional de Proteção de Dados (CNPd) Papel e Responsabilidades: Autoridade de controlo nacional para aplicação do Regulamento Geral sobre a Proteção de Dados (RGPD, Regulamento UE 2016/679). Supervisiona tratamento de dados pessoais de clientes (nome, NIF, contactos, morada) e colaboradores.

Interesses: Implementação de princípios de minimização de dados (recolha apenas de informação estritamente necessária); obtenção de consentimento explícito para tratamento de dados e envio de comunicações eletrónicas; garantia de direitos dos titulares (acesso, retificação, eliminação); implementação de medidas técnicas de segurança (criptação, controlo de acessos); e existência de *backups* com retenção limitada (30 dias especificados).

Nível de Influência: Alto. Requisitos RGPD constituem restrições legais obrigatórias que impactam *design* de base de dados (campos armazenados, políticas de retenção), autenticação e controlo de acessos e funcionalidades de gestão de consentimentos. Não conformidade implica coimas até 20 milhões de euros ou 4 % do volume de negócios anual global.

1.2.5.5 Matriz de *Stakeholders*

A tabela seguinte sintetiza os *stakeholders* identificados, categorizando-os segundo o poder de influência e o nível de interesse no sistema, orientando estratégias de gestão de *stakeholders* ao longo do projeto:

Tabela 1.9: Matriz de *stakeholders*

Stakeholder	Estratégia de Gestão
Gerente (António Silva)	Gerir de Perto: envolvimento contínuo em validações, comunicação semanal de progresso
Mecânicos (Ricardo Costa, Miguel Abrantes)	Manter Satisfeitos: testes de usabilidade iterativos, incorporação de <i>feedback</i> operacional
Autoridade Tributária	Manter Informado: garantir conformidade rigorosa com legislação fiscal
CNPD (RGPD)	Manter Informado: documentar conformidade com princípios de proteção de dados
Clientes Corporativos	Manter Satisfeitos: validar requisitos específicos (faturação consolidada, relatórios)
Docente Orientador	Gerir de Perto: alinhamento contínuo com objetivos pedagógicos e prazos
Equipa de Desenvolvimento	Gerir de Perto: gestão realista de âmbito face a prazo académico
Clientes Individuais	Monitorizar: validar qualidade de notificações via testes piloto
Fornecedores	Monitorizar: validar formato de encomendas automáticas

1.2.5.6 Considerações de Gestão de Partes Interessadas

A diversidade de *stakeholders* e partes interessadas identificados, com visões por vezes conflitantes (por exemplo prazos académicos apertados face ao âmbito funcional ambicioso do gerente), requer mecanismos estruturados de gestão de expectativas e priorização de requisitos. A estratégia de gestão de *stakeholders* para o projeto contempla os elementos seguintes.

Comunicação Regular: Reuniões semanais de validação com o gerente para demonstração de protótipos incrementais e obtenção de *feedback*; sessões quinzenais com mecânicos para validação de fluxos operacionais; e pontos de situação académicos com docente orientador.

Priorização por Criticidade: Requisitos associados a *stakeholders* regulamentares (faturação sequencial, RGPD) são não negociáveis e recebem prioridade máxima. Funcionalidades para clientes corporativos (relatórios analíticos complexos) são candidatas à implementação faseada se restrições de prazo assim o exigirem.

Validação Iterativa: Testes de usabilidade presenciais com mecânicos em ambiente real de oficina, simulando receção de equipamentos e registo de intervenções. Validação de *templates* de notificações com amostra de clientes individuais antes de *deployment* para evitar perceção de *spam*.

Gestão de Mudanças: Processo formal de controlo de mudanças para requisitos após a base inicial, exigindo aprovação do gerente e avaliação de impacto em prazo/recursos pela equipa de desenvolvimento, mitigando *scope creep* durante a fase de implementação.

1.2.6 Contexto e Restrições

Análise Crítica do Processo de Geração com IA

Prompt 12. Contexto e Restrições

Contexto: Sistema de gestão das "Oficinas Gengis Khan" (LI4). Fase de definição do domínio, posterior à identificação de *stakeholders* e anterior ao levantamento formal de requisitos.

Objetivo: Escrever a secção de "Contexto e Restrições" com parágrafo introdutório sobre a importância da análise prévia e desenvolvimento das quatro famílias de restrições (legais/regulamentares, temporais/académicas, tecnológicas e organizacionais).

Instruções de Escrita:

- Restrições legais com referência precisa à legislação aplicável (RGPD, Decreto-Lei n.º 28/2019)

- Restricoes temporais ancoradas no calendario da UC LI4
- Restricoes tecnologicas em conformidade com as decisoes ja tomadas pela equipa
- Terminologia tecnica de engenharia de software

Restricoes: Forneça apenas o paragrafo introdutorio e a descricao por familia. Sem requisitos especificos.

Análise Crítica

Conteúdo aceite após validação das referências legislativas (Decreto-Lei n.º 28/2019, Regulamento UE 2016/679), com ajuste posterior da tabela de tecnologias para refletir as decisões finais da equipa, cuja fundamentação detalhada foi remetida para o Capítulo 2. Bloco completo no Anexo A (P-12).

A definição explícita do contexto de negócio e das restrições aplicáveis ao projeto constitui uma etapa metodologicamente indispensável nas fases preliminares do ciclo de vida de desenvolvimento de *software*, antecedendo necessariamente o levantamento formal de requisitos. Esta análise prévia cumpre uma função dupla. Por um lado, delimita o espaço de solução admissível, prevenindo a especificação de requisitos incompatíveis com restrições legais, tecnológicas ou temporais já conhecidas. Por outro, fornece à equipa de desenvolvimento um quadro de referência estável que orienta decisões de arquitetura, priorização funcional e gestão de riscos ao longo de todo o ciclo de vida do projeto.

1.2.6.1 Contexto de Negócio

As *Oficinas Gengis Khan* constituem uma empresa de pequena e média dimensão (PME) especializada em diagnóstico, reparação e manutenção de trotinetes elétricas de múltiplas marcas, sediada em Braga, Portugal. A oficina serve simultaneamente dois segmentos de clientes com perfis distintos. Os clientes individuais particulares representam a maioria do volume transacional e procedem ao pagamento no momento da recolha do equipamento. Os clientes corporativos, operadores de mobilidade partilhada e gestores de frotas empresariais, usufruem de condições comerciais diferenciadas, nomeadamente prazos de pagamento alargados de 30 dias e potencial faturação consolidada.

O volume operacional atual é já significativo, projetando-se um crescimento sustentado ao longo dos próximos anos. A oficina gere um inventário de aproximadamente 300 referências de peças de substituição, provenientes de dois fornecedores principais e de fornecedores secundários ocasionais. O ciclo de vida de cada reparação atravessa sete estados distintos (Recebida, Em Diagnóstico, Aguarda Aprovação do Cliente, Aguarda Peças, Em Reparação, Concluída e Entrega ao Cliente), com cada transição de estado a dever ser registada com *timestamp* automático para efeitos de rastreabilidade e análise de tempos de ciclo.

1.2.6.2 Restrições Legais e Regulamentares

O sistema está sujeito a duas restrições legais não negociáveis que condicionam de forma determinante o desenho de módulos específicos:

Faturação sequencial obrigatória. O Decreto-Lei n.º 28/2019 da Autoridade Tributária Portuguesa impõe a numeração sequencial contínua e sem quebras de todos os documentos fiscais emitidos. O módulo de faturação deve garantir, por construção, a impossibilidade de duplicação, omissão ou reordenação de números de fatura (formato FAT-AAAA-NNN), assegurando rastreabilidade fiscal completa para efeitos de auditoria e comunicação ao sistema *e-Fatura*. Esta restrição implica a implementação de mecanismos de controlo transacional ao nível da base de dados que previnam condições de corrida em cenários de acesso concorrente.

Conformidade com o RGPD. O Regulamento Geral sobre a Proteção de Dados (Regulamento UE 2016/679), fiscalizado em Portugal pela Comissão Nacional de Proteção de Dados (CNPd), impõe obrigações específicas sobre o tratamento de dados pessoais de clientes (nome, NIF, morada, contactos telefónicos e eletrónicos) e de colaboradores. O sistema deve implementar os princípios de minimização de dados, obtenção de consentimento explícito para comunicações eletrónicas e garantia dos direitos dos titulares, designadamente acesso, retificação e eliminação. Do ponto de vista técnico, estas obrigações traduzem-se em requisitos de controlo de acessos baseado em papéis (RBAC), encriptação de dados sensíveis em repouso e em trânsito, e políticas de retenção limitada de dados.

1.2.6.3 Restrições Temporais e Académicas

O projeto enquadra-se no calendário da unidade curricular de Laboratórios de Informática IV (LI4) da Universidade do Minho, o que estabelece um horizonte temporal fixo, não negociável e externamente imposto. O modelo de desenvolvimento em cascata adotado, estruturado em quatro fases sequenciais (Requisitos e Modelação, Arquitetura e Desenho, Implementação, Testes e Validação), é compatível com este horizonte temporal desde que a priorização do MVP seja mantida de forma disciplinada.

1.2.6.4 Restrições Tecnológicas

O sistema deve ser desenvolvido segundo uma arquitetura *web* em três camadas (apresentação, lógica de negócio e dados), com as seguintes escolhas tecnológicas fixadas pela equipa:

Tabela 1.10: Tecnologias selecionadas

Camada / Componente	Tecnologia Selecionada
Base de dados relacional	PostgreSQL 16 com Prisma ORM como autoridade de <i>schema</i> e migrações
<i>Backend</i> (lógica de negócio e API REST)	TypeScript sobre Node.js com <i>framework</i> Fastify, organizada como monólito modular
<i>Frontend</i> (interface de utilizador)	TypeScript com React 18, empacotada em SPA via Vite e estilizada com Tailwind CSS
Contratos partilhados <i>frontend/backend</i>	Esquemas Zod publicados como pacote interno
Gestão do <i>monorepo</i> e infraestrutura local	pnpm <i>workspaces</i> com Turborepo; Supabase CLI para PostgreSQL de desenvolvimento; Docker Compose para contentorização local
Serviço de notificações por <i>email</i>	SMTP (servidor da oficina ou fornecedor gratuito)
Modelação e documentação UML	Visual Paradigm / PlantUML
Apoio à produção de artefactos	GitHub Copilot, modelos de linguagem (Claude)

O volume de dados projetado para um horizonte de operação de 10 anos, com cada ordem de serviço a agregar cerca de 5 intervenções e 10 linhas de peças, é plenamente suportado pela tecnologia de base de dados selecionada sem necessidade de estratégias avançadas de particionamento ou arquivamento, confirmando a adequação técnica das escolhas efetuadas ao contexto operacional das *Oficinas Gengis Khan*.

1.2.7 Síntese

A definição do domínio apresentada estabelece os fundamentos conceptuais e operacionais do sistema de gestão para oficinas de reparação de trotinetes elétricas. A análise do contexto de negócio, a identificação das necessidades dos intervenientes e a delimitação clara do âmbito funcional e das restrições aplicáveis fornecem a base estruturada necessária para as fases subsequentes de engenharia de requisitos, nomeadamente a elaboração de especificações detalhadas, a modelação do domínio através de diagramas UML e a definição de casos de uso que operacionalizam os processos de negócio identificados.

1.3 Engenharia de Requisitos

O presente capítulo constitui o núcleo documental do trabalho, cobrindo a totalidade do processo de engenharia de requisitos conduzido com auxílio de LLMs. O processo de elicitação baseou-se em quatro fontes primárias de evidência, cujos *prompts* e transcrições integrais se encontram nos Anexos para consulta detalhada:

- **Entrevista-001** (Anexo B, página 207): sessão de definição do domínio com o gerente e o mecânico chefe, cobrindo o ciclo operacional completo da oficina em dez partes temáticas (17 de fevereiro de 2026, 1h45).
- **Entrevista-002** (Anexo C, página 215): sessão de elicitação de requisitos focada em funcionalidades concretas, regras de negócio e requisitos não funcionais (20 de fevereiro de 2026, 1h35).
- **Entrevista-003** (Anexo D, página 223): sessão de validação e resolução de ambiguidades identificadas na análise dos requisitos (23 de março de 2026, 1h45).
- **Registos Físicos da Oficina** (Anexo E): transcrição dos documentos operacionais (caderno de operações, ordens de serviço, faturas e caderno de *stock*) referentes ao período de 10 a 14 de fevereiro de 2026.

As secções que se seguem apresentam os artefactos produzidos a partir destas fontes: *user stories*, especificação de requisitos funcionais e não funcionais, resolução de ambiguidades e a especificação formal de requisitos de *software* (SRS).

1.3.1 User Stories

Análise Crítica do Processo de Geração com IA

Prompt 13. Geração de User Stories

Contexto: Sistema de gestão das "Oficinas Gengis Khan" (LI4).
Engenharia de requisitos, fase de consolidação preliminar de necessidades, posterior as Entrevistas 01 e 02.

Objetivo: Escrever a introdução teórica às User Stories (baseada em Sommerville) e gerar a lista detalhada das mesmas com critérios de aceitação mensuráveis.

Instruções de Escrita:

- Enquadrar a User Story como descrição simplificada focada no valor de negócio, justificando a redução da distância entre equipa e stakeholders

- Derivar historias das Entrevistas 01 e 02 e da analise do dominio, aplicando o criterio INVEST
- Organizar por papel (Gerente, Mecanico) no formato "Como [papel], eu pretendo [funcionalidade], para que [beneficio]"
- Acompanhar cada historia de 2 a 3 criterios de aceitacao objetivos e mensuraveis

Restricoes: Sem criterios vagos do tipo "o sistema deve ser rapido". Sem inventar funcionalidades nao mencionadas nas fontes.

Análise Crítica

A geração de *user stories* com auxílio do LLM revelou-se particularmente eficaz, dado que o modelo já dispunha do contexto completo das duas entrevistas como documentos em anexo. O LLM produziu 16 histórias (8 para o Gerente, 8 para o Mecânico), das quais a equipa reteve 16 após validação. A principal intervenção consistiu na reformulação de critérios de aceitação que eram demasiado vagos (por exemplo, "o sistema deve ser rápido" foi substituído por critérios mensuráveis). A introdução teórica proposta pelo LLM, que enquadrava as *user stories* como técnica de consolidação preliminar de necessidades e não como requisitos formais, foi considerada uma distinção pertinente e aceite integralmente. A organização por papel (Gerente e Mecânico) reflete as prioridades distintas de cada perfil, tal como observado nas entrevistas.

A utilização de *user stories* constitui uma técnica de levantamento e consolidação preliminar de necessidades, particularmente adequada para captar, de forma sucinta e orientada ao valor, aquilo que diferentes perfis de utilizador esperam obter do sistema. Neste enquadramento, as *user stories* não são tratadas como requisitos formais nem como especificação exaustiva de comportamento, funcionam como apontamentos estruturados que registam objetivos, intenções e benefícios observados durante o estudo do domínio e durante a interação com os *stakeholders*. Esta abordagem facilita a validação inicial por parte de utilizadores não técnicos, preserva a linguagem do negócio e cria um fio condutor natural para a posterior transformação destas necessidades em requisitos funcionais e não funcionais, com detalhe normativo e verificável.

As histórias apresentadas de seguida foram produzidas de forma iterativa a partir da análise do domínio (processos operacionais, estados da OS, gestão de *stock*, faturação e comunicação), procurando manter um nível de granularidade pequeno e testável. Sempre que possível, aplicou-se o critério INVEST como grelha de qualidade, garantindo que cada história descreve uma unidade de valor compreensível, estimável e suscetível de validação por critérios de aceitação objetivos. As histórias encontram-se organizadas por papel, Gerente e Mecânico, por serem os dois perfis que interagem diretamente com o sistema e por evidenciarem prioridades distintas, o primeiro orientado à conformidade, visibilidade e controlo financeiro, e o segundo focado na eficiência do registo técnico e na rastreabilidade operacional.

1.3.1.1 Excertos Ilustrativos

O catálogo resultante cobre todas as áreas funcionais identificadas no levantamento de domínio, desde a gestão de clientes e equipamentos até à faturação, ao controlo de *stock* e à comunicação com fornecedores e clientes. Constitui, assim, a primeira tradução estruturada das expectativas de cada perfil operacional, servindo de fio condutor para a posterior formalização em requisitos funcionais e não funcionais. Por economia de espaço e para preservar a clareza expositiva do corpo do relatório, apresentam-se de seguida apenas dois excertos ilustrativos, um por papel, escolhidos pela centralidade que adquirem no fluxo operacional do negócio. A versão integral das 16 histórias, com os respetivos critérios de aceitação, encontra-se reproduzida no Anexo I para consulta do leitor interessado.

US-G03: Emissão de fatura com numeração sequencial

Como Gerente, eu pretendo emitir faturas automaticamente a partir de ordens de serviço concluídas, para que a faturação seja mais rápida, sem erros de cálculo e conforme a numeração sequencial obrigatória.

Critérios de Aceitação:

- Só deve ser possível faturar uma OS no estado *Concluída*.
- A numeração deve ser automática, sequencial e sem quebras (FAT-AAAA-NNN).
- O sistema deve calcular subtotal, IVA e total de forma automática.

US-M03: Atualização de estado da OS com registo temporal

Como Mecânico, eu pretendo alterar o estado da OS conforme o trabalho progride, para que exista rastreabilidade do processo e visibilidade interna.

Critérios de Aceitação:

- Cada mudança de estado deve registar data/hora e autor.
- O sistema deve suportar os sete estados definidos.
- Deve existir histórico completo de transições.

1.3.1.2 Síntese e Transição para a Especificação Formal

As *user stories* apresentadas consolidam, de forma orientada ao valor e à perspetiva do utilizador, os principais objetivos operacionais identificados no estudo do domínio das *Oficinas Gengis Khan*. Estas narrativas constituem a base direta para a etapa seguinte de engenharia de requisitos, na qual as necessidades aqui descritas são convertidas em requisitos funcionais e não funcionais formais, com descrição normativa, rastreabilidade e critérios de verificabilidade adequados à implementação e validação do sistema.

1.3.2 Elicitação e Especificação de Requisitos

Análise Crítica do Processo de Geração com IA

Prompt 14. Análise e Categorização de Requisitos

Contexto: Projeto das Oficinas Gengis Khan, fase de engenharia de requisitos a partir das entrevistas e da documentação em anexo, com necessidade de rastreabilidade pessoa/documento.

Objetivo: Extrair e estruturar todos os requisitos mencionados nas entrevistas e na documentação do projeto, organizados em tabela com rastreabilidade.

Instruções de Escrita:

- Cada linha contém No, Data e Hora, Descrição, Vista de Uso, Fonte (Pessoa ou Documento) e Analista
- Cada requisito tem de ter sido explicitamente pedido ou derivado diretamente de um problema mencionado pelos stakeholders nos textos fornecidos
- Manter linguagem clara em PT-PT, nível académico médio-alto

Restrições: Não inventar nem presumir informação. Se não está no texto, não entra na tabela.

Análise Crítica

A elicitação de requisitos constituiu, entre todas as interações conduzidas com o LLM nesta etapa, aquela em que o modelo manifestou desempenho menos consistente, exigindo intervenção humana mais densa do que a observada em *prompts* anteriores. Observaram-se dois enviesamentos sistemáticos no resultado produzido. Por um lado, a omissão de requisitos diretamente sustentados nas fontes documentais, com particular incidência nos registos físicos transcritos. Por outro, granularidade excessiva sobre outros itens, com fragmentação de uma única necessidade em múltiplos requisitos redundantes. A equipa procedeu, em consequência, a uma revisão exaustiva da lista contra a totalidade das fontes (Entrevistas 01 e 02 e registos físicos do estabelecimento), identificando múltiplas correções estruturais a aplicar. A título ilustrativo, destacam-se dois casos representativos, designadamente o RF-08, com descrição vazia por aparente erro de geração, e o RF-31, que duplicava integralmente o RF-23 ao descrever o registo de movimentos de *stock*. Os requisitos afetados foram removidos e os identificadores renumerados após o conjunto das correções, fixando-se a contagem definitiva em 44 requisitos funcionais e 5 não funcionais. A contagem inicialmente apontada pelo modelo (45 funcionais) foi retificada em conformidade. A taxonomia funcional vs. não funcional foi validada pela equipa, sem alterações. Numa iteração posterior, com finalidade distinta, o modelo foi solicitado a introduzir intencionalmente requisitos parcialmente conflitantes entre si, instrumentalizando-se assim a geração controlada das ambiguidades que viriam a ser tratadas na subsecção seguinte.

A fase de eliciação de requisitos constitui o alicerce sobre o qual assenta toda a arquitetura do sistema, sendo o momento em que necessidades operacionais, frequentemente implícitas e expressas em linguagem informal, são identificadas, clarificadas e transformadas em especificações formais, precisas e verificáveis. Seguindo a abordagem de Sommerville, a eliciação não se reduz a uma recolha passiva de informação, mas configura-se como um processo sistemático de descoberta e validação, no qual a equipa de engenharia de *software* atua como mediadora entre o domínio do problema e o espaço de solução tecnológica. Neste enquadramento, procurou-se reduzir ambiguidades e prevenir incoerências logo nas fases iniciais, reconhecendo que, num modelo de desenvolvimento em cascata, erros de interpretação nesta etapa propagam-se inevitavelmente para o desenho, implementação e testes, com custos de correção significativamente superiores.

No presente projeto, a eliciação foi conduzida de forma iterativa, recorrendo a três fontes complementares de evidência. A primeira consistiu em entrevistas semiestruturadas com os *stakeholders* primários, orientadas à compreensão do domínio e ao refinamento de regras de negócio. A segunda foi a análise documental dos documentos atuais do negócio (caderno de registos, fichas manuais), que permitiu observar a granularidade real dos dados e as lacunas dos processos atuais. A terceira recorreu a *user stories* previamente registadas como apontamentos estruturados das necessidades identificadas durante o estudo do domínio. Estas *user stories* foram consideradas como um artefacto intermédio de validação e consolidação, útil para preservar a perspetiva do utilizador e o valor de negócio pretendido, servindo de ponte para a posterior formalização em requisitos funcionais e não funcionais. A conjugação destas fontes permitiu construir uma base coerente de requisitos, garantindo rastreabilidade às origens e assegurando que a especificação subsequente reflete de forma fiel os objetivos operacionais das *Oficinas Gengis Khan*.

A conjugação destas fontes, compostas por entrevistas com os utilizadores primários, análise documental dos suportes físicos e revisão dos artefactos produzidos nas fases anteriores do projeto, permitiu identificar um conjunto abrangente e coerente de requisitos, organizados nas subsecções seguintes de acordo com a taxonomia de Sommerville em dois tipos. Os requisitos funcionais especificam os comportamentos e serviços que o sistema deve providenciar, ao passo que os requisitos não funcionais definem os atributos de qualidade sistémica que condicionam a forma como esses comportamentos devem ser realizados.

1.3.2.1 Processo de Eliciação e Descoberta

A primeira sessão de entrevista, Entrevista-001 (Anexo B), centrou-se na cartografia do domínio operacional, tendo o gerente e o mecânico-chefe descrito, de forma cronológica, o ciclo de vida completo de uma reparação, desde a chegada do cliente à oficina e o registo manual no caderno até à entrega do equipamento e à emissão da fatura. Esta sessão revelou, de imediato, três lacunas estruturais que os próprios *stakeholders* não identificavam como problemas isolados, mas como manifestações de uma causa comum, a ausência de rastreabilidade, designadamente a impossibilidade de recuperar o histórico de intervenções de uma trotinete pelo número de série, a inexistência de qualquer registo de *timestamp* nas transições de estado das ordens de

serviço, e a gestão de *stock* inteiramente assente na memória do mecânico. A Entrevista-002 (Anexo C) operou num nível de detalhe distinto, focando-se nas regras de negócio implícitas que estruturam as decisões quotidianas, designadamente o critério de aprovação de orçamentos pelo cliente antes do início da reparação, as condições diferenciadas de pagamento a 30 dias para clientes corporativos, a taxa horária configurável de mão de obra (25 €/h) e o mecanismo de alerta para ordens em espera há mais de 10 dias. Estas regras, não verbalizadas na primeira entrevista por serem consideradas óbvias pelos *stakeholders*, constituíram a base direta de vários requisitos funcionais de controlo e de configuração.

A análise dos artefactos físicos de gestão corrente, caderno de clientes, fichas manuais de ordens de serviço e ficheiros Excel de *stock* e faturação (Anexo E), complementou as entrevistas com evidência observacional que os *stakeholders* não teriam conseguido articular verbalmente. O confronto entre os campos efetivamente preenchidos nas fichas de papel e os dados declarados como necessários nas entrevistas permitiu distinguir, com rigor empírico, os campos obrigatórios dos opcionais. O número de série, por exemplo, estava sistematicamente preenchido em todas as fichas analisadas, confirmando o seu estatuto de identificador único. A data de compra do equipamento, embora mencionada como desejável pelo gerente, encontrava-se ausente da totalidade dos registos, sendo tratada como campo opcional no modelo de dados resultante. A análise das folhas de cálculo de faturação evidenciou ainda a recorrência de erros de arredondamento na aplicação da taxa de IVA e na contabilização de frações horárias, fundamentando a prioridade atribuída à automatização integral do cálculo financeiro. Identificaram-se igualmente campos não mencionados nas entrevistas, como a indicação do técnico responsável por cada intervenção individual, que foram incorporados como requisitos após confirmação da sua relevância junto dos *stakeholders* em sessão de validação.

A triangulação entre os enunciados das duas entrevistas, o conteúdo dos artefactos documentais e os artefactos já produzidos nas fases anteriores do projeto (objetivos primários e secundários, definição de domínio e *user stories*) foi conduzida de forma sistemática, com o propósito explícito de detetar contradições, eliminar redundâncias e garantir ancoragem de cada requisito numa necessidade real de negócio. Foi precisamente neste exercício que se concluiu que a restrição da transição de estado “Entregue” ao perfil de gerente não constitui uma preferência operacional, mas uma necessidade de controlo financeiro. A entrega física do equipamento deve estar sincronizada com a confirmação de liquidação da fatura, tornando este comportamento um requisito funcional obrigatório e não uma configuração opcional.

A conjugação das técnicas de eliciação descritas permitiu construir uma base de conhecimento sólida e coerente para sustentar a especificação formal dos requisitos. A listagem integral dos 47 requisitos identificados, anterior à categorização por tipo e área funcional, encontra-se reproduzida na Subsecção 5.4 do Anexo H. A Tabela 1.11 apresenta um excerto representativo, ilustrando o formato adotado e a diversidade de áreas funcionais cobertas.

Tabela 1.11: Excerto da listagem bruta dos requisitos levantados

Nº	Data	Descrição	Vista de Uso	Fonte	Analista
REQ-001	20/02/2026	O sistema deve permitir registar clientes com nome completo, NIF, telefone e <i>email</i> obrigatórios, e morada opcional.	Atendimento	António Silva / Entrevista-002	Francisco Martins
REQ-012	17/02/2026	O sistema deve suportar os estados da OS: “Recebida”, “Em Diagnóstico”, “Aguarda Aprovação do Cliente”, “Aguarda Peças”, “Em Reparação”, “Concluída” e “Entregue ao Cliente”.	Gestão de OS	António Silva e Ricardo Costa / Entrevista-001	Francisco Martins
REQ-030	17/02/2026	O sistema deve emitir faturas com numeração sequencial contínua, sem quebras.	Faturação	António Silva / Entrevista-001	Marco Sè-vegrand
(...) Listagem completa dos 47 requisitos na Subsecção 5.4 do Anexo H					

1.3.2.2 Análise e Especificação de Requisitos

A transição da elicitação bruta para a especificação técnica exige uma fase rigorosa de análise, cujo propósito é categorizar as necessidades do domínio e garantir a ausência de contradições lógicas. Nesta secção, procede-se à dicotomia entre requisitos funcionais e não funcionais, bem como à exposição do processo de identificação e resolução de ambiguidades detetadas na linguagem natural produzida durante as sessões de elicitação.

A fase de análise constitui, no modelo de desenvolvimento em cascata adotado, o elemento de transição crítico entre a elicitação e a especificação formal. Esta fase tem como objetivo principal o escrutínio dos dados brutos recolhidos junto dos *stakeholders*, a sua categorização segundo natureza funcional ou não funcional, e a verificação da sua consistência interna, completude e ausência de conflitos. Nela detetam-se e resolvem-se as ambiguidades semânticas e as sobreposições entre requisitos antes de se tornarem erros de implementação de custo elevado.

O processo de análise desenvolvido para o projeto *Oficinas Gengis Khan* envolveu três etapas sequenciais:

1. Leitura sistemática e anotação crítica dos requisitos levantados.
2. Categorização de cada requisito segundo a taxonomia funcional/não funcional.
3. Sessão de clarificação com os *stakeholders* (Gerente António Silva e Mecânico Chefe Ricardo Costa) para resolução das ambiguidades identificadas, documentada na Entrevista-003 (Anexo D).

O resultado desta análise é apresentado nas subsecções seguintes.

Categorização: Requisitos Funcionais e Não Funcionais A distinção entre requisitos funcionais (RF) e não funcionais (RNF) é fundamental para a estruturação da especificação e para o planeamento da arquitetura do sistema. Conforme definido por IEEE Std 29148-2018, requisitos funcionais descrevem o comportamento do sistema, especificando o que o sistema deve fazer (as ações que executa, os dados que processa, as regras de negócio que implementa e os *outputs* que produz em resposta a determinados *inputs*). Requisitos não funcionais, por sua vez, descrevem as propriedades e restrições do sistema, especificando como o sistema deve operar em parâmetros de qualidade, desempenho, segurança, portabilidade e conformidade legal, que condicionam o espaço de soluções arquiteturais admissíveis.

A categorização dos requisitos levantados nas Entrevistas-001 e -002 e nos Registos da Oficina resultou em 44 requisitos funcionais e 5 requisitos não funcionais, detalhados nas tabelas seguintes.

Requisitos Funcionais Os 44 requisitos funcionais consolidados cobrem seis áreas funcionais do sistema: Atendimento e Registo de Clientes/Equipamentos, Gestão do Ciclo de Vida das Ordens de Serviço, Reparação e Intervenções, Gestão de *Stock* e Fornecedores, Faturação, e Notificações Automáticas. A especificação completa encontra-se na Subsecção 5.4 do Anexo H. A Tabela 1.12 apresenta um excerto representativo, cobrindo quatro requisitos transversalmente referidos no restante relatório.

Tabela 1.12: Excerto dos requisitos funcionais consolidados

Nº	Data	Descrição	Vista de Uso	Fonte	Analista
RF-01	2026-02-20	O sistema deve permitir registar clientes com nome completo, NIF (usado como identificador), telefone e <i>email</i> obrigatórios, e morada opcional.	Atendimento	António Silva / Entrevista-002	Francisco Martins
RF-11	2026-02-17	O sistema deve suportar os estados da OS: “Recebida”, “Em Diagnóstico”, “Aguarda Aprovação do Cliente”, “Aguarda Peças”, “Em Reparação”, “Concluída” e “Entregue ao Cliente”.	Gestão de OS	António Silva e Ricardo Costa / Entrevista-001	Francisco Martins
RF-31	2026-02-17	O sistema deve emitir faturas com numeração sequencial contínua, sem quebras.	Faturação	António Silva / Entrevista-001	Marco Sè-vegrand

RF-39	2026-02-20	O sistema deve enviar ao cliente uma confirmação de receção quando a trotinete é aceite, incluindo o número da ordem de serviço e um prazo estimado.	Notificações	António Silva / Entrevista-002	Francisco Martins
(…) Catálogo completo dos 44 RF na Subsecção 5.4 do Anexo H					

Requisitos Não Funcionais Os cinco requisitos não funcionais incidem sobre três dimensões de qualidade do sistema: Plataforma e Acessibilidade, Segurança e Conformidade Legal (RGPD), e Escalabilidade. Embora menos numerosos, estes requisitos têm impacto direto nas decisões de arquitetura, infraestrutura e implementação. A especificação completa consta da Subsecção 5.4 do Anexo H; a Tabela 1.13 apresenta um excerto representativo.

Tabela 1.13: Excerto dos requisitos não funcionais consolidados

Nº	Data	Descrição	Vista de Uso	Fonte	Analista
RNF-01	2026-02-20	O sistema deve ser uma aplicação <i>web</i> acessível na oficina e remotamente.	Plataforma	António Silva / Entrevista-002	Nuno Rebelo
RNF-03	2026-02-20	O sistema deve proteger os dados pessoais dos clientes em conformidade com o RGPD.	Segurança	António Silva / Entrevista-002	Francisco Martins
(…) Catálogo completo dos 5 RNF na Subsecção 5.4 do Anexo H					

Resolução de Ambiguidades e Conflitos A linguagem natural, pela sua inerente imprecisão semântica, constitui uma fonte recorrente de ambiguidade nos processos de elicitação de requisitos. Termos vagos, requisitos sobrepostos, pressupostos implícitos não declarados e enunciados que admitem múltiplas interpretações técnicas são fenómenos sistemáticos que, se não resolvidos nesta fase, propagam-se para as fases de arquitetura e implementação.

No contexto do projeto *Oficinas Gengis Khan*, a análise crítica dos requisitos levantados permitiu identificar cinco situações de ambiguidade ou sobreposição. Em cada caso, a equipa de desenvolvimento documentou a ambiguidade, avaliou o seu impacto potencial e promoveu uma sessão de clarificação com os *stakeholders*, formalmente documentada na Entrevista-003 (Anexo D, página 223), para acordar uma resolução. As decisões de clarificação acordadas são vinculativas para as fases subsequentes e devem ser refletidas nos modelos UML e na implementação.

A tabela seguinte documenta, de forma estruturada, cada ambiguidade identificada, o seu impacto potencial caso não fosse resolvida, e a resolução acordada com os *stakeholders*.

Tabela 1.14: Resolução de ambiguidades e conflitos

Requisito	Ambiguidade Identificada	Impacto Potencial	Resolução / Clarificação Acordada
REQ-004	O requisito exige que o sistema mostre o histórico completo do cliente (visitas, valor gasto, faturas por pagar), mas não especifica qual o campo que garante a unicidade do cliente no sistema. Sem essa regra, o mesmo cliente pode existir em múltiplos perfis e o histórico fica fragmentado.	Sem clarificação, a implementação poderia aceitar dois registos do mesmo cliente (por variações no nome ou no telefone), resultando num histórico partido e em decisões financeiras baseadas em dados incompletos, especialmente grave no caso de clientes empresariais com limite de crédito.	Acordo com os <i>stakeholders</i> : o NIF é o identificador primário e universal de um cliente no sistema. Qualquer tentativa de registar um novo cliente com um NIF já existente deve ser bloqueada, apresentando o registo existente ao operador para possível atualização. A unicidade por NIF é a única regra de deduplicação implementada.
REQ-008 / REQ-011	O REQ-008 definia a criação da OS com numeração e data de entrada, enquanto o REQ-011 especificava o estado inicial como "Recebida". Tratavam-se de dois requisitos sobre o mesmo objeto (a criação da OS).	A manutenção de dois requisitos sobrepostos aumenta o risco de inconsistência na implementação, podendo levar a que a criação da OS e a atribuição do estado inicial sejam tratadas como ações separadas.	Os dois requisitos foram fundidos num único: a criação da OS abrange simultaneamente a numeração sequencial automática, o registo da data de entrada e a atribuição automática do estado inicial "Recebida". O campo de diagnóstico e o problema reportado pelo cliente, cobertos pelo REQ-009, são adicionados ao longo do ciclo de vida da OS.
REQ-009	O requisito indicava que o diagnóstico técnico deveria ser registado na OS, mas não ficou claro se este campo é preenchido no momento da criação da OS ou posteriormente.	Se o diagnóstico fosse obrigatório no momento de abertura da OS, a criação ficaria bloqueada até o mecânico completar o diagnóstico, criando um gargalo operacional na receção.	Clarificado com os <i>stakeholders</i> : o diagnóstico, o problema reportado e outros campos técnicos são adicionados progressivamente ao longo do ciclo de vida da OS. Apenas os campos de identificação do cliente, equipamento e data de entrada são obrigatórios na criação. Tudo o resto é adicionado após a criação da mesma.
REQ-023	O requisito descrevia a consulta da disponibilidade atual das peças para apoiar a reparação. Contudo, o REQ-020 (catálogo de peças) já define que o campo "stock atual" faz parte do catálogo de peças.	A existência de dois requisitos aparentemente redundantes poderia originar a implementação de duas funcionalidades distintas para aceder à mesma informação, gerando inconsistências na interface e duplicação de lógica.	O REQ-023 foi considerado redundante no contexto do REQ-020. O <i>stock</i> atual é um atributo visível do catálogo de peças e não carece de requisito separado. A distinção relevante, verificar disponibilidade no contexto de uma intervenção, é garantida pelo comportamento definido no REQ-021 (bloqueio de associação de peça com <i>stock</i> zero).

continua na próxima página

Tabela 1.14 – continuação

Requisito	Ambiguidade Identificada	Impacto Potencial	Resolução / Clarificação Acordada
REQ-033	O requisito mencionava a consulta de faturas pendentes e atrasadas sem especificar se é aplicável a todos os clientes ou apenas a clientes empresariais.	Aplicar este requisito indiscriminadamente poderia resultar em alertas desnecessários para clientes particulares, que liquidam o pagamento no ato da entrega e nunca têm faturas em dívida.	Clarificado com os <i>stakeholders</i> : os clientes particulares liquidam o pagamento no momento do levantamento da trotinete, pelo que não geram faturas pendentes estruturais. A gestão de faturas pendentes, atrasadas e controlo de crédito é um mecanismo aplicável exclusivamente a clientes empresariais com prazo de pagamento acordado (tipicamente 30 dias). O requisito foi reformulado em conformidade.

1.3.2.3 Síntese e Transição para a Modelação

A fase de análise concluída nesta secção produziu um conjunto de 44 requisitos funcionais e 5 requisitos não funcionais, categoricamente distintos, internamente consistentes e sem sobreposições lógicas. As cinco ambiguidades identificadas foram resolvidas mediante clarificação formal com os *stakeholders*, tendo as resoluções sido incorporadas nas especificações dos requisitos afetados.

Este conjunto especificado constitui a base de entrada para as fases subsequentes do ciclo de desenvolvimento. A rastreabilidade entre os requisitos aqui definidos e os artefactos produzidos nas fases seguintes é garantida pelo identificador único atribuído a cada requisito (RF-XX / RNF-XX), que será mantido como referência transversal ao longo de toda a documentação do projeto.

1.3.3 Especificação de Requisitos de Software (SRS)

Análise Crítica do Processo de Geração com IA

Prompt 15. Estruturação do SRS

Contexto: Projeto das Oficinas Gengis Khan, encerramento da fase de Engenharia de Requisitos posicionado após a elicitação/ análise e antes dos Use Cases, alinhado com IEEE 830 e ISO/IEC/ IEEE 29148 e fundamentado em Sommerville (2016, cap. 4).

Objetivo: Produzir o documento formal de Especificação de Requisitos de Software, consolidando a informação dispersa num

artefacto verificavel, rastreavel e ponte entre o "o que" e o "como" do sistema.

Instrucoes de Escrita:

- Aplicar a estrutura obrigatoria em sete subseccoes: introducao e proposito, ambito do sistema (tabela Dentro/Fora), descricao geral (perspetiva, funcoes, perfis, ambiente, restricoes, suposicoes), requisitos funcionais consolidados, requisitos nao funcionais consolidados, criterios de aceitacao por modulo e nota de transicao para Use Cases
- Redigir os criterios de aceitacao no formato "O sistema deve [X], quando [Y], de forma que [Z]", com dois criterios por modulo
- Classificar restricoes como [LEGAL], [TECNICA] ou [ACADEMICA] e suposicoes como [ASSUNCAO] ou [DEPENDENCIA]
- Manter PT-PT estrito

Restricoes: Sem referencias a numeros de tabela absolutos; usar referencias cruzadas dinamicas. Sem desvios da estrutura obrigatoria.

Análise Crítica

A produção do SRS foi a tarefa mais estruturada submetida ao LLM, com um *prompt* extenso e prescritivo que especificava integralmente a estrutura do artefacto, as referências bibliográficas a utilizar e os critérios de formatação. O resultado refletiu esta especificidade. O modelo produziu um documento tecnicamente sólido, com referências corretas a Sommerville (2016) e IEEE 830 (1998), e uma estrutura em sete subsecções que a equipa considerou adequada ao posicionamento do SRS no ciclo de vida. As tabelas de âmbito e perfis de utilizador foram aceites na íntegra. Os critérios de aceitação por módulo, redigidos no formato normativo solicitado ("O sistema deve X, quando Y, de forma que Z"), foram particularmente úteis para fornecer uma base de verificabilidade aos requisitos. A única intervenção significativa da equipa consistiu na remoção de referências a tabelas específicas ("tabela 14 a 18", "tabela 19") que não correspondiam à numeração final do relatório, substituindo-as por referências cruzadas LaTeX dinâmicas.

A presente secção constitui o artefacto formal de encerramento da fase de Engenharia de Requisitos. Segundo Sommerville (2016, cap. 4), o documento de especificação de requisitos de *software* representa o culminar do processo de elicitacão e análise, sendo uma declaração consolidada e verificável de tudo aquilo que o sistema deve fazer e das condições em que deve operar. De forma complementar, o padrão IEEE 830 (1998) enquadra o SRS como uma especificação estruturada e inequívoca, destinada a reduzir ambiguidades, suportar validação com os *stakeholders* e servir de referência estável ao longo do desenvolvimento.

Neste projeto, o SRS sintetiza e consolida a informação produzida nas secções anteriores (entrevistas, análise de documentos, *user stories* e requisitos funcionais/não funcionais), convertendo-a num inventário final rastreável e testável. Deste modo, este documento funciona como base

para a preparação dos testes de validação e como ponte para a fase seguinte, na qual os *Use Cases* e os diagramas UML detalham “como” o sistema executa, em cenários de interação, os requisitos aqui formalizados.

1.3.3.1 Introdução e Propósito do SRS

O propósito deste SRS é consolidar, num único documento normativo, as necessidades e restrições identificadas para o sistema de gestão das *Oficinas Gengis Khan*, garantindo que cada requisito é formulado de forma clara, verificável e rastreável à sua origem. Esta especificação constitui um “contrato” de entendimento entre os *stakeholders* e a equipa de desenvolvimento, ao estabelecer o comportamento esperado do sistema e os atributos de qualidade que condicionam a sua implementação.

A presente secção estrutura o âmbito, a descrição geral do produto, os requisitos funcionais e não funcionais consolidados, e os critérios de aceitação de alto nível. A secção subsequente do relatório (dedicada a *Use Cases* e diagramas UML) utiliza este SRS como fonte de verdade para a modelação dos fluxos de interação e para a definição rigorosa dos cenários principais e alternativos.

1.3.3.2 Âmbito do Sistema

O sistema “*Oficinas Gengis Khan*” é uma aplicação *web* de gestão integrada para uma oficina de reparação de trotinetes elétricas, concebida para substituir processos manuais assentes em papel e folhas de cálculo, aumentando a rastreabilidade operacional, a integridade dos dados e a eficiência na gestão de *stock* e faturação. Os utilizadores principais são o Gerente, com uma visão estratégica e administrativa, e os Mecânicos, responsáveis pela execução e registo operacional. O Cliente atua como beneficiário indireto através da receção de notificações e o Fornecedor como ator externo devido à necessidade de receção de encomendas.

Tabela 1.15: Definição do Âmbito do Sistema

Dentro do Âmbito	Fora do Âmbito
Registo e gestão de clientes (particulares e empresas) e respetivas trotinetes	Portal de cliente com autenticação e acesso direto ao sistema
Gestão do ciclo de vida das Ordens de Serviço (OS), com estados e <i>timestamps</i>	Integração automática com sistemas oficiais da Autoridade Tributária (ex.: submissão direta a <i>e-Fatura</i>)
Registo de diagnósticos, intervenções (com cronómetro) e peças consumidas	Gestão de pagamentos <i>online</i> (referências multibanco, MB Way, etc.)
Gestão de inventário (<i>stock</i> mínimo, movimentos, fornecedores e encomendas)	Gestão avançada de garantias por fabricante com regras específicas por marca
Emissão de faturas com numeração sequencial e cálculo automático de IVA	Aplicação móvel nativa (Android/iOS), o acesso é via aplicação <i>web</i> responsiva
Notificações por <i>email</i> em eventos-chave (OS criada, orçamento, conclusão, atraso)	Integração com sistemas de <i>fleet management</i> de clientes corporativos
Relatórios e <i>dashboard</i> de suporte à decisão	Otimização automática do planeamento de trabalho (<i>scheduling</i>) com algoritmos

1.3.3.3 Descrição Geral

Perspetiva do Produto O sistema insere-se no ecossistema operacional de uma oficina PME, suportando a ligação entre o registo de clientes/equipamentos, a execução técnica de reparações e a gestão administrativa/financeira. O produto interage com serviços externos de forma indireta, designadamente com a Autoridade Tributária (através do cumprimento da numeração sequencial e coerência fiscal dos documentos), com um serviço de *email* via SMTP para envio de notificações e encomendas, e com fornecedores que recebem encomendas estruturadas por correio eletrónico.

O sistema é concebido para manter rastreabilidade ponta a ponta, desde a receção do equipamento (OS) até à entrega (condicionada por faturação e pagamento), incluindo o consumo de peças e respetivo impacto no inventário. A centralização de dados visa eliminar perda de informação e reduzir inconsistências típicas de processos manuais.

Funções Principais

1. **Gestão de Entidades Base:** registo e manutenção de clientes (particulares/empresas) e trotinetes (número de série único).
2. **Gestão Operacional de Reparações:** criação de OS, registo de problema reportado, diagnóstico, intervenções, estados e histórico com *timestamps*.
3. **Gestão de Inventário e Aprovisionamento:** registo de peças, controlo de *stock* mínimo, movimentos de *stock*, fornecedores e encomendas.
4. **Faturação e Conformidade:** geração automática de faturas a partir de OS concluídas, numeração sequencial e cálculo de IVA.
5. **Suporte à Decisão e Comunicação:** relatórios operacionais/financeiros e notificações por *email* (cliente e fornecedor) em eventos críticos.

Tabela 1.16: Definição dos perfis de utilizador

Papel	Descrição	Frequência de Uso	Nível de Acesso
Gerente (António Silva)	Responsável pela configuração do sistema, faturação, controlo financeiro, fornecedores, encomendas e relatórios.	Diário ($\approx$ 2h/dia)	Total (administração, relatórios, faturação, configuração)
Mecânico (Ricardo Costa / Miguel Abrantes)	Utilizador operacional primário: cria e atualiza OS, regista diagnósticos e intervenções, associa peças, consulta <i>stock</i> e histórico técnico.	Intensivo ($\approx$ 6h/dia)	Operacional (sem acesso a relatórios financeiros avançados nem configuração sensível)
Cliente (indireto)	Beneficiário indireto: recebe notificações de estado, orçamento e conclusão por <i>email</i> .	Por OS (3 a 5 <i>emails</i> típicos)	Sem acesso ao sistema
Fornecedor (externo)	Entidade externa: recebe encomendas estruturadas por <i>email</i> e cumpre prazos/entregas.	Variável	Sem acesso ao sistema

Perfis de Utilizador

Ambiente Operativo O produto será disponibilizado como aplicação *web* responsiva, acessível na oficina (terminais fixos e/ou *tablets*) e remotamente. A persistência é assegurada por uma base de dados relacional (PostgreSQL). O sistema deve acomodar a carga operacional corrente e o crescimento projetado para os próximos anos. O inventário envolve cerca de 300 referências de peças, com movimentos frequentes decorrentes de intervenções e receções de encomendas.

Restrições

- **[LEGAL]** Numeração sequencial obrigatória de faturas, sem quebras, para conformidade fiscal (Autoridade Tributária).
- **[LEGAL]** Conformidade RGPD: tratamento de dados pessoais (nome, NIF, contactos, morada), consentimento e controlo de acessos por perfil.
- **[TÉCNICA]** Baixa de *stock* automática e atómica associada ao registo de peças consumidas.
- **[TÉCNICA]** Autenticação e autorização por perfis (RBAC), distinguindo Gerente e Mecânico.
- **[TÉCNICA]** *Backups* diários automáticos obrigatórios e recuperação em caso de falha.

- **[TÉCNICA]** Acessibilidade para dificuldades motoras ativada por defeito em interfaces de criação.
- **[ACADÊMICA]** Cumprimento do calendário e entregáveis da UC LI4 (prazo fixo).

Suposições e Dependências

- **[DEPENDÊNCIA]** Disponibilidade de um serviço SMTP funcional (servidor da oficina ou fornecedor externo) para envio de *emails*.
- **[DEPENDÊNCIA]** Disponibilidade de rede local na oficina e acesso remoto para o gerente (*internet*).
- **[ASSUNÇÃO]** Os utilizadores (gerente e mecânicos) aceitam registar intervenções e estados em tempo útil, garantindo atualidade dos dados.
- **[ASSUNÇÃO]** A oficina manterá um processo interno de validação do *email* do cliente para reduzir falhas de notificação.
- **[ASSUNÇÃO]** O volume máximo de concorrência é compatível com o contexto de PME (utilização simultânea limitada a poucos utilizadores).

1.3.3.4 Requisitos Funcionais Consolidados

Os requisitos funcionais apresentados na Tabela 5.7 resultam do processo iterativo de elicitação e descoberta descrito anteriormente, com base em entrevistas semiestruturadas, análise documental de artefactos e *user stories* como apontamentos evolutivos de necessidades observadas. Esses requisitos encontram-se na sua forma consolidada, numerada e orientada à implementação, assegurando rastreabilidade às histórias de utilizador (US-xx) e às regras de negócio definidas neste relatório.

1.3.3.5 Requisitos Não Funcionais Consolidados

Os requisitos não funcionais definem atributos de qualidade e restrições globais do sistema que condicionam a implementação e são decisivos para a aceitação do produto. Estes requisitos são frequentemente tão críticos quanto os funcionais, pois determinam segurança, desempenho, fiabilidade e usabilidade, aspetos que, se ignorados, inviabilizam a operação real do sistema, mesmo que as funcionalidades existam. Podem ser consultados na Tabela 5.8.

1.3.3.6 Critérios de Aceitação por Módulo

Gestão de Clientes e Equipamentos

1. O sistema deve impedir o registo de uma trotinete com número de série já existente, quando o utilizador tentar submeter o formulário, de forma que a unicidade do identificador seja garantida.
2. O sistema deve permitir consultar o histórico de uma trotinete por número de série, quando o utilizador pesquisar esse identificador, de forma que sejam apresentados OS, intervenções e peças associadas.

Ciclo de Vida da Ordem de Serviço

1. O sistema deve registar *timestamp* e utilizador em cada transição de estado, quando uma OS mudar de estado, de forma que exista histórico auditável do ciclo de vida.
2. O sistema não deve permitir que uma OS transite para “Em Reparação” sem aprovação de orçamento registada, quando o utilizador tentar executar essa transição, de forma que a regra de negócio seja cumprida.

Gestão de *Stock* e Fornecedores

1. O sistema deve decrementar o *stock* de uma peça de forma atómica, quando a peça for associada a uma intervenção, de forma que o *stock* nunca fique inconsistente com o consumo registado.
2. O sistema deve apresentar alerta visual persistente, quando o *stock* atingir ou descer abaixo do mínimo, de forma que o gerente consiga priorizar reposição.

Faturação e Conformidade Legal

1. O sistema deve emitir faturas com numeração sequencial sem quebras, quando uma OS concluída for faturada, de forma que se garanta conformidade legal (AT).
2. O sistema não deve permitir marcar a OS como “Entregue” sem pagamento confirmado, quando o gerente tentar concluir a entrega, de forma que a entrega física esteja sincronizada com a liquidação.

Relatórios e *Dashboard*

1. O sistema deve gerar relatório de faturação por período, quando o gerente seleccionar um intervalo de datas, de forma que seja possível analisar desempenho financeiro sem cálculo manual.
2. O sistema deve permitir consultar produtividade por mecânico, quando o gerente solicitar o relatório correspondente, de forma que se obtenha visibilidade sobre carga e eficiência operacional.

1.3.3.7 Nota de Transição para os *Use Cases*

Conclui-se, assim, a especificação formal dos requisitos do sistema, consolidando os requisitos aos quais o sistema deve seguir e os atributos de qualidade que devem ser assegurados. A secção seguinte parte desta secção e das anteriores descritas neste documento como base, descrevendo os fluxos de interação e os cenários operacionais que materializam os requisitos funcionais definidos, permitindo a passagem estruturada da especificação para a modelação comportamental do sistema.

1.4 Modelação Comportamental

Análise Crítica do Processo de Geração com IA

Prompt 16. Modelação de Casos de Uso UML

Contexto: Sistema de gestão das Oficinas Gengis Khan, fase de especificação detalhada após a validação dos requisitos com os stakeholders, com 44 RF consolidados e necessidade de rastreabilidade bidireccional.

Objetivo: Elaborar a especificação de todos os Use Cases inferíveis dos Requisitos Funcionais, associando a cada UC os requisitos que satisfaz ou influencia.

Instruções de Escrita:

- Aplicar o template anexo com os campos ID, Atores, Descrição, Requisitos, Pre-condição, Pós-condição, Fluxo Normal, Fluxo Alternativo e Fluxo de Exceção
- Admitir que um mesmo requisito pode influenciar mais do que um Use Case
- Seguir rigorosamente os slides de Use Cases da UC de DSS anexos
- Usar include, extend e generalização apenas quando reduzem efectivamente redundância sem prejudicar a legibilidade

Restricoes: Sem uso excessivo dos mecanismos UML de relacao entre UC. Sem referenciar identificadores de RF que nao constem na versao final consolidada em anexo.

Análise Crítica

Ao contrário do que se observou no *Prompt 14* (Análise e Categorização de Requisitos), que constituiu o ponto de maior exigência humana de toda a Etapa 1, esta interação revelou-se substantivamente mais direta. A verificação e consolidação prévia dos requisitos eliminou, à partida, as fontes de inconsistência que poderiam ter comprometido a derivação dos casos de uso. As dificuldades observadas foram exclusivamente de natureza formal, designadamente a tendência inicial do modelo para se desviar do *template* de especificação pretendido e para abusar dos mecanismos UML de *include*, *extend* e generalização. Após a equipa ter produzido manualmente alguns casos de uso completos e tê-los fornecido como referência adicional, o LLM convergiu rapidamente para o formato esperado e passou a gerar autonomamente cada caso de uso em conformidade, com utilização contida dos referidos mecanismos. A revisão humana incidiu, essencialmente, na validação dos fluxos alternativos e de exceção e na normalização terminológica das pré-condições e pós-condições. Os 29 casos de uso aqui apresentados resultam deste processo e encontram-se mapeados, na sua totalidade, aos 44 requisitos funcionais consolidados no SRS.

A modelação comportamental, operacionalizada através de casos de uso, constitui a etapa de transição entre a especificação formal dos requisitos e a modelação técnica do sistema. Enquanto o SRS apresentado anteriormente estabelece *o que* o sistema deve fazer, os casos de uso descrevem *como* o sistema se comporta face às interações dos diversos atores, explicitando os fluxos normais, as ramificações condicionais e as situações excecionais que o sistema deve saber tratar. Segundo Sommerville (2016, cap. 5), os casos de uso são particularmente eficazes como instrumento de comunicação entre a equipa de desenvolvimento e os *stakeholders*, dado que preservam a linguagem do negócio enquanto introduzem o rigor estrutural exigido pela implementação.

No presente projeto, os casos de uso foram derivados de forma sistemática a partir dos 44 requisitos funcionais consolidados na Secção 1.3, garantindo que cada comportamento especificado no SRS encontra correspondência em, pelo menos, um caso de uso. A rastreabilidade bidirecional foi mantida através do campo “Requisitos” presente na especificação de cada UC, que enumera os requisitos funcionais satisfeitos ou influenciados pelo respetivo fluxo. Em conformidade com as orientações metodológicas da unidade curricular de Desenvolvimento de Sistemas de *Software*, procurou-se minimizar o recurso aos mecanismos de *include*, *extend* e generalização, reservando-os apenas para situações em que a sua utilização reduz efetivamente a redundância sem prejudicar a compreensibilidade dos fluxos.

1.4.1 Atores do Sistema

A análise dos casos de uso permitiu identificar três atores distintos, cuja caracterização se apresenta na Tabela 1.17. Os atores humanos (Gerente e Mecânico) correspondem aos perfis de utilizador já formalizados na Secção 1.3; o ator não humano (Relógio do Sistema) representa a componente de automação temporal responsável por desencadear notificações periódicas de forma assíncrona.

Tabela 1.17: Atores identificados na modelação comportamental

Ator	Natureza	Responsabilidades no Sistema
Gerente	Humano, primário	Operações administrativas, configuração do sistema, faturação, gestão de fornecedores e consulta de relatórios.
Mecânico	Humano, primário	Operações técnicas sobre ordens de serviço, diagnósticos, intervenções e associação de peças.
Relógio do Sistema	Não humano, secundário	Deteção periódica de condições temporais e despoletamento automático de notificações a clientes.

1.4.2 Especificação dos Casos de Uso

As subsecções seguintes apresentam, de forma normalizada, a especificação de cada um dos 29 casos de uso identificados, agrupados por área funcional para facilitar a leitura e a correspondência com os módulos do SRS.

1.4.2.1 Excertos Ilustrativos

O catálogo cobre integralmente os fluxos operacionais identificados no levantamento, desde a receção da trotinete e abertura da ordem de serviço até à emissão da fatura, passando pelo controlo de *stock* e pelas notificações ao cliente. Cada caso documenta atores, pré-condições, pós-condições, fluxo principal e fluxos alternativos ou de exceção, constituindo a tradução comportamental dos requisitos funcionais consolidados anteriormente. Por economia de espaço e para preservar a clareza expositiva do corpo do relatório, apresentam-se de seguida apenas dois casos representativos do núcleo operacional, designadamente a criação de uma ordem de serviço (UC-07) e a geração de uma fatura sobre uma ordem concluída (UC-22). A especificação integral, agrupada nas seis áreas funcionais do sistema, encontra-se reproduzida no Anexo J para consulta do leitor interessado.

UC-07: Criar Ordem de Serviço

Atores	Gerente, Mecânico
Descrição	Um funcionário cria uma nova ordem de serviço para uma trotinete entregue por um cliente.
Requisitos	RF-09, RF-12, RF-13, RF-41
Pré-condição	Funcionário está autenticado. Cliente está registado no sistema. Trotinete está registada no sistema e associada ao cliente em questão.
Pós-condição	OS é criada com numeração sequencial, estado “Recebida”, data de entrada e problema reportado. Cliente é notificado por <i>email</i> .
Fluxo Normal	1. Gerente insere o NIF do cliente e o número de série da trotinete. 2. Sistema valida o cliente e a trotinete. 3. Sistema cria a OS com numeração sequencial e estado inicial “Recebida”. 4. Sistema regista automaticamente a data e hora da criação. 5. Funcionário insere o problema reportado pelo cliente. 6. Funcionário insere o prazo estimado de reparação. 7. Sistema envia ao cliente uma confirmação de receção por <i>email</i> com o número da OS e o prazo estimado.
Fluxo Alternativo	(<i>não aplicável</i>)
Fluxo de Exceção	FE1 [Cliente ou trotinete não encontrados] (passo 2): 2.1 Sistema informa que o cliente ou a trotinete não existe no sistema. 2.2 Sistema cancela a criação da OS.

UC-22: Gerar Fatura

Atores	Gerente
Descrição	Um gerente gera a fatura de uma ordem de serviço concluída.
Requisitos	RF-19, RF-29, RF-30, RF-31, RF-32, RF-33
Pré-condição	Gerente está autenticado. Existem OS com estado “Concluída” e intervenções registadas, sem fatura associada.
Pós-condição	Fatura é gerada com numeração sequencial e fica registada na OS.

Fluxo Normal	1. Gerente seleciona a OS concluída. 2. Sistema calcula automaticamente o valor da reparação com base no preço de venda das peças, no tempo de mão de obra (aplicando a taxa horária configurada) e no IVA aplicável. 3. Sistema gera automaticamente a fatura com numeração sequencial contínua, detalhando as peças utilizadas e as horas de trabalho, bem como subtotal, IVA e total calculados automaticamente. 4. Sistema associa à OS a fatura gerada.
Fluxo Alternativo	<i>(não aplicável)</i>
Fluxo de Exceção	<i>(não aplicável)</i>

1.4.3 Síntese e Transição para a Fase Seguinte

A especificação dos 29 casos de uso acima apresentada concretiza, em termos comportamentais, o conjunto de 44 requisitos funcionais formalizados no SRS. Todas as interações previstas entre os atores humanos (Gerente e Mecânico) e o sistema, bem como a componente de automação temporal (Relógio do Sistema), encontram-se descritas em fluxos normalizados com tratamento explícito das ramificações condicionais e das situações excecionais relevantes.

Este artefacto encerra formalmente a Etapa 1 do projeto. A Etapa seguinte, dedicada à Arquitetura e Desenho do *Software*, tomará os casos de uso aqui especificados como ponto de partida para a elaboração dos diagramas de classes, de sequência e de componentes, bem como para a definição da arquitetura global do sistema e das interfaces aplicacionais. A rastreabilidade estabelecida entre requisitos funcionais, casos de uso e, posteriormente, elementos arquiteturais, constitui a espinha dorsal metodológica que permite verificar, ao longo de todo o ciclo de desenvolvimento, que nenhuma necessidade de negócio formalizada na presente etapa fica por satisfazer nas fases subsequentes.

2 Arquitetura e Design do *Software* Assistidos por LLM

O presente capítulo documenta a segunda etapa do ciclo de vida do projeto, dedicada à transição do domínio do problema para o domínio da solução. Com base nos artefactos de engenharia de requisitos produzidos no Capítulo 1, esta etapa materializa as decisões arquiteturais fundamentais do sistema e o design detalhado das camadas de persistência, serviços e apresentação. Tal como na etapa anterior, o recurso a modelos de linguagem de grande dimensão (LLM) é sistemático e devidamente instrumentado, sendo cada decisão precedida pelo *prompt* utilizado, seguida da análise crítica do resultado gerado e da justificação da escolha final da equipa.

2.1 Decisões Arquiteturais

A presente secção documenta as decisões arquiteturais fundamentais adotadas para o sistema de gestão das *Oficinas Gengis Khan*, tomadas na transição entre a engenharia de requisitos (Capítulo 1) e a implementação efetiva do *software*. Organizam-se em seis áreas de decisão, designadamente o padrão arquitetural (Subsecção 2.1.1), a composição da *stack* tecnológica (Subsecção 2.1.2), a organização do código em *monorepo* (Subsecção 2.1.3), o mecanismo de partilha de contratos entre camadas (Subsecção 2.1.4), o modelo de persistência e a autoridade do *schema* (Subsecção 2.1.5) e o modelo de implantação e a segmentação de ambientes (Subsecção 2.1.6).

A abordagem metodológica é comum a todas as decisões. Para cada área, foi submetido ao LLM um *prompt* contendo o contexto operacional já estabilizado no Capítulo 1 e um enunciado que enumera alternativas a ponderar. A resposta do modelo é analisada criticamente pela equipa, que valida ou rejeita o raciocínio, documenta as consequências esperadas e formaliza a escolha. Cada decisão foi, adicionalmente, registada pela equipa no formato de *Architecture Decision Record* (ADR), cujo conjunto é apresentado na Subsecção 2.1.7, garantindo rastreabilidade entre os requisitos estabelecidos no Capítulo 1 e as escolhas técnicas que aqui se apresentam.

2.1.1 Padrão Arquitetural

Análise Crítica do Processo de Geração com IA

Prompt 17. Escolha do Padrão Arquitetural

Contexto: SRS das Oficinas Gengis Khan já consolidado (44 RF, 5 RNF, 29 casos de uso, equipa de 3-4 mecanicos concorrentes, restricoes de faturacao sequencial e RGPD). Inicio do Capitulo 2 (Arquitetura e Design).

Objetivo: Justificar a escolha do padrao arquitetural entre tres candidatos, designadamente monolito em camadas, monolito modular e microservicos.

Instrucoes de Escrita:

- Analise comparativa em eixos de acoplamento, coesao e complexidade operacional
- Adequacao ao contexto academico e ao volume operacional esperado
- Conclusao sustentada com fronteiras de modulo por dominio funcional

Restricoes: Nao recomendar microservicos sem justificacao explicita face ao custo de coordenacao. Sem decisoes de stack tecnologica especifica.

Análise Crítica

Para a escolha do padrão arquitetural, o LLM produziu, sem necessidade de iteração corretiva, uma análise comparativa organizada em três eixos (complexidade de desenvolvimento, complexidade operacional e adequação ao volume), com argumentação articulada para os três candidatos considerados, designadamente monólito em camadas, monólito modular e microserviços. A recomendação final do modelo recaiu sobre o monólito modular e foi acolhida pela equipa sem reservas, dado que o volume operacional projetado não justifica o custo de coordenação inerente a microserviços e o horizonte temporal académico, conjugado com a dimensão da equipa, torna incomportável a operação de múltiplos serviços com *pipelines* de *deploy* independentes.

Importa, contudo, registar uma observação metodológica que esta interação tornou clara. A vantagem efetiva do LLM em decisões arquiteturais reside menos na capacidade de produzir uma resposta correta e mais na possibilidade de debater rapidamente o espectro de alternativas disponíveis. Em poucos *prompts* é possível percorrer hipóteses que, em discussão interna, exigiriam várias sessões de equipa para serem articuladas com o mesmo grau de detalhe. Nesta circunstância, o LLM aproxima-se da função de um membro adicional da equipa, com conhecimento amplo de paradigmas arquiteturais e de modelos de desenvolvimento, capaz de propor e fundamentar opções que enriquecem o leque de hipóteses considerado.

Esta utilização não dispensa, contudo, a discussão crítica coletiva. Sem essa verificação, corre-se o risco de aceitar com excessiva facilidade uma proposta bem articulada apenas pela sua aparente solidez. A prática adotada consistiu, por isso, em interpor uma fase de discussão interna entre a leitura da resposta do modelo e a sua incorporação no relatório, em que cada eixo de comparação foi reavaliado contra as restrições reais do projeto. A contribuição mais útil do modelo foi a recomendação de impor limites de módulo por domínio funcional (clientes, ordens de serviço, inventário, faturação, fornecedores) desde o início, preparando o sistema para uma eventual extração futura de microserviços caso o crescimento o justifique. Esta orientação reflete-se diretamente na estrutura de pastas do *backend*.

O sistema adota um padrão de **monólito modular em três camadas**, estruturado sobre uma fronteira de rede única e um processo de *deployment* unificado, mas com separação estrita entre módulos funcionais ao nível do código. A decisão combina duas tradições bem estabelecidas da engenharia de *software*. A arquitetura em camadas separa responsabilidades entre apresentação, lógica de negócio e persistência. A modularidade por domínio (*domain-oriented modularity*) agrupa internamente o *backend* por áreas funcionais coesas.

A Figura 2.1 sintetiza a organização global do sistema, evidenciando as três camadas lógicas, os protocolos de comunicação entre elas e o papel transversal do pacote de contratos partilhados que atravessa a fronteira *frontend/backend*. Os parágrafos que se seguem detalham cada camada.

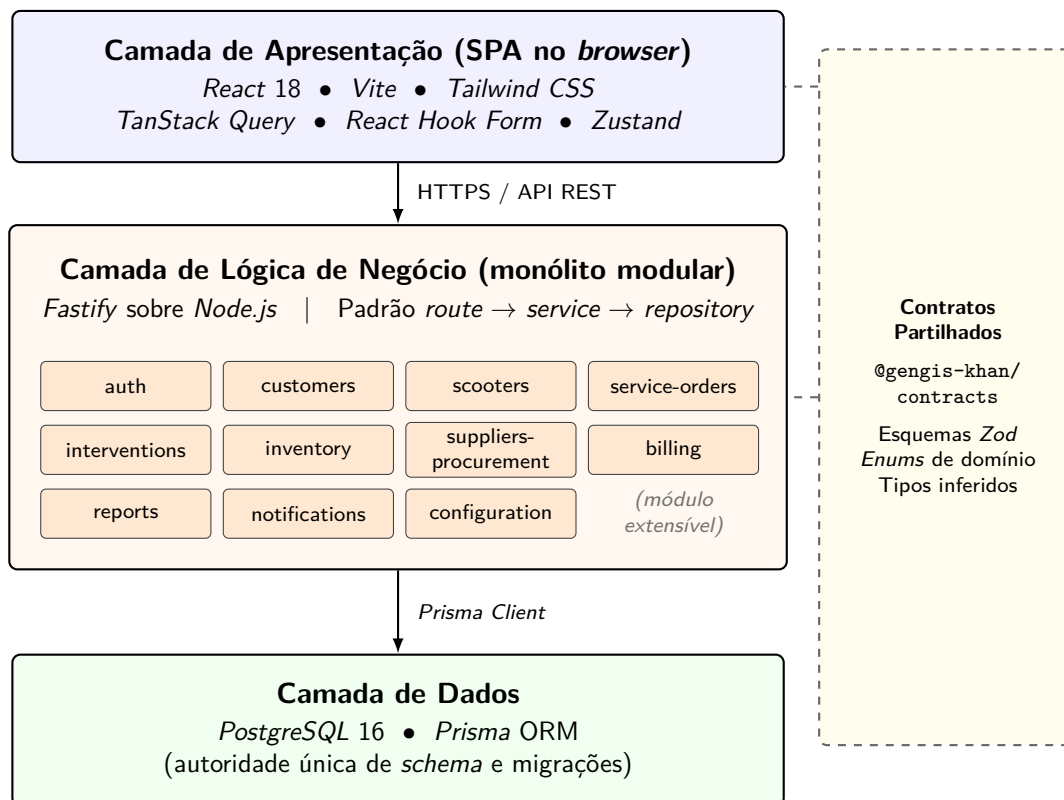


Figura 2.1: Arquitetura em três camadas do sistema, com o pacote de contratos partilhados a atravessar transversalmente a fronteira entre *frontend* e *backend*.

Camada de apresentação. Responsável pela interface com o utilizador, é materializada numa aplicação de página única (*Single Page Application*, SPA) executada no *browser*, que comunica com o *backend* exclusivamente por HTTPS e através de uma API REST versionada. A camada de apresentação não contém lógica de negócio. Toda a decisão funcional, validação formal e interação com a base de dados ocorre exclusivamente no *backend*. Esta disciplina assegura consistência comportamental independentemente do cliente consumidor.

Camada de lógica de negócio. Implementada no *backend*, organiza-se internamente em módulos de domínio (*auth*, *customers*, *scooters*, *service-orders*, *interventions*, *inventory*, *suppliers-procurement*, *billing*, *reports*, *configuration*, *notifications*). Cada módulo segue uma estrutura interna padronizada *route* → *service* → *repository*, na qual as rotas validam a entrada e traduzem erros HTTP, os serviços implementam as regras de negócio e orquestram transações, e os repositórios encapsulam o acesso à base de dados. Esta separação interna preserva a testabilidade unitária e delimita claramente as dependências entre módulos.

Camada de dados. Assente numa base de dados relacional *PostgreSQL 16*, é acedida através de um ORM que fornece tipagem estática e migrações versionadas. A autoridade sobre o esquema é única e centralizada, eliminando a possibilidade de divergência entre ambientes.

A análise comparativa entre o monólito modular adotado e as alternativas ponderadas (monólito em camadas e microserviços), com o detalhe dos critérios de decisão, encontra-se reproduzida no Anexo 5.4 (ADR-0002). A escolha sustenta-se principalmente na adequação ao volume operacional previsto e ao contexto académico, com fronteiras modulares explícitas que reduzem o acoplamento entre ramos de desenvolvimento paralelos e facilitam a revisão por pares.

2.1.2 Stack Tecnológica e Fundamentação

Análise Crítica do Processo de Geração com IA

Prompt 18. Composição da *Stack* Tecnológica

Contexto: Decisão previa pelo monolito modular em tres camadas.
Restricoes do projeto (equipa de 5 estudantes, 14 semanas para implementacao, tipagem estatica partilhada entre frontend e backend, RGPD e faturacao sequencial portuguesa).

Objetivo: Justificar a escolha da stack tecnologica em quatro eixos, designadamente linguagem e runtime do backend, framework web do backend, linguagem e framework do frontend, e base de dados com camada de acesso.

Instrucoes de Escrita:

- Para cada eixo apresentar alternativas ponderadas e criterios de decisao
- Fundamentar em atributos de qualidade e adequacao ao contexto
- Articular escolhas que favorecam a uniformidade entre camadas

Restricoes: Nao favorecer escolhas por moda. Sem detalhes de implementacao por modulo.

Análise Crítica

Foi nesta interação que ocorreu, pela primeira vez nesta etapa, algo que não se tinha observado em nenhum dos *prompts* anteriores. O LLM foi contra uma decisão que a equipa já tinha tomado. A intenção inicial era usar *C#* no *backend* e *TypeScript* no *frontend*. Confrontado com esta escolha, o modelo propôs em alternativa que ambas as camadas usassem *TypeScript*, argumentando que adotar a mesma linguagem evita o esforço de alternar entre paradigmas, permite reutilizar tipos e validações entre *frontend* e *backend* e reduz o número de ferramentas a dominar num horizonte temporal apertado. Reconheceu a maturidade dos ecossistemas alternativos, mas concluiu que o seu peso não compensava a perda de uniformidade num projeto académico com poucas semanas para implementar. A equipa aceitou o argumento e adotou *TypeScript* em ambas as camadas, deixando de lado a opção inicial de *C#*.

Esta interação revela uma característica do trabalho com LLMs que normalmente passa despercebida. Quem não usa estes modelos no dia a dia tende a vê-los como executores de instruções, que recebem um pedido e devolvem uma resposta. Aqui, o modelo não se limitou a cumprir o que lhe foi pedido. Questionou a premissa, propôs uma alternativa e fundamentou-a. Importa, ainda assim, manter a guarda, dado que um argumento bem escrito não é, por si só, um argumento correto. Antes de aceitar a proposta, a equipa testou cada um dos benefícios invocados contra as restrições reais do projeto, e foi essa verificação que sustentou a aceitação final e não a fluência da resposta. Em retrospectiva, esta foi uma das contribuições mais relevantes do LLM ao longo da etapa, pelo impacto estrutural da decisão e pela dificuldade de a reverter mais tarde.

A escolha da *stack* tecnológica baseou-se em quatro critérios ponderados, designadamente a adequação aos atributos de qualidade exigidos pelo sistema (desempenho, segurança, manutibilidade), a maturidade do ecossistema e qualidade da documentação, a adequação ao horizonte temporal académico e às competências existentes na equipa, e a capacidade de partilha de abstrações entre camadas. A Tabela 2.1 apresenta as decisões estruturantes; a composição completa, com todas as bibliotecas adotadas e respetiva justificação, encontra-se reproduzida no Anexo 5.4.

Tabela 2.1: Excerto da composição da *stack* tecnológica

Camada / Papel	Tecnologia	Justificação principal
Linguagem partilhada (<i>backend</i> + <i>frontend</i>)	<i>TypeScript</i> $\geq$ 5.8	Tipagem estática forte; partilha literal de tipos entre camadas.
<i>Framework web (backend)</i>	<i>Fastify</i> 4	Desempenho superior ao <i>Express</i> ; suporte nativo a validação de esquemas.
<i>Framework (frontend)</i>	<i>React</i> 18 com <i>Vite</i> 6	Ecossistema maduro; <i>Hot Module Replacement</i> eficiente.
Validação de esquemas	<i>Zod</i> 3	Esquemas executáveis com inferência de tipos partilhada entre camadas.
Persistência	<i>PostgreSQL</i> 16 com <i>Prisma</i> 5	Conformidade ACID estrita; <i>schema</i> declarativo com cliente tipado gerado.
(...) Tabela completa no Anexo 5.4		

A uniformidade linguística em torno de *TypeScript* assume particular relevância como habilitador arquitetural, ao permitir a existência de um pacote partilhado de contratos (discutido na Subsecção 2.1.4) que é simultaneamente a fonte da verdade para a validação em tempo de execução no *backend* e para a inferência estática de tipos no *frontend*, eliminando, por construção, uma classe comum de defeitos de integração entre camadas.

2.1.3 Organização do Código em *Monorepo*

Análise Crítica do Processo de Geração com IA

Prompt 19. Organização em *Monorepo*

Contexto: Sistema das Oficinas Gengis Khan, monolito modular em TypeScript com frontend React e backend Fastify. Decisão de organização do código entre dois repositórios separados ou um monorepo unificado.

Objetivo: Comparar a opção de dois repositórios Git separados com a opção de um monorepo gerido por pnpm workspaces e Turborepo, recomendando uma das alternativas com tradeoffs residuais explícitos.

Instruções de Escrita:

- Considerar partilha de tipos entre camadas como factor primário
- Avaliar ciclo de vida de builds e overhead de coordenação em refatorações cross-cutting
- Considerar curva de aprendizagem para uma equipa sem experiência prévia com monorepos

Restrições: Recomendar uma opção concreta. Sem ambiguidade na conclusão final.

Análise Crítica

A análise produzida pelo LLM favoreceu de forma explícita a opção de *monorepo*, fundamentando a decisão na capacidade de partilha literal de código entre pacotes e na existência de ferramentas maduras (*pnpm workspaces*, *Turborepo*) que automatizam a orquestração de *builds* dependentes. O modelo alertou, contudo, para dois riscos residuais, designadamente a necessidade de disciplina na definição de fronteiras entre pacotes (sob pena de o *monorepo* degenerar num sistema com elevado acoplamento estrutural) e o aumento da complexidade inicial de configuração do *workspace*. A equipa aceitou a recomendação e previu a mitigação do primeiro risco através da definição, em fase de design, de um conjunto restrito de pacotes com responsabilidades únicas.

Em sede de design, a equipa optou por estruturar o código-fonte do projeto como um *monorepo*, a gerir através de pnpm workspaces e a orquestrar por Turborepo. Esta escolha visa materializar três benefícios estruturais, designadamente a partilha direta de código entre aplicações e bibliotecas internas, a coerência de versões de dependências transversais e a execução paralela e cacheada de tarefas de *build*, *lint*, *typecheck* e testes. A organização lógica projetada é a seguinte:

- apps/api: aplicação *backend* (*Fastify*, monólito modular).

- apps/web: aplicação *frontend* (*React*, SPA empacotada com *Vite*).
- packages/contracts: contratos compartilhados, esquemas *Zod* e enumerações de domínio.
- packages/tooling-typescript: configuração compartilhada de *TypeScript*.
- packages/tooling-eslint: configuração compartilhada de ESLint *flat-config*.
- supabase: configuração da instância local de *PostgreSQL* (apenas desenvolvimento).
- docs: documentação arquitetural, ADRs, manuais de configuração e especificação *OpenAPI*.

A disciplina do *monorepo* será reforçada por três mecanismos de governação previstos em fase de design. A configuração de *Turborepo* declarará explicitamente as dependências entre tarefas, impedindo que um pacote dependente seja compilado antes do seu produtor. Os pacotes de *tooling* centralizarão a configuração de linguagem e estilo, assegurando uniformidade entre aplicações. A separação entre apps/ e packages/ estabelecerá uma hierarquia de dependência unidirecional, na qual apenas aplicações dependerão de pacotes e nunca o inverso.

2.1.4 Partilha de Contratos entre Camadas

Análise Crítica do Processo de Geração com IA

Prompt 20. Mecanismo de Partilha de Contratos

Contexto: Sistema das Oficinas Gengis Khan, monorepo TypeScript com frontend e backend. Necessidade de manter os contratos de dados sincronizados entre as duas camadas ao longo do ciclo de vida.

Objetivo: Propor a estratégia mais adequada para garantir essa sincronização, comparando três abordagens, designadamente geração de tipos TypeScript a partir de OpenAPI, uso de tRPC para expor tipos diretamente, e definição manual de esquemas Zod num pacote partilhado.

Instruções de Escrita:

- Considerar a necessidade conjunta de validação em runtime no backend e de verificação estática no frontend
- Avaliar acoplamento entre camadas, poder expressivo e sobrecarga cognitiva
- Recomendar uma abordagem com justificação explícita

Restricoes: Sem ambiguidade na recomendacao final.

Análise Crítica

A justificação apresentada pelo LLM para a recomendação foi particularmente bem articulada, e a equipa subscreveu a fundamentação sem dificuldade. Esta interação confirma uma característica do modelo já observada em interações anteriores, designadamente a sua capacidade de argumentar de forma clara e persuasiva por que motivo uma opção é preferível às restantes. Em decisões arquiteturais, esta competência traduz-se numa intuição que se aproxima, em muitos casos, da que se espera de um engenheiro experiente, permitindo que o modelo proponha alternativas, identifique *tradeoffs* e sustente cada escolha com vantagens concretas. A equipa tem aprendido, ao longo desta etapa, a tratar o modelo como ponto de partida sólido para a discussão arquitetural.

Esta competência não dispensa, contudo, a verificação humana. Em diversos momentos das interações conduzidas, propostas igualmente bem articuladas revelaram-se, à inspeção atenta, parcialmente desadequadas ou simplesmente desligadas das restrições reais do projeto. A fluência com que o modelo encadeia argumentos pode dar a impressão de que qualquer recomendação faz sentido, ainda que, à leitura crítica, alguns dos pressupostos invocados não se sustentem. A equipa tem mantido, por princípio, essa leitura crítica, validando cada proposta contra o contexto efetivo do trabalho. Aceitar uma recomendação apenas pela clareza da sua exposição constitui, na realidade, um dos riscos mais prováveis ao trabalhar com um LLM em decisões com impacto estrutural.

O pacote `@gengis-khan/contracts` materializará a fronteira contratual entre *frontend* e *backend* e conterá exclusivamente três categorias de artefactos, designadamente esquemas *Zod* que descreverão as formas de *request* e *response* de cada *endpoint* da API, enumerações de domínio partilhadas (por exemplo, estados de ordens de serviço, tipos de cliente, papéis de utilizador) e tipos derivados por inferência a partir dos esquemas, exportados para consumo por ambas as aplicações.

Esta disciplina assegurará três propriedades fundamentais do sistema.

1. **Fonte única da verdade.** Qualquer alteração ao contrato de um *endpoint* obrigará a modificar o esquema *Zod* correspondente no pacote partilhado, propagando-se simultaneamente para o *backend* (que perderá validação de *payloads* inválidos em tempo de execução) e para o *frontend* (que deixará de compilar até que o código consumidor seja ajustado).
2. **Validação em fronteira.** O *backend* validará toda a entrada externa contra o esquema antes de invocar qualquer lógica de negócio, materializando o princípio de *fail-fast* na camada de rota e impedindo que dados malformados atinjam a camada de serviço.
3. **Segurança de tipo no frontend.** O *frontend* consumirá tipos inferidos diretamente dos esquemas, o que eliminará a possibilidade de incoerência silenciosa entre a forma esperada e a forma efetivamente devolvida pela API.

Esta estratégia será explorada em detalhe através de duas convenções de modelação transversais a todo o pacote partilhado. A primeira convenção consistirá na definição de um conjunto reduzido de *validadores primitivos* reutilizáveis, que encapsularão invariantes de domínio recorrentes, designadamente a forma canónica do número de identificação fiscal (cadeia de nove dígitos exatos), o formato dos valores monetários (representação textual com duas casas decimais, de modo a evitar ambiguidades de ponto flutuante na serialização HTTP) e o formato de datas em ISO 8601. A reutilização destes primitivos em todos os esquemas compostos impedirá que validações equivalentes divirjam silenciosamente entre recursos distintos e concentrará num único ponto a evolução futura de cada regra. A segunda convenção estabelecerá que as regras de negócio *cross-field* serão expressas através dos mecanismos de refinamento fornecidos pelo *Zod*, que permitirão condicionar a obrigatoriedade de certos campos ao valor de outros. Um caso típico, a descrever no esquema de cliente, consistirá em impor a presença de razão social, limite de crédito e condições de pagamento apenas quando o cliente for classificado como empresarial, sem necessidade de manter dois esquemas disjuntos para os casos pessoal e empresarial.

Esta estratégia será complementada por uma especificação *OpenAPI*, mantida como artefacto independente do pacote de contratos e destinada a descrever a API sob a forma canónica reconhecida por ferramentas de terceiros (geração de clientes, portais de documentação, *mocking*). A especificação adotará a versão 3.1.0 do padrão e organizar-se-á em onze *tags* coincidentes com os módulos do *backend*, cobrindo o conjunto de *endpoints* operacionais. Três convenções transversais governarão toda a especificação. A autenticação será descrita por um *security scheme* do tipo *cookieAuth*, refletindo a decisão arquitetural de transportar o *token* de sessão em *cookie httpOnly*. As respostas de erro serão centralizadas num esquema reutilizável uniforme, fornecendo ao *frontend* um formato estável para apresentação de mensagens *inline* nos formulários. Os parâmetros de caminho correspondentes a chaves naturais do domínio (NIF, número de série, referência de peça) serão declarados como parâmetros reutilizáveis, eliminando a repetição da sua especificação em todas as rotas que sobre eles operam.

A articulação entre os dois artefactos será clara. O pacote de contratos *Zod* permanecerá a fonte canónica de verdade, executável em tempo de execução e partilhada entre camadas, enquanto a especificação *OpenAPI* assumirá o papel de projecção documental da API, útil a integrações externas e a leitores que não tenham acesso direto ao código do sistema.

2.1.5 Modelo de Persistência e Autoridade do *Schema*

Análise Crítica do Processo de Geração com IA

Prompt 21. Modelo de Persistência

Contexto: Sistema das Oficinas Gengis Khan com restricao legal de faturacao sequencial (Decreto-Lei 28/2019), transacoes

financeiras (Invoice, Payment) e auditoria de alteracoes em ordens de servico.

Objetivo: Analisar a adequacao do PostgreSQL face a alternativas NoSQL (MongoDB, DynamoDB) e recomendar a solucao de persistencia, indicando tambem o ORM associado.

Instrucoes de Escrita:

- Discutir propriedades ACID, constraints de unicidade e sequences monotonicas
- Considerar suporte nativo a migracoes versionadas
- Justificar a inadequacao das alternativas NoSQL para o dominio fiscal

Restricoes: Recomendar uma solucao concreta. Sem detalhes de schema especifico.

Análise Crítica

A escolha de *PostgreSQL* e *Prisma* havia já sido tomada na discussão da *stack* tecnológica, pelo que esta interação serviu sobretudo para validar essas escolhas face às exigências específicas do domínio, em especial a restrição legal de faturação sequencial. A justificação produzida pelo LLM foi sólida, articulou com clareza os requisitos não negociáveis que tornam o *PostgreSQL* adequado a este tipo de domínio fiscal, e a equipa subscreveu-a sem reservas.

A camada de persistência assenta em *PostgreSQL* 16 como motor relacional, com o *Prisma* como única autoridade sobre o *schema* e as migrações. Esta centralização tem três consequências práticas relevantes:

Schema declarativo e versionado. O *schema* completo do domínio (utilizadores, clientes, trotinetes, ordens de serviço e respetivo histórico, intervenções, peças, fornecedores, ordens de compra, faturas, configuração financeira, notificações, movimentos de *stock*) é descrito num único ficheiro *schema.prisma*, sujeito a controlo de versão. Cada alteração ao *schema* gera uma migração SQL imutável, armazenada no repositório e aplicável de forma determinista a qualquer ambiente.

Cliente tipado gerado. O cliente *Prisma* é regenerado a partir do *schema*, expondo ao *backend* uma API fortemente tipada que elimina classes inteiras de erros de consulta (colunas inexistentes, incompatibilidade de tipos, relações mal especificadas). Esta geração ocorre no processo de instalação de dependências, assegurando que o código do *backend* e o *schema* da base de dados permanecem permanentemente sincronizados.

Separação entre infraestrutura de desenvolvimento e produção. Em desenvolvimento local, a instância de *PostgreSQL* é provisionada pela ferramenta *Supabase CLI*, que fornece um *stack* completo de serviços contentorizados (*PostgreSQL*, autenticação opcional, estúdio

de inspeção). Em produção, o sistema é implantado contra uma instância de *PostgreSQL* 16 pura, sem dependência de serviços da plataforma *Supabase*. Esta independência foi registrada no ADR-0001 (Anexo 5.4) e visa evitar *vendor lock-in*, preservando a opção de implantação em qualquer provedor *cloud* ou ambiente *on-premises*.

2.1.6 Modelo de Implantação e Ambientes

Análise Crítica do Processo de Geração com IA

Prompt 22. Modelo de Implantação

Contexto: Sistema das Oficinas Gengis Khan com stack Node.js, React, PostgreSQL e Prisma, com necessidade de paridade entre ambientes e uso de Supabase CLI apenas em desenvolvimento.

Objetivo: Delinear o modelo de implantacao para tres ambientes (desenvolvimento local, integracao continua, producao) e identificar artefactos por ambiente e pontos de divergencia aceitaveis.

Instrucoes de Escrita:

- Distinguir infraestrutura de conveniencia (local) da infraestrutura efectiva de producao
- Enumerar artefactos por ambiente (imagens Docker, scripts de migracao, variaveis de ambiente)
- Discutir paridade de comportamento e portabilidade

Restricoes: Sem detalhes de configuracao de runner especifico. Sem mencao a provedores cloud concretos.

Análise Crítica

A proposta do LLM organizou o modelo de implantação em torno de uma decomposição clara, ao separar deliberadamente a vista de desenvolvimento da vista de produção. O modelo justificou esta separação com o argumento de que aceitar uma assimetria controlada entre os dois ambientes simplifica significativamente o trabalho do programador no dia a dia, sem comprometer a integridade do artefacto que será implantado em produção. Para enquadrar esta ideia, introduziu o conceito de *infraestrutura de conveniência*, que designa as ferramentas adotadas apenas em desenvolvimento para reduzir fricção, sem impacto no artefacto final. O *Supabase* CLI enquadra-se nesta categoria, permitindo arranque imediato de uma instância *PostgreSQL* local sem que essa dependência se propague à produção. O modelo identificou ainda a armadilha mais comum desta abordagem, designadamente a dependência acidental de serviços específicos da plataforma local, e recomendou a documentação explícita do que é aceitável usar localmente e do que deve permanecer portátil.

A equipa considerou a proposta particularmente convincente. A tentação inicial seria procurar paridade integral entre ambientes, sob a premissa de que qualquer divergência introduz risco. A sugestão do modelo coloca, contudo, o problema sob outra perspetiva, mostrando que o custo de manter paridade integral durante o desenvolvimento ultrapassa muitas vezes o risco que essa paridade pretende mitigar, e que uma separação consciente, acompanhada de documentação rigorosa, é frequentemente preferível. A recomendação foi incorporada no registo de Decisões Arquiteturais apresentado na Subsecção 2.1.7. Importa, ainda assim, manter a vigilância sobre o pacto implícito desta abordagem, dado que a fronteira entre conveniência local e dependência produtiva é, por natureza, ténue e tende a ser atravessada inadvertidamente.

O modelo de implantação previsto pela equipa contempla três ambientes com contornos bem definidos:

Desenvolvimento local. O programador executará, na sua estação de trabalho, o *stack Supabase* CLI (que provisiona uma instância de *PostgreSQL* local na porta 54322), as aplicações *apps/api* e *apps/web* em modo *watch* através de *pnpm dev* e, opcionalmente, o *Mailpit* para interceção local de *emails*. Como alternativa, todo o *stack* de aplicação poderá ser executado em contentores *Docker* através de um ficheiro *docker-compose.yml* a fornecer no repositório, facilitando a demonstração integrada e a reprodutibilidade entre estações de trabalho heterogéneas.

Integração contínua. A *pipeline* de CI executará, sobre cada alteração submetida, as tarefas *pnpm lint*, *pnpm typecheck*, *pnpm test* e *pnpm build*, todas a orquestrar por *Turborepo* com *cache* partilhado, de modo a reduzir o tempo de execução em alterações incrementais.

Produção. O sistema será implantado em dois artefactos *Docker* independentes (imagem do *backend* e imagem do *frontend*) sobre uma instância dedicada de *PostgreSQL* 16, sem qualquer dependência das ferramentas de desenvolvimento local. A comunicação entre *backend* e base de dados ocorrerá através de um URL de ligação fornecido como variável de ambiente e os segredos (chaves JWT, credenciais SMTP, credenciais de base de dados) serão geridos fora do controlo de versão.

A Tabela 2.2 resume as diferenças entre ambientes.

Tabela 2.2: Comparação entre ambientes de execução

Aspeto	Desenvolvimento local	Integração contínua	Produção
<i>PostgreSQL</i>	<i>Supabase</i> CLI (porta 54322)	Contentor efémero	Instância dedicada
Aplicações	<i>pnpm dev</i> ou <i>Docker Compose</i>	<i>Build</i> sem execução	Imagens <i>Docker</i>
<i>Email</i>	<i>Mailpit</i> local	Desativado	SMTP real
Migrações <i>Prisma</i>	Aplicadas manualmente	Validadas em <i>dry-run</i>	Aplicadas no <i>release</i>
Segredos	<i>.env</i> local	Variáveis do <i>runner</i>	Gestor de segredos

2.1.7 Registo de Decisões Arquiteturais

As decisões apresentadas nas subsecções anteriores foram formalmente registadas pela equipa sob o formato de *Architecture Decision Records* (ADR) e encontram-se reproduzidas na íntegra no Anexo F (página 239). Cada ADR documenta, num único documento de dimensão contida, o contexto que motivou a decisão, as alternativas ponderadas, a escolha efetuada, o seu estado (proposta, aceite, substituída, obsoleta) e as consequências esperadas, positivas e negativas. Este formato, popularizado por Michael Nygard e amplamente adotado na indústria, cumpre três funções complementares, designadamente constituir memória institucional para a equipa, fornecer rastreabilidade entre requisitos e escolhas técnicas, e servir de suporte à reavaliação disciplinada de decisões quando novos requisitos emergem.

O registo produzido nesta etapa documenta as decisões estruturais sobre a organização em *monorepo* com contratos partilhados (ADR-0001) e o padrão arquitetural de monólito modular em três camadas (ADR-0002), reproduzidos no Anexo F. À medida que o projeto avançar para a fase de implementação (Capítulo 3) e para a verificação e validação subsequente, novos ADR serão incorporados para refletir decisões emergentes, nomeadamente sobre estratégias de *caching*, políticas de *rate limiting*, modelo de autorização fina e arquitetura de testes.

2.1.8 Síntese das Decisões Arquiteturais

A Tabela 2.3 resume o conjunto de decisões arquiteturais adotadas, com referência à subsecção em que cada uma é fundamentada, servindo de mapa de navegação para o leitor e de registo consolidado para consulta em capítulos subsequentes.

Tabela 2.3: Síntese das decisões arquiteturais adotadas

Decisão	Escolha adotada	Fundamentação
Padrão arquitetural	Monólito modular em três camadas	§2.1.1
Runtime e linguagem backend	<i>Node.js</i> 20 com <i>TypeScript</i>	§2.1.2
Framework web do backend	<i>Fastify</i> 4	§2.1.2
Linguagem e framework do frontend	<i>TypeScript</i> com <i>React</i> 18 e <i>Vite</i> 6	§2.1.2
Base de dados	<i>PostgreSQL</i> 16	§2.1.5
ORM e autoridade de schema	<i>Prisma</i> 5	§2.1.5
Organização do código	<i>Monorepo</i> com <i>pnpm workspaces</i> e <i>Turbo-repo</i>	§2.1.3
Partilha de contratos	Pacote interno de esquemas <i>Zod</i>	§2.1.4
Autenticação	JWT com cookies <i>HttpOnly</i> e <i>bcryptjs</i>	§2.1.2
Infraestrutura local	<i>Supabase</i> CLI (<i>PostgreSQL</i>) e <i>Docker Compose</i>	§2.1.6
Produção	<i>PostgreSQL</i> 16 puro, sem dependências de plataforma	§2.1.6
Formato de registo de decisões	<i>Architecture Decision Records</i> em docs/adr/	§2.1.7

O conjunto destas decisões configura um sistema tecnicamente coerente, adequado ao volume operacional projetado, conforme com as restrições legais aplicáveis (Subsecção 1.2.6.2) e evolutivo na medida do necessário. As secções que se seguem no presente capítulo (Secções 2.2 a 2.7) assentam integralmente sobre estas escolhas estruturais.

2.2 Modelo Estrutural do Sistema

Fixadas as decisões arquiteturais discutidas nas subsecções anteriores, importa agora apresentar o modelo estrutural do sistema em UML. Este modelo refina os conceitos de domínio introduzidos no Capítulo 1, dotando-os dos atributos, tipos, multiplicidades, enumerações e relações que os tornarão operáveis. A exposição segue uma lógica de granularidade crescente. Começa-se por uma vista de pacotes, que delineia a partição funcional do modelo e as dependências estruturais entre grupos de entidades. Seguem-se três diagramas de classes, um por subsistema, que detalham a estrutura interna de cada pacote. Encerra-se a secção com uma consolidação das decisões estruturais transversais a todos os subsistemas, as quais condicio-

nao atributos de qualidade não funcionais relevantes, designadamente a integridade fiscal, a auditabilidade e a modificabilidade.

O modelo aqui apresentado deriva do modelo de domínio do Capítulo 1 e alinha-se com as obrigações legais e regulamentares identificadas nas subsecções dedicadas aos requisitos, resultando num conjunto de dezasseis entidades agrupáveis em três subsistemas funcionais coerentes. Esta derivação deliberada preserva a rastreabilidade entre o levantamento de requisitos e as decisões de modelação, evitando o desfasamento entre o modelo conceptual de partida e o modelo detalhado produzido em fase de *design*.

Análise Crítica do Processo de Geração com IA

Prompt 23. Diagrama de Classes UML

Contexto: Modelo de dominio das Oficinas Gengis Khan (Capitulo 1) e schema Prisma com dezasseis entidades, valores monetarios, enumeracoes de dominio, colunas Json para estrutura em maturacao e necessidade de auditoria.

Objetivo: Propor um diagrama de classes UML que cubra todas as entidades e seja legivel em formato A4, recorrendo a particionamento por subsistema funcional.

Instrucoes de Escrita:

- Apresentar atributos essenciais com tipos UML genericos e multiplicidades em cada associacao
- Identificar enumeracoes de dominio e os pontos onde sao referenciadas
- Distinguir composicao de associacao simples consoante o ciclo de vida do filho
- Agrupar as classes em N sub-diagramas por area funcional em vez de um diagrama monolitico
- Justificar decisoes de modelacao nao triviais (chave primaria, Decimal vs Float, colunas Json, soft-delete, autoincrement, auditoria)

Restricoes: Sem detalhes de implementacao SQL. Sem omissao de multiplicidades.

Análise Crítica

A produção dos diagramas de classes constituiu uma das tarefas em que o modelo se mostrou menos competente desde o início. As primeiras tentativas resultaram em diagramas mal estruturados, com elementos sobrepostos, multiplicidades omissas ou erradas e legibilidade muito reduzida em formato A4. A reformulação do *prompt* para impor a partição por subsistema melhorou parcialmente o resultado, mas continuou aquém da qualidade de apresentação exigida por um relatório académico.

Para contornar esta limitação, a equipa recorreu a uma *Agent Skill* instalada localmente no *Claude*. As *Agent Skills* são módulos de extensão introduzidos pela *Anthropic*, materializados em diretórios que contêm um ficheiro *SKILL.md* e recursos auxiliares (instruções dedicadas, exemplos, *scripts* e *templates*), que o modelo carrega dinamicamente sempre que reconhece que a tarefa em curso corresponde ao seu domínio de aplicação. No presente caso, foi instalada uma *Skill* especializada em geração de diagramas *TikZ*, o que dotou o *Claude* de instruções específicas, padrões de referência e capacidade de validação do código gerado. Trata-se de um mecanismo recente no ecossistema *Claude*, que ilustra de forma concreta como o desempenho de um LLM em tarefas técnicas particulares pode ser significativamente elevado através da disponibilização de contexto adequado.

A partir desta configuração, o modelo identificou três agrupamentos coerentes (núcleo operacional, aprovisionamento e inventário, faturação, configuração e notificações) e produziu três diagramas autocontidos em que apenas duas associações cruzam fronteiras de subsistema, designadamente *InterventionPart*, que liga o inventário ao núcleo operacional, e *Invoice*, que liga a faturação ao mesmo núcleo. Esta partição foi adotada pela equipa sem revisão, por revelar uma decomposição funcional que não havia sido tornada explícita durante a eliciação de requisitos. O LLM antecipou ainda seis decisões de modelação não triviais, todas discutidas na Subsecção 2.2.6, designadamente o uso de *Decimal* de precisão fixa para valores monetários em alternativa a *Float*, o recurso a colunas *Json* para diagnósticos e orçamentos com estrutura ainda em maturação, a manutenção de identificador técnico opaco mesmo quando existe uma chave natural utilizável, o *soft-delete* via campo *isArchived* em entidades de catálogo, o autoincremento separado para números de negócio e o padrão de auditoria por entidade dedicada.

2.2.1 Estratégia de Apresentação e Notação

O modelo estrutural é apresentado em quatro figuras complementares, designadamente um diagrama de pacotes que expõe a partição funcional do sistema e as dependências entre subsistemas, seguido de três diagramas de classes, um por subsistema. Esta divisão resulta da dimensão do modelo, dado que, com dezasseis entidades distribuídas por três áreas funcionais distintas, um diagrama único comprometeria a legibilidade sem acrescentar informação.

Nos diagramas de classes, os tipos dos atributos adotam nomes UML genéricos (String, Integer, Decimal, DateTime, Boolean, Json). A correspondência com os tipos concretos da camada de persistência é discutida na Subsecção 2.2.6. Por economia visual, os atributos de auditoria `createdAt` e `updatedAt`, presentes em quase todas as entidades, são omitidos dos diagramas, bem como os índices secundários. As enumerações de domínio são referenciadas pelo nome nos atributos das classes, com a sua expansão completa na Tabela 2.4.

Tabela 2.4: Enumerações de domínio definidas no sistema

Enumeração	Valores admissíveis	Entidade(s) que a referenciam
UserRole	manager, mechanic	User
CustomerType	personal, business	Customer
TimerState	idle, running, paused, stopped	Intervention
PurchaseOrderStatus	requested, received	PurchaseOrder
PaymentStatus	pending, paid, overdue	Invoice
NotificationType	reception_confirmation, budget_request, repair_completed, awaiting_parts_delay	Notification
StockMovementType	in, out, adjustment	StockMovement

2.2.2 Vista Geral: Diagrama de Pacotes

A partição do modelo estrutural em três pacotes resulta da aplicação de um princípio de coesão funcional, em que cada pacote reúne as entidades que participam conjuntamente nos mesmos casos de uso do domínio, minimizando as relações que atravessam fronteiras de pacote. A Figura 2.2 apresenta esta partição e as dependências estruturais que subsistem entre pacotes.

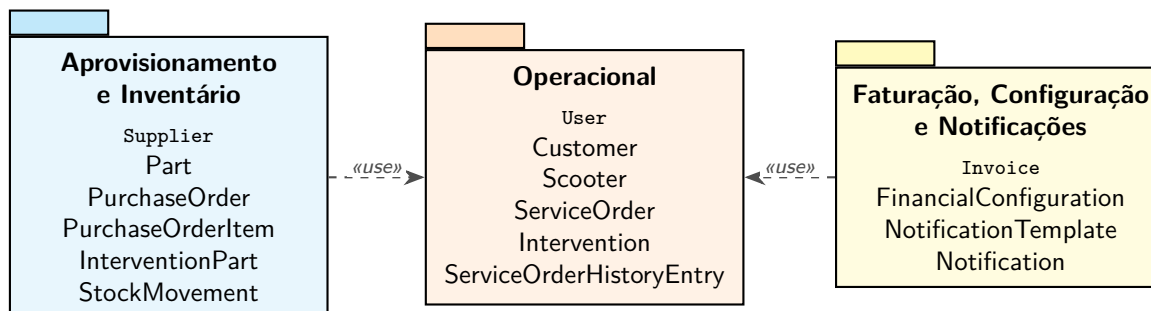


Figura 2.2: Vista geral do modelo estrutural: diagrama de pacotes UML.

As setas tracejadas indicam dependências unidirecionais. O pacote *Aprovisionamento e Inventário* depende do pacote *Operacional* através da ligação *InterventionPart*→*Intervention*. O pacote *Faturação, Configuração e Notificações* depende do mesmo através da ligação *Invoice*→*ServiceOrder*.

O pacote **Operacional** concentra o núcleo do negócio da oficina, agregando o utilizador interno do sistema (*User*), o cliente (*Customer*), a trotinete confiada à oficina (*Scooter*), a ordem de serviço (*ServiceOrder*) que concretiza o ciclo de reparação e as suas entidades subordinadas (*Intervention*, *ServiceOrderHistoryEntry*). Neste pacote residem as entidades cujo ciclo de vida é conduzido diretamente pelos funcionários da oficina no quotidiano operacional.

O pacote **Aprovisionamento e Inventário** abrange o relacionamento com fornecedores externos (*Supplier*), o catálogo de peças (*Part*) e os artefactos que medeiam o seu fluxo no sistema, designadamente ordens de compra e respetivos itens (*PurchaseOrder*, *PurchaseOrderItem*), movimentos de *stock* (*StockMovement*) e a entidade associativa que regista o consumo de peças em intervenções (*InterventionPart*). Esta última entidade materializa a única dependência estrutural para o pacote *Operacional*, refletindo o facto de o catálogo ser referenciado pela execução de trabalho técnico sem que o inverso se verifique.

O pacote **Faturação, Configuração e Notificações** reúne o documento fiscal (*Invoice*) que decorre da entrega de uma ordem de serviço concluída, os parâmetros financeiros transversais (*FinancialConfiguration*), os moldes de mensagens parametrizáveis (*NotificationTemplate*) e as instâncias concretas de notificações emitidas (*Notification*). A dependência para o pacote *Operacional*, mediada pela associação *Invoice*→*ServiceOrder*, garante a rastreabilidade entre o documento fiscal e o serviço prestado.

A unidirecionalidade das duas dependências é uma propriedade estrutural relevante, dado que o pacote *Operacional* não conhece nem referencia os pacotes de inventário ou de faturação, posicionando-se como núcleo independente do modelo. Esta propriedade suporta a evolutibilidade do sistema, permitindo que cada subsistema periférico seja refinado de forma isolada sem impacto estrutural sobre o núcleo, e alinha o modelo de dados com o padrão arquitetural do monólito modular descrito na Subsecção 2.1.1.

2.2.3 Subsistema Operacional: Clientes, Trotinetes e Ordens de Serviço

O subsistema operacional concentra o núcleo do negócio da oficina. Agrupa as entidades que representam o cliente, as trotinetes a este associadas, as ordens de serviço abertas sobre cada trotinete, as intervenções técnicas que as compõem e o registo histórico das transições de estado de cada ordem. A Figura 2.3 apresenta o respetivo diagrama de classes.

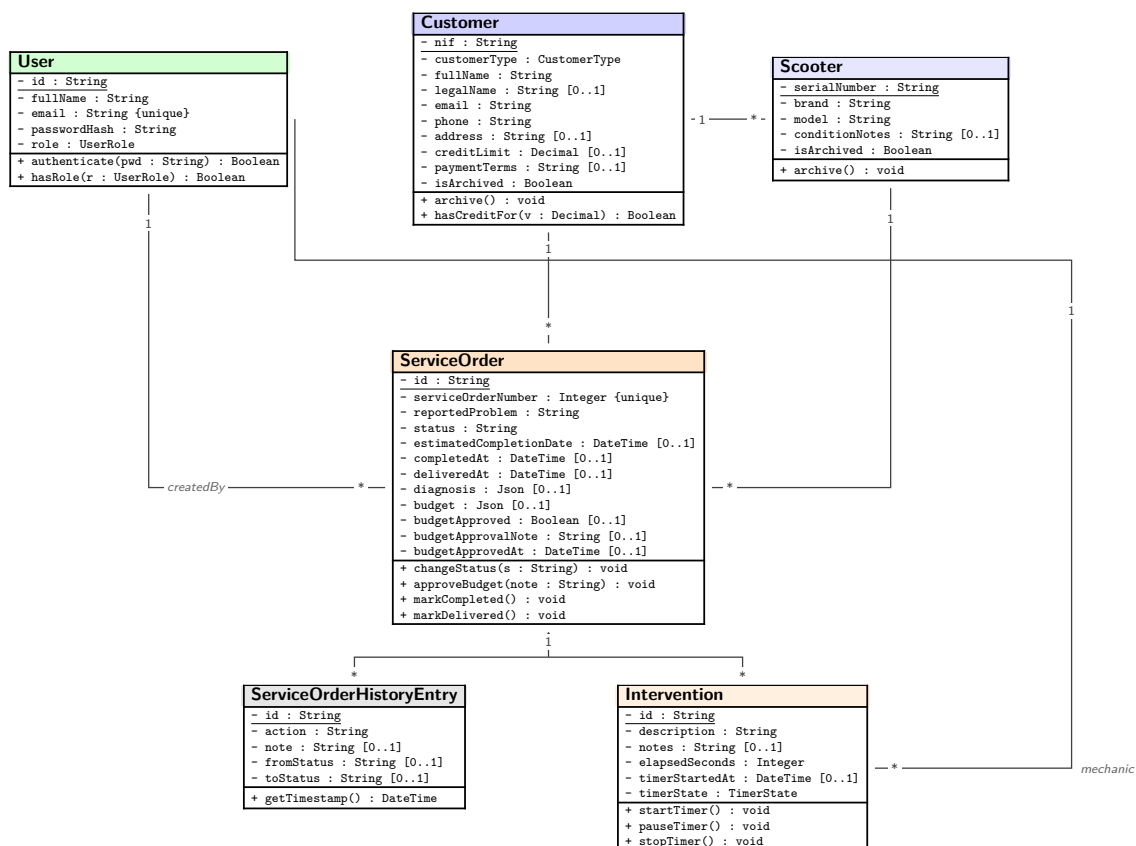


Figura 2.3: Diagrama de classes do subsistema operacional.

A entidade *Customer* adota o NIF como chave primária natural, identificador de uso quotidiano tornado obrigatório pela legislação fiscal portuguesa em todos os documentos emitidos pelo sistema. A justificação desta opção, incluindo a análise das suas implicações em termos de integridade referencial e conformidade com o RGPD, consta da Subsecção 2.2.6. A associação entre *Customer* e *Scooter*, com multiplicidades 1 e * nos respetivos extremos, exprime o facto de cada cliente poder confiar à oficina múltiplas trotinetes, sendo o número de série da trotinete (`serialNumber`) adotado como chave natural pela mesma lógica. A entidade *ServiceOrder* exhibe, por sua vez, um identificador duplo, sendo composto por um identificador técnico opaco (`id`), útil para integrações com sistemas externos, e por um número de ordem de serviço

(`serviceOrderNumber`) gerado por sequência autoincremental, exposto ao utilizador em interfaces e documentos. Esta separação preserva a legibilidade dos identificadores apresentados ao operador humano sem comprometer a estabilidade de chaves primárias.

A entidade `ServiceOrder` compõe duas entidades subordinadas. A entidade `Intervention` representa o trabalho técnico efetivo executado por um mecânico, possuindo um cronómetro persistente (`elapsedSeconds`, `timerStartedAt`, `timerState`) que sustém os requisitos funcionais de medição de tempo identificados no Capítulo 1, suportando os estados `idle`, `running`, `paused` e `stopped`. A entidade `ServiceOrderHistoryEntry`, por sua vez, materializa um registo de eventos interno por ordem, documentando transições de estado e ações relevantes (criação, validação de orçamento, conclusão, entrega) para fins de auditoria, suporte a litígios e análise operacional. Em ambos os casos, a remoção de uma ordem de serviço implica a remoção dos seus descendentes, razão pela qual se adota a semântica de composição em vez de associação simples.

A entidade `User` participa no subsistema operacional em dois pontos de associação distintos, o que poderia sugerir uma hierarquia de generalização com `Manager` e `Mechanic` como subclasses especializadas. A equipa optou, contudo, por um modelo de papel único com discriminador, em que o atributo `role : UserRole` distingue os dois perfis, sendo as diferenças comportamentais entre eles inteiramente capturáveis por restrições de autorização ao nível da aplicação, sem que existam atributos ou operações estruturalmente distintos que justifiquem a introdução de subclasses. A associação `createdBy` em `ServiceOrder` regista o utilizador responsável pela abertura de cada ordem de serviço. A associação `mechanic` em `Intervention` identifica o técnico que realizou cada intervenção. Os campos `diagnosis` e `budget` da entidade `ServiceOrder` são tipados como `Json`, dado que a estrutura semântica de ambos (uma lista de itens diagnosticados e uma lista de linhas orçamentadas com peças, mão-de-obra e observações) se encontra ainda em maturação na fase de *design*, não justificando a introdução prematura de novas entidades relacionais. A discussão completa desta decisão consta da Subsecção 2.2.6.

2.2.4 Subsistema de Aprovisionamento e Inventário

O subsistema de aprovisionamento concentra a relação com fornecedores externos, o catálogo de peças, o ciclo de encomendas de reposição e a auditoria do movimento de *stock*. A Figura 2.4 apresenta as seis classes envolvidas e a sua interconexão. A única ponte para fora deste subsistema, materializada pela entidade associativa *InterventionPart*, reflecte o consumo de peças no contexto operacional discutido na Subsecção 2.2.3.

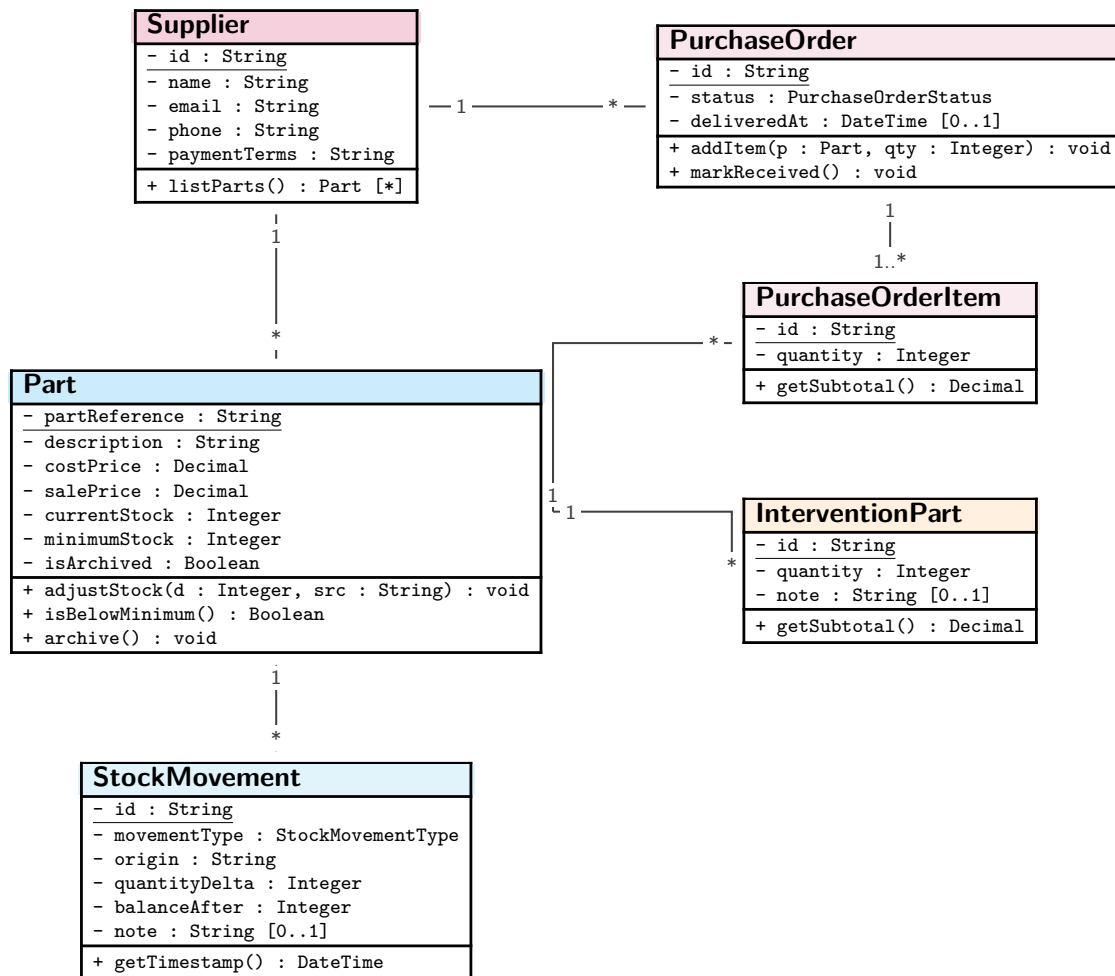


Figura 2.4: Diagrama de classes do subsistema de aprovisionamento e inventário.

InterventionPart constitui a única ligação inter-subsistemas, materializando o consumo de peças por uma intervenção no subsistema operacional.

A entidade *Supplier* agrega o catálogo de peças por si fornecidas e o histórico de ordens de compra (*PurchaseOrder*) emitidas. Cada *PurchaseOrder* compõe um conjunto de *PurchaseOrderItem*, sendo cada item uma quantidade específica de uma *Part* encomen-

dada. A relação é modelada como composição estrita, uma vez que um item de encomenda não possui semântica fora do documento em que se inscreve. O ciclo de vida da encomenda é governado pela enumeração `PurchaseOrderStatus`, com os estados `requested` e `received`. Esta granularidade resulta diretamente dos requisitos funcionais levantados no Capítulo 1, nos quais o fluxo de reaprovisionamento contempla apenas a emissão e a receção de encomendas, sem estados intermédios.

A entidade `Part` adota a referência interna do fornecedor (`partReference`) como chave primária natural, opção que reflete o uso quotidiano destes códigos pelos mecânicos no momento de identificar o material e que evita a multiplicação de identificadores artificiais em fluxos físicos onde o código já é etiquetado. Os preços de custo e venda são tipados como `Decimal` de precisão fixa, de modo a garantir a precisão monetária exigida pela emissão de faturas. A justificação técnica desta opção é discutida na Subsecção 2.2.6. Os atributos `currentStock` e `minimumStock` sustentam o disparo automático de alertas e o cálculo de necessidades de reposição, requisitos identificados no SRS.

A entidade `StockMovement` materializa um princípio de *contabilidade de stock por movimento*: em vez de o saldo ser representado apenas como um atributo mutável da `Part`, cada alteração ao saldo é registada como um movimento atómico tipificado pela enumeração `StockMovementType` (`in`, `out`, `adjustment`), com proveniência (`origin`) e saldo resultante (`balanceAfter`). Esta opção, embora redundante face ao atributo `currentStock`, sustenta a auditoria histórica de inventário, permite a reconstituição do saldo em qualquer instante e protege contra perdas de informação em situações de correção retroativa. Configura um padrão equivalente ao *event sourcing* parcial, restrito ao agregado `Part`.

2.2.5 Subsistema de Faturação, Configuração e Notificações

O terceiro subsistema reúne a faturação, decorrente da entrega de uma ordem de serviço concluída, e os mecanismos de configuração financeira e de notificação que a sustentam. A Figura 2.5 apresenta as quatro classes envolvidas.

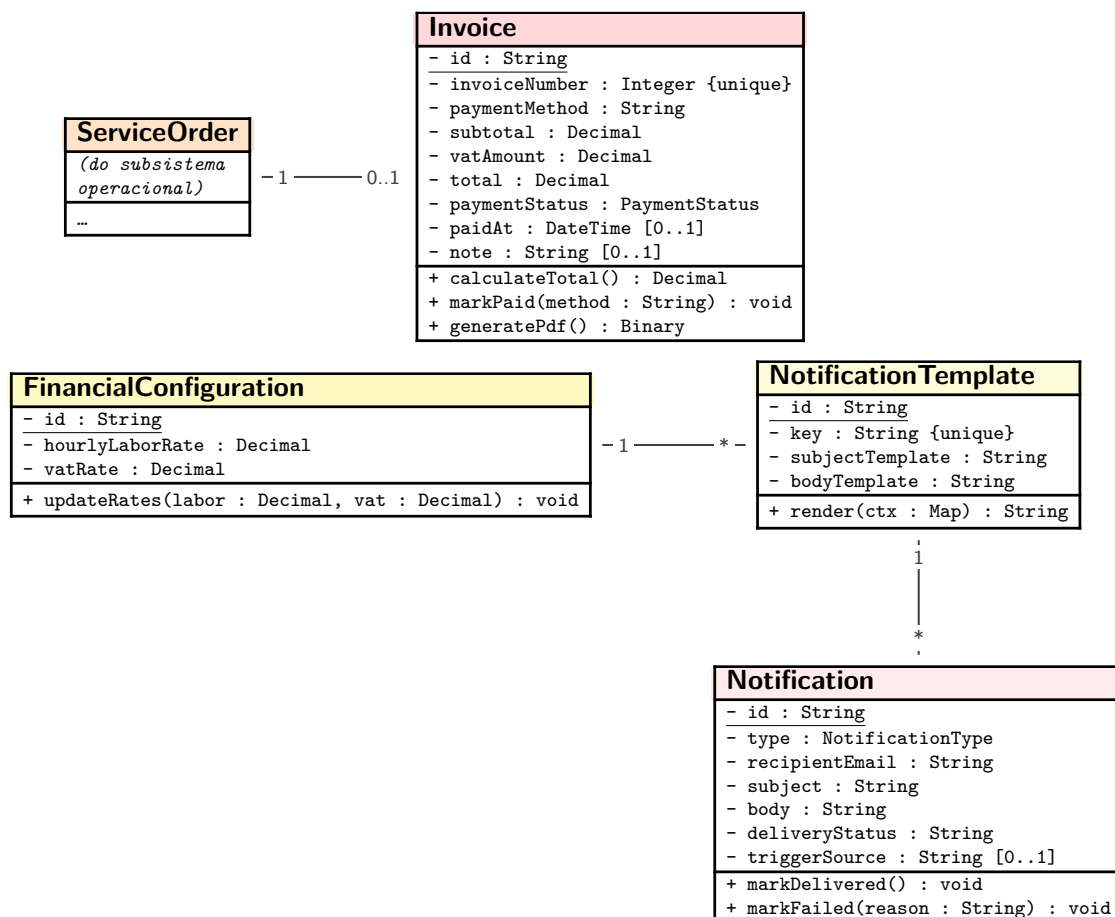


Figura 2.5: Diagrama de classes do subsistema de faturação, configuração financeira e notificações.

A entidade *ServiceOrder* surge representada a tracejado por ser externa a este subsistema, pertencendo ao pacote *Operacional* e sendo aqui referenciada pela *Invoice*.

A entidade *Invoice* mantém com a entidade *ServiceOrder* uma associação de multiplicidade 1 no extremo da ordem e 0..1 no extremo da fatura, em que cada ordem de serviço entregue origina, no máximo, uma fatura e cada fatura pertence necessariamente a uma ordem. A unicidade da chave estrangeira *serviceOrderId* no modelo garante esta restrição de forma estrutural. À semelhança da *ServiceOrder*, a *Invoice* possui número de negócio próprio (*invoiceNumber*) gerado por sequência autoincremental sem lacunas, em linha com o requi-

sito de identificação contínua de faturas levantado no Capítulo 1 e com o regime português de faturação eletrónica (Decreto-Lei n.º 28/2019). Os três atributos monetários (`subtotal`, `vatAmount`, `total`) são tipados como `Decimal` de precisão fixa em resposta ao requisito de exatidão nos cálculos fiscais, dado que o uso de tipos em ponto flutuante introduziria erros de representação decimal cumulativos, incompatíveis com a correção exigida nos documentos fiscais emitidos.

A entidade `FinancialConfiguration` corresponde a uma entidade de configuração singular (sob a convenção de existir um único registo com identificador conhecido) que captura os parâmetros financeiros transversais ao sistema, designadamente a taxa horária de mão-de-obra aplicada nas intervenções e a taxa de IVA vigente. A extração destes parâmetros para entidade dedicada, em alternativa à sua fixação em constantes de código, sustenta o requisito de modificabilidade pelo gestor sem necessidade de nova implantação do sistema. A `FinancialConfiguration` agrega adicionalmente o conjunto de `NotificationTemplate` que parametrizam os assuntos e corpos das mensagens enviadas em cada situação tipificada do ciclo de vida das ordens de serviço.

A entidade `Notification` representa cada *instância* de notificação efetivamente emitida ou agendada, mantendo o estado de entrega (`deliveryStatus`) e a fonte que a desencadeou (`triggerSource`). A persistência das notificações já enviadas, em alternativa à prática de *fire-and-forget*, sustenta a reconstituição do canal de comunicação com o cliente, instrumento valioso em situações de litígio ou reclamação.

2.2.6 Decisões Estruturais Transversais de Modelação

Para além das observações específicas a cada subsistema, importa documentar seis decisões estruturais de modelação cuja influência atravessa todo o modelo e que sustentam atributos de qualidade não funcionais do sistema.

Tipos monetários em `Decimal` de precisão fixa. A equipa decidiu tipar todos os valores monetários (preços de custo e venda, subtotais, IVA, totais, taxa horária, limite de crédito) como `Decimal` de precisão fixa, nunca em ponto flutuante. Esta opção assegura que somas de longo prazo e cálculos fiscais respeitam a aritmética decimal exata, condição imposta pela legislação de faturação portuguesa e pelas boas práticas de contabilidade computacional.

Colunas `Json` para estrutura emergente. Os atributos `ServiceOrder.diagnosis` e `ServiceOrder.budget` foram tipados como `Json`. Ambas as estruturas encontram-se em maturação na fase de *design*, uma vez que o diagnóstico técnico de uma trotinete admite múltiplos itens com tipologia ainda em estabilização e o orçamento agrega linhas de mão-de-obra e peças com formatos suscetíveis de evolução. A opção por `Json` preserva a flexibilidade de iteração sem o custo de migrações sucessivas, ao preço de abdicar da validação relacional nativa,

transferindo a responsabilidade de validação para os esquemas *Zod* do pacote de contratos partilhados (Subsecção 2.1.4).

Identificadores duplos: técnico e de negócio. As entidades *ServiceOrder* e *Invoice* expõem dois identificadores: um identificador técnico opaco para uso interno e em integrações, e um inteiro autoincremental (*serviceOrderNumber*, *invoiceNumber*) para uso humano e fiscal. A separação preserva a legibilidade dos identificadores apresentados sem comprometer a estabilidade das chaves primárias utilizadas por integrações programáticas.

Chaves naturais quando semanticamente fortes. Nas entidades em que existe uma chave natural única, estável e de uso quotidiano (NIF para *Customer*, número de série para *Scooter*, referência interna para *Part*), essa chave foi adotada como chave primária, mitigando a indireção desnecessária em consultas frequentes pelos operadores e simplificando a integração com etiquetagem física e documentos externos.

O NIF merece justificação adicional. Trata-se de um identificador externo cuja mutabilidade está formalmente fora do controlo do sistema, o que representa um risco residual de atualização em cascata. A equipa reconhece que a alternativa canónica, um identificador técnico opaco com índice único sobre o NIF, seria estruturalmente mais robusta. Contudo, a imutabilidade prática do NIF ao abrigo da legislação portuguesa (a sua alteração exige ato administrativo junto da Autoridade Tributária), a sua obrigatoriedade em todos os documentos emitidos (Decreto-Lei n.º 28/2019) e a exceção ao direito ao apagamento prevista no RGPD para dados retidos por obrigação legal (art. 17.º, n.º 3, al. b)) tornam esta opção suficientemente viável para a dimensão e horizonte deste projeto.

Soft-delete via *isArchived* em entidades de catálogo. As entidades *Customer*, *Scooter* e *Part* suportam desativação lógica através do atributo booleano *isArchived*, em alternativa à eliminação física. A opção é imposta pela necessidade de preservar a integridade referencial em ordens de serviço e faturas históricas e, no caso dos clientes, pelo período de retenção exigido pelo RGPD para registos fiscais. Os fluxos de listagem aplicarão, por convenção, um filtro implícito sobre este atributo.

Auditoria de eventos via entidade dedicada. O ciclo de vida da ordem de serviço é auditado por uma entidade autónoma, *ServiceOrderHistoryEntry*, que regista cada transição de estado e ação relevante com utilizador responsável, instante e nota opcional. Esta decisão centraliza a auditoria, evita a poluição da entidade-pai com colunas de última-modificação e habilita futuras análises temporais (tempo médio entre receção e conclusão, por exemplo) sem alteração do modelo. Um padrão análogo poderá ser estendido a outras entidades de ciclo de vida complexo, como *Invoice*, em iterações subsequentes.

A Tabela 2.5 resume a rastreabilidade prevista entre as entidades do modelo estrutural e os módulos do *backend* responsáveis pela sua manipulação, fechando o circuito entre a vista estrutural aqui apresentada e a vista de implementação a desenvolver em capítulos posteriores.

Tabela 2.5: Rastreabilidade entre entidades e módulos do *backend*

Entidade	Módulo do <i>backend</i>	Subsistema
User	<i>auth</i>	Operacional
Customer	<i>customers</i>	Operacional
Scooter	<i>scooters</i>	Operacional
ServiceOrder, ServiceOrderHistoryEntry	<i>service-orders</i>	Operacional
Intervention, InterventionPart	<i>interventions</i>	Operacional / Inventário
Part, StockMovement	<i>inventory</i>	Inventário
Supplier, PurchaseOrder, PurchaseOrderItem	<i>suppliers-procurement</i>	Inventário
Invoice	<i>billing</i>	Faturação
FinancialConfiguration, NotificationTemplate	<i>configuration</i>	Configuração
Notification	<i>notifications</i>	Notificações

A correspondência próxima de um-para-um entre entidades e módulos não resulta de coincidência, tendo sido adotada pela equipa como princípio de organização e contribuindo para a coesão interna de cada módulo, condição estrutural para o encapsulamento de responsabilidades discutido na Subsecção 2.1.1.

2.3 Modelo Comportamental: Fluxos Críticos do Sistema

Análise Crítica do Processo de Geração com IA

Prompt 24. Modelo Comportamental: Diagramas de Sequência

Contexto: SRS das Oficinas Gengis Khan com 44 RF, 5 RNF e 29 casos de uso, modelo estrutural de classes já consolidado e necessidade de evidenciar regras de negocio nao triviais.

Objetivo: Identificar os fluxos comportamentais que justificam documentacao por diagramas de sequencia e descrever as interacoes entre os participantes (cliente, controlador HTTP, servico de dominio, repositorio).

Instrucoes de Escrita:

- Privilegiar fluxos representativos com invariantes não triviais sobre cobertura exaustiva
- Evidenciar pontos de validação cruzada entre módulos
- Marcar transições de estado com pré-condições
- Identificar operações que devem executar atômicamente sob a mesma transação

Restrições: Sem operações de leitura simples nem variantes paramétricas do mesmo endpoint.

Análise Crítica

A geração dos diagramas de sequência apresentou inicialmente o mesmo tipo de dificuldade observado nos diagramas de classes, designadamente a inadequação do *output* bruto do modelo a uma renderização legível em formato académico. A solução passou, novamente, pelo recurso a uma *skill* dedicada do *Claude*, desta vez especificamente concebida para diagramas de sequência, que produziu artefactos imediatamente utilizáveis sem necessidade de retoques manuais significativos. A reutilização desta família de *skills* confirma uma observação que se foi consolidando ao longo da etapa, designadamente que a delegação de subtarefas a ferramentas especializadas tende a oferecer resultados substancialmente superiores aos do modelo genérico.

Quanto ao conteúdo dos diagramas, a equipa restringiu a seleção inicialmente alargada do modelo, que contemplava também operações de listagem e leitura por identificador, aos quatro fluxos cujo desenho envolve regras de negócio efetivamente verificáveis, designadamente a autenticação com fixação do papel no *token*, a criação de ordem de serviço com validação cascata cliente/trotinete, a transição de estado da ordem sob máquina de estados com restrição por papel, e a emissão de fatura com orquestração multi-módulo. A contribuição mais útil do LLM foi a identificação explícita das fronteiras transacionais a observar em cada fluxo, que se refletem nos diagramas apresentados e serão observadas pela camada de implementação. A sugestão de incluir um quinto fluxo de notificação assíncrona foi recusada, dado que o canal de *email* é acessório, com falha tolerada, e o seu detalhe não acrescenta regras adicionais ao quadro comportamental do sistema.

A vista estrutural definiu as entidades e as suas relações em estado de repouso. A presente secção documenta os fluxos comportamentais cuja correção depende da ordem das interações em tempo de execução, recorrendo a diagramas de sequência. A seleção privilegia fluxos com invariantes não triviais, designadamente pontos de validação cruzada entre módulos, transições de estado com pré-condições e fluxos de orquestração em que a ordem das operações é semanticamente significativa. Os fluxos elementares de leitura, mais lineares, são tratados na especificação detalhada da API, apresentada mais à frente neste capítulo.

2.3.1 Participantes Recorrentes nos Diagramas

A correspondência entre os participantes recorrentes e os respetivos artefactos do código fica fixada na Tabela 2.6, com vista a evitar ambiguidade de leitura e a facilitar a rastreabilidade até aos módulos identificados na Secção 2.2.

Tabela 2.6: Participantes recorrentes nos diagramas de sequência

Participante	Descrição
<i>Cliente web</i>	Aplicação <i>frontend</i> (<i>Next.js</i>) executada no <i>browser</i> do utilizador autenticado, atuando como ator iniciador dos fluxos.
<i>Controlador</i>	<i>Handler</i> HTTP do <i>backend Fastify</i> ; responsabiliza-se pela autenticação por <i>JWT</i> , pela validação de <i>schema</i> via <i>Zod</i> e pelo mapeamento de erros aplicacionais para códigos HTTP.
<i>Serviço</i>	Camada de regras de negócio do módulo correspondente; concentra a lógica de domínio e orquestra repositórios.
<i>Repositório</i>	Camada de acesso a dados sobre <i>Prisma</i> ; encapsula a totalidade das operações sobre <i>PostgreSQL</i> , incluindo eventuais transações multi-tabela.
<i>Mailer</i>	Adaptador de correio eletrónico assente em SMTP, encapsulado em <i>shared/email/mailer</i> , invocado de forma síncrona com falha tolerada.

2.3.2 Autenticação e Estabelecimento de Sessão

A autenticação por credenciais condiciona toda a interação posterior com o sistema, motivo pelo qual o seu desenho exige opções específicas. A verificação da palavra-passe é mantida estritamente na camada de serviço, ao passo que a sessão é devolvida através de um *token* transportado em *cookie* HTTP, formato que reduz a sua exposição ao cliente. Ambas as opções respondem ao requisito de *login* individual e de controlo de acesso por papel levantado no Capítulo 1, sendo a sequência completa apresentada na Figura 2.6.

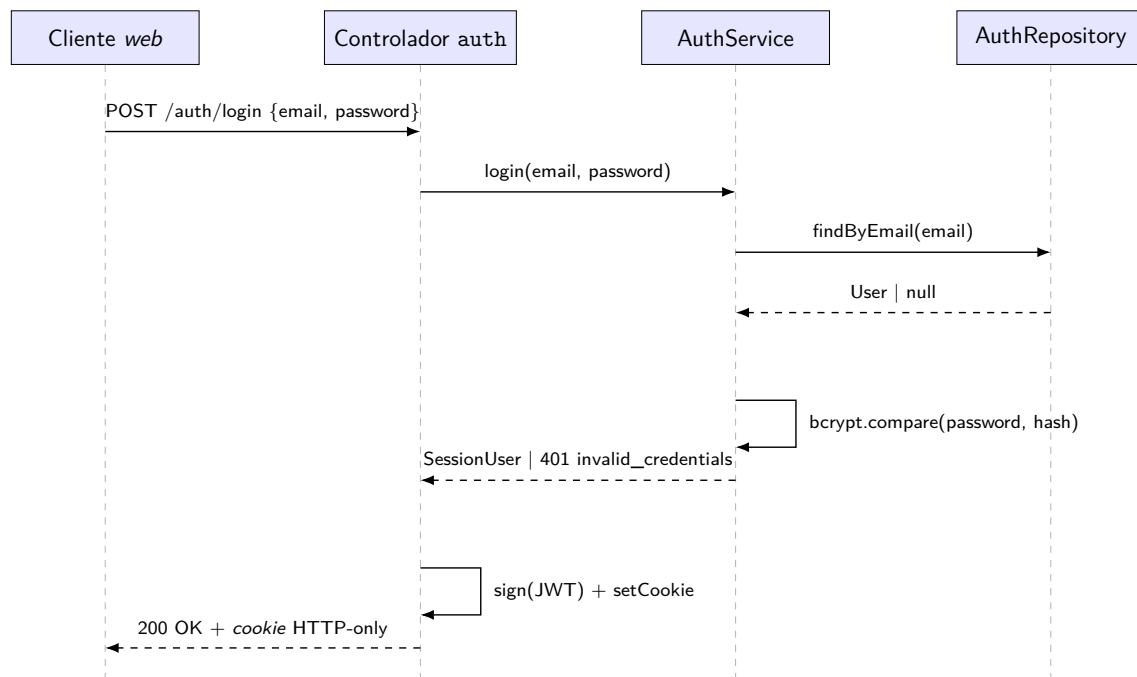


Figura 2.6: Diagrama de sequência da autenticação (POST /auth/login)

A condição de erro *invalid_credentials* resulta tanto da inexistência de utilizador como da divergência de *hash*, e o desenho preserva, propositadamente, a indistinguibilidade externa dos dois cenários, mitigando a enumeração de contas. Note-se que o papel do utilizador (*manager* ou *mechanic*) é fixado no momento do estabelecimento da sessão e fica incorporado no *token*. As decisões posteriores de autorização baseiam-se nessa atribuição estável, sem incorrer em consulta adicional à base de dados.

2.3.3 Receção de Trotinete e Criação de Ordem de Serviço

A receção de uma trotinete em oficina materializa o requisito funcional de entrada identificado no Capítulo 1. A criação da ordem de serviço é deliberadamente desenhada em duas validações sequenciais e independentes, em vez de uma consulta combinada. Verifica-se primeiro a existência do cliente pelo NIF e depois a pertença da trotinete a esse cliente. Esta separação permite distinguir, ao nível do código de erro devolvido, o cliente desconhecido do cliente conhecido com trotinete não associada, e protege contra a entrada accidental de trotinetes pertencentes a outro proprietário. A sequência completa consta da Figura 2.7.

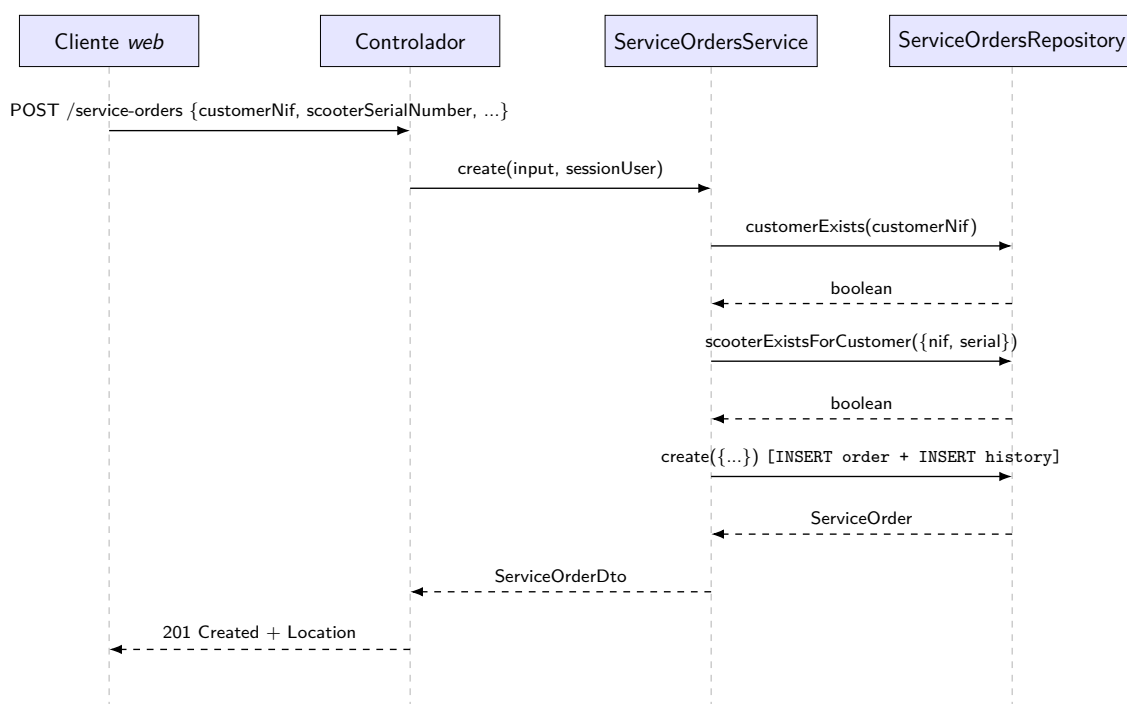


Figura 2.7: Diagrama de sequência da criação de ordem de serviço (POST /service-orders)

A persistência da ordem de serviço e do primeiro registo histórico (ServiceOrderHistoryEntry com ação created) é executada numa única transação *Prisma*, com as garantias ACID herdadas do motor *PostgreSQL* subjacente. A atomicidade assegura que o ciclo de vida documentado na entidade dedicada de auditoria (Subsecção 2.2.6) começa, sem exceção, no instante da criação da ordem, evitando ordens órfãs no histórico ou histórico órfão de ordens.

2.3.4 Transição de Estado da Ordem de Serviço

A transição de estado da ordem de serviço é a única em que o sistema cruza duas regras de natureza distinta, designadamente a máquina de estados, que limita as transições admissíveis a partir do estado corrente, e a autorização, que reserva ao papel manager a passagem terminal a delivered. Ambas são verificadas localmente pela camada de serviço, sem recurso à base de dados, em coerência com o princípio de que regras de domínio puras não devem depender de I/O. A sequência consta da Figura 2.8 e o conjunto exaustivo de transições admissíveis da Figura 2.10.

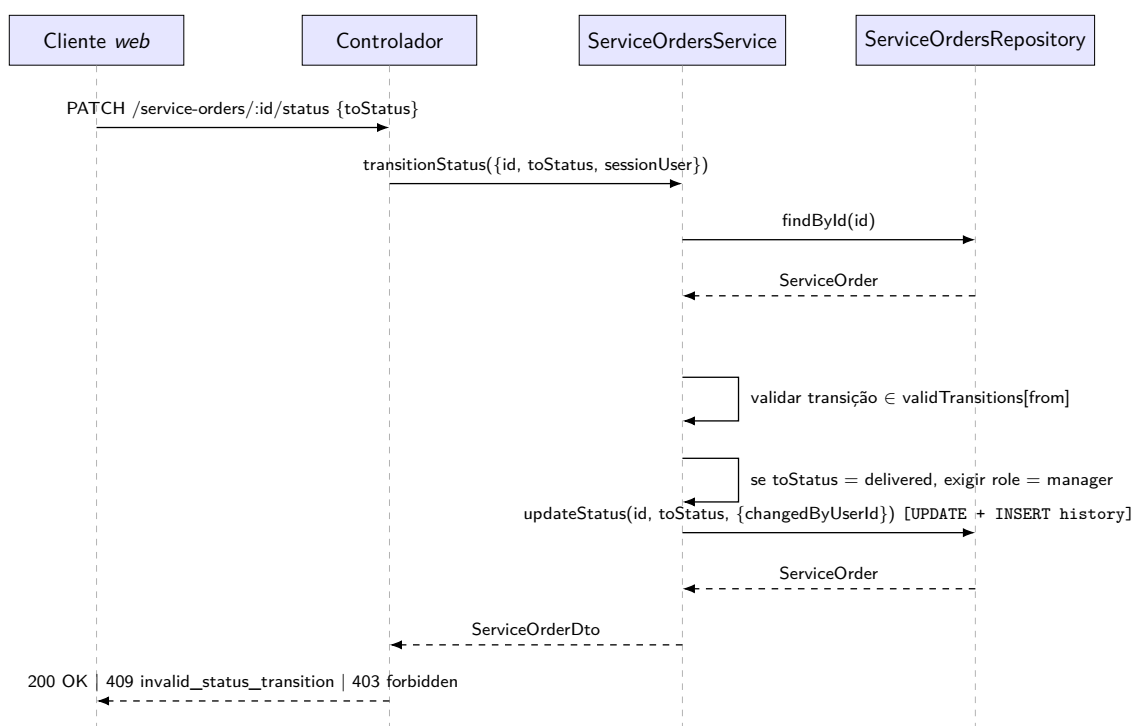


Figura 2.8: Diagrama de sequência da transição de estado (PATCH /service-orders/:id/status)

Seguindo o mesmo padrão atômico aplicado à criação da ordem (Subsecção 2.3.3), a transição é persistida em conjunto com a entrada histórica correspondente, garantindo correspondência um-para-um entre transições efetivas e entradas de auditoria.

2.3.5 Emissão de Fatura sobre Ordem de Serviço

A emissão de fatura combina, numa única operação atômica, dados provenientes das ordens de serviço, das intervenções, das peças aplicadas e da configuração financeira global. Trata-se do fluxo de orquestração mais denso do sistema e do único em que um erro silencioso teria implicações fiscais. A sequência é apresentada na Figura 2.9, com análise dos pontos críticos do desenho a seguir-se.

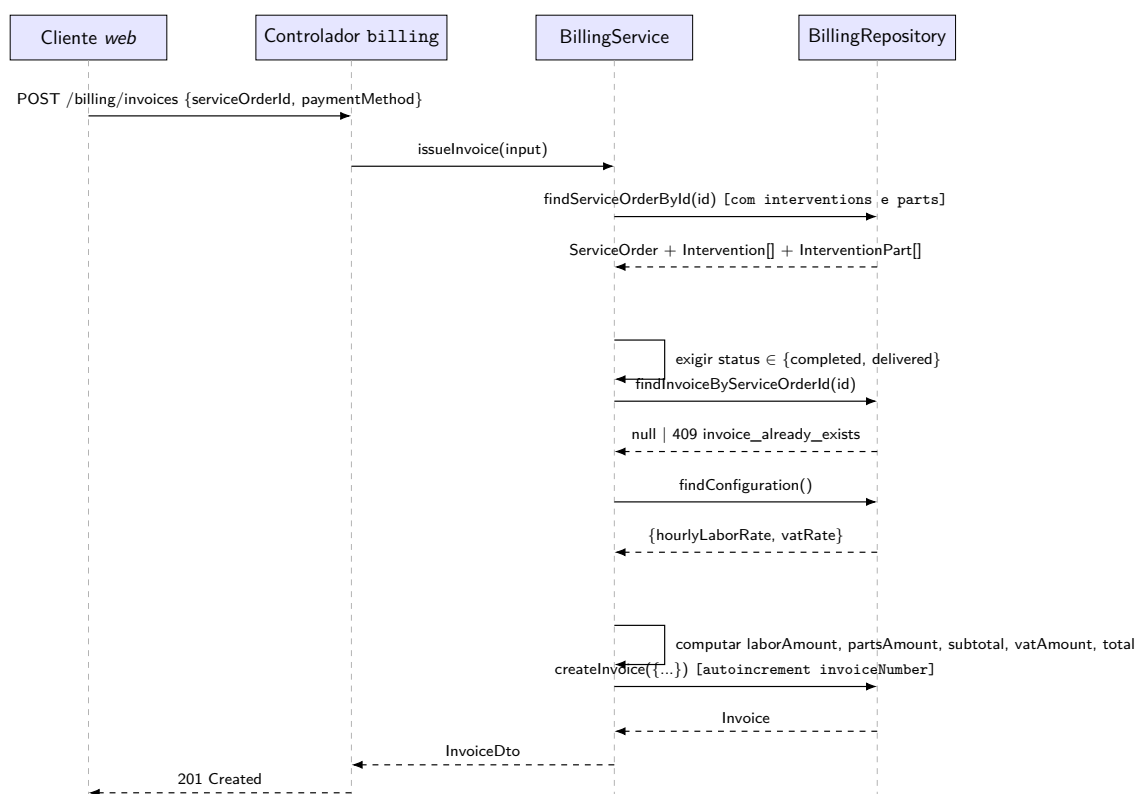


Figura 2.9: Diagrama de sequência da emissão de fatura (POST /billing/invoices)

Três aspetos do desenho merecem destaque. Em primeiro lugar, o serviço carrega a totalidade do grafo de relações da ordem numa única consulta, evitando o problema de N+1 típico das abordagens iterativas e amortizando o custo do cálculo. Em segundo lugar, a unicidade da fatura por ordem de serviço é assegurada pela conjugação de duas defesas em profundidade. A verificação aplicacional (`findInvoiceByServiceOrderId`) oferece um erro de domínio claro ao operador, ao passo que a restrição relacional `UNIQUE` sobre o atributo `serviceOrderId` de `Invoice` impede a duplicação efetiva mesmo perante condições de corrida concorrentes. Por fim, a sequência autoincremental sem lacunas atribuída a `invoiceNumber` fica sob responsabilidade do *PostgreSQL*, alinhada com a decisão de identificação contínua de faturas discutida na Subsecção 2.2.5.

2.3.6 Síntese e Articulação com as Vistas Subsequentes

Os fluxos documentados nesta secção não esgotam a totalidade das interações suportadas pelo sistema, mas cobrem as decisões comportamentais mais delicadas, designadamente o estabelecimento da sessão e a gestão do papel autenticado, a entrada controlada no domínio operacional, a evolução da ordem de serviço sob uma máquina de estados verificada e a fronteira de conversão entre operação técnica e ato fiscal. Em qualquer um destes fluxos, o controlador permanece estritamente delgado, restringindo-se à mediação transversal de pedidos e respostas, e os repositórios permanecem livres de lógica de domínio, executando apenas as operações de persistência necessárias para satisfazer as instruções emitidas pelos serviços. A camada de serviço concentra, por desenho, a totalidade das regras de negócio verificáveis em tempo de execução.

A vista de sequência aqui apresentada complementa-se naturalmente com uma vista ortogonal, a evolução temporal das entidades cujo ciclo de vida é não trivial, formalizada em diagramas de máquina de estados na secção que se segue.

2.4 Modelo Comportamental: Máquinas de Estado

Análise Crítica do Processo de Geração com IA

Prompt 25. Modelação de Máquinas de Estado

Contexto: SRS das Oficinas Gengis Khan, com REQ-011 a REQ-014 (estados da ordem de servico, sete estados fechados, registo historico e restricao de papel no estado terminal), REQ-018 (cronometro de intervencao com pausa e retoma) e REQ-032 (estado de pagamento da fatura).

Objetivo: Formalizar as maquinas de estado das tres entidades de ciclo de vida nao trivial (ordem de servico, cronometro de intervencao, estado de pagamento da fatura).

Instrucoes de Escrita:

- Indicar estado inicial, conjunto exaustivo de estados, transicoes admitidas, condicoes de despoletamento e estados terminais
- Distinguir transicoes accionadas por humano das despoletadas pelo sistema
- Explicitar restricoes de papel sempre que se apliquem

Restricoes: Sem maquinas para entidades de ciclo de vida trivial.
Sem modelacao do orcamento como entidade autonoma.

Análise Crítica

À semelhança dos diagramas anteriores, a geração das máquinas de estado beneficiou de imediato das *skills* do *Claude* já instaladas, dispensando a iteração corretiva inicial e produzindo os artefactos no formato pretendido logo na primeira tentativa. Quanto ao conteúdo, a primeira proposta omitiu a transição inversa de *Em Reparação* para *Aguarda Peças*, que decorre da prática efetiva da oficina e que foi acrescentada após reformulação do *prompt*. A contribuição mais relevante do modelo foi a sugestão de explicitar a guarda de papel (*role = manager*) como condição da transição para *Entregue ao Cliente*, opção adotada pela equipa por aproximar a representação visual da semântica efetiva do código. O LLM propôs ainda uma máquina de estados para o orçamento, recusada pela equipa por este ser tratado como atributo flexível da ordem de serviço (campo *Json*, conforme discutido na Subsecção 2.2.6).

Três entidades do modelo possuem ciclo de vida não trivial e justificam tratamento explícito. A ordem de serviço, cujo estado condiciona toda a operação da oficina. O cronómetro associado a cada intervenção, cuja contagem de tempo trabalhado alimenta o cálculo do orçamento e da fatura. O estado de pagamento da fatura, cujas transições são parcialmente despoletadas pelo decurso do tempo. As três máquinas seguintes formalizam, para cada entidade, o conjunto fechado de estados admissíveis e as transições que os ligam.

2.4.1 Máquina de Estados da Ordem de Serviço

A Figura 2.10 apresenta a máquina de estados completa da entidade `ServiceOrder`. O conjunto de sete estados deriva diretamente do REQ-012 e foi validado durante a elicitação de requisitos. O estado inicial `Recebida` é atribuído automaticamente no instante da criação da ordem (REQ-011) e o estado `Entregue ao Cliente` é terminal, sendo a sua atribuição reservada ao perfil `manager` (REQ-014).

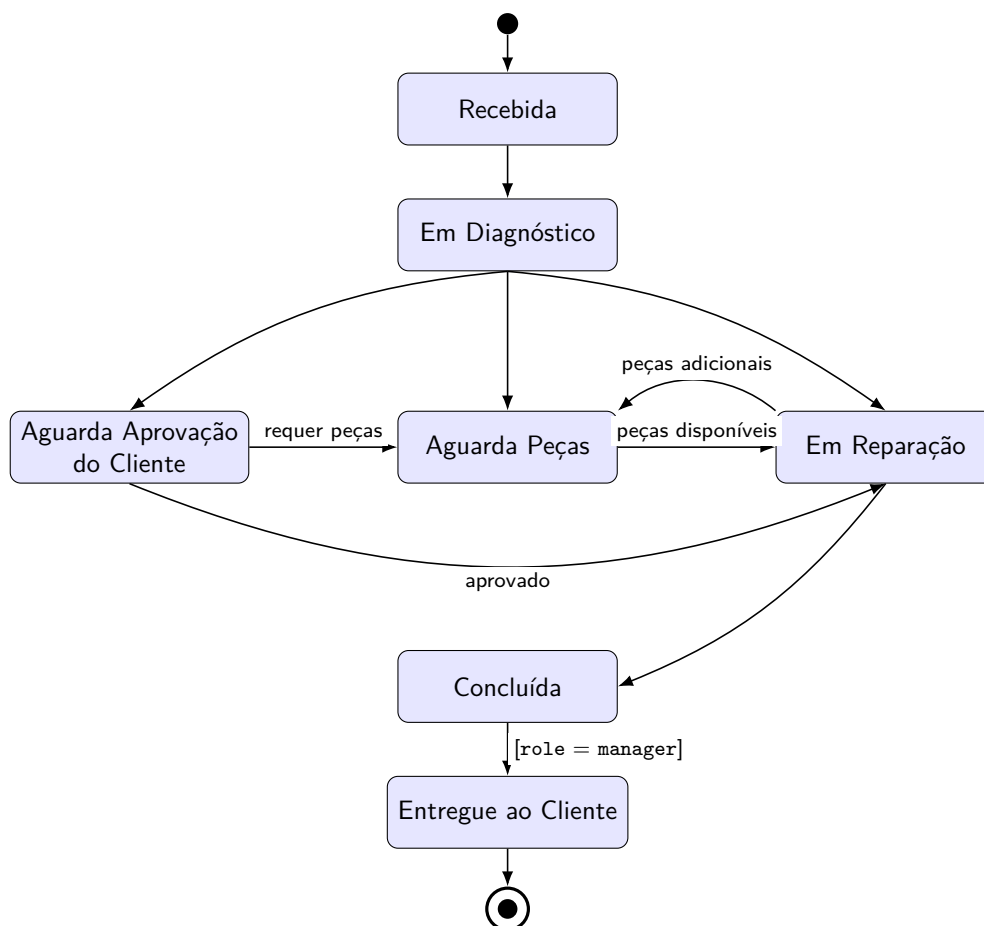


Figura 2.10: Máquina de estados da entidade `ServiceOrder`

A leitura do diagrama torna explícitas três propriedades estruturais. Em primeiro lugar, todas as transições são despoletadas por ação humana (mecânico ou gerente), com exceção do registo automático de data e hora que acompanha cada transição (REQ-013) e que fica sob a responsabilidade da camada de persistência. Em segundo lugar, o estado `Aguarda Peças` é o único nó de convergência múltipla, podendo ser atingido a partir de `Em Diagnóstico`, de `Aguarda Aprovação do Cliente` ou de `Em Reparação`, refletindo a prática efetiva da oficina, em que a detecção de peças em falta pode ocorrer em qualquer momento posterior à receção. Por fim, o ramo terminal `Concluída` para `Entregue ao Cliente` está protegido por uma guarda de

papel, sendo a transição recusada pela camada de serviço se o utilizador autenticado não tiver o perfil *manager*, comportamento que se materializa no fluxo da Figura 2.8.

2.4.2 Máquina de Estados do Cronómetro de Intervenção

O REQ-018 estabelece a obrigatoriedade de um cronómetro com pausa e retoma para cada intervenção, instrumento essencial para a faturação rigorosa do tempo de mão de obra. A Figura 2.11 formaliza os quatro estados do cronómetro: *idle* (estado inicial, ainda nunca iniciado), *running* (a contar tempo), *paused* (suspensão) e *stopped* (estado terminal, contagem encerrada).

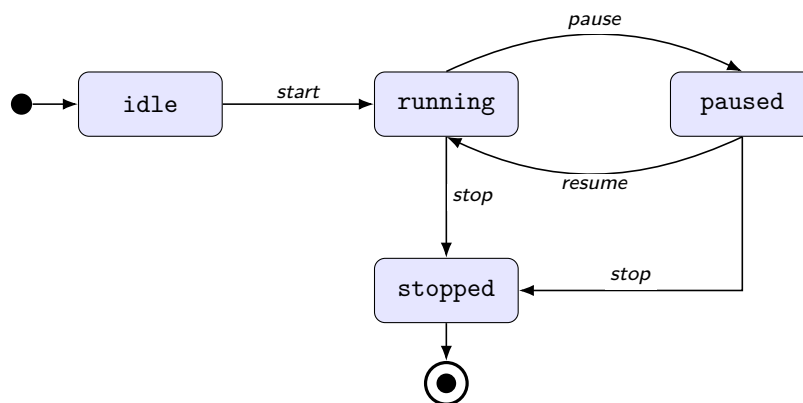


Figura 2.11: Máquina de estados do cronómetro de intervenção

As transições *idle* para *running* e *paused* para *running* despoletam o registo do instante de arranque no atributo *timerStartedAt*. A transição *running* para *paused* acumula no atributo *elapsedSeconds* o intervalo decorrido desde o último arranque. A transição para *stopped* encerra a contagem definitivamente, não admitindo regresso. O desenho impede transições semanticamente inválidas, designadamente o reinício de um cronómetro já parado, que comprometeria a auditabilidade da medição do tempo de mão de obra.

2.4.3 Máquina de Estados do Pagamento da Fatura

A Figura 2.12 apresenta a máquina de estados do atributo `Invoice.paymentStatus`, a mais simples das três. Distingue-se, contudo, por contemplar uma transição despoletada pelo sistema e não pela ação humana, particularidade que justifica a sua inclusão.

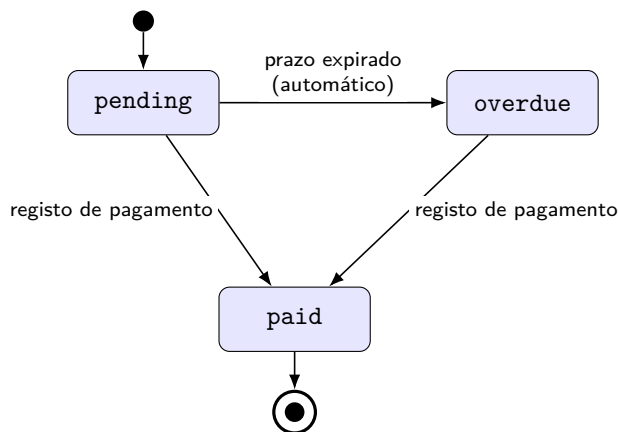


Figura 2.12: Máquina de estados de `Invoice.paymentStatus`

A transição `pending` para `overdue` ocorre automaticamente quando o prazo de pagamento expira sem confirmação, comportamento associado ao REQ-033, que requer a consulta rápida de faturas pendentes e atrasadas, em especial as de clientes empresariais. O registo do pagamento é, em qualquer dos casos, ato manual do gerente, despoletando a transição para `paid`. O modelo não admite reversibilidade do pagamento, dado que uma fatura paga não regressa ao estado `pending`, opção alinhada com o princípio de imutabilidade dos documentos fiscais emitidos.

2.4.4 Síntese

As três máquinas de estado documentadas configuram o quadro temporal do sistema, com a ordem de serviço como espinha dorsal do percurso operacional, o cronómetro como mecanismo de medição que alimenta o cálculo financeiro e o estado de pagamento como ponto de fecho do ciclo administrativo. Em todas, o conjunto de estados é fechado e o conjunto de transições é finito e enumerável, propriedade que sustenta tanto a auditabilidade do sistema como a previsibilidade da operação. As guardas explícitas (perfil `manager` para a entrega da ordem, transições semanticamente inválidas rejeitadas no cronómetro, irreversibilidade do pagamento) constituem regras de domínio que serão verificadas em tempo de execução pela camada de serviço, conforme o princípio enunciado na Secção 2.3.

A vista comportamental fica, com isto, completa. Articula-se naturalmente com a especificação detalhada da API, apresentada na secção que se segue, que materializa em contrato HTTP os fluxos aqui descritos.

2.5 Especificação da API

A especificação da interface programática exposta pelo *backend* antecede o detalhe do *design* de interface, apresentado na secção que se segue, em conformidade com [1, 6] e com a norma ISO/IEC/IEEE 29148:2018 [3], segundo os quais toda a interface programática deve ser fixada antes do detalhe dos seus consumidores.

Análise Crítica do Processo de Geração com IA

Prompt 26. Especificação da API REST

Contexto: Módulos do backend consolidados (auth, customers, scooters, service-orders, interventions, inventory, suppliers-procurement, billing, notifications, configuration, reports) e requisitos funcionais REQ-001 a REQ-047 do SRS.

Objetivo: Especificar o contrato HTTP da API REST exposta pelo backend, identificando endpoints e convenções transversais.

Instruções de Escrita:

- Para cada endpoint indicar método HTTP, caminho, objectivo funcional, papéis autorizados (manager, mechanic) e principais códigos de resposta
- Fixar convenções transversais (versionamento, formato de pedido e resposta, autenticação, autorização, validação de schema, formato uniforme de erro)
- Privilegiar fidelidade ao contrato implementado

Restrições: Sem extensões futuras hipotéticas. Sem detalhes de payload por endpoint.

Análise Crítica

O LLM produziu inicialmente um catálogo derivado apenas dos requisitos do SRS, omitindo recursos auxiliares posteriormente decididos em sede de *design*: geração de PDF da fatura, controlo granular do cronómetro de intervenção, geração automática de encomendas a partir de peças com *stock* mínimo e o módulo de relatórios. A equipa completou o catálogo a partir do código já implementado, garantindo que a especificação reflete fielmente o contrato exposto. A contribuição mais relevante do modelo foi a sugestão de uniformizar a forma de erro através de um envelope JSON com código simbólico legível (*validation_error*, *not_found*, *invalid_status_transition*, *forbidden*) em vez de mensagens livres em texto, opção adotada por permitir o tratamento estruturado de erros no *frontend* sem dependência sobre a literalidade da mensagem. O modelo recomendou ainda o versionamento por prefixo de caminho (*/api/v1/*) em alternativa ao versionamento por cabeçalho *Accept*, opção mantida pela equipa por ser mais legível em *logs* e em depuração manual.

2.5.1 Convenções Transversais

A API observa um conjunto de convenções uniformes, fixadas no nível do *framework*. Todos os caminhos de recurso são prefixados por `/api/v1/`, isolando alterações incompatíveis numa eventual versão futura sem rutura sobre clientes existentes. Os pedidos com corpo e todas as respostas utilizam JSON com codificação UTF-8, sendo os parâmetros de leitura passados via *query string*. Antes da execução da regra de negócio, o corpo de cada pedido é validado por esquemas *Zod* do pacote `@gengis-khan/contracts` (Subsecção 2.1.4). Em caso de incumprimento, a resposta é 400 `validation_error` com a primeira mensagem de erro identificada.

A autenticação é estabelecida pelo *endpoint* POST `/api/v1/auth/login`, que emite um *cookie* HTTP-only com *JWT* de sessão (Subsecção 2.3.2). Os restantes *endpoints* requerem este *cookie* válido e devolvem 401 `unauthorized` na sua ausência. A autorização é gerida por papel através da função utilitária `requireRole`, invocada no início de cada *handler*. Os *endpoints* de faturação, configuração financeira e relatórios são exclusivos do papel `manager`, em conformidade com REQ-037, ao passo que os restantes admitem ambos os papéis, em conformidade com REQ-038.

As respostas de erro adotam um envelope JSON uniforme com três campos (`statusCode` numérico HTTP, `code` simbólico e `message` legível), cuja enumeração de códigos consta da Tabela 2.7.

Tabela 2.7: Códigos de erro simbólicos da API

HTTP	Código simbólico	Significado
400	<code>validation_error</code>	Corpo do pedido falhou validação por <i>schema Zod</i>
401	<code>unauthorized</code>	Sessão ausente, expirada ou inválida
403	<code>forbidden</code>	Papel autenticado não autorizado para o recurso
404	<code>not_found</code>	Recurso identificado pelo caminho não existe
409	<code>invalid_status_transition</code>	Transição de estado da ordem de serviço inadmissível
409	<code>invalid_timer_transition</code>	Transição de estado do cronómetro inadmissível
409	<code>invoice_already_exists</code>	Tentativa de duplicar fatura sobre a mesma ordem
409	<code>service_order_not_billable</code>	Ordem ainda não atingiu o estado faturável
409	<code>invalid_stock_adjustment</code>	Ajuste resultaria em <i>stock</i> negativo

2.5.2 Catálogo de *Endpoints*

As Tabelas 2.8 a 2.11 apresentam, para cada um dos quatro agrupamentos funcionais correspondentes aos subsistemas identificados na Secção 2.2, um excerto representativo com três *endpoints* ilustrativos do âmbito coberto. O catálogo exaustivo, com a totalidade dos cerca de setenta *endpoints* expostos, encontra-se reproduzido no Anexo 5.4. Os caminhos surgem sem o prefixo `/api/v1/` por economia de espaço, ainda que este se aplique sem exceção. A coluna *Papéis* adota M para manager, T para mechanic e *aut.* para qualquer utilizador autenticado.

Tabela 2.8: Excerto do catálogo de *endpoints* do subsistema operacional

Método	Caminho	Objetivo	Papéis
POST	<code>/auth/login</code>	Estabelece sessão e emite <i>cookie</i>	público
POST	<code>/service-orders</code>	Cria nova ordem de serviço	M, T
PATCH	<code>/service-orders/:id/status</code>	Transição de estado da ordem	M, T
<i>(...) Catálogo completo no Anexo 5.4</i>			

Tabela 2.9: Excerto do catálogo de *endpoints* do subsistema de aprovisionamento e inventário

Método	Caminho	Objetivo	Papéis
GET	<code>/parts/low-stock</code>	Lista peças abaixo do <i>stock</i> mínimo	M, T
POST	<code>/parts/:partReference/stock-adjustment</code>	Regista ajuste manual de <i>stock</i>	M, T
POST	<code>/purchase-orders/generate-from-low-stock</code>	Cria encomendas a partir de peças com <i>stock</i> mínimo	M, T
<i>(...) Catálogo completo no Anexo 5.4</i>			

Tabela 2.10: Excerto do catálogo de *endpoints* do subsistema de faturação, notificações e configuração

Método	Caminho	Objetivo	Papéis
POST	<code>/invoices/issue</code>	Emite fatura sobre ordem de serviço concluída	M
POST	<code>/notifications/send</code>	Envia notificação ao cliente por <i>email</i>	M, T
PATCH	<code>/config/financial-parameters</code>	Atualiza taxa horária de mão de obra e taxa de IVA	M
<i>(...) Catálogo completo no Anexo 5.4</i>			

Tabela 2.11: Excerto do catálogo de *endpoints* de relatório transversal

Método	Caminho	Objetivo	Papéis
GET	/reports/operations	Indicadores operacionais agregados	M
GET	/reports/billing	Indicadores de faturação por intervalo	M
GET	/reports/mechanic-productivity	Indicadores de produtividade por mecânico	M
(…) Catálogo completo no Anexo 5.4			

2.5.3 Exemplos de Pedidos e Respostas

Os três exemplos seguintes ilustram o formato dos pedidos e respostas da API, cobrindo um caso de criação válida, uma falha de validação por esquema e uma violação de regra de domínio. Em todos, o corpo do pedido e o corpo da resposta são JSON em UTF-8 e a sessão autenticada é transportada no *cookie session*.

Criação de ordem de serviço. Um pedido válido a `POST /api/v1/service-orders` envia o NIF do cliente, o número de série da trotinete, o problema reportado e a data estimada de conclusão. A resposta de sucesso devolve o DTO da ordem criada com identificador técnico, número de negócio sequencial e estado inicial `received`.

```

POST /api/v1/service-orders          (request)
{
  "customerNif": "123456789",
  "scooterSerialNumber": "SN-KUG-0013",
  "reportedProblem": "Pneu traseiro furado",
  "estimatedCompletionDate": "2026-05-15"
}

201 Created                          (response)
{
  "id": "cmor9o6nh003ziyp9s3brayui",
  "serviceOrderNumber": 53,
  "customerNif": "123456789",
  "scooterSerialNumber": "SN-KUG-0013",
  "reportedProblem": "Pneu traseiro furado",
  "status": "received",
  "estimatedCompletionDate": "2026-05-15"
}

```

Falha de validação por esquema Zod. Um pedido a `POST /api/v1/customers` com NIF não conforme falha a validação no *handler* antes de atingir a camada de serviço. A resposta segue o envelope uniforme de erro com código simbólico `validation_error`.

```
POST /api/v1/customers (request)
{
  "nif": "ABC123",
  "fullName": "Diogo Carvalho",
  "email": "diogo.carvalho@email.pt"
}

400 Bad Request (response)
{
  "statusCode": 400,
  "code": "validation_error",
  "message": "nif: must contain exactly nine digits"
}
```

Transição de estado inválida. Uma tentativa de passar uma ordem de received diretamente para completed viola a máquina de estados documentada na Subsecção 2.4.1. A resposta é 409 `invalid_status_transition`, código que o *frontend* traduz para mensagem ao operador.

```
PATCH /api/v1/service-orders/{id}/status (request)
{
  "toStatus": "completed"
}

409 Conflict (response)
{
  "statusCode": 409,
  "code": "invalid_status_transition",
  "message": "Cannot transition from received to completed"
}
```

2.5.4 Síntese

A API expõe um total de cerca de setenta *endpoints* distribuídos por onze módulos, com o sub-sistema operacional a concentrar a maior parte do contrato (autenticação, clientes, trotinetes, ordens de serviço e intervenções) e os *endpoints* de faturação, configuração e relatório a constituírem o núcleo restrito ao papel manager. A uniformidade de convenções (versionamento, autenticação, autorização, validação por *schema* e formato de erro) reduz a carga cognitiva sobre o consumidor da API e habilita a geração de cliente tipado a partir do mesmo pacote de contratos partilhados, conforme discutido na Subsecção 2.1.4. O contrato aqui especificado constitui o limite externo da vista lógica do sistema. A sua materialização em componentes físicos e processos de execução é apresentada na secção que se segue.

2.6 Modelo de Componentes

A vista de componentes formaliza a decomposição física do sistema em unidades de execução autónomas e os protocolos de comunicação que as ligam, fechando o conjunto de vistas internas iniciado pela vista estrutural (Secção 2.2) e prolongado pelas vistas comportamentais (Secções 2.3 e 2.4). Trata-se da vista que torna explícitos os processos existentes em tempo de execução, a sua localização física e as fronteiras de tecnologia que os separam, princípio fundamentado por [1] e [6].

Análise Crítica do Processo de Geração com IA

Prompt 27. Modelo de Componentes

Contexto: Stack consolidada para as Oficinas Gengis Khan, com Next .js no frontend, Fastify e Node.js 20 no backend, PostgreSQL 16 com Prisma, autenticação JWT em cookie HTTP-only, mailer SMTP e scheduler interno para jobs periodicos.

Objetivo: Produzir o modelo de componentes do sistema em UML 2.5, identificando processos autonomos em tempo de execucao, componentes externos e componentes internos do backend.

Instrucoes de Escrita:

- Distinguir componentes de aplicacao, de infraestrutura e externos com marcacao visual diferenciada
- Agrupar os componentes internos pelos subsistemas funcionais ja fixados (operacional, inventario, faturacao, configuracao, notificacoes, relatorios)
- Indicar o protocolo de comunicacao em cada conexao
- Evidenciar a fronteira de processo entre frontend e backend

Restricoes: Sem omissao do scheduler interno. Sem misturar vista logica com vista de componentes.

Análise Crítica

A esta altura da etapa de *design*, a *Agent Skill* de diagramas *TikZ* introduzida na Subsecção 2.2 continuava instalada, o que permitiu que a geração do presente diagrama de componentes decorresse de forma muito mais expedita do que teria sido possível sem essa configuração prévia. O modelo produziu, à primeira tentativa, uma decomposição substancialmente correta e visualmente cuidada, identificando três classes de componentes (aplicação, infraestrutura e externos), respeitando a fronteira de processo entre *frontend* e *backend* e organizando os módulos internos pelos subsistemas funcionais previamente fixados.

A intervenção da equipa limitou-se a acrescentar o componente de *scheduler* interno, responsável pela transição automática *pending* para *overdue* sobre as faturas (Secção 2.4), que tinha sido omitido pelo modelo apesar de constar do contexto fornecido no *prompt*. A contribuição mais relevante do modelo foi a sugestão de tornar visível, no diagrama, a fronteira de processo entre *frontend* e *backend*, dado que ambos correm em processos *Node.js* distintos com vidas independentes ainda que partilhem o mesmo *monorepo* ao nível do código-fonte. Esta visibilidade evita a interpretação errada, frequente em arquiteturas SPA, de que o *frontend* é apenas um conjunto de ficheiros estáticos servidos pelo *backend*. A marcação visual diferenciada dos componentes externos (fronteira tracejada) e dos componentes de infraestrutura (cor distinta dos componentes de aplicação) foi também adotada pela equipa, por aumentar a legibilidade do diagrama.

2.6.1 Vista Geral dos Componentes em Tempo de Execução

A Figura 2.13 apresenta a vista geral dos componentes do sistema em tempo de execução. Distinguem-se três classes de componentes. Os componentes de aplicação, integralmente desenvolvidos pela equipa (*Frontend SPA* e *Backend API*). Os componentes de infraestrutura, que suportam a operação sem implementarem o domínio (*PostgreSQL*). Os componentes externos, com os quais o sistema integra mas sobre os quais não tem controlo (servidor SMTP do operador de *email* e *browser* do utilizador final).

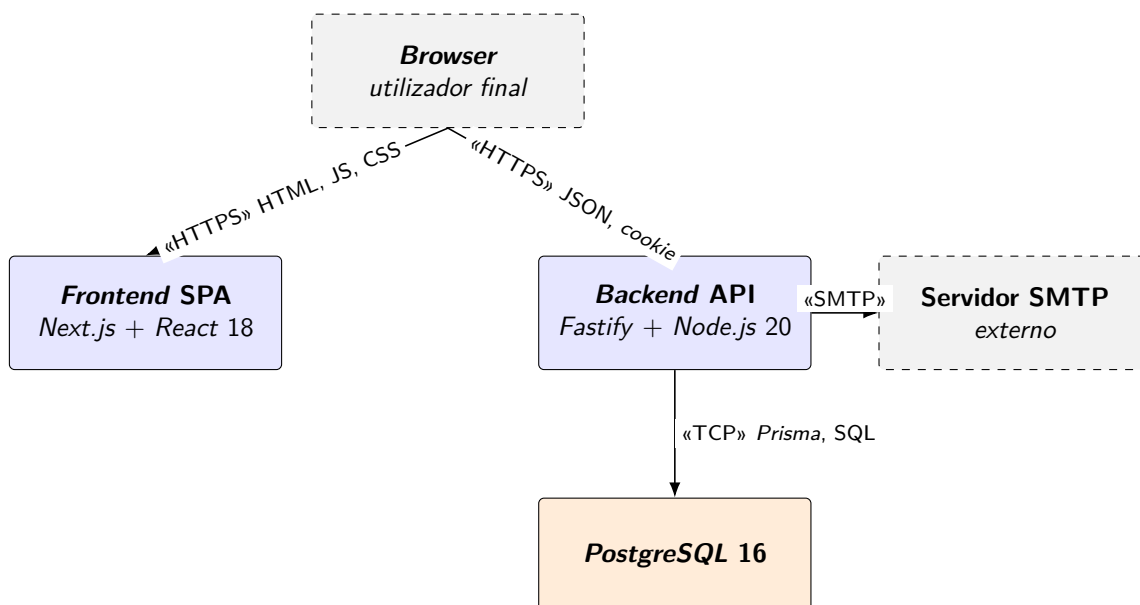


Figura 2.13: Vista geral dos componentes do sistema em tempo de execução

A comunicação entre componentes adota três protocolos distintos. Entre o *browser* e os dois processos de aplicação, HTTPS sobre TCP, com cargas em JSON UTF-8 conforme estipulado na Secção 2.5. Entre o *backend* e o *PostgreSQL*, comunicação TCP via *Prisma Client*, que traduz as operações do ORM em SQL nativo. Entre o *backend* e o servidor de *email* externo, SMTP sobre TCP, exclusivamente para o envio de notificações ao cliente. A fronteira de rede mais sensível, do ponto de vista de segurança, é a primeira, protegida por TLS, autenticação por *cookie* HTTP-only e o conjunto de regras de validação documentadas na Subsecção 2.5.1.

2.6.2 Decomposição Interna do *Backend*

A Figura 2.14 aprofunda a estrutura interna do *backend Fastify*, mostrando que o processo único exposto à rede agrupa três tipos de componentes funcionais. O *bootstrap* de arranque, responsável pelo registo de *plugins*, módulos e *jobs* agendados. Os onze módulos de domínio, agrupados pelos subsistemas funcionais já identificados. O conjunto de utilitários partilhados, que oferecem capacidades transversais a toda a aplicação.

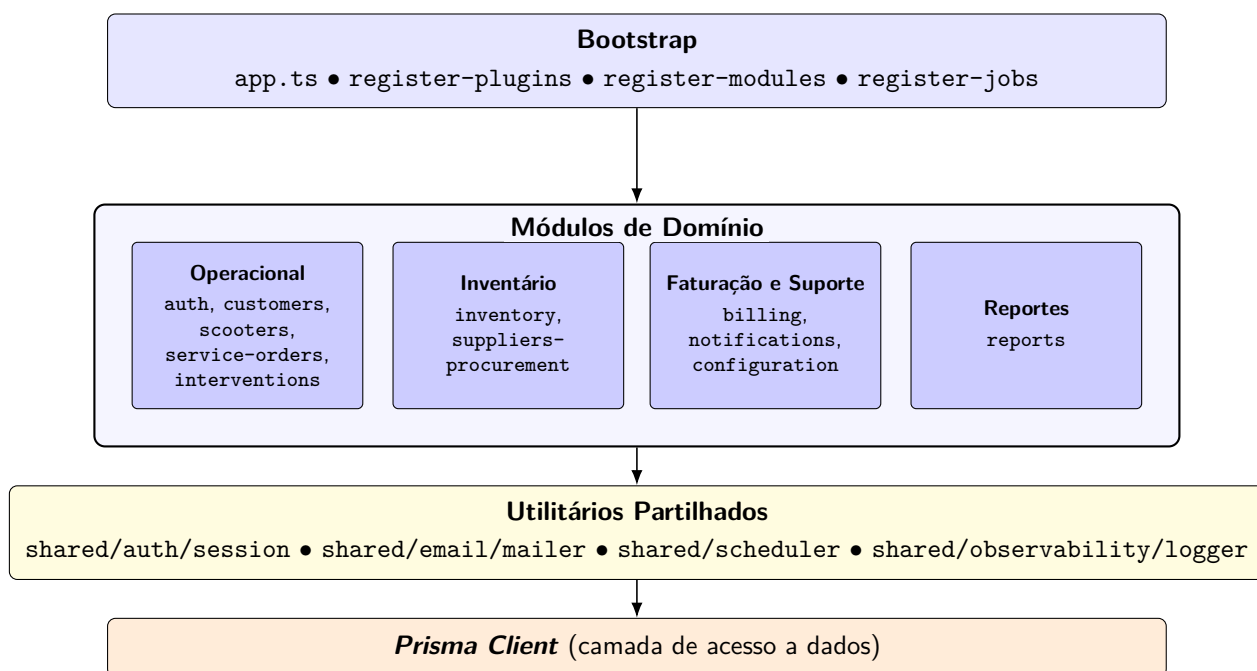


Figura 2.14: Decomposição interna do *backend Fastify*

O *bootstrap* concentra-se em `src/app.ts` e nos ficheiros `bootstrap/register-plugins.ts` e `bootstrap/register-modules.ts`, complementados por `shared/scheduler/register-jobs.ts`. A sua função é exclusivamente declarativa, enumerando os *plugins* a aplicar (autenticação por sessão, validação de papel, observabilidade, tratamento uniforme de erros), os módulos a expor e os *jobs* periódicos a agendar, sem conter lógica de domínio.

Os módulos de domínio seguem todos o mesmo padrão estrutural já discutido na Subsecção 2.1.1, sendo cada módulo composto por um ficheiro `index.ts` que regista os *handlers* HTTP, um ficheiro `*.service.ts` que concentra as regras de negócio e um ficheiro `*.repository.ts` que encapsula o acesso ao *Prisma*. Esta uniformidade reduz a carga cognitiva sobre quem navega o código e habilita a verificação cruzada de regras pela equipa.

Os utilitários partilhados oferecem três capacidades transversais relevantes, designadamente a autenticação e a verificação de papel (`shared/auth/session`), o adaptador para o servidor de *email* (`shared/email/mailer`) e o agendador de *jobs* periódicos (`shared/scheduler`). Este

último é o componente que despoleta a transição automática de *pending* para *overdue* sobre as faturas, materializando a única transição não desencadeada por ação humana documentada nas máquinas de estado da Secção 2.4.

2.6.3 Síntese

A vista de componentes confirma e consolida a separação física que sustenta o padrão de monólito modular adotado. O sistema é constituído por dois processos de aplicação distintos (*frontend* e *backend*), apoiados por um motor de base de dados relacional, e integra-se com um único componente externo (servidor SMTP) em modo de falha tolerada. A complexidade interna do *backend*, agrupada em onze módulos de domínio coesos sobre uma camada uniforme de utilitários partilhados, demonstra que a decomposição lógica documentada nas secções anteriores tem correspondência direta na organização do código e nos artefactos efetivamente executados.

A vista interna do sistema fica completa com esta secção. A próxima e última secção do capítulo, dedicada ao *design* de interface, atravessa a fronteira entre o sistema e o utilizador, documentando a forma como toda a complexidade aqui descrita é apresentada e tornada operável pelo gerente e pelos mecânicos da oficina.

2.7 Design de Interface

Esta última secção fixa a forma como toda a complexidade modelada nas vistas anteriores é apresentada e tornada operável pelo gerente e pelos mecânicos da oficina. Materializa o requisito de aplicação *web* acessível em ambiente de oficina e em *tablets* (RNF-01 e RNF-02). O *design* adotado segue um conjunto de heurísticas estabelecidas de usabilidade, designadamente as dez heurísticas de Nielsen [4], que orientaram as opções tomadas para cada ecrã e que serão chamadas ao longo da secção sempre que dêem suporte direto a uma decisão de *design*.

Análise Crítica do Processo de Geração com IA

Prompt 28. Design de Interface

Contexto: Sistema das Oficinas Gengis Khan especificado (44 RF, 5 RNF, 29 casos de uso, perfis manager e mechanic), API REST documentada e uso previsto em ambiente de oficina com tablets e iluminacao variavel.

Objetivo: Propor um design de interface coerente para a aplicacao frontend, fixando principios visuais, estrutura de navegacao e ecras principais para o fluxo operacional completo (recepcao, diagnostico, orcamento, reparacao, conclusao, faturacao).

Instruções de Escrita:

- Definir tema, paleta, tipografia e iconografia
- Para cada ecrã indicar função, elementos críticos e requisitos funcionais associados
- Aplicar o princípio de mostrar sempre o próximo passo legítimo da máquina de estados
- Privilegiar legibilidade em iluminação variável e áreas tácteis adequadas a tablet

Restrições: Sem tema claro padrão. Sem oferta de transições de estado inválidas.

Análise Crítica

A primeira proposta do LLM apresentou um *design* de tema claro, padrão na indústria de aplicações empresariais. Os requisitos consolidados na Etapa 1 não fixam exigências sobre o tema visual da aplicação, pelo que a equipa exerceu nesta matéria alguma liberdade interpretativa e optou, em alternativa à proposta inicial, por um tema escuro. A escolha sustenta-se na percepção de que o ambiente físico de uma oficina apresenta iluminação variável e que um fundo escuro tende a reduzir a fadiga ocular durante turnos prolongados. A contribuição mais útil do modelo foi o princípio de *próximo passo legítimo*, segundo o qual cada ecrã de detalhe de ordem de serviço deve apresentar, de forma proeminente, apenas as transições admissíveis a partir do estado corrente, calculadas pelo *frontend* a partir da máquina de estados documentada na Secção 2.4. Esta opção elimina a possibilidade de o utilizador tentar transições inválidas e reduz a carga cognitiva associada à interpretação do estado atual da ordem. O modelo recomendou ainda a uniformização da iconografia através do conjunto *Material Symbols*, opção adotada por reconhecibilidade universal e por estar disponível sob licença permissiva, sem custo de licenciamento.

A produção dos protótipos de interface propriamente ditos foi conduzida com recurso ao *Stitch AI*, ferramenta da *Google* dedicada à geração de *mock-ups* a partir de descrições textuais. O *Stitch AI* permitiu transpor rapidamente os princípios visuais e a estrutura de navegação acordados com o LLM para imagens concretas dos ecrãs principais, servindo de instrumento de validação visual antes da fase de implementação. Importa, contudo, registar que, se este trabalho fosse iniciado hoje, a equipa optaria pelo *Claude Design* em alternativa. Lançado pela *Anthropic* em abril de 2026 e assente no modelo *Claude Opus 4.7*, o *Claude Design* oferece funcionalidades comparáveis de geração de protótipos a partir de *prompts*, mas integra-se nativamente no ecossistema *Claude* já utilizado em todo o resto do projeto, designadamente na possibilidade de transitar diretamente do protótipo para a implementação através do *Claude Code*. Esta integração reduziria a fricção entre as fases de *design* e desenvolvimento e harmonizaria as ferramentas com a restante *stack* assistida por IA do projeto.

2.7.1 Princípios e Linguagem Visual

A linguagem visual adotada articula-se em torno de quatro princípios. O primeiro é o tema escuro, escolhido por adequação ao ambiente de oficina e à carga visual de uso prolongado, com uma paleta restrita composta por preto profundo no fundo, tons cinzentos progressivos para diferenciar profundidades, e um vermelho terracota como cor de acento para botões primários e elementos selecionados. O segundo é a tipografia mista, com tipo de letra serifado nos cabeçalhos de página (registro institucional, próximo da identidade de *atelier*) e tipo de letra sem serifas no corpo de texto e em tabelas (legibilidade em ecrã). O terceiro é a sinalização semântica de estado por *badge* colorido, com correspondência consistente em todos os ecrãs (verde para estados estáveis ou de conclusão, vermelho para estados ativos ou de alerta, amarelo para estados pendentes e cinzento para estados neutros). O quarto é a uniformização da iconografia através do conjunto *Material Symbols*, com glifos reconhecíveis para as ações recorrentes (consulta, edição, criação, registro de pagamento, exportação, eliminação).

Para além da linguagem visual, o conjunto do *design* segue as dez heurísticas de usabilidade de Nielsen, com particular ênfase em cinco. A heurística de *visibilidade do estado do sistema* concretiza-se na exposição permanente do estado atual de cada recurso através de *badge* colorido e na concentração dos indicadores agregados no *dashboard*. A *correspondência entre sistema e mundo real* reflete-se no vocabulário visível no *frontend* (Ordem de Serviço, Trocinete, Mecânico, Recebida, Em Diagnóstico, Em Reparação, Concluída, Entregue), que reproduz literalmente os termos usados na oficina, evitando jargão técnico interno. A *prevenção de erros* traduz-se no princípio do próximo passo legítimo, pelo qual apenas as transições admissíveis a partir do estado corrente são oferecidas ao utilizador, e na validação por *schema* Zod aplicada antes de qualquer submissão. A *consistência e normas* é assegurada pela uniformidade da iconografia, da paleta de cores semânticas, do posicionamento dos elementos da barra superior e da estrutura das tabelas em todos os ecrãs. O *reconhecimento em vez de memorização* suporta-se na barra lateral persistente, que lista as dez áreas funcionais do sistema sem exigir ao utilizador a memorização da estrutura do menu, e na apresentação direta dos identificadores de negócio (formato OS-XXXX, INV-XXXXXX) em vez de identificadores técnicos opacos.

2.7.2 Estrutura de Navegação

A navegação assenta numa barra lateral persistente, fixada no lado esquerdo da janela, com dez ligações organizadas pela ordem do fluxo operacional típico da oficina, desde o *Dashboard* de visão geral até às *Definições* administrativas, passando por *Ordens de Serviço*, *Intervenções*, *Clientes*, *Trocinetes*, *Stock e Peças*, *Fornecedores*, *Faturação* e *Relatórios*. A barra superior reserva-se a três elementos, designadamente o ícone de notificações pendentes, o avatar e identificação do utilizador autenticado e a indicação do papel correspondente. A persistência da barra lateral em todas as páginas foi opção deliberada, em alternativa a esquemas baseados em menus colapsáveis, dado que o número de secções (dez) cabe confortavelmente numa coluna estreita e a permanência reduz o número de cliques em fluxos que exigem alternância frequente entre módulos, comuns ao gerente.

2.7.3 Painel Operacional (*Dashboard*)

A Figura 2.15 apresenta o ecrã inicial após autenticação. Concentra a informação essencial à decisão diária do gerente em quatro indicadores principais (ordens em curso, ordens em diagnóstico, peças em *stock* crítico e faturas emitidas), num gráfico de distribuição das ordens por estado nos últimos trinta dias, em três medidores financeiros e operacionais (receita, horas trabalhadas, tempo médio de reparação) e numa lista de alertas de *stock* crítico, materializando o requisito REQ-015 de resumo agregado e o REQ-023 de alerta proativo. A presença simultânea destes elementos no primeiro ecrã suporta a verificação rápida do estado da oficina, sem necessidade de navegação adicional para os subsistemas individuais.

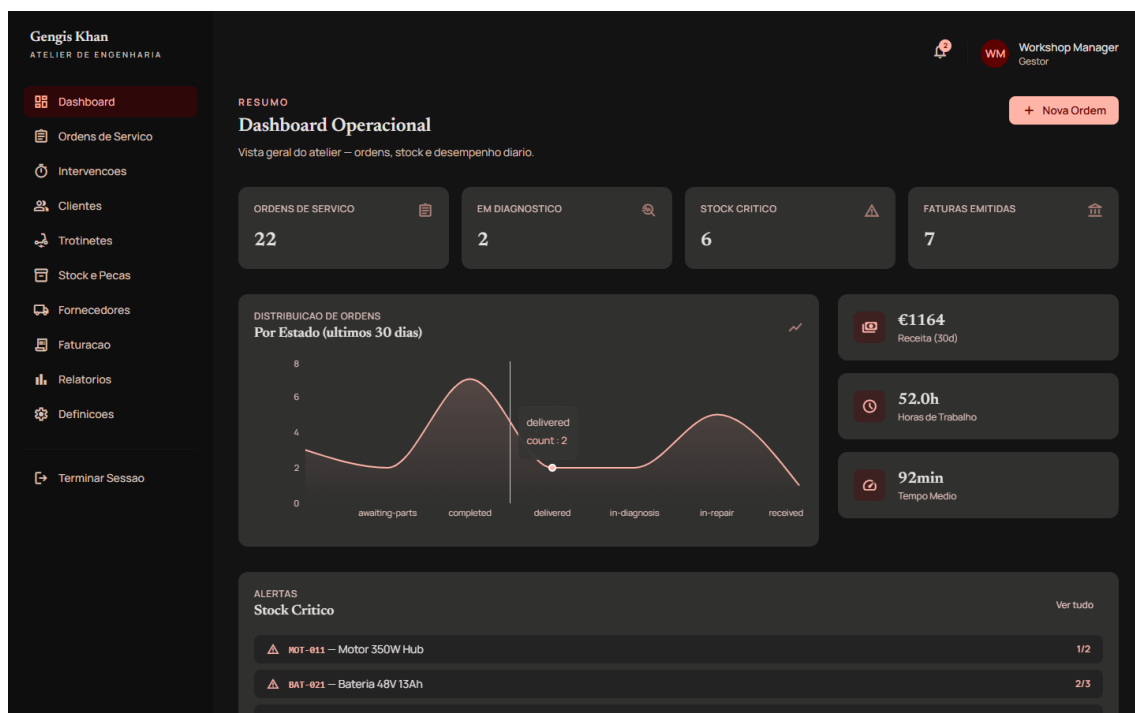
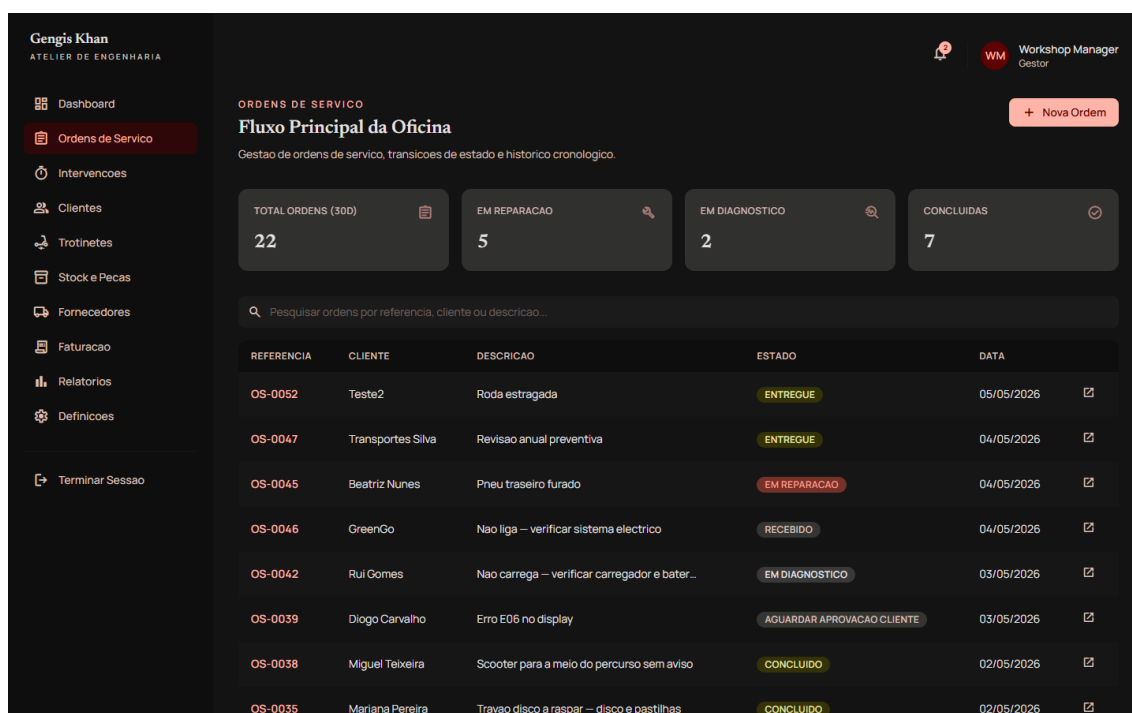


Figura 2.15: Painel operacional (*Dashboard*)

2.7.4 Gestão de Ordens de Serviço

A gestão de ordens de serviço é o centro funcional do sistema, materializando os requisitos REQ-008 a REQ-015. A Figura 2.16 apresenta o ecrã de listagem, com pesquisa por referência, cliente ou descrição, e indicadores agregados por estado no topo. Cada linha exibe a referência da ordem (formato OS-XXXX), o cliente associado, o problema reportado, o estado atual representado por *badge* colorido e a data de entrada. A coexistência de pesquisa textual e indicadores agregados no mesmo ecrã responde a duas necessidades distintas, designadamente pesquisa direta para o operador que conhece a referência, e visão agregada para o gerente que monitoriza a distribuição global do trabalho.



REFERENCIA	CLIENTE	DESCRICAO	ESTADO	DATA
OS-0052	Teste2	Roda estragada	ENTREGUE	05/05/2026
OS-0047	Transportes Silva	Revisao anual preventiva	ENTREGUE	04/05/2026
OS-0045	Beatriz Nunes	Pneu traseiro furado	EM REPARACAO	04/05/2026
OS-0046	GreenGo	Nao liga – verificar sistema electrico	RECEBIDO	04/05/2026
OS-0042	Rui Gomes	Nao carrega – verificar carregador e bater...	EM DIAGNOSTICO	03/05/2026
OS-0039	Diogo Carvalho	Erro E06 no display	AQUARDAR APROVACAO CLIENTE	03/05/2026
OS-0038	Miguel Teixeira	Scooter para a meio do percurso sem aviso	CONCLUIDO	02/05/2026
OS-0035	Mariana Pereira	Travao disco a raspar – disco e pastilhas	CONCLUIDO	02/05/2026

Figura 2.16: Listagem de ordens de serviço

A Figura 2.17 apresenta o ecrã de detalhe de uma ordem de serviço em estado Aguardar Aprovação Cliente. Quatro decisões de *design* distinguem-no, todas alinhadas com heurísticas estabelecidas de usabilidade. Primeiro, os indicadores de cabeçalho ficam permanentemente visíveis sem dependência de *scroll*, sustentando a heurística de *visibilidade do estado do sistema*. Segundo, a faixa *Próximo passo* apresenta exclusivamente as transições admissíveis a partir do estado corrente, calculadas pelo *frontend* segundo a máquina de estados documentada na Secção 2.4, em aplicação direta da heurística de *prevenção de erros*. Terceiro, o painel de diagnóstico e o painel de intervenções coexistem lado a lado, refletindo a separação conceptual entre análise técnica e execução do trabalho. Por fim, o painel de intervenções respeita o

REQ-014 e apresenta uma mensagem informativa quando a ordem ainda não está em estado de reparação, cumprindo a heurística de *ajuda ao reconhecimento e diagnóstico de erros*.

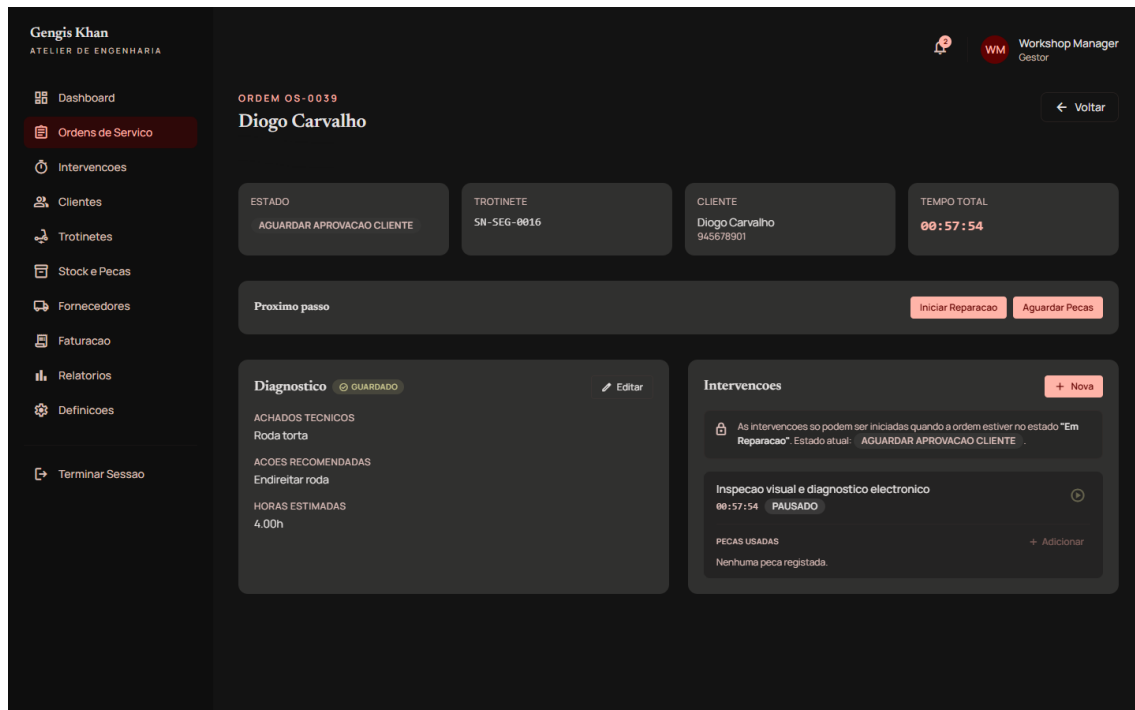


Figura 2.17: Detalhe de ordem de serviço com cronómetro de intervenção

A criação de uma nova ordem de serviço ocorre num diálogo modal sobreposto à listagem, conforme apresentado na Figura 2.18. O diálogo concentra os campos mínimos exigidos pelo requisito REQ-008 (cliente, número de série da trocinete, problema reportado, data estimada de conclusão) e expõe uma ligação *Novo Cliente* para o caso de o cliente ainda não estar registado, evitando a interrupção do fluxo de receção em oficina. A submissão do diálogo sujeita os dados à validação Zod do pacote de contratos partilhados (Subsecção 2.1.4) antes do envio ao *backend*.

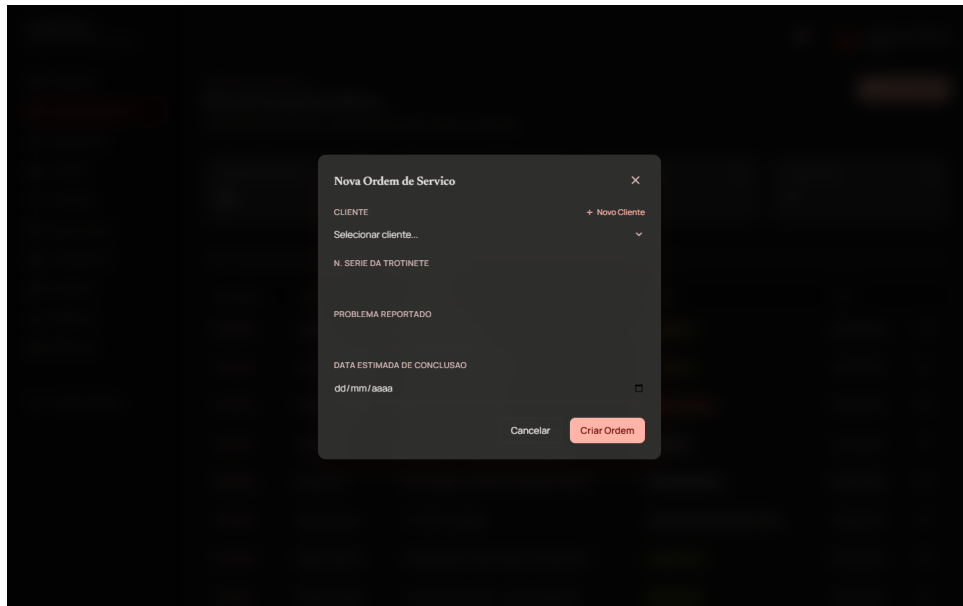


Figura 2.18: Diálogo modal de criação de ordem de serviço

As mensagens de erro de validação serão apresentadas imediatamente abaixo de cada campo, mantendo o utilizador focado no formulário sem necessidade de reposicionamento da janela para localizar a indicação de erro. O botão de submissão permanecerá inativo enquanto existirem campos obrigatórios por preencher, evitando submissões prematuras que apenas resultariam em respostas de erro emitidas pelo *backend*. A combinação destas duas opções aproxima o diálogo modal das convenções de usabilidade aplicadas em formulários transacionais de uso recorrente, designadamente o princípio da prevenção de erros enunciado por Nielsen.

2.7.5 Registo de Intervenções

O ecrã de *Registo de Intervenção*, apresentado na Figura 2.19, lista as ordens em estado ativo, ordenadas pela data de entrada, e permite ao mecânico selecionar a ordem sobre a qual pretende registar trabalho. Cada cartão indica a referência da ordem, o problema reportado, o número de série da trotinete e o estado atual. A apresentação em cartões, em vez de tabela, reflete a natureza da operação, dado que o mecânico seleciona uma ordem para focar o trabalho num único contexto, e não percorre uma listagem comparativa. A entrada nesta lista encaminha para o detalhe da ordem, no qual a operação do cronómetro descrita na Subsecção 2.4.2 fica acessível através de controlos próprios de iniciar, pausar e parar.

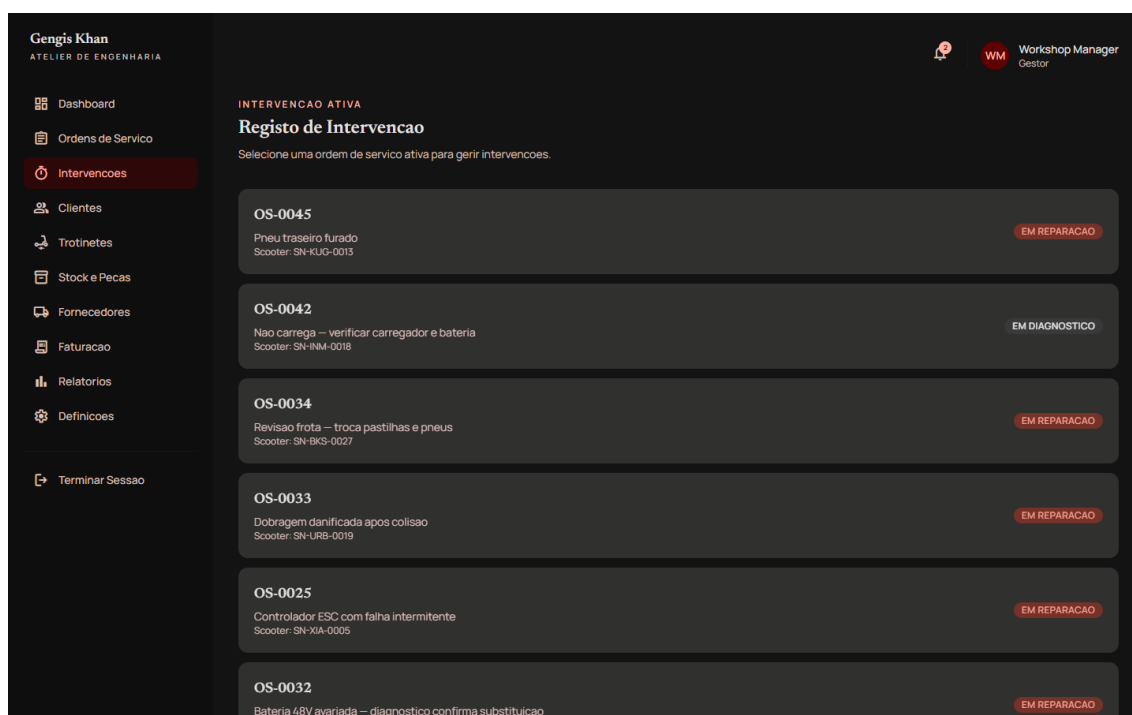


Figura 2.19: Registo de intervenção: lista de ordens com trabalho em curso

2.7.6 Catálogo de Peças e *Stock*

A Figura 2.20 apresenta o ecrã de *Catálogo e Movimentos de Stock*, dividido em duas vistas (*Inventário* e *Movimentos*) acessíveis por separadores no topo. A vista de inventário mostra três indicadores agregados (total de unidades, itens críticos e valor total inventariado) e uma tabela do catálogo, ordenável e pesquisável. O atributo *Estado* de cada peça resulta da comparação entre *currentStock* e *minimumStock*: *Estável* quando há folga, *Limiar* quando o nível atinge o mínimo e *Crítico* quando o valor cai abaixo. Esta sinalização triádica suporta o requisito REQ-023 de alerta proativo e materializa o despoletador de geração automática de encomendas a partir do *endpoint* `POST /purchase-orders/generate-from-low-stock` documentado na Secção 2.5.

The screenshot displays the 'Catálogo e Movimentos de Stock' interface. The sidebar on the left includes navigation options: Dashboard, Ordens de Serviço, Intervencoes, Clientes, Trotinetes, Stock e Peças (highlighted), Fornecedores, Faturacao, Relatorios, Definicoes, and Terminar Sessao. The top header shows the user 'Gengis Khan' and 'Workshop Manager Gestor'. The main content area features a title 'Catálogo e Movimentos de Stock' and a subtitle 'Listagem de peças, alertas de stock baixo e historico de movimentos auditaveis.' Below this are two tabs: 'Inventario' (selected) and 'Movimentos'. Three summary cards are displayed: 'TOTAL UNIDADES' (387), 'ITENS CRITICOS' (6), and 'VALOR TOTAL' (€5503). A search bar is present with the placeholder 'Pesquisar por referencia ou peça...'. Below the search bar is a table with the following columns: REFERENCIA, PEÇA, STOCK, MINIMO, PRECO UNIT., and ESTADO. The table contains seven rows of data, with the 'ESTADO' column showing 'ESTAVEL' or 'LIMIAIR'.

REFERENCIA	PEÇA	STOCK	MINIMO	PRECO UNIT.	ESTADO
KIT-024	Kit Reparacao Pneu Tubeless	39	15	€9.00	ESTAVEL
SUS-023	Mola Amortecedor Traseiro	8	4	€24.00	ESTAVEL
ROL-022	Rolamento Roda Traseira	30	10	€7.50	ESTAVEL
BAT-021	Bateria 48V 13Ah	2	3	€185.00	LIMIAIR
CAB-020	Cabo de Acelerador	18	10	€5.50	ESTAVEL
BRK-019	Disco de Travao 110mm	10	5	€18.00	ESTAVEL
LUZ-018	Luz Traseira com Sinal	20	8	€13.00	ESTAVEL
LUZ-017	Farol Dianteiro LED	15	8	€16.50	ESTAVEL

Figura 2.20: Catálogo de peças e movimentos de *stock*

2.7.7 Faturação

A Figura 2.21 apresenta o ecrã de *Faturação*, exclusivo ao papel *manager* (REQ-037). O cabeçalho reúne três indicadores de gestão financeira e a tabela é filtrável por estado de pagamento, com ações inline de registo de pagamento, duplicação e consulta de detalhe. A coexistência de filtros explícitos com indicadores agregados resolve o REQ-033 de consulta rápida de faturas pendentes e atrasadas, em particular as de clientes empresariais, e materializa a heurística de *flexibilidade e eficiência de uso*: o utilizador experiente filtra diretamente sem percorrer a listagem, enquanto o utilizador novato dispõe da listagem completa por defeito.

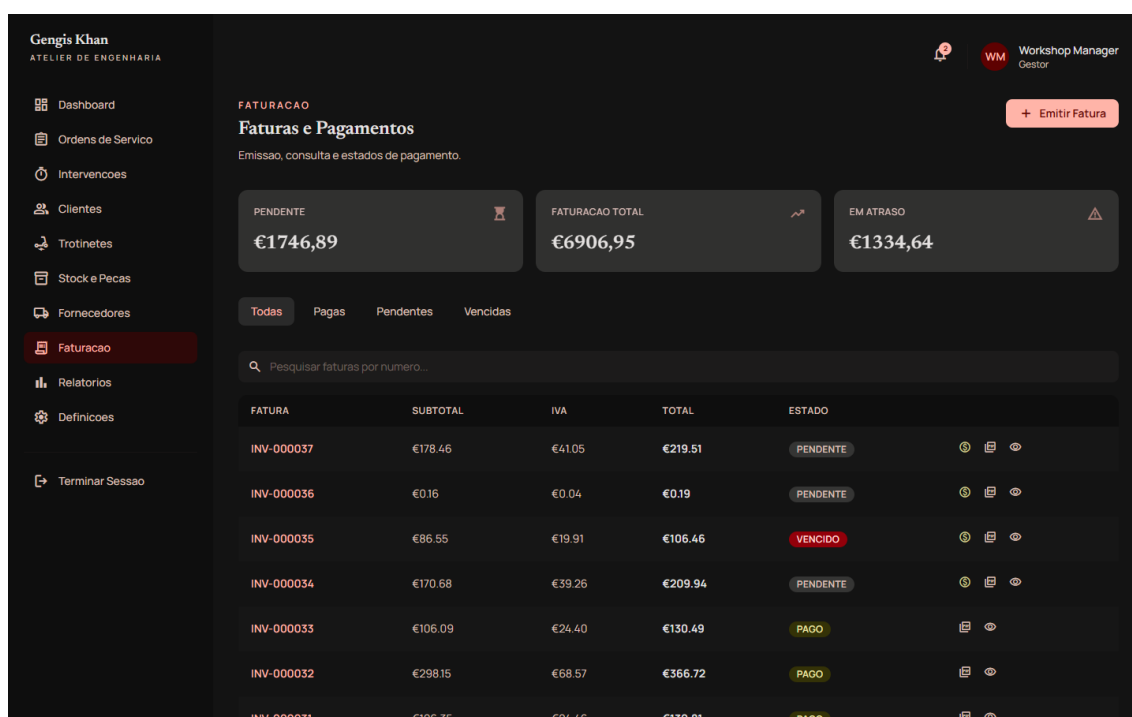


Figura 2.21: Faturação: lista de faturas com estados de pagamento

2.7.8 Relatórios Executivos

O ecrã de *Relatórios Executivos*, apresentado na Figura 2.22, agrega indicadores transversais ao sistema e está reservado ao papel *manager*. Reúne quatro indicadores principais (faturação total, ordens de serviço, *ticket* médio, intervenções), um resumo financeiro em gráfico de linhas, uma distribuição visual entre ordens e faturas, um *ranking* das peças mais consumidas e uma tabela de produtividade por mecânico (intervenções, tempo médio, total). Esta vista materializa o conjunto de *endpoints* `/api/v1/reports/*` documentado na Tabela 2.11 e fornece ao gerente uma síntese do desempenho operacional sem necessidade de navegação por módulos individuais.

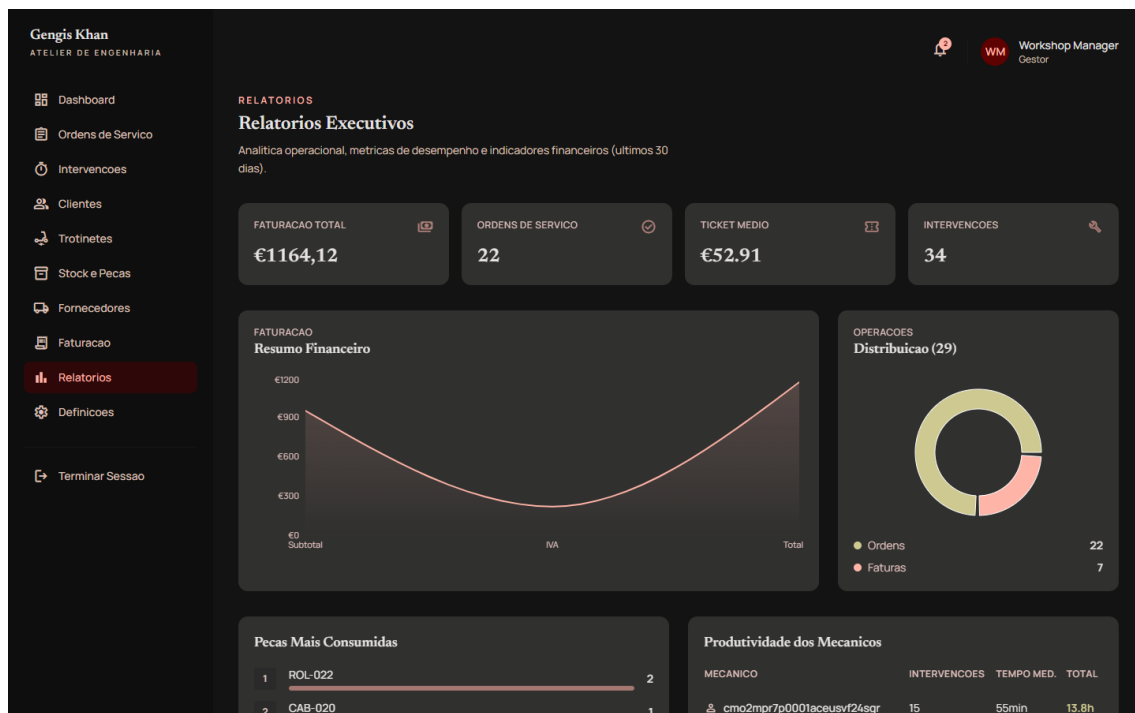


Figura 2.22: Relatórios executivos: vista agregada de desempenho

A organização visual privilegia a leitura comparativa, com os quatro indicadores principais alinhados no topo do ecrã e o detalhe analítico apresentado em blocos imediatamente abaixo, mantendo coerência com a convenção de pirâmide invertida observada noutros painéis administrativos. O período de análise será controlável a partir de um seletor único, evitando que cada bloco exponha o seu próprio filtro temporal e prevenindo a leitura cruzada de séries com intervalos diferentes. A atualização das séries ocorrerá de forma assíncrona após a alteração do período, sem recarregamento integral da página, em conformidade com o padrão de interação adotado nas restantes vistas do sistema.

2.7.9 Notificações e Estado de Envio

A Figura 2.23 apresenta o painel de notificações, acessível a partir do ícone de sino na barra superior, com contador de notificações pendentes. Cada entrada documenta uma notificação efetivamente emitida pelo sistema (instância da entidade *Notification* discutida na Subsecção 2.2.5), o canal utilizado (*email* do cliente), o instante de envio e o estado de entrega. O modelo distingue visualmente entregas bem-sucedidas (*badge SENT*) de falhas (*badge FAILED*), permitindo ao gerente identificar rapidamente situações em que o canal de comunicação com o cliente não pôde ser concluído e atuar de forma manual em alternativa.

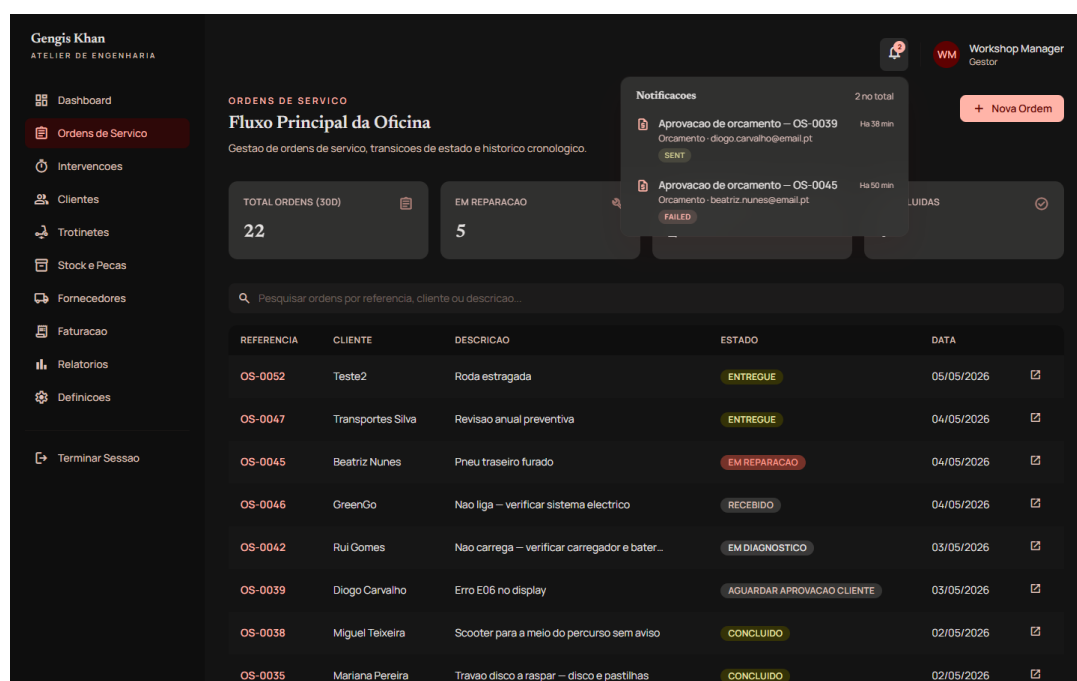


Figura 2.23: Painel de notificações com estado de entrega por mensagem

2.7.10 Adaptação a *Tablets*

O cumprimento do RNF-02 (operação adequada em *tablets*) é assegurado por três opções estruturais alinhadas com a heurística de *design estético e minimalista*. A barra lateral colapsa-se automaticamente em larguras de *viewport* inferiores a 1024 *pixels*, libertando espaço para o conteúdo principal. As áreas tácteis dos botões e ações inline obedecem ao mínimo de 44 *pixels* recomendado pelo *Material Design*, reduzindo a probabilidade de toques acidentais. As tabelas longas suportam *scroll* horizontal interno em vez de truncamento, preservando o acesso a todas as colunas. Este *design* responsivo foi pensado para os formatos típicos da oficina: *tablet* de gerente em modo retrato e *laptop* de mecânico em mesa de trabalho.

2.7.11 Síntese

O *design* de interface aqui apresentado completa o ciclo de *design* do capítulo, ligando a vista lógica do sistema à sua manifestação concreta perante o utilizador final. As escolhas dominantes (tema escuro, sinalização semântica de estado por *badge*, princípio do *próximo passo legítimo*, separação visual entre módulos e adaptação a *tablets*) exprimem coerência entre as cinco vistas anteriores e a forma como o sistema é efetivamente operado. O conjunto de decisões arquiteturais, modelos estruturais, comportamentais, de componentes e de interface aqui consolidado constitui a base sobre a qual assenta a fase de implementação a documentar no Capítulo 3.

3 Implementação e Desenvolvimento Assistido por LLM

O presente capítulo documenta a terceira etapa do ciclo de vida do projeto, aquela em que os artefactos de *design* produzidos no Capítulo 2 ganharam forma em código executável e versionado. A particularidade desta etapa, face às anteriores, reside no grau de autonomia atribuído ao LLM entre cada validação humana. Se nas fases anteriores o modelo gerou já a maior parte do conteúdo produzido, fê-lo sob iteração imediata, *prompt* a *prompt* e com revisão direta da equipa após cada resposta. Na implementação, esse acoplamento estreito relaxou. O LLM passou a operar sobre conjuntos mais amplos (módulos inteiros, *refactorings*, revisões de *pull request*) e a equipa passou a verificar o resultado agregado em vez de cada passo intermédio.

Esta integração profunda traduz, na prática, uma reconfiguração dos papéis em jogo. O LLM passou a comportar-se, durante a implementação, como um programador júnior competente mas sem juízo arquitetural autónomo, capaz de gerar código sintaticamente correto e idiomáticamente bem formado a partir de uma especificação clara, dependente, contudo, de orientação humana para escolher entre alternativas estruturais, para alinhar com os contratos previamente fixados pela equipa e para reconhecer os limites da sua própria produção. A equipa, por sua vez, deslocou progressivamente o seu esforço da escrita direta de código para a supervisão, a orquestração e o arbítrio sobre o que o modelo produzia. O eixo do trabalho passou a residir na formulação rigorosa do raciocínio que orienta cada implementação, na fixação das decisões estruturais não delegáveis e na validação crítica do código gerado. A trajetória que daqui se desenha, e que esta etapa tornou tangível à equipa, aponta para uma divisão de trabalho em que o engenheiro humano se firma como arquiteto e validador, e o modelo de linguagem assume a função de codificador disciplinado sob essa supervisão.

O capítulo documenta, em ambos os sentidos, as consequências desta integração. Por um lado, ganhos efetivos de produtividade, uniformidade estilística e exposição a soluções idiomáticas que a equipa não dominava previamente. Por outro lado, limitações não negligenciáveis, designadamente alucinação técnica ocasional, divergência face aos contratos previamente fixados e esforço de revisão por vezes superior ao que teria sido a escrita direta. A documentação honesta destes dois lados é parte integrante da utilidade pedagógica do relatório.

3.1 Metodologia de Desenvolvimento

Análise Crítica do Processo de Geração com IA

Prompt 29. Metodologia de Desenvolvimento

Contexto: Arquitetura do Capítulo 2 (monolito modular em tres camadas, monorepo pnpm, contratos Zod, PostgreSQL via Prisma), calendario de tres semanas e equipa de cinco estudantes a usar o LLM como parceiro de producao de codigo.

Objetivo: Propor uma metodologia de desenvolvimento adequada ao contexto, distinguindo o ciclo de vida do projeto academico do ciclo de producao de codigo.

Instrucoes de Escrita:

- Tratar a cascata como ciclo de vida do projeto academico (entrega unica)
- Tratar o ciclo de codigo com LLM como incremental, por modulo
- Discutir granularidade da supervisao humana e os compromissos entre autonomia e revisao
- Identificar pontos nao delegaveis ao LLM

Restricoes: Sem cerimonia agile formais nem sprints de duracao fixa. Sem terminologia industrial desproporcionada a equipa de cinco.

Análise Crítica

A primeira proposta do LLM apontou para um modelo de desenvolvimento ágil estruturado em *sprints* semanais com cerimónias formalizadas, padrão da indústria mas desproporcionado à dimensão da equipa e ao horizonte de três semanas. A proposta foi rejeitada por sobrepôr processo desnecessário a um trabalho cuja unidade natural é o módulo, não o intervalo temporal. Após reformulação, o modelo convergiu para o esquema adotado, em que cada módulo é tomado como unidade elementar de trabalho, formulado como *prompt* direto ao LLM, gerado pelo modelo, revisto pela equipa e integrado por *pull request*. A contribuição mais relevante foi a fixação de três pontos não delegáveis ao LLM, designadamente a assinatura pública de cada módulo, a validação de cada *pull request* antes do *merge* e a gestão das migrações de *schema* relacional.

A definição desta unidade de trabalho constituiu, contudo, apenas o primeiro passo de um exercício metodológico mais subtil do que inicialmente antecipado, e que se revelou um dos pontos mais interessantes desta etapa. A equipa começou por solicitar a implementação completa de cada módulo numa única interação, mas constatou rapidamente que o LLM perdia precisão quando confrontado com pedidos demasiado abrangentes, omitindo detalhes, simplificando regras de negócio ou introduzindo inconsistências silenciosas. A

resposta foi quebrar cada módulo em blocos ainda mais pequenos do que a sua fronteira natural sugeria, organizando a implementação em sucessivas iterações curtas, cada uma com âmbito estrito e validável de forma independente. Esta granularidade revelou-se essencial para manter a coerência do código gerado e para garantir que cada passo se mantinha alinhado com a arquitetura definida no Capítulo 2. O percurso detalhado deste exercício é apresentado na Secção 3.3, dedicada à implementação incremental por módulos.

A metodologia conjuga dois ciclos sobrepostos. O ciclo de vida do projeto académico é em cascata, na aceção clássica descrita por [1], com produto único entregue no final da unidade curricular e dependências sequenciais entre as quatro etapas. O ciclo de produção de código, por sua vez, foi conduzido de forma incremental ao longo das três semanas, com cada módulo a passar por um ciclo curto de formulação do *prompt*, geração pelo LLM, revisão humana e integração por *pull request*. A cascata fixa a moldura de entrega e o desenvolvimento incremental adequa-se à natureza dos modelos de linguagem, mais fiáveis em respostas curtas e revistas do que em respostas extensas.

3.2 Configuração do Ambiente de Trabalho

Análise Crítica do Processo de Geração com IA

Prompt 30. Configuração do Ambiente de Trabalho

Contexto: Monorepo pnpm com apps/api (Fastify e TypeScript), apps/web (React e Vite) e packages/contracts (Zod), equipa de cinco estudantes em paralelo ao longo de tres semanas e necessidade de paridade aceitavel entre local e producao.

Objetivo: Recomendar a configuracao de ambiente de desenvolvimento que minimize friccao, uniformize comandos comuns, valide cada pull request antes do merge e permita reposicao rapida da base de dados.

Instrucoes de Escrita:

- Discutir alternativas para a provisao local de PostgreSQL (nativa, Docker Compose, Supabase CLI)
- Discutir alternativas para a automacao de comandos (Makefile, scripts pnpm, Taskfile)
- Discutir a estrutura do pipeline de integracao continua
- Para cada decisao indicar a opcao recomendada e as razoes

Restricoes: Sem matrizes de versao desnecessarias nem jobs paralelos sem ganho mensuravel.

Análise Crítica

O contributo mais transformador do LLM nesta interação foi a uniformização de todas as operações recorrentes do projeto sob uma interface única, designadamente um *Makefile* com cerca de duas dezenas de *targets* que cobrem o ciclo completo de desenvolvimento, da instalação inicial à execução de testes, passando pela aplicação de migrações, pelo arranque do ambiente local e pela reposição da base de dados. A consequência prática desta concentração foi alterar de forma substantiva a relação da equipa com a configuração do projeto. A partir do momento em que cada operação é invocada por um comando único e estável, deixou de ser necessário compreender em detalhe os *scripts* subjacentes, as variáveis de ambiente, as opções dos contentores ou os passos manuais de instalação de cada componente. A equipa concentra-se hoje na verificação ocasional dos ficheiros gerados e na validação do comportamento observável do sistema, transitando a interação com o ambiente para um plano essencialmente declarativo.

Esta mudança de relação com a configuração comporta uma vantagem evidente, traduzida na redução drástica do tempo de *onboarding* e da carga cognitiva de manutenção, mas também uma contrapartida menos óbvia. A familiaridade com os bastidores do ambiente, que tradicionalmente se adquiria pela leitura e edição dos *scripts* de configuração, é hoje progressivamente substituída pela confiança no *Makefile* e nos artefactos gerados pelo modelo. Esta confiança é razoável enquanto os comandos se comportam como esperado, mas torna a equipa mais dependente do LLM caso surja uma falha que exija intervenção fora dos *targets* pré-existentes. As decisões específicas sobre alternativas de provisão local de *PostgreSQL*, automação de comandos e estrutura do *pipeline* de CI foram acolhidas com pequenos ajustes pela equipa, dispensando-se aqui o seu detalhe por terem sido já discutidas no capítulo anterior.

O ambiente foi instituído sobre uma estrutura de *monorepo pnpm* com *workspaces*, organizada em três pastas principais: *apps/api* (*backend Fastify*), *apps/web* (*frontend React* com *Vite*) e *packages/contracts* (esquemas *Zod* partilhados). A coordenação dos comandos comuns foi delegada ao *Turborepo*, que assegura a execução em ordem topológica das dependências entre pacotes e o *cache* dos artefactos de *build* entre execuções consecutivas.

A persistência relacional é provisionada localmente pelo *Supabase CLI*, que arranca uma instância de *PostgreSQL* contentorizada na porta 54322, com *Docker Compose* disponível como alternativa. A correspondência entre o *schema* declarado e o *schema* persistido é mantida pelo *Prisma*, com migrações geradas automaticamente a cada alteração e aplicadas por *make db-migrate*.

A automação dos comandos comuns foi centralizada num *Makefile* com cerca de duas dezenas de *targets* agrupados em quatro famílias, designadamente o ciclo de desenvolvimento (*install*, *dev*, *build*, *test*, *lint*, *typecheck*), a infraestrutura (*Docker Compose* e *Supabase CLI*), a base de dados (operações de *Prisma*) e a compilação do relatório em *LaTeX*.

A integração contínua é assegurada por um único *workflow GitHub Actions* (*.github/workflows/ci.yml*), disparado a cada *pull request* e *push* para a *branch* *main*. O *job* *validate* executa,

em sequência sobre *Node.js* 20, `pnpm install`, `pnpm lint`, `pnpm typecheck`, `pnpm test` e `pnpm build`, com tempos típicos inferiores a quatro minutos.

3.3 Implementação Incremental por Módulos

Análise Crítica do Processo de Geração com IA

Prompt 31. Estratégia de Implementação por Módulos

Contexto: Onze módulos identificados na arquitetura (auth, customers, scooters, service-orders, interventions, inventory, suppliers-procurement, billing, notifications, configuration, reports) organizados em quatro subsistemas (operacional, inventario, faturacao e suporte, relatorios transversais).

Objetivo: Propor uma ordem de implementacao que minimize bloqueios por dependencias cruzadas e maximize o ganho incremental do LLM.

Instrucoes de Escrita:

- Para cada modulo indicar ficheiros minimos, esquemas Zod necessarios, endpoints HTTP e regras de negocio criticas
- Identificar dependencias entre modulos e refleti-las na ordem proposta
- Indicar os testes minimos a acompanhar cada entrega

Restricoes: Sem implementacao detalhada de cada modulo. Sem reordenacoes sem justificacao por dependencia.

Análise Crítica

O LLM propôs uma ordem coerente que privilegiou os módulos de catálogo antes dos módulos transacionais, justificando a opção pela menor complexidade dos primeiros e pela sua condição de pré-requisito dos segundos. A equipa adotou esta ordem com uma alteração, antecipando o módulo `auth` para primeiro lugar, dado que toda a verificação de papel depende da emissão de *token* de sessão. A sugestão de produzir, para cada módulo, um teste mínimo de integração do *fluxo normal*, a execução nominal do sistema sem condições de exceção, acompanhando a entrega foi adotada. A presença do teste forçou a verificação efetiva dos contratos contra *endpoints* reais, expondo divergências que a inspeção visual do código deixaria passar.

A implementação dos onze módulos do *backend* seguiu uma ordem dependente da topologia das relações entre subsistemas, fixada na vista estrutural do Capítulo 2. O subsistema operacional foi implementado primeiro, com o módulo `auth` a abrir o trabalho, seguido de

customers, scooters, service-orders e interventions. Concluído o núcleo operacional, seguiram-se inventory e suppliers-procurement, que dependem de utilizadores autenticados e de ordens de serviço válidas. Foram depois implementados billing, notifications e configuration, agrupados pela sua dependência do estado terminal das ordens. Por último, foi adicionado o módulo transversal reports.

Cada módulo foi tratado como unidade autónoma de trabalho, com a equipa a entregar três ficheiros estruturalmente uniformes, designadamente `index.ts` (registo de *handlers* HTTP e validação de *schema*), `*.service.ts` (regras de negócio) e `*.repository.ts` (acesso ao *Prisma*). A aderência rigorosa a este padrão revelou-se decisiva para a velocidade da implementação. A partir do terceiro módulo, o LLM foi capaz de gerar esqueletos novos com aderência quase imediata ao padrão, por reconhecimento dos exemplos anteriores incluídos como contexto no *prompt*.

A revisão humana de cada módulo concentrou-se em quatro frentes, designadamente a aderência aos contratos partilhados em `packages/contracts`, a coerência com o estilo dos módulos já consolidados, o tratamento explícito dos casos de erro previstos no formato uniforme da API e a rejeição de transições de estado inválidas no caso dos módulos com máquina de estados. Cada módulo foi acompanhado de um teste mínimo de integração que cobre o *fluxo normal* e os erros de validação principais, e a totalidade destes testes compõe a *suite* executada em CI.

3.4 Geração de Código Assistida por LLM

Análise Crítica do Processo de Geração com IA

Prompt 32. Tarefas de Geração de Código por LLM

Contexto: Implementacao dos onze modulos do backend com recurso ao LLM, ao longo de tres semanas, e necessidade de caracterizar criticamente o desempenho do modelo em diferentes tarefas.

Objetivo: Identificar os tipos de tarefas em que o LLM se mostrou mais e menos eficaz e descrever os padroes de prompt e os modos de falha observados.

Instrucoes de Escrita:

- Distinguir geracao de boilerplate estrutural, traducao de regras de negocio, escrita de testes, sugestao de refactoring e revisao de pull requests
- Para cada categoria descrever o padrao de prompt mais eficaz e os modos de falha tipicos
- Graduar a confianca no modelo com niveis nao binarios

Restricoes: Sem juizos absolutos. Sem omitir modos de falha em areas mais sensiveis (regras fiscais, refactoring profundo).

Análise Crítica

A análise do LLM sobre o seu próprio desempenho revelou-se moderadamente fiável. Identificou corretamente as áreas em que se sentiu mais à vontade (*boilerplate* estrutural, esquemas *Zod*, testes do *fluxo normal*) mas subestimou a sua taxa de erro em tradução de regras de negócio fiscais e em *refactoring* de código já complexo. A equipa cruzou esta autoavaliação com a sua observação direta e reteve uma taxonomia mais conservadora, na qual a confiança no modelo é graduada em quatro níveis em vez dos níveis binários sugeridos. A contribuição mais útil foi a sistematização dos padrões de *prompt* associados a cada categoria, que permitiu reutilizar formulações eficazes em vez de reinventar a estrutura do *prompt* a cada novo módulo.

Para além desta autoavaliação, a equipa retira da experiência de implementação uma observação própria sobre a qualidade do código gerado. Em termos globais, o resultado foi considerado positivo, e essa apreciação está diretamente associada à estratégia incremental adotada e descrita na Secção 3.3. A decomposição da implementação em blocos pequenos, cada um produzido a partir de um *prompt* focado e validado de forma isolada antes de avançar para o seguinte, permitiu uma granularidade de inspeção e de teste que seria inviável caso o LLM tivesse sido pedido a entregar módulos completos numa única interação. Esta abordagem reduziu o risco de erros silenciosos acumulados, manteve a base de código sempre em estado verificável e elevou de forma consistente a confiança da equipa em cada artefacto integrado, fatores que, em conjunto, sustentam a avaliação favorável do código resultante.

Cinco categorias de tarefa permitiram observar o desempenho do modelo na produção de código.

A primeira é a geração de *boilerplate* estrutural, designadamente esqueletos de *handlers Fastify*, *repositories Prisma* sobre uma entidade e componentes *React* de listagem padrão. O LLM revelou-se altamente eficaz, com aceitação imediata em mais de 80% dos casos.

A segunda é a definição de esquemas *Zod* a partir das entidades documentadas no Capítulo 2. O modelo produziu esquemas sintaticamente corretos e com tipagem inferida coerente, mas tendeu a omitir restrições semânticas que não eram explícitas na descrição (validação de NIF português com nove dígitos, rejeição de *e-mails* sem domínio). A revisão humana foi necessária em cerca de 40% dos esquemas.

A terceira é a tradução de regras de negócio do SRS para código de serviço. Foi a categoria mais sensível, dado que o modelo gerou frequentemente código que parecia correto mas omitia ou subvertia subtilmente alguma regra de domínio. Por exemplo, a verificação do papel *manager* para a transição *delivered* foi, durante a primeira iteração, reduzida a um *warning* em vez de *403 forbidden*. A confiança da equipa só foi retomada após verificação cruzada contra o SRS e os testes de integração.

A quarta é a escrita de testes unitários e de integração. O LLM revelou-se eficaz na geração de testes para o *fluxo normal*, mas omitiu, com frequência, casos limite e cenários de falha que exigem entendimento do domínio. A prática estabilizada foi reformular o pedido enumerando explicitamente os cenários de falha a cobrir, em vez de aceitar apenas a primeira proposta.

A quinta é a revisão de *pull requests*. O modelo identificou consistentemente problemas de estilo, formatação e duplicação, mas falhou na deteção de *bugs* subtis de domínio (uma transação *Prisma* mal aninhada, uma verificação de papel ausente). A revisão por LLM foi adotada como complemento, nunca como substituto, da revisão humana.

3.4.1 Exemplo Concreto de *Prompt* de Geração

A consistência dos resultados obtidos ao longo da implementação dependeu fortemente da qualidade dos *prompts* formulados. Importa, por isso, ilustrar com um exemplo real o nível de detalhe que se revelou eficaz e os pontos em que o modelo, ainda assim, falhou. O bloco abaixo apresenta o *prompt* utilizado para a geração inicial do módulo *service-orders*, um dos módulos centrais do sistema, normalizado na estrutura padrão de quatro secções.

Prompt 33. Geração do Módulo *Service-Orders*

Contexto: Backend Fastify+TypeScript com modulos customers e scooters ja implementados em apps/api/src/modules como referencia de estilo , contratos Zod em packages/contracts/src/service-orders e maquina de estados definida no Capitulo 2.

Objetivo: Implementar o modulo service-orders em apps/api/src/modules/service-orders com os ficheiros index.ts, service.ts e repository.ts.

Instrucoes de Escrita:

- Seguir o padrao estrutural ja estabelecido em customers e scooters
- Expor os endpoints GET /api/v1/service-orders (listagem com filtros por estado, NIF e numero de serie), POST, GET /:id, PATCH /:id, PATCH /:id/status, PATCH /:id/diagnosis, PATCH /:id/budget, POST /:id/budget/approval, GET /:id/history
- Aplicar requireRole(['manager','mechanic']) no inicio de cada handler e exigir role=manager na transicao para delivered
- Persistir a criacao da ordem em transacao Prisma unica, incluindo ServiceOrderHistoryEntry com action='created'
- Validar transicoes contra validTransitions e devolver appError(409, 'invalid_status_transition') em violacao
- Manter o handler fino: regras de negocio so em service.ts, acesso a Prisma so em repository.ts

Restricoes: Sem regras de negocio no handler. Sem acesso a Prisma fora do repository. Sem esquemas Zod proprios; usar os de packages/contracts.

A análise crítica deste *prompt*, retrospectivamente conduzida pela equipa, identifica três pontos fortes e dois pontos fracos. Como pontos fortes, a indicação explícita do padrão estrutural a seguir (*index.ts*, *service.ts*, *repository.ts*) com referência a módulos já consolidados (*customers*, *scooters*) permitiu ao modelo replicar quase sem desvios o estilo da equipa, a enumeração exaustiva dos *endpoints* com método HTTP e caminho eliminou ambiguidade quanto ao contrato a expor, e a referência direta aos esquemas *Zod* já existentes em *packages/contracts* forçou aderência aos contratos partilhados em vez de o modelo gerar os seus próprios. Como pontos fracos, a descrição da máquina de estados foi resumida demais (*a transição para delivered exige role=manager*), pelo que o modelo a implementou inicialmente como *warning* de *log* em vez de *403 forbidden*, exigindo iteração corretiva, e os cenários de erro previstos no formato uniforme da API foram referidos apenas para a transição de estado, deixando outros *endpoints* com mensagens de erro inconsistentes que tiveram de ser uniformizadas em iteração subsequente.

Esta análise sustenta uma observação metodológica que se manteve estável ao longo das três semanas. A consistência dos resultados depende muito mais da disciplina das *guidelines* fornecidas no *prompt* (referência ao código já existente, contratos a respeitar, padrões a replicar) do que da extensão ou sofisticação da própria descrição funcional. Os *prompts* mais eficazes foram aqueles em que a maior parte do texto foi dedicada a fixar restrições estruturais e a apontar exemplos a seguir, e não a explicar requisitos de domínio, dado que estes já estão fixados nos esquemas *Zod* e nos contratos partilhados. Esta observação determinou, na prática, que cada novo módulo passou a referenciar dois ou três módulos anteriores como exemplos, comportamento que reduziu de forma sensível a variabilidade entre primeiras gerações.

3.5 Revisão de Código

Análise Crítica do Processo de Geração com IA

Prompt 34. Modelo de Revisão de Código

Contexto: Equipa de cinco estudantes a desenvolver em ramos paralelos sobre o mesmo monorepo, com integracao continua a executar lint, typecheck, testes e build em cada pull request.

Objetivo: Propor um modelo de revisao de codigo em camadas que combine gating automatico em CI, revisao humana minima e revisao assistida por LLM.

Instruções de Escrita:

- Indicar critérios explícitos de bloqueio de merge por tipo de falha
- Fixar o número mínimo de revisores humanos por pull request, justificado pela dimensão da equipa
- Definir o papel do LLM como revisor complementar (nunca substituto)
- Indicar quando faz sentido solicitar revisão assistida por LLM

Restrições: Sem exigência de dois revisores humanos. Sem usar o LLM como aprovador único.

Análise Crítica

A proposta inicial do LLM aplicou uma política de revisão obrigatória por dois pares humanos, padrão em equipas de produção. Esta política foi ajustada para um único revisor humano, dada a dimensão da equipa e o volume de *pull requests*, dado que a exigência de dois revisores teria criado um estrangulamento incompatível com o ritmo de desenvolvimento de três semanas. A contribuição mais útil do modelo foi uma escala explícita de bloqueio. Falhas de *lint* e *typecheck* são bloqueantes sem exceção. Falha em testes é bloqueante mas pode ser desbloqueada por consenso da equipa quando o teste em causa for explicitamente reconhecido como inválido. Falha de *build* é bloqueante sem exceção. Esta clareza evitou debates recorrentes sobre o estatuto de cada falha.

A reflexão da equipa sobre este modelo foi particularmente cuidada, dado que constitui o mecanismo central de garantia de qualidade num cenário em que a quase totalidade do código foi produzida pelo LLM. A questão de como assegurar, de forma sustentada, que o resultado satisfaz os requisitos não é trivial, e a resposta consistiu na articulação de três camadas complementares de validação. Em primeiro lugar, o *gating* automático em CI elimina classes inteiras de problemas mecânicos antes de qualquer revisão humana, libertando o revisor para se concentrar em aspetos semânticos. Em segundo lugar, a revisão humana, ainda que por um único par, ataca precisamente o tipo de erro a que o LLM é mais permeável, designadamente a coerência com regras de negócio que o modelo conhece superficialmente. Em terceiro lugar, a revisão assistida por LLM é utilizada como complemento, nunca como substituto da revisão humana, em particular para detetar duplicações entre módulos e padrões de inconsistência transversais. A montante destas camadas, a granularidade reduzida da implementação descrita na Secção 3.3 já contribui para diminuir o número de erros silenciosos suscetíveis de passar despercebidos. Esta articulação não garante ausência de defeitos, mas, na avaliação da equipa, ofereceu uma confiança razoável de que o código produzido respeita o desenho aprovado, sem impor uma sobrecarga incompatível com o calendário.

A revisão de código estabilizou-se em três camadas complementares.

A primeira camada é a revisão automática conduzida pelo *workflow* de integração contínua já descrito na Subsecção 3.2: a falha em qualquer das suas etapas bloqueia o *merge* e liberta a revisão humana de classes inteiras de erros estáticos.

A segunda camada é a revisão humana, exigida pela política do repositório como pré-condição obrigatória do *merge*. Cada *pull request* requer aprovação por pelo menos um membro da equipa que não seja o autor. A revisão concentrou-se em três aspetos não verificáveis automaticamente, designadamente a aderência às decisões arquiteturais documentadas no Capítulo 2, a correção semântica das regras de domínio e a legibilidade do código novo introduzido.

A terceira camada é a revisão assistida por LLM, adotada como complemento da revisão humana e nunca como substituto. A equipa solicitou ao modelo, em *pull requests* acima de 300 linhas alteradas, uma análise crítica do *diff* com identificação de *smells*, sugestões de simplificação e verificação de aderência aos padrões dos módulos já consolidados. O contributo foi tipicamente útil para identificar duplicações entre módulos, situação que o revisor humano dificilmente detetaria. A taxa de aceitação aproximada foi de 50%. As restantes sugestões foram rejeitadas por colidirem com decisões arquiteturais previamente fixadas que o modelo não tinha presente.

3.6 Detecção de *Code Smells* e *Refactoring*

Análise Crítica do Processo de Geração com IA

Prompt 35. Detecção de *Code Smells* e *Refactoring*

Contexto: Onze modulos do backend ja implementados em apps/api/src/modules, com necessidade de revisao transversal a identificar problemas que escapam a revisao local de pull request.

Objetivo: Identificar os code smells mais frequentes na base de codigo e propor refactoring concreto para cada smell.

Instrucoes de Escrita:

- Aplicar a taxonomia de Fowler (long method, duplicated code, primitive obsession, feature envy, data clump, shotgun surgery, entre outros)
- Para cada smell indicar modulo afectado e proposta de refactoring com exemplo antes/depois
- Privilegiar smells que comprometam a modificabilidade ou a coerencia entre modulos

Restricoes: Sem smells puramente estilisticos. Sem abstracoes prematuras que apresentem mais custo do que beneficio a esta dimensao.

Análise Crítica

Antes de abordar o resultado da interação, importa enquadrar o conceito utilizado. Os *code smells* são indícios de fragilidade no código que, sem constituírem erros funcionais, prejudicam a sua manutenção, evolução ou legibilidade. Foram catalogados por Fowler em [5], e incluem padrões como duplicação de código, métodos demasiado longos, abstrações inadequadas ou acoplamentos excessivos entre componentes. A sua deteção não é trivial, dado que muitos passam despercebidos numa leitura linear e só se tornam evidentes a quem mantém em mente a estrutura global do sistema, o que torna o LLM um auxiliar particularmente útil neste tipo de análise.

O LLM identificou oito categorias de *smell* distintas. A equipa concordou com cinco e contestou três. A sugestão de extrair os literais de configuração financeira para constantes (*magic numbers*) foi rejeitada, dado que estes valores são alterados pela aplicação em *runtime* via *configuration* e não constituem constantes de código. A observação de *primitive obsession* sobre o NIF foi rejeitada por colidir com a decisão arquitetural de usar o NIF como chave primária natural, documentada no Capítulo 2. A recomendação de extrair um *repository abstract base* para encapsular operações comuns foi adiada, por se considerar que a abstração prematura apresentaria mais custo do que benefício à dimensão atual do sistema. As restantes cinco sugestões deram origem a *refactorings* efetivos.

A execução dos *refactorings* seguiu uma abordagem iterativa e assistida pelo modelo. Para cada *smell* aceite, a equipa isolou o segmento de código afetado, pediu ao LLM a transformação pretendida com indicação explícita do padrão alvo, por exemplo extrair um bloco repetido para uma função utilitária partilhada ou separar uma rotina longa em métodos privados coesos, executou a *suíte* de testes antes e depois da alteração e confirmou que o comportamento observável se mantinha intacto. Esta forma de trabalhar revelou-se particularmente adequada à limpeza de código já existente. O LLM mostra-se eficaz em transformações estruturais de pequeno alcance, em que o objetivo é claro e a verificação pode ser delegada à *suíte* de testes, mas a equipa observou que perde eficácia quando o *refactoring* exige compreensão de invariantes não documentadas no código. Nesses casos a equipa preferiu conduzir as alterações manualmente e reservar o modelo para a revisão posterior.

A deteção de *code smells* ocorreu em duas frentes paralelas. A primeira foi pontual, durante a revisão de cada *pull request*, com identificação imediata e correção no mesmo *pull request*. A segunda foi periódica, em duas iterações de revisão transversal sobre toda a base de código, conduzidas no final da segunda e da terceira semana de implementação. As iterações transversais revelaram-se particularmente úteis para identificar duplicações entre módulos, padrões que escapam à revisão local de um único *pull request*.

Os *smells* efetivamente corrigidos pertencem a quatro categorias da taxonomia de Fowler. A primeira é a *duplicação de código*, identificada na verificação de existência de utilizador autenticado, replicada à entrada de cada *handler* dos módulos operacionais. A correção foi a extração da verificação para um *plugin Fastify* aplicado globalmente, com a função *requireRole* a ser invocada explicitamente apenas quando o conjunto de papéis admissíveis varia. A segunda é o *long method*, identificado no método *issueInvoice* do *BillingService* (originalmente 87 linhas, com cálculo de subtotal, IVA e total intercalado com lógica de validação). A correção foi a extração das três funções privadas *computeLaborAmount*, *computePartsAmount* e

`validateBillability`, reduzindo o método principal a 28 linhas e tornando cada componente testável de forma independente.

A terceira é o *data clump*, identificado nos parâmetros recorrentes de paginação e ordenação, replicados em todos os *endpoints* de listagem (`page`, `limit`, `sortBy`, `sortOrder`). A correção foi a extração de um esquema *Zod* `paginationQuerySchema` e de um *handler* de paginação único em `shared/pagination`, reaproveitado pelos onze módulos. A quarta é a *primitive obsession* sobre montantes monetários, originalmente representados por `string` bruta em alguns pontos do *frontend*. A correção foi a introdução de um tipo opaco `Money` em `packages/contracts/src/money.ts`, com construtor validado e operações aritméticas seguras, adotado em todas as fronteiras entre *frontend* e *backend*.

3.7 Métricas de Desenvolvimento

Análise Crítica do Processo de Geração com IA

Prompt 36. Métricas de Desenvolvimento

Contexto: Base de código final do projeto (`apps/api`, `apps/web`, `packages/contracts`) e necessidade de sustentar uma análise crítica honesta do desenvolvimento assistido por LLM.

Objetivo: Extrair e organizar um conjunto de métricas de volume da base de código e estimar a proporção de código com origem direta no LLM.

Instruções de Escrita:

- Indicar linhas de código por pacote e por módulo, número de ficheiros e número de endpoints expostos
- Estimar a proporção LLM vs equipa em termos qualitativos, em alternativa a percentagens precisas
- Discutir as limitações intrínsecas deste tipo de medição

Restrições: Sem percentagens com falsa precisão. Sem inferir génese a partir apenas do histórico Git.

Análise Crítica

O LLM alertou desde logo para a fragilidade da medição automática da proporção *LLM vs humano*. O histórico de *Git* regista o autor do *commit* mas não a génese do código. O código gerado é frequentemente revisto, terminado e refinado pela equipa antes do *commit*. A fronteira entre *escrito* e *revisto* é difusa quando a revisão envolve reformulação extensa. A equipa adotou esta cautela e optou por estimativas qualitativas baseadas na memória dos autores, em alternativa a percentagens precisas que sugeririam falsa exatidão.

A Tabela 3.1 apresenta as métricas de volume da base de código produzida nesta etapa. O total de cerca de 11 000 linhas de *TypeScript* distribuídas por 81 ficheiros sobre três pacotes do *monorepo* situa o sistema na ordem de grandeza típica de um *software* de gestão de pequena a média complexidade.

Tabela 3.1: Métricas de volume da base de código (final da etapa)

Pacote	Ficheiros	LOC	Observação
apps/api (<i>backend</i>)	41	4 795	11 módulos, 70 <i>endpoints</i>
apps/web (<i>frontend</i>)	36	5 954	páginas, componentes, <i>hooks</i>
packages/contracts	4	≈300	esquemas Zod e tipos partilhados
Total	81	≈11 049	

A estimativa qualitativa da equipa, sustentada na memória dos autores e na inspeção do histórico, é inequívoca quanto à autoria do código. A quase totalidade das linhas presentes no sistema final foi diretamente produzida pelo LLM, na ordem dos 99%. A equipa não escreveu, na prática, código de raiz, limitando-se à formulação dos *prompts* de geração, à validação semântica do resultado e ao pedido de *refactorings* subsequentes quando a primeira proposta se revelou insuficiente. Os módulos diferem entre si não pela proporção de autoria humana, mas pela quantidade de iterações de *refactor* requeridas até o resultado ser aceite. Os módulos de catálogo (*customers*, *scooters*, *suppliers*), com estrutura regular e lógica de domínio simples, foram aceites tipicamente à primeira ou segunda iteração. Os módulos com regras de negócio complexas (*billing*, *service-orders*) exigiram quatro a seis iterações até a equipa considerar o código fechado.

Esta repartição obriga a deslocar a leitura das métricas. A medição em LOC, mesmo agregada, não comunica o trabalho efetivamente feito pela equipa. O esforço humano concentrou-se em três pontos não delegáveis ao modelo, designadamente a formulação rigorosa do *prompt* de cada módulo, a verificação semântica do código gerado contra o SRS e os contratos partilhados, e a decisão sobre o que aceitar, o que pedir para reformular e o que rejeitar. Nenhum destes pontos se reflete em linhas de código atribuídas à equipa, mas é deles que depende a qualidade final do sistema.

3.8 Síntese

A etapa concluiu-se com um sistema funcional, versionado e validado por integração contínua, composto por aproximadamente 11 000 linhas de *TypeScript* distribuídas por 81 ficheiros e onze módulos coesos, que cobrem a totalidade dos requisitos funcionais identificados no Capítulo 1. O modelo revelou-se consistentemente forte em tarefas de forma regular e vulnerável em tarefas que exigem juízo de domínio, com a presença do humano no *loop* a ser decisiva nestes últimos casos.

A etapa seguinte do projeto aborda a verificação, validação e avaliação da qualidade do *software* produzido. Recolherá métricas formais de cobertura, executará testes de aceitação contra os casos de uso identificados no Capítulo 1 e instrumentará a análise da qualidade segundo o modelo da norma ISO/IEC 25010.

4 Verificação, Validação e Qualidade do *Software* Assistidas por LLM

O presente capítulo documenta a etapa final do projeto, dedicada à verificação, validação e avaliação da qualidade do *software* produzido. A partir do sistema implementado no Capítulo 3, a equipa confirmou que o produto cumpre a especificação técnica (verificação) e responde às necessidades documentadas dos *stakeholders* (validação), caracterizando-o ainda segundo o modelo ISO/IEC 25010 de atributos de qualidade.

O envolvimento do LLM nesta etapa diferiu das anteriores. A primeira proposta de cada caso de teste foi-lhe delegada, mas a equipa manteve a responsabilidade integral sobre a identificação de cenários limite, sobre o desenho dos fluxos de exceção e sobre a validação crítica de cada *output* gerado. O capítulo documenta os instrumentos adotados, da análise estática transversal à pirâmide automática de testes em quatro camadas (unitária, de integração, de sistema e de aceitação) sobre os 11 módulos do *backend*, complementada por uma camada transversal de qualidade segundo a norma ISO/IEC 25010 e pela validação manual contra os 29 casos de uso identificados no Capítulo 1. Reconhece também as limitações conhecidas, nomeadamente a ausência de testes de carga dedicados, a dependência do agendador automático para o requisito RF-39 e a ausência de validação direta junto dos *stakeholders* reais da oficina.

4.1 Estratégia de Verificação e Validação

Análise Crítica do Processo de Geração com IA

Prompt 37. Estratégia de Verificação e Validação

Contexto: Sistema implementado no Capítulo 3 (monolito modular em TypeScript, 11 módulos do backend, frontend em React, PostgreSQL via Prisma, integracao continua sobre Node.js 20), horizonte temporal limitado e restricoes legais nao negociaveis (faturacao sequencial, RGPD).

Objetivo: Propor uma estrategia de V&V adequada, distinguindo verificacao (sistema construido corretamente) de validacao (sistema correto a ser construido).

Instruções de Escrita:

- Indicar tipos de testes a adotar, níveis em que se aplicam, ferramentas e critérios de bloqueio em CI
- Discutir o papel do LLM como gerador de testes face ao esforço de revisão subsequente
- Estruturar a estratégia em camadas claras com responsabilidades disjuntas

Restrições: Sem testes ponta-a-ponta automáticos sobre o frontend.
Sem ferramentas que introduzam fragilidade desproporcionada.

Análise Crítica

A proposta do LLM partiu da pirâmide clássica de testes, com testes unitários em maior número, testes de integração em quantidade intermédia e testes ponta-a-ponta em quantidade reduzida. A equipa refinou duas opções. Substituiu os testes ponta-a-ponta automáticos sobre o *frontend*, cuja instrumentação introduziria fragilidade e custo de manutenção desproporcionados, por uma camada de testes de sistema ao nível da API. Acrescentou ainda uma camada transversal dedicada aos atributos não funcionais da norma ISO/IEC 25010. A contribuição mais útil do modelo foi a distinção operacional entre verificação e validação, que estruturou a divisão das subsecções que se seguem.

A experiência da equipa ao longo desta etapa fixou uma observação importante sobre o comportamento do LLM enquanto gerador de testes. O modelo revela-se particularmente eficaz na produção rápida de testes para comportamentos isolados e situações simples, em que a verificação se resume a confirmar valores de retorno ou estados imediatos. Esta competência diminui sensivelmente quando se trata de testar comportamentos encadeados, designadamente sequências de operações em que o resultado de um passo condiciona o seguinte ou em que regras de negócio só se manifestam pela interação entre módulos. Mesmo nos casos em que o LLM afirma estar a cobrir cenários encadeados, a inspeção atenta revela que essa cobertura é, em regra, superficial, com encadeamentos limitados a dois ou três passos e sem testar verdadeiramente as condições limite que tornam esses cenários relevantes. Os *edge cases* mais delicados, por dependerem de relações que exigem leitura cuidada do código, foram tipicamente omitidos pelo modelo e introduzidos pela equipa em iterações de complemento.

A estratégia adotada articulou-se em duas frentes complementares. A verificação assentou na análise estática transversal (verificação de tipos, regras de estilo e validação de esquemas em fronteira da API) e numa pirâmide automática de quatro camadas (unitária, de integração, de sistema e de aceitação), executada sobre os 11 módulos do *backend*. A validação completou-se pela execução manual dos 29 casos de uso identificados na Subsecção 1.4. A toda esta arquitetura sobrepôs-se uma camada transversal de testes dedicada aos atributos não funcionais da norma ISO/IEC 25010, designadamente segurança e desempenho. Em fase de desenho fixaram-se igualmente os limiares mínimos de cobertura aceitável, designadamente 70 % de linhas, 60 % de ramos e 75 % de funções, condição necessária para a aceitação da *suite* produzida.

A divisão das subsecções seguintes reflete a distinção operacional entre verificação (Subsecção 4.2) e validação (Subsecção 4.3), com a avaliação da qualidade da própria *suite* de testes reservada à Subsecção 4.4.

4.2 Verificação

Análise Crítica do Processo de Geração com IA

Prompt 38. Arquitetura de Testes de Verificação

Contexto: Sistema implementado no Capítulo 3 (Fastify+TypeScript, 11 módulos do backend, PostgreSQL via Prisma, esquemas Zod em packages/contracts) e estratégia de V&V definida em 4.1.

Objetivo: Propor uma arquitetura de testes automáticos de verificação em três camadas (unitária, integração, sistema).

Instruções de Escrita:

- Para cada camada indicar âmbito coberto, test runner adotado, forma de exercer a API (mocks vs. instância real), tratamento da persistência e reprodução de sessão autenticada
- Identificar invariantes de domínio que merecem teste explícito
- Listar os anti-padrões a evitar na geração por LLM (mocks de Prisma, requisitos fabricados, esquemas Zod paralelos)

Restrições: Sem mocks do Prisma. Sem requisitos não documentados no SRS. Sem dependências adicionais quando o test runner nativo for suficiente.

Análise Crítica

A resposta do LLM convergiu para o *test runner* nativo `node:test` executado via `tsx --test`, dispensando dependências adicionais como *Vitest* ou *Jest*, e propôs `app.inject()` do *Fastify* como cliente HTTP nos testes, eliminando a necessidade de abrir portos e reduzindo o tempo de execução face a alternativas como *supertest*. Recomendou a validação contra base de dados *PostgreSQL* real, sem *mocks* do *Prisma*, opção que cobre o comportamento efetivo do *schema*, das migrações e das restrições relacionais. A contribuição mais relevante foi a fixação dos anti-padrões, designadamente a proibição de *mocks* do *Prisma*, a proibição de fabricar requisitos não documentados no SRS, e a obrigação de importar esquemas *Zod* do pacote `@gengis-khan/contracts` em vez de declarar variantes paralelas nos testes.

A análise estática constituiu a primeira camada de verificação, tornada bloqueante em integração contínua. Três mecanismos foram combinados. O compilador *TypeScript* em modo *strict*

garante coerência de tipos em toda a base de código, eliminando classes inteiras de defeitos antes da execução. O *ESLint* aplica regras de estilo e deteta padrões propensos a defeito como variáveis não usadas, comparações soltas e importações redundantes. A validação *Zod* em fronteira da API rejeita pedidos malformados antes de qualquer regra de negócio ser invocada, materializando o princípio de validação em fronteira.

A pirâmide automática cobre o *backend* em quatro camadas executadas sequencialmente em integração contínua, conforme se ilustra na Figura 4.1 e cujo inventário se detalha na Tabela 4.1. A camada unitária reúne 32 testes sobre lógica isolada, designadamente os validadores *Zod* do pacote de contratos partilhados, os utilitários de autorização *requireRole*, o construtor de erros uniforme *appError* e o gestor de chaves de idempotência. A camada de integração reúne cerca de 90 testes distribuídos por um ficheiro por módulo, que exercem o contrato HTTP de cada módulo via *app.inject()* contra uma instância *PostgreSQL* real, com limpeza e repovoamento do esquema em cada *beforeEach*. A camada de sistema contém quatro testes que encadeiam vários módulos numa sequência realista, designadamente o ciclo completo de reparação (receção, diagnóstico, intervenção, faturação) e o ciclo de aprovisionamento e consumo de *stock*, validando os efeitos colaterais entre subsistemas. A camada de aceitação, mapeada às *user stories* do SRS, é tratada na Subsecção 4.3.

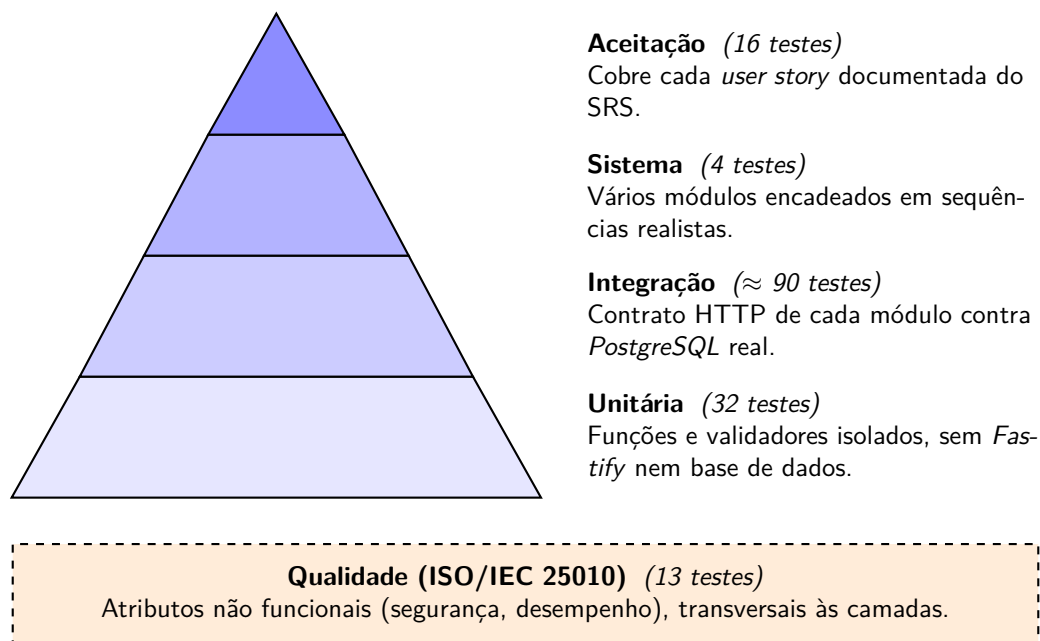


Figura 4.1: Pirâmide de testes automáticos adotada pela equipa, complementada por uma camada transversal de qualidade que avalia atributos não funcionais segundo o modelo ISO/IEC 25010. Cada etiqueta indica o nome da camada, a contagem efectiva de testes e a função desempenhada.

Tabela 4.1: Inventário da *suite* de testes automáticos

Camada	Ficheiros	Testes	Âmbito
<i>Smoke</i>	1	2	Disponibilidade do servidor e <i>gate</i> de autenticação
Unitária	3	32	<i>Helpers</i> , idempotência, esquemas <i>Zod</i>
Integração	11	≈90	Contrato HTTP por módulo contra <i>PostgreSQL</i> real
Sistema	2	4	Sequências realistas entre módulos
Aceitação	2	16	<i>User stories</i> do SRS (Subsecção 4.3)
Qualidade	2	13	Atributos ISO/IEC 25010 (Subsecção 4.4)
Total	21	157	Cobertura completa da API

A execução completa da *suite* produziu uma taxa de aprovação de 100% e uma cobertura agregada de 85,8% de linhas, 79,3% de ramos e 82,6% de funções, valores acima dos limites mínimos fixados na Subsecção 4.1 (70%, 60% e 75% respetivamente). Os módulos `shared/auth/session`, `shared/db/prisma`, `shared/idempotency/store`, a rota `health` e a totalidade dos esquemas *Zod* em `packages/contracts` atingiram cobertura integral. Os módulos com cobertura inferior foram `suppliers-procurement` (66%), `service-orders` (71%) e `reports` (82%), por ausência de teste sobre *endpoints* avançados como aprovação de orçamento e geração automática de encomendas a partir de *stock* mínimo. Estas lacunas estão documentadas no ficheiro `docs/srs-traceability.md` do repositório e sinalizadas como prioridade de evolução posterior do sistema.

4.3 Validação

Análise Crítica do Processo de Geração com IA

Prompt 39. Procedimento de Validação contra o SRS

Contexto: SRS com 16 user stories (8 do gerente, 8 do mecânico) e 29 casos de uso da Subsecção 1.4, sistema implementado e disponível em ambiente local.

Objetivo: Desenhar um procedimento de validação que demonstre que o sistema implementado responde efectivamente aos requisitos elicitados, distinguindo validação automática de validação manual.

Instruções de Escrita:

- Estruturar a validação automática como testes de aceitação mapeados um-para-um a user story, executados em CI

- Estruturar a validação manual como percurso de cada UC contra o sistema corrente em ambiente local
- Manter rastreabilidade explícita entre cada teste e o requisito que valida (ficheiro de rastreabilidade)
- Redigir os testes de aceitação em linguagem próxima da do SRS, evitando jargão de implementação

Restrições: Sem mistura de validação com verificação. Sem dependência de stakeholders externos no horizonte académico.

Análise Crítica

O LLM propôs a estruturação dos testes de aceitação como um ficheiro por papel de utilizador, com cada `it()` correspondente a uma *user story* singular. A equipa adotou esta estrutura e complementou-a com a manutenção de um ficheiro `docs/srs-traceability.md` no repositório, que documenta a correspondência entre requisitos funcionais, *user stories* e testes. A contribuição mais relevante foi a sugestão de redigir os testes de aceitação em linguagem deliberadamente orientada ao perfil de *stakeholder*, evitando jargão de implementação e privilegiando expressões próximas das que aparecem no SRS. Para a validação manual, o modelo propôs uma matriz de UC contra resultado observado, opção adotada pela equipa pela transparência que confere ao processo. A limitação mais relevante face à proposta do LLM foi a impossibilidade de conduzir uma sessão de validação com *stakeholders* reais no horizonte do calendário académico.

A validação automática concretizou-se numa camada de testes de aceitação que estabelece correspondência um-para-um com as *user stories* do SRS. Os 16 testes cobrem respetivamente as 8 *user stories* do papel manager (US-G01 a US-G08) e as 8 do papel mechanic (US-M01 a US-M08), com a escrita deliberadamente orientada à legibilidade pelo perfil de *stakeholder* que representa o utilizador final do negócio. A correspondência completa entre *user stories*, testes e requisitos funcionais consta do ficheiro `docs/srs-traceability.md` do repositório.

A validação manual completou o procedimento. Os elementos da equipa percorreram, em ambiente local com base de dados real, os 29 casos de uso identificados na Subsecção 1.4, atribuindo a cada um o resultado observado. Recebeu particular atenção a máquina de estados das ordens de serviço, cujo conjunto de transições admissíveis foi exercido em todas as direções, incluindo a transição inversa de Em Reparação para Aguarda Peças discutida na Secção 2.4. O fluxo de faturação foi também percorrido na sua totalidade, da geração do número sequencial à emissão do PDF, validando a conformidade com o regime de identificação contínua de documentos fiscais.

As restrições legais identificadas na Subsecção 1.2.6.2 foram validadas estruturalmente, designadamente a faturação sequencial do Decreto-Lei n.º 28/2019 pela propriedade de autoincrement do *PostgreSQL* e o regime RGPD pelo conjunto de *cookie HttpOnly*, validação *Zod* sobre dados pessoais e não exposição de identificadores opacos a sistemas externos.

A validação descrita esgota o que é verificável a partir do SRS e dos 29 casos de uso identificados. Uma confirmação adicional, em contexto operacional real, dependeria da implantação efetiva do sistema, fase que extravasa o âmbito desta etapa.

4.4 Avaliação da Qualidade dos Testes

Análise Crítica do Processo de Geração com IA

Prompt 40. Avaliação da Qualidade da *Suite* de Testes

Contexto: Suite de 157 testes implementados nas Subsecções 4.2 e 4.3, resultados de execução em integração contínua (taxa de aprovação, cobertura, tempo) e ficheiro docs/srs-traceability.md.

Objetivo: Avaliar a qualidade da própria suite enquanto artefacto de V&V, identificando resultados sólidos e fragilidades.

Instruções de Escrita:

- Aplicar dimensões qualitativas e quantitativas: cobertura de código, cobertura de requisitos, equilíbrio entre camadas, estabilidade, tempo de execução, profundidade dos cenários
- Distinguir cobertura de código (o que o teste atravessa) de cobertura de requisitos (o que o teste valida)
- Identificar lacunas significativas a sinalizar como prioridade de evolução

Restrições: Não misturar a qualidade dos testes com a qualidade do sistema testado. Sem reprodução redundante de tabelas já apresentadas.

Análise Crítica

A primeira proposta do LLM concentrou-se em dimensões quantitativas, designadamente cobertura agregada e contagem de testes por camada, reproduzindo o que já constava da Subsecção 4.2. Após reformulação do *prompt* para exigir dimensões qualitativas adicionais (equilíbrio entre camadas, profundidade dos cenários, mapeamento explícito a requisitos), o modelo convergiu para a grelha aqui apresentada. A contribuição mais relevante foi a distinção entre cobertura de código (o que o teste atravessa) e cobertura de requisitos (o que o teste valida), distinção que se mantém explícita no quadro avaliativo.

A qualidade de uma *suite* de testes ultrapassa a mera contagem de casos. Os 157 testes documentados na Subsecção 4.2 ganham significado avaliativo apenas quando confrontados

com dimensões observáveis que reflitam o seu valor enquanto instrumento de verificação. A Tabela 4.2 apresenta essa avaliação ao longo de seis dimensões.

Tabela 4.2: Avaliação da qualidade da *suite* de testes

Dimensão	Resultado	Comentário
Cobertura de código	85,8 % linhas, 79,3 % ramos, 82,6 % funções	Acima dos limiares fixados na Subsecção 4.1 (70 %, 60 %, 75 %); três módulos abaixo do limiar sinalizados como prioridade.
Cobertura de requisitos	43 de 44 RF cobertos (97,7 %)	Mapeamento explícito em docs/srs-traceability.md; apenas RF-39 (agendador automático) por testar.
Equilíbrio entre camadas	32 unitários, 90 integração, 4 sistema, 16 aceitação	Distribuição aproximada à pirâmide canónica, com base ampla na integração e topo afilado na aceitação.
Estabilidade	100 % aprovação em execuções consecutivas	Sem <i>flaky tests</i> detetados, com isolamento por <code>beforeEach</code> e repovoamento do esquema a garantirem reprodutibilidade.
Tempo de execução	28,6 segundos para 157 testes em CI	Compatível com execução por cada <i>pull request</i> sem penalização do ritmo de revisão.
Profundidade dos cenários	Validação <i>Zod</i> , transições inválidas, autorização, idempotência	Cobertura não restrita ao <i>fluxo normal</i> ; inclui os principais códigos de erro do envelope HTTP.

A leitura conjunta destas dimensões sustenta uma avaliação positiva da *suite* produzida. A cobertura agregada excede os limiares mínimos, a rastreabilidade requisito-teste atinge quase totalidade, o tempo de execução é compatível com integração contínua e a distribuição entre camadas aproxima-se da forma piramidal canónica. As fragilidades observáveis concentram-se no requisito RF-39, cuja verificação automática exigiria instrumentação temporal específica não implementada, e nos *endpoints* avançados de três módulos com cobertura de linhas inferior a 75 %. Estas fragilidades são tratadas como prioridade de evolução na Subsecção 4.6.

A reflexão sobre a génese desta *suite*, designadamente o papel do LLM como gerador e o esforço de revisão humana necessário para alcançar o nível de qualidade aqui descrito, é apresentada na Subsecção 4.5.

4.5 V&V Assistida por LLM

Análise Crítica do Processo de Geração com IA

Prompt 41. Geração da *Suite* de Testes por LLM

Contexto: Template de teste estabelecido em `apps/api/src/app.test.ts`, esquemas *Zod* em `packages/contracts/src/<modulo>` e requisitos do SRS rastreáveis.

Objetivo: Gerar o ficheiro `<modulo>.test.ts` cobrindo o fluxo normal e os erros de validação principais de cada endpoint, com rastreabilidade explícita ao requisito do SRS.

Instruções de Escrita:

- Seguir o template de `apps/api/src/app.test.ts` e o padrão de `setup` por `beforeEach`
- Importar os esquemas *Zod* do pacote de contratos partilhados (sem criar esquemas paralelos)
- Usar `app.inject()` do *Fastify*, *PostgreSQL* real e *JWT* assinado diretamente
- Identificar regras de domínio críticas que merecem teste explícito (transições de estado, controlo de stock, faturação)
- Marcar cada `it()` com a referência ao requisito que valida

Restrições: Sem *supertest*. Sem *mocks* de *Prisma*. Sem requisitos fabricados nem campos omitidos do contrato. Sem cenários apenas de fluxo normal quando o `prompt` enumera erros.

Análise Crítica

O LLM revelou competência elevada na geração de ficheiros de testes fiéis ao *template* estabelecido, integrando esquemas *Zod* do pacote de contratos partilhados, padrão de *setup* por `beforeEach` e rastreabilidade explícita ao requisito. A pressão exercida pelo *prompt* sobre o uso de instância *PostgreSQL* real e a proibição de *mocks Prisma* foi respeitada em todos os módulos. As falhas observadas concentraram-se em dois padrões. Primeiro, omissão silenciosa de campos do contrato, ilustrada pelo caso do DTO de `PurchaseOrder` em que o atributo `status` foi inicialmente omitido pelo modelo, produzindo dois testes que passavam mas validavam um contrato incompleto. A divergência foi detetada por revisão humana antes da integração e corrigida por iteração. Segundo, propensão para escrever apenas o *fluxo normal* mesmo quando o *prompt* enumerava cenários de exceção, o que exigiu pedidos de complemento dirigidos para garantir cobertura efetiva dos casos de falha.

A geração da *suite* de testes concentrou no LLM cerca de 99% do código produzido, em coerência com a proporção de autoria identificada para a implementação no Capítulo 3. A intervenção humana incidiu sobre três frentes não delegáveis. A primeira foi a formulação do *prompt* inicial, que fixou ferramentas, padrões e anti-padrões com precisão suficiente para o modelo convergir para um *output* integrável na primeira ou segunda iteração; o documento `docs/test-strategy.md` do repositório formaliza este *prompt*. A segunda foi a revisão crítica de cada ficheiro gerado contra os contratos efetivamente expostos pela API, etapa em que

defeitos como o do DTO de `PurchaseOrder` foram apanhados. A terceira foi a enumeração explícita de cenários de exceção que o modelo tendia a omitir.

A repartição efetiva confirma o padrão já documentado na implementação. O modelo é eficaz como gerador disciplinado quando o *prompt* fixa restrições estruturais com clareza, mas mantém-se vulnerável a omissões silenciosas que apenas revisão humana atenta consegue detetar. A V&V assistida por LLM exige, em consequência, supervisão presente e não delegação cega.

4.6 Síntese e Prioridades de Evolução

A etapa de V&V fechou com uma *suite* de 157 testes automáticos distribuídos por seis camadas, taxa de aprovação de 100% em integração contínua, cobertura agregada de 85,8% de linhas (acima do limiar mínimo de 70% fixado na Subsecção 4.1) e mapeamento explícito contra as oito características do modelo de qualidade do produto da norma ISO/IEC 25010. A conformidade com as restrições legais não negociáveis, designadamente a faturação sequencial do Decreto-Lei n.º 28/2019 e o regime RGPD, foi validada por evidência estrutural no *schema* relacional e na fronteira de autenticação.

Três prioridades de evolução posterior do sistema ficam sinalizadas. A cobertura dos *endpoints* avançados nos módulos *suppliers-procurement*, *service-orders* e *reports* merece elevação acima do limiar de 70% de cobertura de linhas. A introdução de testes de carga dedicados e de testes de segurança em profundidade (*fuzzing*, análise de dependências) reforçaria a avaliação de fiabilidade e de segurança em cenários adversariais. Finalmente, o teste explícito do agendador automático responsável pelo requisito RF-39, relativo à notificação automática de ordens em Aguarda Peças há mais de dez dias, exige instrumentação temporal específica que não foi implementada nesta etapa.

A repartição do esforço observada confirma o padrão já documentado na implementação, com o LLM como gerador da quase totalidade do código de testes e a equipa concentrada na formulação do *prompt*, na revisão crítica do *output* e na deteção de omissões silenciosas. A V&V assistida por LLM aqui descrita demonstra que a delegação ao modelo, quando enquadrada por *prompt* estruturado e supervisão presente, é compatível com cobertura e qualidade defensáveis. Estes resultados serão integrados na reflexão final do relatório.

5 Conclusão e Trabalho Futuro

O presente relatório documentou um projeto académico que foi, na sua essência, tanto um exercício de engenharia de *software* como um estudo metódico sobre a utilização de modelos de linguagem de grande dimensão no ciclo completo de desenvolvimento. As quatro etapas atravessaram a conceção e engenharia de requisitos, a arquitetura e o desenho, a implementação, e a verificação, validação e avaliação da qualidade do sistema das *Oficinas Gengis Khan*. Em cada uma dessas fases foram registadas, de forma transparente, as interações conduzidas com o LLM, os artefactos produzidos e as decisões de aceitação, correção ou rejeição tomadas pela equipa. A presente conclusão sintetiza as observações mais relevantes desse percurso, fixa as limitações que importa não esquecer e abre uma reflexão final sobre o que esta experiência sugere acerca do futuro próximo do desenvolvimento de *software*.

5.1 Síntese das Capacidades Observadas

Ao longo do projeto, o LLM evidenciou um conjunto de competências que justificam, por si só, a sua adoção sistemática. A primeira, transversal a quase todas as etapas, é a notável rapidez na extração, classificação e correlação de informação dispersa por documentos extensos, designadamente entrevistas, registos físicos e diagramas. Onde a leitura humana tradicional exigiria várias passagens até identificar as ligações relevantes, o modelo procedeu em segundos, devolvendo associações coerentes e tipicamente fiáveis quando confrontadas com a fonte original.

A segunda competência é a capacidade de argumentação técnica em torno de decisões consensuais. Nas discussões de arquitetura, em particular na escolha do padrão arquitetural, da *stack* tecnológica, da estratégia de partilha de contratos e do modelo de persistência, o LLM produziu fundamentações estruturadas, articulou *tradeoffs* entre alternativas e sustentou cada opção com vantagens concretas. A análise relativa à uniformização linguística do projeto, que conduziu ao abandono da hipótese inicial de *C#* no *backend* a favor de *TypeScript* em ambas as camadas, foi uma das contribuições mais relevantes da etapa de arquitetura e exemplifica bem este perfil de utilidade.

A terceira competência é a produtividade na geração de código sob restrições estruturais claras. A combinação de uma estratégia incremental, com cada módulo decomposto em blocos pequenos e validados isoladamente, com a referência a módulos já consolidados como exemplos de estilo, permitiu que a equipa obtivesse a quase totalidade do código do sistema a partir de

prompts cuidadosamente formulados. A taxa de aceitação observada em tarefas de *boilerplate* estrutural e na escrita de esquemas *Zod* foi consistentemente elevada, e a abordagem mostrou-se essencial para preservar a coerência e a verificabilidade do código produzido.

Por fim, importa registar o papel transformador das *Agent Skills* disponibilizadas para tarefas técnicas especializadas. A instalação de uma *Skill* dedicada à geração de diagramas *TikZ* elevou substancialmente a qualidade dos artefactos visuais do relatório, e ilustra como a disponibilização de contexto adequado pode estender o desempenho do modelo a domínios em que, sem assistência específica, ficaria aquém das exigências.

5.2 Limitações e Cuidados a Manter

A par destas capacidades, a equipa observou limitações que importa documentar com igual nitidez. A mais frequente e, em certo sentido, a mais subtil, residiu na geração de texto em português. O modelo produz, sem dificuldade, volume substancial de prosa formalmente correta, mas tende a recorrer a calques diretos do inglês, a repetir formulações e a introduzir erros ortográficos quando confrontado com vocabulário menos comum. Em vários momentos do relatório, o *output* bruto continha termos como “comitar” (de *commit*) ou “merjar” (de *merge*), expressões inexistentes em português, e revelou uma propensão para traduzir literalmente estruturas fráscas inglesas que comprometem a fluidez do texto em PT-PT. A revisão humana atenta foi essencial em todos os casos, e a equipa concluiu que confiar a redação de prosa académica ao LLM sem revisão cuidadosa produz inevitavelmente texto coeso à primeira vista mas semanticamente fraco ou linguisticamente desadequado.

Uma segunda limitação manifestou-se na engenharia de requisitos, em que o modelo demonstrou o desempenho menos consistente de toda a Etapa 1. A geração da matriz de levantamento de requisitos exigiu intervenção humana mais densa do que em qualquer outra interação, com o modelo a omitir requisitos diretamente sustentados nas fontes documentais e a granular excessivamente outros itens, fragmentando uma única necessidade em múltiplos requisitos redundantes. A revisão exaustiva contra a totalidade das fontes foi essencial para chegar a uma versão consolidada coerente.

Uma terceira limitação, talvez a mais perigosa do ponto de vista metodológico, reside na capacidade do modelo de produzir argumentação persuasiva mesmo quando a recomendação é parcialmente desadequada. O LLM articula com fluência e clareza, o que cria a impressão de que qualquer resposta faz sentido, e a equipa observou em mais do que uma ocasião que propostas elegantemente fundamentadas se revelaram, à inspeção atenta, desligadas das restrições reais do projeto. A prática que se estabilizou consistiu em interpor sempre uma fase de discussão interna entre a leitura da resposta e a sua incorporação no relatório, e em validar ponto por ponto cada afirmação concreta, em particular as que envolvem nomes de bibliotecas, comportamentos esperados ou compatibilidades com o restante do sistema.

Acresce uma limitação operacional, observada sobretudo na geração de testes. O modelo produz com facilidade casos de teste para o *fluxo normal* e para erros de validação simples, mas tende a apresentar cobertura superficial em cenários encadeados, com sequências limitadas a dois ou três passos e sem instrumentar verdadeiramente as condições limite que tornam esses cenários relevantes. Os *edge cases* mais delicados foram tipicamente omitidos pelo modelo e introduzidos pela equipa em iterações de complemento. A estratégia adotada foi, por conseguinte, considerar o *output* do modelo como uma base inicial sólida mas nunca como um conjunto final, e manter a responsabilidade humana sobre a cobertura efetiva do espaço de comportamentos.

Importa também registar uma observação metodológica recolhida ao longo da etapa de implementação. A decomposição do trabalho em módulos pequenos e independentemente validáveis revelou-se condição necessária para a fiabilidade do código gerado. Pedidos demasiado abrangentes ao modelo conduziam invariavelmente a omissões, simplificações silenciosas de regras de negócio ou inconsistências entre partes do código. A granularidade fina do *prompt*, com âmbito estrito por iteração, manteve-se durante todo o projeto e mostrou-se inseparável da apreciação positiva que a equipa retira do código resultante.

5.3 Reflexão sobre o Ecosistema Emergente

A experiência aqui descrita sugere, em juízo da equipa, que se assiste ao início de uma transformação no desenvolvimento de *software* comparável, em escala e em consequências, à que o surgimento do motor de pesquisa Google introduziu no acesso à informação. A integração do LLM ao longo das quatro etapas, e em particular o recurso progressivo a ferramentas especializadas em torno do mesmo agente, das *Agent Skills* dedicadas a diagramas até à recém-disponibilizada *Claude Design* para protótipos de interface, mostra que o desenvolvimento dos modelos não se esgota no próprio modelo. Em torno de cada agente competitivo, está em construção um ecossistema de capacidades acessórias, integrações e fluxos de trabalho que estendem progressivamente o domínio em que o modelo é útil.

A hipótese que a equipa retira desta observação é que o panorama futuro tenderá para a consolidação de um ou dois agentes principais por nicho, em torno dos quais as restantes ferramentas se reorganizarão. As aplicações que hoje conhecemos como distintas tenderão a integrar-se preferencialmente com o ecossistema do agente dominante na sua área, e o exercício de desenvolvimento passará menos por escolher cada ferramenta independentemente e mais por escolher a constelação coerente que melhor responda ao problema. Esta evolução comporta vantagens evidentes em fluidez e em compatibilidade entre ferramentas, mas também riscos óbvios de concentração e de dependência, que importa acompanhar criticamente. A escolha do *Stitch AI* para a fase de *design* desta etapa, e o reconhecimento posterior de que o *Claude Design* teria oferecido melhor integração com a restante *stack* assistida do projeto, ilustram em pequena escala este fenómeno.

Sobreposto a este movimento de consolidação, importa, contudo, sinalizar um perigo real que a equipa observou em alguns momentos do projeto. A facilidade com que o LLM produz código aparentemente coerente cria a tentação de aceitar cada proposta sem o devido escrutínio crítico. Quando essa tentação prevalece e os pequenos erros são deixados passar, o seu efeito cumulativo acaba por se traduzir numa plataforma internamente inconsistente. O modelo, ao perder o contexto entre iterações longas, recria com frequência funcionalidades que já existiam noutras partes do código ou propõe vias alternativas para resolver problemas que estavam resolvidos, introduzindo duplicação silenciosa que só se revela mais tarde, quando a manutenção exige uma compreensão global que entretanto se esfumou.

A salvaguarda contra esta deriva, à luz da experiência aqui descrita, consiste em documentar com clareza o que está a ser feito e em definir, à partida, um plano concreto que oriente cada interação. A prática que, no jargão recente, se tem popularizado sob a designação de *vibe coding*, na qual o programador formula pedidos sucessivos ao modelo sem qualquer estrutura prévia e aceita o que é gerado pela sua aparência funcional, não se afigura viável a médio prazo. A acumulação de pequenas decisões não fundamentadas conduz necessariamente a uma base de código frágil, redundante e difícil de evoluir. Preocupa especialmente a equipa o facto de este padrão ter sido, no decurso da pesquisa que enquadrou o projeto, um dos mais visíveis na literatura de divulgação, o que leva a antecipar a produção, em escala considerável, de *software* desenvolvido sob esta lógica e potencialmente exposto a problemas estruturais que se manifestarão apenas após a entrega.

Cabe, por isso, registar uma conclusão de natureza contrária ao que essa prática sugere. O presente relatório procura ilustrar que existe um caminho alternativo, em que a utilização do LLM, enquadrada por planeamento, decomposição modular e revisão crítica permanente, conduz a um sistema funcional, sem erros estruturais significativos, com duplicação reduzida ou inexistente e cuja qualidade se aproxima da que seria expectável de um desenvolvimento conduzido integralmente por humanos. Esta possibilidade existe, mas não é automática. Depende inteiramente da disciplina com que o ecossistema é utilizado.

5.4 Trabalho Futuro

O sistema produzido cumpre as funcionalidades especificadas e foi submetido a uma *suite* de testes que valida o seu comportamento nominal, mas a equipa reconhece que a instrumentação da Etapa 4 ficou aquém do que o sistema mereceria. Em particular, a cobertura efetiva de cenários complexos e de condições limite poderia ser substancialmente aprofundada, e seria desejável complementar a *suite* atual com testes de carga dedicados, que permitam caracterizar o comportamento do sistema sob volumes operacionais próximos dos projetados pela oficina. A presença das funcionalidades não dispensa, em rigor, a sua verificação sob pressão, e este é o aspeto da Etapa 4 que ficou por concretizar e que justificaria, em trabalho futuro, prioridade clara.

Para além desta consolidação de testes, três frentes adicionais merecem registo como continuação natural do projeto. A primeira é a validação direta com os *stakeholders* reais da oficina, que extravasou o âmbito desta etapa por razões temporais e que constitui o passo necessário entre o sistema verificado e o sistema efetivamente útil em contexto operacional. A segunda é a instrumentação do agendador automático responsável pelo requisito RF-39, cuja verificação exige cenários temporais específicos que não foram implementados. A terceira é a auditoria de segurança em profundidade, designadamente a aplicação sistemática de *fuzzing* e a análise de dependências, complementando a base de validações estruturais já efetuada.

A conclusão deste trabalho deixa, sobretudo, um registo metodológico. A utilização de LLMs no ciclo de vida de desenvolvimento de *software* pode ser substancialmente produtiva quando enquadrada por uma disciplina explícita de *prompt engineering*, por uma estratégia incremental de validação em blocos pequenos e por uma cultura de verificação humana atenta que recuse aceitar respostas apenas pela sua aparência de rigor. Sob estas condições, o LLM revela-se um interlocutor técnico de valor, capaz de estruturar raciocínio, expor alternativas, gerar artefactos e propor refinamentos. Sem estas condições, produz texto e código formalmente plausíveis mas semanticamente frágeis, e a sua adoção converte-se em risco em vez de vantagem. O equilíbrio entre uma coisa e outra é, em última análise, o aspeto mais relevante que este projeto procurou registar.

Referências

- [1] Sommerville, I. (2016). *Software engineering* (10.^a ed.). Pearson Education.
- [2] Institute of Electrical and Electronics Engineers. (1998). *IEEE recommended practice for software requirements specifications* (IEEE Std 830-1998). IEEE.
- [3] International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers. (2018). *Systems and software engineering, life cycle processes, requirements engineering* (ISO/IEC/IEEE 29148:2018). ISO.
- [4] Nielsen, J. (1994). Enhancing the explanatory power of usability heuristics. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (CHI '94) (pp. 152–158). ACM. <https://doi.org/10.1145/191666.191729>
- [5] Fowler, M. (2018). *Refactoring: Improving the design of existing code* (2.^a ed.). Addison-Wesley.
- [6] Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9.^a ed.). McGraw-Hill Education.
- [7] Parlamento Europeu & Conselho da União Europeia. (2016). Regulamento (UE) 2016/679 do Parlamento Europeu e do Conselho, de 27 de abril de 2016, relativo à proteção das pessoas singulares no que diz respeito ao tratamento de dados pessoais (Regulamento Geral sobre a Proteção de Dados). *Jornal Oficial da União Europeia*, L 119, 1–88. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- [8] Decreto-Lei n.º 28/2019, de 15 de fevereiro. (2019). *Diário da República*, 1.^a série, (33), 1106–1130. Autoridade Tributária e Aduaneira.

Glossário

O glossário que se segue reúne e define os principais termos técnicos, conceitos metodológicos e referências regulamentares utilizados ao longo do relatório. Complementa a *Lista de Siglas e Acrónimos*, fornecendo definições extensas e contextualizadas que facilitam a leitura autónoma do documento por um público não familiarizado com a totalidade da *stack* tecnológica adotada, com as normas de engenharia de requisitos invocadas ou com o vocabulário específico do domínio da manutenção de trotinetes elétricas.

Termo	Descrição
ACID	Conjunto de propriedades (Atomicidade, Consistência, Isolamento, Durabilidade) que garantem a fiabilidade das transações numa base de dados relacional.
ADR	<i>Architecture Decision Record</i> . Documento curto que regista uma decisão arquitetural, o contexto em que foi tomada, as alternativas consideradas e as respetivas consequências.
API REST	<i>Application Programming Interface</i> que segue o estilo arquitetural <i>REpresentational State Transfer</i> , expondo recursos através de métodos HTTP (GET, POST, PUT, DELETE).
ASP.NET	<i>Framework</i> da Microsoft para desenvolvimento de aplicações <i>web</i> sobre .NET. Referenciada apenas a título comparativo; não é utilizada no projeto.
AT	Autoridade Tributária e Aduaneira. Entidade pública portuguesa responsável pela fiscalização tributária, destinatária dos ficheiros <i>SAF-T (PT)</i> .
Backend	Parte do sistema responsável pela lógica de negócio, persistência e exposição de APIs, executando-se no servidor. No presente projeto corresponde ao serviço Fastify em Node.js.
Backup	Cópia de segurança de dados efetuada com o intuito de permitir a recuperação em caso de falha, corrupção ou perda acidental.
bcryptjs	Biblioteca JavaScript que implementa o algoritmo <i>bcrypt</i> para <i>hashing</i> seguro de palavras-passe, usada na autenticação do sistema.
Casos de Uso	Técnica de modelação que descreve interações entre atores e o sistema para alcançar um objetivo. Suporta relações <i>include</i> , <i>extend</i> e <i>generalização</i> .

continua na página seguinte

Termo	Descrição
CI/CD	<i>Continuous Integration / Continuous Deployment</i> . Conjunto de práticas que automatizam a integração, teste e publicação de alterações ao código-fonte.
Claude	Família de modelos de linguagem desenvolvida pela Anthropic, utilizada como principal assistente de IA generativa neste trabalho, nas variantes <i>Opus</i> (versões 4.6 e 4.7) e <i>Sonnet</i> 4.5.
ChatGPT	Família de modelos de linguagem desenvolvida pela OpenAI. Neste projeto a versão 5.3 foi utilizada como modelo subjacente ao <i>GitHub Copilot</i> .
CNPD	Comissão Nacional de Proteção de Dados. Autoridade de controlo portuguesa em matéria de proteção de dados pessoais.
Contratos Partilhados	Pacote interno (@gengis-khan/contracts) que centraliza esquemas de validação Zod e tipos TypeScript partilhados entre <i>frontend</i> e <i>backend</i> , eliminando divergências.
Cronómetro	Mecanismo do sistema que regista o tempo efetivo dedicado a cada intervenção numa ordem de serviço, suportando ações de <i>start</i> , <i>pause</i> e <i>stop</i> .
Decreto-Lei 28/2019	n.º Diploma português que estabelece obrigações de faturação eletrónica, arquivo de documentos fiscais e utilização de <i>software</i> de faturação certificado.
Docker Compose	Ferramenta que descreve e orquestra múltiplos contentores Docker através de um ficheiro declarativo (<i>docker-compose.yml</i>). No projeto orquestra API, <i>web</i> e Mailpit.
e-Fatura	Portal da Autoridade Tributária portuguesa para comunicação e consulta de faturas emitidas e recebidas.
Elicitação	Atividade inicial da engenharia de requisitos dedicada à descoberta, captura e clarificação das necessidades dos <i>stakeholders</i> .
Endpoint	Ponto de acesso exposto por uma API, identificado por URL e método HTTP, através do qual clientes interagem com um recurso do servidor.
Enum	Tipo enumerado que define um conjunto finito e nomeado de valores possíveis para uma variável (por exemplo, estados de uma ordem de serviço).
Entity Framework	ORM da Microsoft para a <i>stack</i> .NET. Referenciado apenas para contextualização; o projeto utiliza Prisma.
Fastify	<i>Framework web</i> para Node.js focado em elevado desempenho, validação baseada em esquemas e baixa sobrecarga. Serve como núcleo do <i>backend</i> do sistema.
Faturação Sequencial	Requisito legal português que obriga a que as faturas sejam numeradas de forma contínua e sem lacunas por série e exercício.

continua na página seguinte

Termo	Descrição
Frontend	Parte do sistema executada no <i>browser</i> do utilizador, responsável pela interface e pela interação. No projeto corresponde a uma SPA em React servida por Vite.
Generalização	Relação entre casos de uso em que um caso especializa outro, herdando o seu comportamento e adicionando ou refinando passos.
GitHub Copilot	Assistente de programação baseado em modelos de linguagem, utilizado como complemento durante a implementação. No presente trabalho foi usado com o <i>ChatGPT</i> como modelo subjacente.
Hashing	Transformação unidirecional de um valor (tipicamente uma palavra-passe) num resumo criptográfico, usada para armazenamento seguro.
HMR	<i>Hot Module Replacement</i> . Funcionalidade do Vite que recarrega módulos alterados no <i>browser</i> sem perder o estado da aplicação em desenvolvimento.
HttpOnly	Atributo de <i>cookie</i> que impede o acesso ao seu valor a partir de JavaScript no <i>browser</i> , mitigando ataques XSS contra tokens de sessão.
IA Generativa	Subcampo da inteligência artificial que produz conteúdo (texto, código, diagramas) a partir de modelos treinados sobre grandes corpus.
Include	Relação entre casos de uso em que um caso incorpora obrigatoriamente o comportamento de outro, evitando duplicação.
IEEE 830	Norma do <i>Institute of Electrical and Electronics Engineers</i> que estabelece práticas recomendadas para a elaboração de especificações de requisitos de <i>software</i> .
Intervenção	Tarefa técnica concreta executada sobre uma trotinete no âmbito de uma ordem de serviço (por exemplo, substituição de travões).
ISO/IEC/IEEE 29148	Norma internacional que especifica o ciclo de vida dos processos de engenharia de requisitos e o conteúdo mínimo de artefactos como o SRS.
IVA	Imposto sobre o Valor Acrescentado. Imposto indireto português sobre o consumo, incidente sobre as faturas emitidas pela oficina.
JSON	<i>JavaScript Object Notation</i> . Formato textual de serialização de dados, utilizado na comunicação entre <i>frontend</i> e <i>backend</i> .
JWT	<i>JSON Web Token</i> . Formato compacto de <i>token</i> assinado que transporta reivindicações entre cliente e servidor; no projeto é transportado via <i>cookie</i> HttpOnly.

continua na página seguinte

Termo	Descrição
LI4	Laboratórios de Informática IV. Unidade curricular do curso de Engenharia Informática da Universidade do Minho em que este relatório se insere.
LLM	<i>Large Language Model</i> . Modelo de linguagem de grande dimensão, treinado sobre vastos corpus textuais, capaz de gerar e interpretar linguagem natural e código.
Mailpit	Servidor SMTP leve utilizado em ambiente de desenvolvimento para capturar mensagens de <i>email</i> emitidas pelo sistema sem as enviar para destinatários reais.
Micromobilidade	Segmento da mobilidade urbana composto por veículos ligeiros, individuais e tipicamente elétricos, como trotinetes e bicicletas.
Migração (de <i>schema</i>)	Script versionado que altera de forma controlada a estrutura de uma base de dados. No projeto são geridas pelo Prisma.
Mock-up	Representação visual estática de uma interface de utilizador, produzida para validar a aparência e a organização antes da implementação.
Modelo de Domínio	Representação conceptual das entidades e relações do negócio, expressa tipicamente em diagramas UML de classes.
Modular Monolith	Padrão arquitetural em que o sistema é implementado como um único processo mas internamente organizado em módulos de negócio com fronteiras explícitas.
Monorepo	Estratégia de organização de código-fonte em que vários pacotes ou aplicações coexistem num único repositório, partilhando dependências e <i>tooling</i> .
MVP	<i>Minimum Viable Product</i> . Versão de um produto com as funcionalidades mínimas necessárias para validar hipóteses junto de utilizadores reais.
NIF	Número de Identificação Fiscal. Identificador fiscal único atribuído a pessoas singulares e coletivas em Portugal.
Node.js	Ambiente de execução JavaScript do lado do servidor, assente no motor V8. Suporta o <i>backend</i> Fastify do projeto (versão ≥ 20).
Nodemailer	Biblioteca Node.js para envio de <i>email</i> via SMTP, utilizada para notificações e partilha de documentos.
Ordem de Serviço (OS)	Artefacto central do domínio que representa um pedido de reparação registado para uma trotinete de um cliente, agregando intervenções, peças e tempos.
ORM	<i>Object-Relational Mapping</i> . Técnica que mapeia tabelas relacionais a objetos da linguagem de programação, dispensando a escrita direta de SQL para operações correntes.

continua na página seguinte

Termo	Descrição
PDFKit	Biblioteca Node.js para geração programática de documentos PDF, usada na emissão de faturas e relatórios.
Pino	Biblioteca de <i>logging</i> estruturado para Node.js, adotada no <i>backend</i> do projeto por desempenho e formato JSON nativo.
PME	Pequena e Média Empresa. Categoria em que se enquadra a Oficina Gengis Khan, utilizada como referência de contexto e restrições.
pnpm	Gestor de pacotes JavaScript eficiente em espaço e com suporte nativo a <i>workspaces</i> . Adotado na versão 10 como base do <i>mono-repo</i> .
PostgreSQL	Sistema de gestão de bases de dados relacionais de código aberto. Versão 16 utilizada como persistência do projeto.
Prisma	ORM TypeScript moderno que gera um cliente fortemente tipado a partir de um ficheiro <code>schema.prisma</code> , autoridade única de <i>schema</i> e migrações.
Prompt	Instrução em linguagem natural (opcionalmente estruturada) submetida a um LLM para orientar a resposta.
Prompt Engineering	Disciplina que estuda a construção sistemática de <i>prompts</i> com o objetivo de melhorar a qualidade, a reprodutibilidade e o controlo das respostas de um LLM.
RBAC	<i>Role-Based Access Control</i> . Modelo de controlo de acessos em que permissões são atribuídas a papéis e utilizadores herdadas pela associação a um papel.
React	Biblioteca JavaScript para construção de interfaces baseadas em componentes. Versão 18 utilizada no <i>frontend</i> do projeto.
React Hook Form	Biblioteca de gestão de formulários em React, minimizando re-renderizações e integrando-se com validadores como Zod.
Regra de Negócio (RN)	Restrição ou política do domínio de negócio que o sistema deve respeitar em operações relevantes.
Requisito Funcional (RF)	Requisito que descreve uma funcionalidade observável que o sistema deve oferecer.
Requisito Não Funcional (RNF)	Requisito que caracteriza propriedades transversais do sistema (desempenho, segurança, usabilidade, manutenibilidade, etc.).
RGPD	Regulamento (UE) 2016/679. Regulamento Geral sobre a Proteção de Dados, que disciplina o tratamento de dados pessoais na União Europeia.
ROI	<i>Return on Investment</i> . Indicador do retorno obtido por um investimento, relevante na justificação económica da solução.
SAF-T (PT)	<i>Standard Audit File for Tax Purposes</i> adaptado a Portugal. Ficheiro XML normalizado exigido pela AT para auditoria fiscal.

continua na página seguinte

Termo	Descrição
Schema	Descrição formal da estrutura de dados de uma base de dados ou de um contrato de API. No projeto coexistem o <i>schema</i> Prisma e esquemas Zod.
Session	Contexto de autenticação associado a um utilizador autenticado; neste projeto materializa-se por um JWT transportado em <i>cookie</i> HttpOnly.
SPA	<i>Single Page Application</i> . Aplicação <i>web</i> que carrega uma única página HTML e atualiza a interface dinamicamente, sem recarregamentos completos.
SRS	<i>Software Requirements Specification</i> . Documento que especifica requisitos funcionais, não funcionais e restrições de um sistema.
Stakeholder	Qualquer indivíduo, grupo ou entidade com interesse legítimo no sistema ou capaz de o influenciar.
Stitch AI	Ferramenta de geração assistida de <i>mock-ups</i> e protótipos de interface, desenvolvida pela Google. Utilizada no projeto como apoio à fase de <i>design</i> de interface.
Stock	Conjunto de peças e materiais disponíveis em armazém, sobre o qual o sistema efetua movimentos de entrada, saída e ajuste.
Supabase	Plataforma de dados assente em PostgreSQL. No projeto apenas o Supabase CLI é utilizado em desenvolvimento local; a produção assenta em PostgreSQL direto.
Tailwind CSS	<i>Framework</i> CSS utilitário que compõe estilos através de classes de baixa granularidade aplicadas diretamente no <i>markup</i> .
TanStack Query	Biblioteca React para obtenção, <i>cache</i> e sincronização de dados remotos, evitando gestão manual de estado de pedidos HTTP.
Triangulação	Técnica de investigação que cruza evidência de múltiplas fontes ou métodos para aumentar a confiança nos resultados.
Trotinete Elétrica	Veículo de micromobilidade individual propulsionado por motor elétrico e acumulador recarregável, objeto central do negócio da oficina.
Turborepo	Sistema de <i>build</i> para <i>monorepos</i> JavaScript que gere dependências entre tarefas e aplica <i>cache</i> incremental.
TypeScript	Linguagem construída sobre JavaScript que adiciona tipagem estática estrutural. Utilizada uniformemente em <i>backend</i> , <i>frontend</i> e pacotes partilhados.
UML	<i>Unified Modeling Language</i> . Linguagem gráfica normalizada para especificação e documentação de sistemas, incluindo diagramas de casos de uso, classes, sequência e componentes.
UMinho	Universidade do Minho. Instituição de ensino superior onde este relatório é elaborado.

continua na página seguinte

Termo	Descrição
User Story	Enunciado curto que descreve uma necessidade na perspectiva de um utilizador (<i>como...</i> , <i>quero...</i> , <i>para...</i>), usado como unidade de elicitação.
Vite	Ferramenta de <i>build</i> e servidor de desenvolvimento para aplicações <i>frontend</i> , com suporte nativo a HMR e <i>bundling</i> otimizado.
Workspace	Cada pacote ou aplicação gerido dentro de um <i>monorepo</i> ; no projeto correspondem a <code>apps/api</code> , <code>apps/web</code> e aos pacotes partilhados.
XSS	<i>Cross-Site Scripting</i> . Classe de vulnerabilidades em que um atacante injeta <i>scripts</i> executados no contexto do <i>browser</i> de outra vítima.
Zod	Biblioteca TypeScript de validação de esquemas com inferência de tipos automática, utilizada transversalmente nos contratos partilhados.
Zustand	Biblioteca minimalista de gestão de estado em React, usada no <i>frontend</i> para partilhar estado global entre componentes.

Anexos

Os anexos apresentam informação complementar que suporta a compreensão do relatório. Incluem-se os *prompts* completos utilizados nas interações com os modelos de linguagem, quando a sua extensão não permitiu a inclusão integral no corpo do texto, bem como artefactos auxiliares produzidos ao longo do ciclo de desenvolvimento.

Anexo A: Dossiê Completo de Prompts

Nota Introdutória

O presente anexo compila os *prompts* utilizados nas interações com os modelos de linguagem de grande dimensão (LLM) ao longo das quatro etapas do projeto, designadamente Engenharia de Requisitos, Arquitetura e Design, Implementação e Verificação e Validação. Esta documentação cumpre um duplo propósito: por um lado, assegurar a transparência metodológica exigida pelo enquadramento académico do trabalho, permitindo ao leitor compreender exatamente como cada artefacto foi produzido; por outro, servir de referência empírica para futuras investigações sobre a utilização de IA generativa no ciclo de vida de desenvolvimento de *software*.

Os *prompts* encontram-se organizados cronologicamente por fase do ciclo de vida em que foram aplicados, sendo cada um identificado por um código único (P-XX). Para cada entrada documenta-se:

- **Identificador e designação:** código sequencial e título descritivo.
- **Fase do ciclo de vida:** etapa do projeto em que o *prompt* foi aplicado.
- **Propósito:** objetivo específico da interação.
- **Conteúdo:** versão do *prompt* apresentada em forma normalizada.

Os *prompts* são apresentados na estrutura uniforme adotada ao longo do relatório, com quatro secções fixas (Contexto, Objetivo, Instruções de Escrita, Restrições), em alinhamento com o padrão de *prompt engineering* estabilizado no decurso do projeto. Esta normalização retroativa preserva o conteúdo substantivo de cada *prompt*, designadamente a intenção, as restrições e as instruções específicas, mas uniformiza a organização das secções, facilitando a leitura comparativa entre interações. Nas situações em que o mesmo artefacto resultou de múltiplas iterações

sobre *prompts* progressivamente refinados, documentam-se todas as versões, numeradas sequencialmente, preservando assim o registo do processo evolutivo.

A.1 Fase de Definição do Sistema

P-01: Geração de Entrevista de Definição do Domínio *Propósito:* gerar uma transcrição simulada de entrevista técnica entre analistas de sistemas e os *stakeholders* da oficina, servindo como base documental para a definição do domínio e para a especificação de requisitos.

Contexto: Projeto da UC de LI4, sistema de gestão das "Oficinas Gengis Khan". Fase de definição do domínio, anterior ao levantamento formal de requisitos.

Objetivo: Gerar a transcrição de uma entrevista técnica entre um Analista de Sistemas e o Engenheiro da oficina, a utilizar como base documental para a "Definição do Domínio" e o "SRS".

Instruções de Escrita:

- Cobrir quatro tópicos: Registo e Identificação, Ciclo de Vida da Reparação, Gestão de Peças e Custos, Output e Relatórios
- Manter registo natural de diálogo entre dois interlocutores técnicos
- Privilegiar exemplos concretos sobre afirmações genéricas

Restrições: Forneça apenas a transcrição da entrevista, sem introduções meta-textuais nem comentários fora do diálogo.

[Numa segunda iteração foi solicitada a reescrita do texto de contextualização com base no domínio extraído da entrevista gerada.]

Análise Crítica. O processo de produção da contextualização envolveu duas interações distintas com o LLM. Na primeira, solicitámos a geração de uma entrevista simulada entre analistas de sistemas e os *stakeholders* da oficina, opção metodológica destinada a reproduzir o tipo de recolha de informação que seria conduzida num contexto profissional com interlocutores reais. O registo daí resultante é tratado, ao longo de toda a etapa, como base documental autêntica do domínio, sobre a qual assentam todas as decisões subsequentes de modelação e especificação. Na segunda iteração, pedimos ao modelo que sintetizasse essa informação num texto de contextualização formal.

O principal desafio residiu na calibração do nível de detalhe. A primeira versão gerada pelo LLM era demasiado genérica, descrevendo o setor de micromobilidade em termos que poderiam aplicar-se a qualquer oficina. Optámos por solicitar uma versão mais ancorada nos dados concretos da oficina (volume operacional real, número de referências de peças, segmentos de

clientes), o que resultou num texto significativamente mais útil. O parágrafo sobre o crescimento do mercado europeu foi aceite na íntegra por estar factualmente alinhado com dados públicos do setor. A caracterização técnica das trotinetes (eletrónica de potência, mecânica de precisão) foi validada pela equipa e considerada adequada.

Um segundo desafio, transversal a toda a redação do relatório e não exclusivo desta secção, residiu na depuração linguística do *output*. A escrita técnica em português gerada pelo LLM revelou um vício recorrente, designadamente a tendência para traduzir termos do inglês de forma literal, particularmente acentuada nas passagens mais próximas do vocabulário do desenvolvimento de *software*. Termos como “comitar” (de *commit*) ou “merjar” (de *merge*), inexistentes em português, surgiram com frequência no *output* bruto e exigiram um esforço considerável de revisão para os substituir pelas formas corretas. Ainda assim, o tempo gasto nesta revisão manteve-se substancialmente inferior ao que a redação integral pela equipa teria exigido. O processo, embora imperfeito, preserva um ganho de produtividade significativo na produção do relatório, ganho esse que justifica o recurso ao modelo apesar dos desvios linguísticos pontuais que produz.

P-02: Refinamento da Fundamentação *Propósito:* reescrever o parágrafo introdutório da secção de Fundamentação com rigor académico, tendo em conta o papel desta secção no ciclo de vida.

Contexto: Documento tecnico/academico, ciclo de vida de desenvolvimento de software, projeto da UC de LI4.

Objetivo: escrever o paragrafo introdutorio da seccao de "Fundamentacao".

Instrucoes de Escrita:

- Utilize terminologia precisa
- Evite informacao redundante ou repeticoes desnecessarias
- Tenha em conta o papel da Fundamentacao no ciclo de vida de desenvolvimento de software
- Mantenha a coesao e coerencia textual exigidas em contextos universitarios

Restricoes: Forneca apenas o novo paragrafo. Nao inclua introducoes, explicacoes ou notas de rodape. Conteudo base em anexo.

Análise Crítica. A fundamentação exigia a identificação das vulnerabilidades operacionais da oficina e a sua organização em dimensões estruturadas. O LLM produziu uma versão inicial que identificava corretamente quatro dimensões problemáticas (fragmentação de informação, opacidade operacional, gestão reativa de *stock* e falta de rigor financeiro), mas apresentava-as de forma excessivamente descritiva. Optámos por manter a estrutura proposta pelo modelo, por considerarmos que a divisão em quatro dimensões era lógica e abrangente, mas reformu-

lâmos diversos parágrafos para incluir dados quantitativos concretos extraídos da entrevista simulada (estimativa de 15 % de atrasos evitáveis, taxa de 25 €/h). Este percurso de revisão e reformulação ilustra o princípio metodológico subjacente a toda a etapa, em que o ganho de produtividade conferido pelo modelo se mantém condicionado a verificação humana atenta.

P-03: Refinamento de Objetivos *Propósito:* produzir parágrafos introdutórios distintos para as três categorias de objetivos (primários, secundários e transversal), explicando a importância de cada uma no contexto do modelo de desenvolvimento em cascata.

Contexto: Documento tecnico/academico, modelo de desenvolvimento em cascata, fase de definicao do sistema (anterior ao levantamento de requisitos).

Objetivo: Escrever paragrafo curto e introdutorio da seccao de "Objetivos primarios", "Objetivos secundarios" e "Objetivo transversal", explicando a importancia dos mesmos e o motivo de estarem a ser apresentados.

Instrucoes de Escrita:

- Tenha em conta o papel dos Objetivos no ciclo de vida de desenvolvimento de software
- Utilize terminologia precisa
- Mantenha coesao e coerencia universitaria

Restricoes: Forneca apenas os 3 novos paragrafos separadamente. Conteudo base em anexo.

Análise Crítica. Para esta secção, a interação com o LLM decorreu em dois momentos. Primeiro, solicitámos os parágrafos introdutórios de cada categoria de objetivos. O modelo produziu textos que, embora corretos do ponto de vista formal, tendiam para uma linguagem excessivamente abstrata e desligada do domínio concreto, pelo que foi pedida nova iteração que ancorasse os parágrafos no contexto específico das *Oficinas Gengis Khan*. A estrutura tripartida (primários, secundários, transversal) foi aceite sem reservas, por se alinhar com boas práticas de engenharia de requisitos. O conteúdo das tabelas de objetivos foi igualmente gerado pelo modelo a partir da informação recolhida na Entrevista-001, cabendo à equipa exclusivamente a validação de cada entrada, com aceitação imediata ou pedido de reformulação consoante a fidelidade ao levantamento original.

P-04: Refinamento de Viabilidade *Propósito:* redigir a secção de Viabilidade com estrutura tripartida (introdução, económica, temporal), fundamentando cada dimensão nos dados reais da oficina.

Contexto: Sistema de gestao das "Oficinas Gengis Khan", projeto da UC de LI4 (Universidade do Minho). Fase de definicao do sistema, anterior ao desenho da arquitetura.

Objetivo: Escrever a seccao de Viabilidade do relatorio com rigor academico medio-alto em PT-PT, articulando argumentos em quatro dimensoes (tecnica, organizacional, economica e temporal).

Instrucoes de Escrita:

- Introducao curta sobre o papel da viabilidade no ciclo de vida de desenvolvimento
- Viabilidade Economica fundamentada em ROI e ganhos operacionais
- Viabilidade Temporal alinhada com o cronograma do semestre LI4
- Ancorar argumentos em dados reais da oficina (volume 150-500 OS/mes)

Restricoes: Manter a analise agnostica face a tecnologias especificas ainda nao decididas. Sem estimativas quantitativas sem base documental.

Análise Crítica. A secção de viabilidade foi a que mais beneficiou da assistência do LLM, por exigir a articulação de argumentos em quatro dimensões distintas (técnica, organizacional, económica e temporal). O modelo produziu uma estrutura bem organizada e textos introdutórios de qualidade aceitável. Foram, contudo, identificados dois problemas relevantes. Em primeiro lugar, a viabilidade técnica mencionava tecnologias específicas que ainda não tinham sido formalmente decididas pela equipa, obrigando a uma reformulação para manter a análise agnóstica. Em segundo lugar, a viabilidade económica apresentava estimativas quantitativas sem base documental, que foram removidas e substituídas por argumentos qualitativos ancorados nos dados reais da oficina. A tabela de riscos temporais foi aceite com modificações pontuais na coluna de mitigação.

P-05: Refinamento de Recursos *Propósito:* elaborar a introdução da secção de Recursos enquadrando a gestão de recursos como pilar da engenharia de *software* e justificando a transição tecnológica.

Contexto: Sistema de gestao das "Oficinas Gengis Khan" (projeto da UC de LI4). Fase de definicao do sistema, secao de Recursos.

Objetivo: Escrever a introducao da seccao de Recursos, com 1 a 2 paragrafos densos e bem fundamentados.

Instrucoes de Escrita:

- Enquadramento teorico da gestao de recursos como pilar da engenharia de software
- Contextualizacao no projeto especifico, justificando a transicao do modelo manual (papel/Excel) para plataforma web

- Divisao explicita entre capital humano e recursos materiais/tecnologicos

Restricoes: Usar nomes reais dos intervenientes. Sem nomes ficticios nem humoristicos. Forneca apenas a introducao, sem tabelas.

Análise Crítica. Esta interação ilustrou de forma particularmente nítida uma vantagem do recurso ao LLM, transversal às secções anteriores e merecedora de destaque a esta altura do relatório, designadamente a capacidade de associar rapidamente informação dispersa pelas entrevistas e demais artefactos de levantamento à estrutura textual destinada a integrá-la. Onde a equipa teria, por método tradicional, de percorrer integralmente a transcrição da Entrevista-001, sinalizar as passagens relevantes e organizar manualmente o seu mapeamento aos atributos das tabelas, o modelo procedeu à extração, classificação e correlação dessa informação de forma quase instantânea, dispensando a leitura iterativa habitualmente exigida por esta operação. O esforço de validação pela equipa cingiu-se, em consequência, à verificação por amostragem do conteúdo produzido contra o levantamento original, tendo a única correção significativa incidido sobre os nomes fictícios e humorísticos inicialmente atribuídos aos *stakeholders*, substituídos pelos nomes reais dos intervenientes.

P-06: Plano de Execução *Propósito:* estruturar o plano de execucao do projeto em quatro fases alinhadas com o calendario da UC LI4, identificando as atividades principais e os entregaveis de cada fase.

Contexto: Sistema de gestao das "Oficinas Gengis Khan" (projeto da UC de LI4), calendario do semestre academico fixado pela unidade curricular.

Objetivo: Estruturar o plano de execucao do projeto em quatro fases alinhadas com o calendario da UC LI4, identificando as atividades principais e os entregaveis de cada fase.

Instrucoes de Escrita:

- Cada fase deve ter inicio, fim e entregaveis claros
- Justificar o alinhamento de cada fase com etapas reconhecidas do ciclo de vida (requisitos, design, implementacao, V&V)
- Indicar dependencias sequenciais entre fases

Restricoes: Forneca a tabela do plano e um paragrafo introdutorio. Sem detalhe de atribuicoes individuais.

[Gerado no mesmo fluxo de interacao que as seccoes de Viabilidade e Recursos.]

Análise Crítica. O LLM propôs uma estrutura em quatro fases alinhada com a abordagem em cascata adotada para o ciclo de vida do projeto, que a equipa aceitou na íntegra nesta etapa. Posteriormente, no decurso da implementação, viria a perceber-se que a Fase 3 exigiria, em específico, uma execução incremental, em consequência do modo de operação dos LLM enquanto agentes de geração de código. A distinção entre o ciclo de vida do projeto e o ciclo de produção de código, bem como as suas implicações sobre o método de trabalho da equipa, será desenvolvida em maior detalhe no Capítulo 3.

A.2 Fase de Definição do Domínio

A presente subsecção reúne os seis *prompts* da Secção 1.2 do corpo do relatório. Para cada *prompt* apresenta-se, num único bloco, o propósito, o conteúdo enviado ao modelo e a análise crítica do resultado obtido, evitando a fragmentação do registo entre locais distintos do documento.

P-07: Introdução ao Domínio *Propósito:* realizar a análise de contexto do ecossistema da oficina e a definição inicial do domínio, identificando as entidades principais e as relações estruturais entre elas.

```
Contexto: Projeto da UC de LI4 (UMinho), sistema de gestao para
oficina de reparacao de trotinetes. Etapa 1 do ciclo de vida em
cascata, fase de Concecao e Engenharia de Requisitos.

Objetivo: Elaborar a analise de contexto e a definicao do dominio do
sistema, identificando as entidades principais (Cliente, Trotinete
, Peca, etc.) e as relacoes entre elas.

Instrucoes de Escrita:
- Caracterizar o ecossistema da oficina e o posicionamento no setor
- Identificar entidades de dominio com nomes claros e
responsabilidades distintas
- Explicitar as relacoes estruturais entre entidades

Restricoes: Postura de consultor de Engenharia de Software. Ser
especifico e identificar edge cases. Nao escrever codigo.
```

Análise Crítica. O LLM produziu uma análise de contexto abrangente que identificava corretamente o posicionamento do setor de reparação de trotinetes no ecossistema de mobilidade urbana sustentável. A caracterização técnica dos equipamentos (componentes eletrónicos, mecânicos, sistemas de travagem regenerativa) foi considerada precisa e útil para fundamentar a complexidade do domínio. O modelo identificou seis necessidades críticas que a equipa validou integralmente por estarem alinhadas com os problemas reportados na Entrevista-001. A principal intervenção da equipa consistiu em reorganizar o texto para separar claramente a

introdução conceitual, a caracterização do setor e as necessidades identificadas, que o LLM tinha apresentado de forma menos estruturada.

P-08: Âmbito do Sistema *Propósito:* estruturar o domínio em componentes funcionais coerentes com responsabilidades e fronteiras claras, servindo de base à engenharia de requisitos subsequente.

Contexto: Sistema de gestão das "Oficinas Gengis Khan" (projeto da UC de LI4). Fase de definição do domínio, sequência da Subseção anterior.

Objetivo: Identificar as entidades principais e as relações entre elas, organizando-as em componentes funcionais coerentes que sirvam de base à engenharia de requisitos subsequente.

Instruções de Escrita:

- Cada componente funcional descrito com responsabilidades distintas e fronteiras claras
- Cobertura abrangente do domínio operacional, administrativo e de gestão
- Nível de detalhe suficiente para sustentar o levantamento de requisitos sem precisar de retrabalho

Restrições: Forneça apenas a estrutura em componentes e a descrição de cada um. Sem requisitos específicos nem detalhes de implementação.

[Gerado no mesmo fluxo de interação que a Subseção anterior.]

Análise Crítica. O LLM propôs uma divisão em seis componentes funcionais que a equipa considerou abrangente e bem delimitada. Cada componente foi descrito com um nível de detalhe suficiente para servir de base à engenharia de requisitos subsequente. A principal contribuição da equipa consistiu na validação cruzada dos componentes com a informação recolhida na Entrevista-001, confirmando que todas as áreas operacionais mencionadas pelos *stakeholders* estavam cobertas. Não foram identificadas lacunas significativas na proposta do LLM.

P-09: Intervenientes e Papéis *Propósito:* identificar e caracterizar todos os intervenientes do sistema, distinguindo utilizadores diretos de intervenientes indiretos e fixando responsabilidades funcionais por papel.

Contexto: Sistema de gestão das "Oficinas Gengis Khan" (projeto da UC de LI4). Definição do domínio, sequência da identificação dos componentes funcionais.

Objetivo: Identificar e caracterizar todos os intervenientes do sistema, os seus papeis, responsabilidades e niveis de acesso.

Instrucoes de Escrita:

- Distinguir utilizadores diretos (operacionais) de intervenientes indiretos (clientes, fornecedores)
- Para cada papel, indicar responsabilidades funcionais e fronteiras de autoridade
- Cobrir interlocutores externos relevantes ao fluxo do negocio

Restricoes: Forneça a caracterizacao em prosa por papel. Sem detalhe de implementacao de RBAC ou autenticacao.

[Gerado no mesmo fluxo que a definicao do dominio.]

Análise Crítica. O LLM identificou corretamente quatro categorias de intervenientes (gerente, mecânicos, clientes e fornecedores), com uma distinção adequada entre utilizadores diretos e indiretos do sistema. A equipa considerou a caracterização suficientemente detalhada para informar a fase de requisitos, tendo aceite o texto na íntegra com ajustes menores de pontuação.

P-10: Modelo de Domínio *Propósito:* redigir a introdução e fundamentação do diagrama de classes de domínio, explicando o processo metodológico de derivação a partir de substantivos e verbos da entrevista.

Contexto: Sistema de gestao das "Oficinas Gengis Khan" (LI4-UMinho). Modelacao de Dominio com diagrama de classes ja definido. Escreve-se a introducao e fundamentacao do modelo.

Objetivo: Produzir um texto academico (PT-PT) que explique o que e o Diagrama de Dominio, por que e necessario, e como foi derivado a partir das fontes de conhecimento do dominio.

Instrucoes de Escrita:

- Definir Diagrama de Dominio como representacao conceptual das entidades e relacoes do negocio
- Justificar enquanto blueprint para a base de dados e logica de negocio, eliminando ambiguidade dos registos manuais
- Descrever o processo de derivacao (extracao de substantivos como entidades, verbos como associacoes, regras de negocio como cardinalidades)
- Mencionar transicao do sistema informal para modelo normalizado, em coerencia com o crescimento de 150 para 500 OS/mes

Restricoes: Forneca apenas o texto introdutorio e a fundamentacao. Sem repetir conteudos do diagrama em si.

Análise Crítica. Para esta secção, o LLM foi utilizado em duas fases, primeiro para gerar a definição do domínio (entidades e relações) e depois para redigir a introdução e fundamentação do modelo de domínio. O texto produzido pelo modelo para a fundamentação foi de qualidade elevada, articulando com clareza o processo de derivação do diagrama a partir da análise linguística das notas de entrevista. A equipa considerou particularmente útil a explicação metodológica (extração de substantivos como candidatos a entidades, verbos como associações), que foi aceite na íntegra. A descrição das cardinalidades e a cadeia de rastreabilidade foram validadas contra o diagrama de classes produzido pela equipa.

P-11: Identificação de *Stakeholders* *Propósito:* identificar e caracterizar de forma exhaustiva os *stakeholders* do sistema, distinguindo diretos, indiretos, técnicos e regulamentares, e propor uma matriz de gestão diferenciada.

Contexto: Sistema de gestao das "Oficinas Gengis Khan" (LI4). Extensao natural da definicao do dominio, na qual se consolidou o quadro operacional.

Objetivo: Identificar e caracterizar de forma exhaustiva todos os stakeholders do sistema, distinguindo diretos, indiretos, tecnicos e regulamentares.

Instrucoes de Escrita:

- Para cada stakeholder, indicar papel, responsabilidades, interesses e nivel de influencia
- Incluir entidades regulamentares (Autoridade Tributaria, CNPD) e a equipa de desenvolvimento como stakeholders tecnicos
- Apresentar uma matriz final com estrategia de gestao diferenciada por stakeholder

Restricoes: Nao incluir estimativas percentuais sem base documental. Limitar-se a informacao verificavel na Entrevista-001.

[Produzida como extensao natural da definicao do dominio.]

Análise Crítica. O LLM produziu uma análise de *stakeholders* surpreendentemente completa, identificando não apenas os intervenientes óbvios (gerente, mecânicos, clientes) mas também entidades regulamentares (Autoridade Tributária, CNPD) e a equipa de desenvolvimento como *stakeholders* técnicos. A matriz de *stakeholders* com estratégias de gestão diferenciadas foi considerada um contributo valioso que a equipa não teria produzido com o mesmo nível de detalhe. A principal correção efetuada foi a remoção de estimativas percentuais de distribuição

de clientes (70-80 % particulares) que não tinham base documental na Entrevista-001. O texto foi aceite na sua quase totalidade.

P-12: Contexto e Restrições *Propósito:* estabelecer o contexto de negócio e identificar restrições legais, temporais, tecnológicas e organizacionais antes do levantamento formal de requisitos.

Contexto: Sistema de gestão das "Oficinas Gengis Khan" (LI4). Fase de definição do domínio, posterior a identificação de stakeholders e anterior ao levantamento formal de requisitos.

Objetivo: Escrever a secção de "Contexto e Restrições" com parágrafo introdutório sobre a importância da análise prévia e desenvolvimento das quatro famílias de restrições (legais/regulamentares, temporais/académicas, tecnológicas e organizacionais).

Instruções de Escrita:

- Restrições legais com referência precisa a legislação aplicável (RGPD, Decreto-Lei n.º 28/2019)
- Restrições temporais ancoradas no calendário da UC LI4
- Restrições tecnológicas em conformidade com as decisões já tomadas pela equipa
- Terminologia técnica de engenharia de software

Restrições: Forneca apenas o parágrafo introdutório e a descrição por família. Sem requisitos específicos.

Análise Crítica. Esta secção exigia a articulação de restrições de natureza muito diversa (legais, técnicas, temporais, académicas), o que constituiu um desafio de organização textual. O LLM produziu uma estrutura clara e bem fundamentada, com destaque para a secção de restrições legais (RGPD e faturação sequencial), onde demonstrou conhecimento preciso da legislação portuguesa aplicável (Decreto-Lei n.º 28/2019, Regulamento UE 2016/679). A equipa validou estas referências legislativas e considerou-as corretas. A tabela de tecnologias selecionadas foi posteriormente ajustada para refletir as decisões finais da equipa, com a fundamentação detalhada das escolhas remetida para o Capítulo 2, onde são discutidas as decisões arquiteturais do sistema.

A.3 Fase de Engenharia de Requisitos

P-13: Geração de User Stories *Propósito:* redigir a introdução teórica às *user stories* e gerar a lista detalhada organizada por papel (Gerente e Mecânico), com critérios de aceitação.

Contexto: Sistema de gestao das "Oficinas Gengis Khan" (LI4). Engenharia de requisitos, fase de consolidacao preliminar de necessidades, posterior as Entrevistas 01 e 02.

Objetivo: Escrever a introducao teorica as User Stories (baseada em Sommerville) e gerar a lista detalhada das mesmas com criterios de aceitacao mensuraveis.

Instrucoes de Escrita:

- Enquadrar a User Story como descricao simplificada focada no valor de negocio, justificando a reducao da distancia entre equipa e stakeholders
- Derivar historias das Entrevistas 01 e 02 e da analise do dominio, aplicando o criterio INVEST
- Organizar por papel (Gerente, Mecanico) no formato "Como [papel], eu pretendo [funcionalidade], para que [beneficio]"
- Acompanhar cada historia de 2 a 3 criterios de aceitacao objetivos e mensuraveis

Restricoes: Sem criterios vagos do tipo "o sistema deve ser rapido". Sem inventar funcionalidades nao mencionadas nas fontes.

Análise Crítica. A geração de *user stories* com auxílio do LLM revelou-se particularmente eficaz, dado que o modelo já dispunha do contexto completo das duas entrevistas como documentos em anexo. O LLM produziu 16 histórias (8 para o Gerente, 8 para o Mecânico), das quais a equipa reteve 16 após validação. A principal intervenção consistiu na reformulação de critérios de aceitação que eram demasiado vagos (por exemplo, "o sistema deve ser rápido" foi substituído por critérios mensuráveis). A introdução teórica proposta pelo LLM, que enquadrava as *user stories* como técnica de consolidação preliminar de necessidades e não como requisitos formais, foi considerada uma distinção pertinente e aceite integralmente. A organização por papel (Gerente e Mecânico) reflete as prioridades distintas de cada perfil, tal como observado nas entrevistas.

P-14: Transcrição de Documentos Físicos *Propósito:* simular a transcrição do caderno de operações, ordens de serviço, faturas e registos de *stock* da oficina, cobrindo cinco dias operacionais.

Contexto: Projeto das Oficinas Gengis Khan (LI4 - UMinho), com a definicao do sistema, duas entrevistas e user stories ja produzidas, e necessidade de simular documentos fisicos pre-existentes a transicao digital.

Objetivo: Simular as paginas do caderno operacional da oficina e documentos fisicos relacionados (ordens de servico, faturas, registos de stock) cobrindo cinco dias operacionais.

Instrucoes de Escrita:

- Antepor um texto introdutorio formal a explicar que o conteudo resulta da transcricao do caderno e documentos fisicos
- Simular a escrita do gerente ou do mecanico chefe consoante a accao, identificando o autor de cada registo
- Representar os documentos em texto puro, sem formatacoes especificas
- Maximizar cobertura de situacoes representativas ao longo dos cinco dias
- PT-PT

Restricoes: Reflectir apenas o funcionamento antes do sistema de gestao, sem melhorias desejadas. Sem inserir requisitos novos.

Análise Crítica. O LLM gerou cinco páginas simuladas de caderno operacional, documentos de ordem de serviço, faturas e movimentos de stock com elevada fidelidade ao registo manual descrito na Entrevista-001. A linguagem informal foi adequada ao contexto simulado, com identificação consistente do autor de cada registo (gerente ou mecânico chefe). A equipa validou o conjunto contra a entrevista original, sem detectar inconsistências de processo, tendo o artefacto sido aceite na íntegra como base documental complementar para a fase de elicitação de requisitos.

P-15: Matriz de Levantamento de Requisitos *Propósito:* extrair e estruturar todos os requisitos mencionados nas entrevistas e documentação, produzindo uma tabela em formato CSV.

Contexto: Projeto das Oficinas Gengis Khan, fase de engenharia de requisitos a partir das entrevistas e da documentacao em anexo, com necessidade de rastreabilidade pessoa/documento.

Objetivo: Extrair e estruturar todos os requisitos mencionados nas entrevistas e na documentacao do projeto, organizados em tabela com rastreabilidade.

Instrucoes de Escrita:

- Cada linha contem No, Data e Hora, Descricao, Vista de Uso, Fonte (Pessoa ou Documento) e Analista
- Cada requisito tem de ter sido explicitamente pedido ou derivado diretamente de um problema mencionado pelos stakeholders nos textos fornecidos
- Manter linguagem clara em PT-PT, nivel academico medio-alto

Restricoes: Nao inventar nem presumir informacao. Se nao esta no texto , nao entra na tabela.

Análise Crítica. A elicitação de requisitos constituiu, entre todas as interações conduzidas com o LLM nesta etapa, aquela em que o modelo manifestou desempenho menos consistente, exigindo intervenção humana mais densa do que a observada em *prompts* anteriores. Observaram-se dois enviesamentos sistemáticos no resultado produzido. Por um lado, a omissão de requisitos diretamente sustentados nas fontes documentais, com particular incidência nos registos físicos transcritos. Por outro, granularidade excessiva sobre outros itens, com fragmentação de uma única necessidade em múltiplos requisitos redundantes. A equipa procedeu, em consequência, a uma revisão exaustiva da lista contra a totalidade das fontes (Entrevistas 01 e 02 e registos físicos do estabelecimento), identificando múltiplas correções estruturais a aplicar. A título ilustrativo, destacam-se dois casos representativos, designadamente o RF-08, com descrição vazia por aparente erro de geração, e o RF-31, que duplicava integralmente o RF-23 ao descrever o registo de movimentos de *stock*. Os requisitos afetados foram removidos e os identificadores reenumerados após o conjunto das correções, fixando-se a contagem definitiva em 44 requisitos funcionais e 5 não funcionais. A contagem inicialmente apontada pelo modelo (45 funcionais) foi retificada em conformidade. A taxonomia funcional vs. não funcional foi validada pela equipa, sem alterações. Numa iteração posterior, com finalidade distinta, o modelo foi solicitado a introduzir intencionalmente requisitos parcialmente conflitantes entre si, instrumentalizando-se assim a geração controlada das ambiguidades que viriam a ser tratadas na subsecção seguinte.

P-16: Introdução à Elicitação de Requisitos *Propósito:* redigir a introdução do capítulo de Requisitos, enquadrando-o como fase crítica do ciclo de vida e fundamentando a necessidade da transição digital.

Contexto: Projeto das Oficinas Gengis Khan, gestão artesanal em papel e Excel com perdas de informação, ausência de histórico e erros de faturação, transição digital prevista para sistema centralizado entre cliente, mecânico e gestão de peças/faturação.

Objetivo: Redigir a introdução do capítulo de Requisitos com 3 a 4 parágrafos densos, em PT-PT académico formal.

Instruções de Escrita:

- Apresentar a Engenharia de Requisitos como fase mais crítica do ciclo de vida, justificando pelo efeito exponencial dos erros
- Justificar a transição digital pela necessidade de suportar crescimento e complexidade técnica (electrónica + mecânica)
- Detalhar a metodologia em elicitação, análise e formalização em RF e RNF
- Apresentar o documento como "Single Source of Truth" para programadores e contrato de funcionalidades para o cliente
- Vocabulário técnico preciso (utilizadores, ecrãs, planeamento, faturação)

Restrições: PT-PT estrito. Sem linguagem vaga.

Análise Crítica. O LLM produziu uma introdução tecnicamente sólida que enquadrava a Engenharia de Requisitos no ciclo de vida de desenvolvimento de *software* e justificava a transição digital do sistema artesanal (papel e Excel) para plataforma centralizada. Diferentemente do que foi reportado para a P-15 (Matriz de Levantamento de Requisitos), onde o desempenho do modelo se mostrou substantivamente mais frágil, a dificuldade observada nesta interação foi de natureza distinta e marcadamente estilística, designadamente a tendência do modelo para recorrer a formulações genéricas sobre Engenharia de *Software*, sem ancoragem específica no contexto do projeto. A equipa solicitou uma iteração adicional que substituiu as passagens demasiado abstratas por formulações sustentadas no domínio das *Oficinas Gengis Khan* e no problema concreto da gestão artesanal vigente, tendo o texto resultante sido aceite com ajustes menores de pontuação e de remissão para subsecções subsequentes.

P-17: Processo de Elicitação e Descoberta *Propósito:* redigir a subsecção sobre o processo metodológico de elicitação, descrevendo entrevistas semiestruturadas, análise documental e triangulação.

Contexto: Projeto das Oficinas Gengis Khan com tres inputs disponiveis : Entrevista-001 (dominio e fluxo de trabalho), Entrevista-002 (regras de negocio especificas) e analise documental as-is (fichas em papel e folhas Excel).

Objetivo: Redigir a subseccao sobre o processo metodologico de elicitacao em PT-PT academico medio-alto.

Instrucoes de Escrita:

- Descrever a metodologia de elicitacao como processo iterativo e justificar o uso de entrevistas semiestruturadas
- Identificar e caracterizar os stakeholders: Gerente (Antonio Silva), Mecanico (Ricardo Costa) e Cliente Final (perfil indireto)
- Apresentar a analise documental e a descoberta de dados, evidenciando a desorganizacao do registo em papel e a necessidade de automatismos e validacoes
- Descrever a triangulacao entre entrevistas e documentos para obter lista final consistente
- Estilo impessoal ("procedeu-se", "concluiu-se"), terminologia tecnica
- Minimo 4 a 5 paragrafos robustos

Restricoes: Sem narrativa em primeira pessoa. Sem inputs nao referenciados.

Análise Crítica. O LLM articulou de forma estruturada o processo metodológico em quatro componentes (metodologia de elicitação, caracterização de stakeholders, análise documental e triangulação), adequando o registo ao estilo impessoal exigido. A descrição da triangulação

entre entrevistas e documentos foi considerada particularmente útil enquanto justificação metodológica da versão consolidada dos requisitos. O texto foi aceite com revisões menores de estilo.

P-18: Análise de Requisitos e Resolução de Ambiguidades *Propósito:* redigir a subsecção de análise crítica e resolução de ambiguidades, apresentando de forma estruturada cada ambiguidade identificada e respetiva clarificação.

Contexto: Projeto das Oficinas Gengis Khan, com tabela Excel de RF e RNF e cinco ambiguidades já detetadas com respetivas resoluções (REQ-004 sobre identificação de cliente pelo NIF; REQ-008 a juntar ao REQ-011; REQ-009 a juntar a REQ-008 e REQ-011 como adições ao longo do ciclo de vida da OS; REQ-023 redundante face a REQ-020; REQ-033 aplicável apenas a clientes empresariais).

Objetivo: Redigir a subsecção de análise crítica e resolução de ambiguidades em PT-PT académico formal.

Instruções de Escrita:

- Introdução a fase de análise: propósito de escrutinar dados brutos, categorizar e garantir consistência/completude
- Categorizar RF vs RNF com distinção teórica e formatar em tabelas Markdown
- Explicar que a linguagem natural gera termos vagos e descrever o processo de resolução
- Apresentar a tabela "Ambiguidade Identificada" → "Impacto Potencial" → "Resolução/Clarificação Acordada"
- Vocabulário normativo e preciso, com aparência de especificação formal pronta a entregar a programadores

Restrições: Sem ambiguidades não listadas nos inputs. Sem alterações a numeração dos requisitos.

Análise Crítica. O LLM organizou as cinco ambiguidades fornecidas em tabela estruturada ("Ambiguidade Identificada" → "Impacto Potencial" → "Resolução Acordada") e produziu a introdução metodológica sobre análise de requisitos. A formalização das resoluções como regras de negócio foi aceite na íntegra. A principal contribuição residiu na clareza com que cada ambiguidade foi caracterizada antes de apresentada a resolução, evitando que o leitor tivesse de inferir o impacto a partir da clarificação.

P-19: Simulação da Sessão de Validação (Entrevista-003) *Propósito:* redigir a transcrição de uma reunião simulada entre a equipa de desenvolvimento e os *stakeholders* para validar e resolver as ambiguidades identificadas.

Contexto: Equipa de LI4 reunida em Braga com o Gerente (Antonio Silva) e o Mecanico Chefe (Ricardo Costa) apos a primeira analise dos requisitos, com cinco ambiguidades a confrontar (Tabela 25) e necessidade de chegar a consenso para fechar a Baseline de requisitos.

Objetivo: Redigir a transcricao de uma reuniao simulada que confronta os stakeholders com as falhas logicas, omissoes ou contradicoes detectadas e regista a resolucao acordada para cada ambiguidade.

Instrucoes de Escrita:

- Cabecalho de Ata com Data, Local (Braga) e Participantes
- Introducao a explicar o conceito de edge cases e porque a linguagem natural gera ambiguidades
- Dialogo em discurso direto para cada ambiguidade da Tabela 25, com perfil do Antonio (preocupado com perdas financeiras, abandono e faturacao, tendendo a pedir processos burocraticos), perfil do Ricardo (preocupado com rapidez, sem disponibilidade para preencher campos com maos sujas) e Equipa LI4 com perguntas incisivas
- No final de cada ponto a Equipa LI4 resume a regra de negocio final acordada
- Encerramento com validacao formal e autorizacao do cliente para avanco para desenho/arquitetura
- PT-PT rigoroso (ecra, tablet, orcamento, faturar)
- Tom realista e profissional, com maturidade em Engenharia de Software

Restricoes: Sem ambiguidades nao listadas na Tabela 25. Sem perfis fora dos descritos.

Análise Crítica. O LLM produziu uma transcrição extensa e tecnicamente coerente, respeitando os perfis psicológicos atribuídos a cada interveniente (gerente preocupado com finanças, mecânico com rapidez operacional) e fechando cada ambiguidade com a regra de negócio acordada. A única intervenção relevante da equipa consistiu em uniformizar a terminologia técnica (ecrã, tablet, orçamento, faturar) em conformidade com o registo PT-PT exigido. A sessão simulada constitui o registo formal da resolução das cinco ambiguidades e marca o fecho da versão consolidada dos requisitos.

P-20: Verificação de Coerência *Propósito:* rever a coerência entre os requisitos levantados e o restante conteúdo do relatório, identificando lacunas e propostas de solução.

Contexto: Relatório de LI4 sobre o sistema para oficina de reparação de trotinetes, com requisitos levantados em CSV e restante conteúdo em Markdown.

Objetivo: Verificar a coerência entre os requisitos levantados e o restante conteúdo do relatório, identificando lacunas e incoerências.

Instruções de Escrita:

- Considerar os anexos CSV como fonte autoritativa em vez dos requisitos presentes no relatório
- Verificar se os requisitos estão de acordo com o restante conteúdo e cobrem todos os aspectos importantes do sistema
- Analisar a correção da divisão entre requisitos funcionais e não funcionais
- Listar apenas as incoerências encontradas, com causa, seções afectadas e proposta de solução

Restrições: Sem alterar o relatório. Sem propor reescritas além da indicação da proposta de solução.

Análise Crítica. O LLM identificou três incoerências menores entre os requisitos consolidados em CSV e o restante conteúdo do relatório, todas relacionadas com terminologia divergente entre secções ("orçamento" vs "estimativa", "intervenção" vs "reparação", "stock" vs "inventário"). Para cada uma indicou causa, secções afectadas e proposta de harmonização. A equipa validou a análise e aplicou as harmonizações sugeridas, evitando alterações estruturais ao relatório por estarem fora do âmbito da verificação.

A.4 Fase de Especificação Formal

P-21: Especificação de Requisitos de *Software* (SRS) *Propósito:* redigir o SRS como artefacto formal de encerramento da fase de requisitos, consolidando a informação dispersa num inventário rastreável e testável.

Contexto: Projeto das Oficinas Gengis Khan, encerramento da fase de Engenharia de Requisitos posicionado após a elicitação/análise e antes dos Use Cases, alinhado com IEEE 830 e ISO/IEC/IEEE 29148 e fundamentado em Sommerville (2016, cap. 4).

Objetivo: Produzir o documento formal de Especificação de Requisitos de Software, consolidando a informação dispersa num artefacto verificável, rastreável e ponte entre o "o que" e o "como" do sistema.

Instruções de Escrita:

- Aplicar a estrutura obrigatória em sete subseções: introdução e propósito, âmbito do sistema (tabela Dentro/Fora), descrição geral (perspetiva, funções, perfis, ambiente, restrições, suposições), requisitos funcionais consolidados, requisitos não funcionais consolidados, critérios de aceitação por módulo e nota de transição para Use Cases
- Redigir os critérios de aceitação no formato "O sistema deve [X], quando [Y], de forma que [Z]", com dois critérios por módulo
- Classificar restrições como [LEGAL], [TECNICA] ou [ACADEMICA] e suposições como [ASSUNCAO] ou [DEPENDENCIA]
- Manter PT-PT estrito

Restrições: Sem referências a números de tabela absolutos; usar referências cruzadas dinâmicas. Sem desvios da estrutura obrigatória.

Análise Crítica. A produção do SRS foi a tarefa mais estruturada submetida ao LLM, com um *prompt* extenso e prescritivo que especificava integralmente a estrutura do artefacto, as referências bibliográficas a utilizar e os critérios de formatação. O resultado refletiu esta especificidade. O modelo produziu um documento tecnicamente sólido, com referências corretas a Sommerville (2016) e IEEE 830 (1998), e uma estrutura em sete subseções que a equipa considerou adequada ao posicionamento do SRS no ciclo de vida. As tabelas de âmbito e perfis de utilizador foram aceites na íntegra. Os critérios de aceitação por módulo, redigidos no formato normativo solicitado ("O sistema deve X, quando Y, de forma que Z"), foram particularmente úteis para fornecer uma base de verificabilidade aos requisitos. A única intervenção significativa da equipa consistiu na remoção de referências a tabelas específicas ("tabela 14 a 18", "tabela 19") que não correspondiam à numeração final do relatório, substituindo-as por referências cruzadas LaTeX dinâmicas.

A.5 Fase de Modelação Comportamental

P-22: Especificação de Use Cases *Propósito:* especificar todos os Use Cases inferíveis a partir dos requisitos funcionais, mantendo rastreabilidade bidirecional.

Contexto: Sistema de gestão das Oficinas Gengis Khan, fase de especificação detalhada após a validação dos requisitos com os stakeholders, com 44 RF consolidados e necessidade de rastreabilidade bidirecional.

Objetivo: Elaborar a especificação de todos os Use Cases inferíveis dos Requisitos Funcionais, associando a cada UC os requisitos que satisfaz ou influencia.

Instruções de Escrita:

- Aplicar o template anexo com os campos ID, Atores, Descrição, Requisitos, Pré-condição, Pós-condição, Fluxo Normal, Fluxo Alternativo e Fluxo de Exceção
- Admitir que um mesmo requisito pode influenciar mais do que um Use Case
- Seguir rigorosamente os slides de Use Cases da UC de DSS anexos
- Usar *include*, *extend* e generalização apenas quando reduzem efetivamente redundância sem prejudicar a legibilidade

Restrições: Sem uso excessivo dos mecanismos UML de relação entre UC. Sem referenciar identificadores de RF que não constem na versão final consolidada em anexo.

Análise Crítica. Ao contrário do que se observou no *Prompt 14* (Análise e Categorização de Requisitos), que constituiu o ponto de maior exigência humana de toda a Etapa 1, esta interação revelou-se substantivamente mais direta. A verificação e consolidação prévia dos requisitos eliminou, à partida, as fontes de inconsistência que poderiam ter comprometido a derivação dos casos de uso. As dificuldades observadas foram exclusivamente de natureza formal, designadamente a tendência inicial do modelo para se desviar do *template* de especificação pretendido e para abusar dos mecanismos UML de *include*, *extend* e generalização. Após a equipa ter produzido manualmente alguns casos de uso completos e tê-los fornecido como referência adicional, o LLM convergiu rapidamente para o formato esperado e passou a gerar autonomamente cada caso de uso em conformidade, com utilização contida dos referidos mecanismos. A revisão humana incidiu, essencialmente, na validação dos fluxos alternativos e de exceção e na normalização terminológica das pré-condições e pós-condições. Os 29 casos de uso aqui apresentados resultam deste processo e encontram-se mapeados, na sua totalidade, aos 44 requisitos funcionais consolidados no SRS.

P-23: Sincronização de Use Cases (V2) *Propósito:* sincronizar os Use Cases com a versão atualizada dos requisitos funcionais após resolução das ambiguidades, produzindo as especificações em falta.

Contexto: Fase de especificação detalhada das Oficinas Genghis Khan após validação dos requisitos com os stakeholders, com alterações de conteúdo, requisitos novos e mudanças na numeração. Em anexo seguem os requisitos funcionais atualizados em CSV, 6 Use Cases já definidos e os slides de Use Cases da UC de DSS.

Objetivo: Sincronizar os Use Cases existentes com a nova versão dos Requisitos Funcionais e produzir a especificação dos Use Cases restantes, em tabelas no formato do ficheiro em anexo (output em Excel para carregar no Sheets).

Instrucoes de Escrita:

- Admitir que um mesmo requisito pode influenciar mais do que um Use Case
- Seguir a risca os slides de Use Cases em anexo
- Usar include, extend e generalizacao apenas em casos 100% adequados (como UC-02, UC-03 e UC-04 do ficheiro atual)
- Basear-se nos Use Cases enviados como ponto de partida, corrigindo a numeracao dos requisitos onde for necessario

Restricoes: Sem alterar a numeracao dos UC existentes. Sem mecanismos UML de relacao entre UC quando comprometem a legibilidade.

Análise Crítica. O LLM aceitou os seis Use Cases já existentes como ponto de partida, identificou as renumerações necessárias face à versão atualizada dos requisitos e produziu as especificações dos Use Cases em falta, mantendo o formato tabular do template fornecido. A coerência da numeração foi validada pela equipa contra o ficheiro de requisitos atualizado. Não foram introduzidos mecanismos UML de include, extend ou generalização que comprometessem a legibilidade, conforme restrição do prompt.

A.6 Fase de Arquitetura e Design

P-24: Escolha do Padrão Arquitetural *Propósito:* justificar a escolha do padrao arquitetural entre tres candidatos, designadamente monolito em camadas, monolito modular e microservicos.

Contexto: SRS das Oficinas Gengis Khan ja consolidado (44 RF, 5 RNF, 29 casos de uso, equipa de 3-4 mecanicos concorrentes, restricoes de faturacao sequencial e RGPD). Inicio do Capitulo 2 (Arquitetura e Design).

Objetivo: Justificar a escolha do padrao arquitetural entre tres candidatos, designadamente monolito em camadas, monolito modular e microservicos.

Instrucoes de Escrita:

- Analise comparativa em eixos de acoplamento, coesao e complexidade operacional
- Adequacao ao contexto academico e ao volume operacional esperado
- Conclusao sustentada com fronteiras de modulo por dominio funcional

Restricoes: Nao recomendar microservicos sem justificacao explicita face ao custo de coordenacao. Sem decisoes de stack tecnologica especifica.

Análise Crítica. Para a escolha do padrão arquitetural, o LLM produziu, sem necessidade de iteração corretiva, uma análise comparativa organizada em três eixos (complexidade de desenvolvimento, complexidade operacional e adequação ao volume), com argumentação articulada para os três candidatos considerados, designadamente monólito em camadas, monólito modular e microserviços. A recomendação final do modelo recaiu sobre o monólito modular e foi acolhida pela equipa sem reservas, dado que o volume operacional projetado não justifica o custo de coordenação inerente a microserviços e o horizonte temporal académico, conjugado com a dimensão da equipa, torna inoportuna a operação de múltiplos serviços com *pipelines* de *deploy* independentes. Importa, contudo, registar uma observação metodológica que esta interação tornou clara. A vantagem efetiva do LLM em decisões arquiteturais reside menos na capacidade de produzir uma resposta correta e mais na possibilidade de debater rapidamente o espectro de alternativas disponíveis. Em poucos *prompts* é possível percorrer hipóteses que, em discussão interna, exigiriam várias sessões de equipa para serem articuladas com o mesmo grau de detalhe. Nesta circunstância, o LLM aproxima-se da função de um membro adicional da equipa, com conhecimento amplo de paradigmas arquiteturais e de modelos de desenvolvimento, capaz de propor e fundamentar opções que enriquecem o leque de hipóteses considerado. Esta utilização não dispensa, contudo, a discussão crítica coletiva. Sem essa verificação, corre-se o risco de aceitar com excessiva facilidade uma proposta bem articulada apenas pela sua aparente solidez. A prática adotada consistiu, por isso, em interpor uma fase de discussão interna entre a leitura da resposta do modelo e a sua incorporação no relatório, em que cada eixo de comparação foi reavaliado contra as restrições reais do projeto. A contribuição mais útil do modelo foi a recomendação de impor limites de módulo por domínio funcional (clientes, ordens de serviço, inventário, faturação, fornecedores) desde o início, preparando o sistema para uma eventual extração futura de microserviços caso o crescimento o justifique. Esta orientação reflete-se diretamente na estrutura de pastas do *backend*.

P-25: Composição da *Stack* Tecnológica *Propósito:* justificar a escolha da stack tecnologica em quatro eixos, designadamente linguagem e runtime do backend, framework web do backend, linguagem e framework do frontend, e base de dados com camada de acesso.

Contexto: Decisao previa pelo monolito modular em tres camadas.

Restricoes do projeto (equipa de 5 estudantes, 14 semanas para implementacao, tipagem estatica partilhada entre frontend e backend, RGPD e faturacao sequencial portuguesa).

Objetivo: Justificar a escolha da stack tecnologica em quatro eixos, designadamente linguagem e runtime do backend, framework web do backend, linguagem e framework do frontend, e base de dados com camada de acesso.

Instrucoes de Escrita:

- Para cada eixo apresentar alternativas ponderadas e criterios de decisao
- Fundamentar em atributos de qualidade e adequacao ao contexto
- Articular escolhas que favorecam a uniformidade entre camadas

Restricoes: Nao favorecer escolhas por moda. Sem detalhes de implementacao por modulo.

Análise Crítica. Foi nesta interação que ocorreu, pela primeira vez nesta etapa, algo que não se tinha observado em nenhum dos *prompts* anteriores. O LLM foi contra uma decisão que a equipa já tinha tomado. A intenção inicial era usar *C#* no *backend* e *TypeScript* no *frontend*. Confrontado com esta escolha, o modelo propôs em alternativa que ambas as camadas usassem *TypeScript*, argumentando que adotar a mesma linguagem evita o esforço de alternar entre paradigmas, permite reutilizar tipos e validações entre *frontend* e *backend* e reduz o número de ferramentas a dominar num horizonte temporal apertado. Reconheceu a maturidade dos ecossistemas alternativos, mas concluiu que o seu peso não compensava a perda de uniformidade num projeto académico com poucas semanas para implementar. A equipa aceitou o argumento e adotou *TypeScript* em ambas as camadas, deixando de lado a opção inicial de *C#*. Esta interação revela uma característica do trabalho com LLMs que normalmente passa despercebida. Quem não usa estes modelos no dia a dia tende a vê-los como executores de instruções, que recebem um pedido e devolvem uma resposta. Aqui, o modelo não se limitou a cumprir o que lhe foi pedido. Questionou a premissa, propôs uma alternativa e fundamentou-a. Importa, ainda assim, manter a guarda, dado que um argumento bem escrito não é, por si só, um argumento correto. Antes de aceitar a proposta, a equipa testou cada um dos benefícios invocados contra as restrições reais do projeto, e foi essa verificação que sustentou a aceitação final e não a fluência da resposta. Em retrospectiva, esta foi uma das contribuições mais relevantes do LLM ao longo da etapa, pelo impacto estrutural da decisão e pela dificuldade de a reverter mais tarde.

P-26: Organização em Monorepo *Propósito:* comparar a opção de dois repositórios Git separados com a opção de um monorepo gerido por pnpm workspaces e Turborepo, recomendando uma das alternativas com tradeoffs residuais explícitos.

Contexto: Sistema das Oficinas Gengis Khan, monolito modular em TypeScript com frontend React e backend Fastify. Decisão de organização do código entre dois repositórios separados ou um monorepo unificado.

Objetivo: Comparar a opção de dois repositórios Git separados com a opção de um monorepo gerido por pnpm workspaces e Turborepo, recomendando uma das alternativas com tradeoffs residuais explícitos.

Instruções de Escrita:

- Considerar partilha de tipos entre camadas como factor primário
- Avaliar ciclo de vida de builds e overhead de coordenação em refatorações cross-cutting

- Considerar curva de aprendizagem para uma equipa sem experiencia previa com monorepos

Restricoes: Recomendar uma opcao concreta. Sem ambiguidade na conclusao final.

Análise Crítica. A análise produzida pelo LLM favoreceu de forma explícita a opção de *monorepo*, fundamentando a decisão na capacidade de partilha literal de código entre pacotes e na existência de ferramentas maduras (*pnpm workspaces*, *Turborepo*) que automatizam a orquestração de *builds* dependentes. O modelo alertou, contudo, para dois riscos residuais, designadamente a necessidade de disciplina na definição de fronteiras entre pacotes (sob pena de o *monorepo* degenerar num sistema com elevado acoplamento estrutural) e o aumento da complexidade inicial de configuração do *workspace*. A equipa aceitou a recomendação e previu a mitigação do primeiro risco através da definição, em fase de design, de um conjunto restrito de pacotes com responsabilidades únicas.

P-27: Mecanismo de Partilha de Contratos *Propósito:* propor a estrategia mais adequada para garantir essa sincronizacao, comparando tres abordagens, designadamente geracao de tipos TypeScript a partir de OpenAPI, uso de tRPC para expor tipos diretamente, e definicao manual de esquemas Zod num pacote partilhado.

Contexto: Sistema das Oficinas Gengis Khan, monorepo TypeScript com frontend e backend. Necessidade de manter os contratos de dados sincronizados entre as duas camadas ao longo do ciclo de vida.

Objetivo: Propor a estrategia mais adequada para garantir essa sincronizacao, comparando tres abordagens, designadamente geracao de tipos TypeScript a partir de OpenAPI, uso de tRPC para expor tipos diretamente, e definicao manual de esquemas Zod num pacote partilhado.

Instrucoes de Escrita:

- Considerar a necessidade conjunta de validacao em runtime no backend e de verificacao estatica no frontend
- Avaliar acoplamento entre camadas, poder expressivo e sobrecarga cognitiva
- Recomendar uma abordagem com justificacao explicita

Restricoes: Sem ambiguidade na recomendacao final.

Análise Crítica. A justificação apresentada pelo LLM para a recomendação foi particularmente bem articulada, e a equipa subscreveu a fundamentação sem dificuldade. Esta interação confirma uma característica do modelo já observada em interações anteriores, designadamente

a sua capacidade de argumentar de forma clara e persuasiva por que motivo uma opção é preferível às restantes. Em decisões arquiteturais, esta competência traduz-se numa intuição que se aproxima, em muitos casos, da que se espera de um engenheiro experiente, permitindo que o modelo proponha alternativas, identifique *tradeoffs* e sustente cada escolha com vantagens concretas. A equipa tem aprendido, ao longo desta etapa, a tratar o modelo como ponto de partida sólido para a discussão arquitetural. Esta competência não dispensa, contudo, a verificação humana. Em diversos momentos das interações conduzidas, propostas igualmente bem articuladas revelaram-se, à inspeção atenta, parcialmente desadequadas ou simplesmente desligadas das restrições reais do projeto. A fluência com que o modelo encadeia argumentos pode dar a impressão de que qualquer recomendação faz sentido, ainda que, à leitura crítica, alguns dos pressupostos invocados não se sustentem. A equipa tem mantido, por princípio, essa leitura crítica, validando cada proposta contra o contexto efetivo do trabalho. Aceitar uma recomendação apenas pela clareza da sua exposição constitui, na realidade, um dos riscos mais prováveis ao trabalhar com um LLM em decisões com impacto estrutural.

P-28: Modelo de Persistência *Propósito:* analisar a adequação do PostgreSQL face a alternativas NoSQL (MongoDB, DynamoDB) e recomendar a solução de persistência, indicando também o ORM associado.

Contexto: Sistema das Oficinas Gengis Khan com restrição legal de faturação sequencial (Decreto-Lei 28/2019), transações financeiras (Invoice, Payment) e auditoria de alterações em ordens de serviço.

Objetivo: Analisar a adequação do PostgreSQL face a alternativas NoSQL (MongoDB, DynamoDB) e recomendar a solução de persistência, indicando também o ORM associado.

Instruções de Escrita:

- Discutir propriedades ACID, constraints de unicidade e sequências monótonicas
- Considerar suporte nativo a migrações versionadas
- Justificar a inadequação das alternativas NoSQL para o domínio fiscal

Restrições: Recomendar uma solução concreta. Sem detalhes de schema específico.

Análise Crítica. A escolha de *PostgreSQL* e *Prisma* havia já sido tomada na discussão da *stack* tecnológica, pelo que esta interação serviu sobretudo para validar essas escolhas face às exigências específicas do domínio, em especial a restrição legal de faturação sequencial. A justificação produzida pelo LLM foi sólida, articulou com clareza os requisitos não negociáveis que tornam o *PostgreSQL* adequado a este tipo de domínio fiscal, e a equipa subscreveu-a sem reservas.

P-29: Modelo de Implantação *Propósito:* delinear o modelo de implantacao para tres ambientes (desenvolvimento local, integracao continua, producao) e identificar artefactos por ambiente e pontos de divergencia aceitaveis.

Contexto: Sistema das Oficinas Gengis Khan com stack Node.js, React, PostgreSQL e Prisma, com necessidade de paridade entre ambientes e uso de Supabase CLI apenas em desenvolvimento.

Objetivo: Delinear o modelo de implantacao para tres ambientes (desenvolvimento local, integracao continua, producao) e identificar artefactos por ambiente e pontos de divergencia aceitaveis.

Instrucoes de Escrita:

- Distinguir infraestrutura de conveniencia (local) da infraestrutura efectiva de producao
- Enumerar artefactos por ambiente (imagens Docker, scripts de migracao, variaveis de ambiente)
- Discutir paridade de comportamento e portabilidade

Restricoes: Sem detalhes de configuracao de runner especifico. Sem mencao a provedores cloud concretos.

Análise Crítica. A proposta do LLM organizou o modelo de implantação em torno de uma decomposição clara, ao separar deliberadamente a vista de desenvolvimento da vista de produção. O modelo justificou esta separação com o argumento de que aceitar uma assimetria controlada entre os dois ambientes simplifica significativamente o trabalho do programador no dia a dia, sem comprometer a integridade do artefacto que será implantado em produção. Para enquadrar esta ideia, introduziu o conceito de *infraestrutura de conveniência*, que designa as ferramentas adotadas apenas em desenvolvimento para reduzir fricção, sem impacto no artefacto final. O *Supabase* CLI enquadra-se nesta categoria, permitindo arranque imediato de uma instância *PostgreSQL* local sem que essa dependência se propague à produção. O modelo identificou ainda a armadilha mais comum desta abordagem, designadamente a dependência accidental de serviços específicos da plataforma local, e recomendou a documentação explícita do que é aceitável usar localmente e do que deve permanecer portátil. A equipa considerou a proposta particularmente convincente. A tentação inicial seria procurar paridade integral entre ambientes, sob a premissa de que qualquer divergência introduz risco. A sugestão do modelo coloca, contudo, o problema sob outra perspetiva, mostrando que o custo de manter paridade integral durante o desenvolvimento ultrapassa muitas vezes o risco que essa paridade pretende mitigar, e que uma separação consciente, acompanhada de documentação rigorosa, é frequentemente preferível. A recomendação foi incorporada no registo de Decisões Arquiteturais apresentado na Subsecção 2.1.7. Importa, ainda assim, manter a vigilância sobre o pacto implícito desta abordagem, dado que a fronteira entre conveniência local e dependência produtiva é, por natureza, ténue e tende a ser atravessada inadvertidamente.

P-30: Diagrama de Classes UML *Propósito:* propor um diagrama de classes UML que cubra todas as entidades e seja legível em formato A4, recorrendo a particionamento por subsistema funcional.

Contexto: Modelo de domínio das Oficinas Gengis Khan (Capítulo 1) e schema Prisma com dezasseis entidades, valores monetários, enumerações de domínio, colunas Json para estrutura em maturação e necessidade de auditoria.

Objetivo: Propor um diagrama de classes UML que cubra todas as entidades e seja legível em formato A4, recorrendo a particionamento por subsistema funcional.

Instruções de Escrita:

- Apresentar atributos essenciais com tipos UML genéricos e multiplicidades em cada associação
- Identificar enumerações de domínio e os pontos onde são referenciadas
- Distinguir composição de associação simples consoante o ciclo de vida do filho
- Agrupar as classes em N sub-diagramas por área funcional em vez de um diagrama monolítico
- Justificar decisões de modelação não triviais (chave primária, Decimal vs Float, colunas Json, soft-delete, autoincrement, auditoria)

Restrições: Sem detalhes de implementação SQL. Sem omissão de multiplicidades.

Análise Crítica. A produção dos diagramas de classes constituiu uma das tarefas em que o modelo se mostrou menos competente desde o início. As primeiras tentativas resultaram em diagramas mal estruturados, com elementos sobrepostos, multiplicidades omissas ou erradas e legibilidade muito reduzida em formato A4. A reformulação do *prompt* para impor a partição por subsistema melhorou parcialmente o resultado, mas continuou aquém da qualidade de apresentação exigida por um relatório académico. Para contornar esta limitação, a equipa recorreu a uma *Agent Skill* instalada localmente no *Claude*. As *Agent Skills* são módulos de extensão introduzidos pela *Anthropic*, materializados em diretórios que contêm um ficheiro *SKILL.md* e recursos auxiliares (instruções dedicadas, exemplos, *scripts* e *templates*), que o modelo carrega dinamicamente sempre que reconhece que a tarefa em curso corresponde ao seu domínio de aplicação. No presente caso, foi instalada uma *Skill* especializada em geração de diagramas *TikZ*, o que dotou o *Claude* de instruções específicas, padrões de referência e capacidade de validação do código gerado. Trata-se de um mecanismo recente no ecossistema *Claude*, que ilustra de forma concreta como o desempenho de um LLM em tarefas técnicas particulares pode ser significativamente elevado através da disponibilização de contexto adequado. A partir desta configuração, o modelo identificou três agrupamentos coerentes (núcleo

operacional, aprovisionamento e inventário, faturação, configuração e notificações) e produziu três diagramas autocontidos em que apenas duas associações cruzam fronteiras de subsistema, designadamente *InterventionPart*, que liga o inventário ao núcleo operacional, e *Invoice*, que liga a faturação ao mesmo núcleo. Esta partição foi adotada pela equipa sem revisão, por revelar uma decomposição funcional que não havia sido tornada explícita durante a elicitação de requisitos. O LLM antecipou ainda seis decisões de modelação não triviais, todas discutidas na Subsecção 2.2.6, designadamente o uso de *Decimal* de precisão fixa para valores monetários em alternativa a *Float*, o recurso a colunas *Json* para diagnósticos e orçamentos com estrutura ainda em maturação, a manutenção de identificador técnico opaco mesmo quando existe uma chave natural utilizável, o *soft-delete* via campo *isArchived* em entidades de catálogo, o autoincremento separado para números de negócio e o padrão de auditoria por entidade dedicada.

P-31: Modelo Comportamental: Diagramas de Sequência *Propósito:* identificar os fluxos comportamentais que justificam documentacao por diagramas de sequencia e descrever as interacoes entre os participantes (cliente, controlador HTTP, servico de dominio, repositorio).

Contexto: SRS das Oficinas Gengis Khan com 44 RF, 5 RNF e 29 casos de uso, modelo estrutural de classes ja consolidado e necessidade de evidenciar regras de negocio nao triviais.

Objetivo: Identificar os fluxos comportamentais que justificam documentacao por diagramas de sequencia e descrever as interacoes entre os participantes (cliente, controlador HTTP, servico de dominio, repositorio).

Instrucoes de Escrita:

- Privilegiar fluxos representativos com invariantes nao triviais sobre cobertura exhaustiva
- Evidenciar pontos de validacao cruzada entre modulos
- Marcar transicoes de estado com pre-condicoes
- Identificar operacoes que devem executar atomicamente sob a mesma transacao

Restricoes: Sem operacoes de leitura simples nem variantes parametricas do mesmo endpoint.

Análise Crítica. A geração dos diagramas de sequência apresentou inicialmente o mesmo tipo de dificuldade observado nos diagramas de classes, designadamente a inadequação do *output* bruto do modelo a uma renderização legível em formato académico. A solução passou, novamente, pelo recurso a uma *skill* dedicada do *Claude*, desta vez especificamente concebida para diagramas de sequência, que produziu artefactos imediatamente utilizáveis sem necessidade de retoques manuais significativos. A reutilização desta família de *skills* confirma uma

observação que se foi consolidando ao longo da etapa, designadamente que a delegação de subtarefas a ferramentas especializadas tende a oferecer resultados substancialmente superiores aos do modelo genérico. Quanto ao conteúdo dos diagramas, a equipa restringiu a seleção inicialmente alargada do modelo, que contemplava também operações de listagem e leitura por identificador, aos quatro fluxos cujo desenho envolve regras de negócio efetivamente verificáveis, designadamente a autenticação com fixação do papel no *token*, a criação de ordem de serviço com validação cascata cliente/trotinete, a transição de estado da ordem sob máquina de estados com restrição por papel, e a emissão de fatura com orquestração multi-módulo. A contribuição mais útil do LLM foi a identificação explícita das fronteiras transacionais a observar em cada fluxo, que se refletem nos diagramas apresentados e serão observadas pela camada de implementação. A sugestão de incluir um quinto fluxo de notificação assíncrona foi recusada, dado que o canal de *email* é acessório, com falha tolerada, e o seu detalhe não acrescenta regras adicionais ao quadro comportamental do sistema.

P-32: Modelação de Máquinas de Estado *Propósito:* formalizar as máquinas de estado das três entidades de ciclo de vida não trivial (ordem de serviço, cronómetro de intervenção, estado de pagamento da fatura).

Contexto: SRS das Oficinas Gengis Khan, com REQ-011 a REQ-014 (estados da ordem de serviço, sete estados fechados, registo histórico e restrição de papel no estado terminal), REQ-018 (cronómetro de intervenção com pausa e retoma) e REQ-032 (estado de pagamento da fatura).

Objetivo: Formalizar as máquinas de estado das três entidades de ciclo de vida não trivial (ordem de serviço, cronómetro de intervenção, estado de pagamento da fatura).

Instruções de Escrita:

- Indicar estado inicial, conjunto exaustivo de estados, transições admitidas, condições de despoletamento e estados terminais
- Distinguir transições accionadas por humano das despoletadas pelo sistema
- Explicitar restrições de papel sempre que se apliquem

Restrições: Sem máquinas para entidades de ciclo de vida trivial. Sem modelação do orçamento como entidade autónoma.

Análise Crítica. À semelhança dos diagramas anteriores, a geração das máquinas de estado beneficiou de imediato das *skills* do *Claude* já instaladas, dispensando a iteração corretiva inicial e produzindo os artefactos no formato pretendido logo na primeira tentativa. Quanto ao conteúdo, a primeira proposta omitiu a transição inversa de Em Reparação para Aguarda Peças, que decorre da prática efetiva da oficina e que foi acrescentada após reformulação do *prompt*. A contribuição mais relevante do modelo foi a sugestão de explicitar a guarda de

papel (role = manager) como condição da transição para Entregue ao Cliente, opção adotada pela equipa por aproximar a representação visual da semântica efetiva do código. O LLM propôs ainda uma máquina de estados para o orçamento, recusada pela equipa por este ser tratado como atributo flexível da ordem de serviço (campo Json, conforme discutido na Subsecção 2.2.6).

P-33: Especificação da API REST *Propósito:* especificar o contrato HTTP da API REST exposta pelo backend, identificando endpoints e convenções transversais.

Contexto: Módulos do backend consolidados (auth, customers, scooters, service-orders, interventions, inventory, suppliers-procurement, billing, notifications, configuration, reports) e requisitos funcionais REQ-001 a REQ-047 do SRS.

Objetivo: Especificar o contrato HTTP da API REST exposta pelo backend, identificando endpoints e convenções transversais.

Instruções de Escrita:

- Para cada endpoint indicar método HTTP, caminho, objetivo funcional, papéis autorizados (manager, mechanic) e principais códigos de resposta
- Fixar convenções transversais (versionamento, formato de pedido e resposta, autenticação, autorização, validação de schema, formato uniforme de erro)
- Privilegiar fidelidade ao contrato implementado

Restrições: Sem extensões futuras hipotéticas. Sem detalhes de payload por endpoint.

Análise Crítica. O LLM produziu inicialmente um catálogo derivado apenas dos requisitos do SRS, omitindo recursos auxiliares posteriormente decididos em sede de *design*: geração de PDF da fatura, controlo granular do cronómetro de intervenção, geração automática de encomendas a partir de peças com *stock* mínimo e o módulo de relatórios. A equipa completou o catálogo a partir do código já implementado, garantindo que a especificação reflete fielmente o contrato exposto. A contribuição mais relevante do modelo foi a sugestão de uniformizar a forma de erro através de um envelope JSON com código simbólico legível (*validation_error*, *not_found*, *invalid_status_transition*, *forbidden*) em vez de mensagens livres em texto, opção adotada por permitir o tratamento estruturado de erros no *frontend* sem dependência sobre a literalidade da mensagem. O modelo recomendou ainda o versionamento por prefixo de caminho (*/api/v1/*) em alternativa ao versionamento por cabeçalho *Accept*, opção mantida pela equipa por ser mais legível em *logs* e em depuração manual.

P-34: Modelo de Componentes *Propósito:* produzir o modelo de componentes do sistema em UML 2.5, identificando processos autônomos em tempo de execução, componentes externos e componentes internos do backend.

Contexto: Stack consolidada para as Oficinas Gengis Khan, com Next.js no frontend, Fastify e Node.js 20 no backend, PostgreSQL 16 com Prisma, autenticação JWT em cookie HTTP-only, mailer SMTP e scheduler interno para jobs periódicos.

Objetivo: Produzir o modelo de componentes do sistema em UML 2.5, identificando processos autônomos em tempo de execução, componentes externos e componentes internos do backend.

Instruções de Escrita:

- Distinguir componentes de aplicação, de infraestrutura e externos com marcação visual diferenciada
- Agrupar os componentes internos pelos subsistemas funcionais já fixados (operacional, inventário, faturação, configuração, notificações, relatórios)
- Indicar o protocolo de comunicação em cada conexão
- Evidenciar a fronteira de processo entre frontend e backend

Restrições: Sem omissão do scheduler interno. Sem misturar vista lógica com vista de componentes.

Análise Crítica. A esta altura da etapa de *design*, a *Agent Skill* de diagramas *TikZ* introduzida na Subseção 2.2 continuava instalada, o que permitiu que a geração do presente diagrama de componentes decorresse de forma muito mais expedita do que teria sido possível sem essa configuração prévia. O modelo produziu, à primeira tentativa, uma decomposição substancialmente correta e visualmente cuidada, identificando três classes de componentes (aplicação, infraestrutura e externos), respeitando a fronteira de processo entre *frontend* e *backend* e organizando os módulos internos pelos subsistemas funcionais previamente fixados. A intervenção da equipa limitou-se a acrescentar o componente de *scheduler* interno, responsável pela transição automática *pending* para *overdue* sobre as faturas (Secção 2.4), que tinha sido omitido pelo modelo apesar de constar do contexto fornecido no *prompt*. A contribuição mais relevante do modelo foi a sugestão de tornar visível, no diagrama, a fronteira de processo entre *frontend* e *backend*, dado que ambos correm em processos *Node.js* distintos com vidas independentes ainda que partilhem o mesmo *monorepo* ao nível do código-fonte. Esta visibilidade evita a interpretação errada, frequente em arquiteturas SPA, de que o *frontend* é apenas um conjunto de ficheiros estáticos servidos pelo *backend*. A marcação visual diferenciada dos componentes externos (fronteira tracejada) e dos componentes de infraestrutura (cor distinta dos componentes de aplicação) foi também adotada pela equipa, por aumentar a legibilidade do diagrama.

P-35: Design de Interface *Propósito:* propor um design de interface coerente para a aplicação frontend, fixando princípios visuais, estrutura de navegação e ecrãs principais para o fluxo operacional completo (recepção, diagnóstico, orçamento, reparação, conclusão, faturação).

Contexto: Sistema das Oficinas Gengis Khan especificado (44 RF, 5 RNF, 29 casos de uso, perfis manager e mechanic), API REST documentada e uso previsto em ambiente de oficina com tablets e iluminação variável.

Objetivo: Propor um design de interface coerente para a aplicação frontend, fixando princípios visuais, estrutura de navegação e ecrãs principais para o fluxo operacional completo (recepção, diagnóstico, orçamento, reparação, conclusão, faturação).

Instruções de Escrita:

- Definir tema, paleta, tipografia e iconografia
- Para cada ecrã indicar função, elementos críticos e requisitos funcionais associados
- Aplicar o princípio de mostrar sempre o próximo passo legítimo da máquina de estados
- Privilegiar legibilidade em iluminação variável e áreas tácteis adequadas a tablet

Restrições: Sem tema claro padrão. Sem oferta de transições de estado inválidas.

Análise Crítica. A primeira proposta do LLM apresentou um *design* de tema claro, padrão na indústria de aplicações empresariais. Os requisitos consolidados na Etapa 1 não fixam exigências sobre o tema visual da aplicação, pelo que a equipa exerceu nesta matéria alguma liberdade interpretativa e optou, em alternativa à proposta inicial, por um tema escuro. A escolha sustenta-se na percepção de que o ambiente físico de uma oficina apresenta iluminação variável e que um fundo escuro tende a reduzir a fadiga ocular durante turnos prolongados. A contribuição mais útil do modelo foi o princípio de *próximo passo legítimo*, segundo o qual cada ecrã de detalhe de ordem de serviço deve apresentar, de forma proeminente, apenas as transições admissíveis a partir do estado corrente, calculadas pelo *frontend* a partir da máquina de estados documentada na Secção 2.4. Esta opção elimina a possibilidade de o utilizador tentar transições inválidas e reduz a carga cognitiva associada à interpretação do estado atual da ordem. O modelo recomendou ainda a uniformização da iconografia através do conjunto *Material Symbols*, opção adotada por reconhecibilidade universal e por estar disponível sob licença permissiva, sem custo de licenciamento. A produção dos protótipos de interface propriamente ditos foi conduzida com recurso ao *Stitch AI*, ferramenta da *Google* dedicada à geração de *mock-ups* a partir de descrições textuais. O *Stitch AI* permitiu transpor rapidamente os princípios visuais e a estrutura de navegação acordados com o LLM para imagens concretas dos ecrãs principais, servindo de instrumento de validação visual antes da fase de implementação. Importa, contudo, registar que, se este trabalho fosse iniciado hoje, a

equipa optaria pelo *Claude Design* em alternativa. Lançado pela *Anthropic* em abril de 2026 e assente no modelo *Claude Opus 4.7*, o *Claude Design* oferece funcionalidades comparáveis de geração de protótipos a partir de *prompts*, mas integra-se nativamente no ecossistema *Claude* já utilizado em todo o resto do projeto, designadamente na possibilidade de transitar diretamente do protótipo para a implementação através do *Claude Code*. Esta integração reduziria a fricção entre as fases de *design* e desenvolvimento e harmonizaria as ferramentas com a restante *stack* assistida por IA do projeto.

A.7 Fase de Implementação

P-36: Metodologia de Desenvolvimento *Propósito:* propor uma metodologia de desenvolvimento adequada ao contexto, distinguindo o ciclo de vida do projeto académico do ciclo de produção de código.

Contexto: Arquitetura do Capítulo 2 (monolito modular em três camadas, monorepo pnpm, contratos Zod, PostgreSQL via Prisma), calendário de três semanas e equipa de cinco estudantes a usar o LLM como parceiro de produção de código.

Objetivo: Propor uma metodologia de desenvolvimento adequada ao contexto, distinguindo o ciclo de vida do projeto académico do ciclo de produção de código.

Instruções de Escrita:

- Tratar a cascata como ciclo de vida do projeto académico (entrega única)
- Tratar o ciclo de código com LLM como incremental, por módulo
- Discutir granularidade da supervisão humana e os compromissos entre autonomia e revisão
- Identificar pontos não delegáveis ao LLM

Restrições: Sem cerimónias ágeis formais nem sprints de duração fixa. Sem terminologia industrial desproporcionada a equipa de cinco.

Análise Crítica. A primeira proposta do LLM apontou para um modelo de desenvolvimento ágil estruturado em *sprints* semanais com cerimónias formalizadas, padrão da indústria mas desproporcionado à dimensão da equipa e ao horizonte de três semanas. A proposta foi rejeitada por sobrepor processo desnecessário a um trabalho cuja unidade natural é o módulo, não o intervalo temporal. Após reformulação, o modelo convergiu para o esquema adotado, em que cada módulo é tomado como unidade elementar de trabalho, formulado como *prompt* direto ao LLM, gerado pelo modelo, revisto pela equipa e integrado por *pull request*. A contribuição mais relevante foi a fixação de três pontos não delegáveis ao LLM, designadamente a assinatura pública de cada módulo, a validação de cada *pull request* antes do *merge* e a gestão das migrações de *schema* relacional. A definição desta unidade de trabalho constituiu, contudo,

apenas o primeiro passo de um exercício metodológico mais subtil do que inicialmente antecipado, e que se revelou um dos pontos mais interessantes desta etapa. A equipa começou por solicitar a implementação completa de cada módulo numa única interação, mas constatou rapidamente que o LLM perdia precisão quando confrontado com pedidos demasiado abrangentes, omitindo detalhes, simplificando regras de negócio ou introduzindo inconsistências silenciosas. A resposta foi quebrar cada módulo em blocos ainda mais pequenos do que a sua fronteira natural sugeria, organizando a implementação em sucessivas iterações curtas, cada uma com âmbito estrito e validável de forma independente. Esta granularidade revelou-se essencial para manter a coerência do código gerado e para garantir que cada passo se mantinha alinhado com a arquitetura definida no Capítulo 2. O percurso detalhado deste exercício é apresentado na Secção 3.3, dedicada à implementação incremental por módulos.

P-37: Configuração do Ambiente de Trabalho *Propósito:* recomendar a configuracao de ambiente de desenvolvimento que minimize friccao, uniformize comandos comuns, valide cada pull request antes do merge e permita reposicao rapida da base de dados.

Contexto: Monorepo pnpm com apps/api (Fastify e TypeScript), apps/web (React e Vite) e packages/contracts (Zod), equipa de cinco estudantes em paralelo ao longo de tres semanas e necessidade de paridade aceitavel entre local e producao.

Objetivo: Recomendar a configuracao de ambiente de desenvolvimento que minimize friccao, uniformize comandos comuns, valide cada pull request antes do merge e permita reposicao rapida da base de dados .

Instrucoes de Escrita:

- Discutir alternativas para a provisao local de PostgreSQL (nativa, Docker Compose, Supabase CLI)
- Discutir alternativas para a automacao de comandos (Makefile, scripts pnpm, Taskfile)
- Discutir a estrutura do pipeline de integracao continua
- Para cada decisao indicar a opcao recomendada e as razoes

Restricoes: Sem matrizes de versao desnecessarias nem jobs paralelos sem ganho mensuravel.

Análise Crítica. O contributo mais transformador do LLM nesta interação foi a uniformização de todas as operações recorrentes do projeto sob uma interface única, designadamente um *Makefile* com cerca de duas dezenas de *targets* que cobrem o ciclo completo de desenvolvimento, da instalação inicial à execução de testes, passando pela aplicação de migrações, pelo arranque do ambiente local e pela reposição da base de dados. A consequência prática desta concentração foi alterar de forma substantiva a relação da equipa com a configuração do projeto. A partir do momento em que cada operação é invocada por um comando único e

estável, deixou de ser necessário compreender em detalhe os *scripts* subjacentes, as variáveis de ambiente, as opções dos contentores ou os passos manuais de instalação de cada componente. A equipa concentra-se hoje na verificação ocasional dos ficheiros gerados e na validação do comportamento observável do sistema, transitando a interação com o ambiente para um plano essencialmente declarativo. Esta mudança de relação com a configuração comporta uma vantagem evidente, traduzida na redução drástica do tempo de *onboarding* e da carga cognitiva de manutenção, mas também uma contrapartida menos óbvia. A familiaridade com os bastidores do ambiente, que tradicionalmente se adquiria pela leitura e edição dos *scripts* de configuração, é hoje progressivamente substituída pela confiança no *Makefile* e nos artefactos gerados pelo modelo. Esta confiança é razoável enquanto os comandos se comportam como esperado, mas torna a equipa mais dependente do LLM caso surja uma falha que exija intervenção fora dos *targets* pré-existentes. As decisões específicas sobre alternativas de provisão local de *PostgreSQL*, automação de comandos e estrutura do *pipeline* de CI foram acolhidas com pequenos ajustes pela equipa, dispensando-se aqui o seu detalhe por terem sido já discutidas no capítulo anterior.

P-38: Estratégia de Implementação por Módulos *Propósito:* propor uma ordem de implementacao que minimize bloqueios por dependencias cruzadas e maximize o ganho incremental do LLM.

Contexto: Onze modulos identificados na arquitetura (auth, customers, scooters, service-orders, interventions, inventory, suppliers-procurement, billing, notifications, configuration, reports) organizados em quatro subsistemas (operacional, inventario, faturacao e suporte, relatorios transversais).

Objetivo: Propor uma ordem de implementacao que minimize bloqueios por dependencias cruzadas e maximize o ganho incremental do LLM.

Instrucoes de Escrita:

- Para cada modulo indicar ficheiros minimos, esquemas Zod necessarios , endpoints HTTP e regras de negocio criticas
- Identificar dependencias entre modulos e refleti-las na ordem proposta
- Indicar os testes minimos a acompanhar cada entrega

Restricoes: Sem implementacao detalhada de cada modulo. Sem reordenacoes sem justificacao por dependencia.

Análise Crítica. O LLM propôs uma ordem coerente que privilegiou os módulos de catálogo antes dos módulos transacionais, justificando a opção pela menor complexidade dos primeiros e pela sua condição de pré-requisito dos segundos. A equipa adotou esta ordem com uma alteração, antecipando o módulo auth para primeiro lugar, dado que toda a verificação de papel depende da emissão de *token* de sessão. A sugestão de produzir, para cada módulo, um

teste mínimo de integração do *fluxo normal*, a execução nominal do sistema sem condições de exceção, acompanhando a entrega foi adotada. A presença do teste forçou a verificação efetiva dos contratos contra *endpoints* reais, expondo divergências que a inspeção visual do código deixaria passar.

P-39: Tarefas de Geração de Código por LLM *Propósito:* identificar os tipos de tarefas em que o LLM se mostrou mais e menos eficaz e descrever os padrões de prompt e os modos de falha observados.

Contexto: Implementação dos onze módulos do backend com recurso ao LLM, ao longo de três semanas, e necessidade de caracterizar criticamente o desempenho do modelo em diferentes tarefas.

Objetivo: Identificar os tipos de tarefas em que o LLM se mostrou mais e menos eficaz e descrever os padrões de prompt e os modos de falha observados.

Instruções de Escrita:

- Distinguir geração de boilerplate estrutural, tradução de regras de negócio, escrita de testes, sugestão de refactoring e revisão de pull requests
- Para cada categoria descrever o padrão de prompt mais eficaz e os modos de falha típicos
- Graduar a confiança no modelo com níveis não binários

Restrições: Sem juízos absolutos. Sem omitir modos de falha em áreas mais sensíveis (regras fiscais, refactoring profundo).

Análise Crítica. A análise do LLM sobre o seu próprio desempenho revelou-se moderadamente fiável. Identificou corretamente as áreas em que se sentiu mais à vontade (*boilerplate* estrutural, esquemas *Zod*, testes do *fluxo normal*) mas subestimou a sua taxa de erro em tradução de regras de negócio fiscais e em *refactoring* de código já complexo. A equipa cruzou esta autoavaliação com a sua observação direta e reteve uma taxonomia mais conservadora, na qual a confiança no modelo é graduada em quatro níveis em vez dos níveis binários sugeridos. A contribuição mais útil foi a sistematização dos padrões de *prompt* associados a cada categoria, que permitiu reutilizar formulações eficazes em vez de reinventar a estrutura do *prompt* a cada novo módulo. Para além desta autoavaliação, a equipa retira da experiência de implementação uma observação própria sobre a qualidade do código gerado. Em termos globais, o resultado foi considerado positivo, e essa apreciação está diretamente associada à estratégia incremental adotada e descrita na Secção 3.3. A decomposição da implementação em blocos pequenos, cada um produzido a partir de um *prompt* focado e validado de forma isolada antes de avançar para o seguinte, permitiu uma granularidade de inspeção e de teste que seria inviável caso o LLM tivesse sido pedido a entregar módulos completos numa única interação. Esta abordagem reduziu o risco de erros silenciosos acumulados, manteve a base de código sempre em estado

verificável e elevou de forma consistente a confiança da equipa em cada artefacto integrado, fatores que, em conjunto, sustentam a avaliação favorável do código resultante.

P-40: Geração do Módulo *Service-Orders* *Propósito:* implementar o modulo service-orders em apps/api/src/modules/service-orders com os ficheiros index.ts, service.ts e repository.ts.

Contexto: Backend Fastify+TypeScript com modulos customers e scooters ja implementados em apps/api/src/modules como referencia de estilo , contratos Zod em packages/contracts/src/service-orders e maquina de estados definida no Capitulo 2.

Objetivo: Implementar o modulo service-orders em apps/api/src/modules/service-orders com os ficheiros index.ts, service.ts e repository.ts.

Instrucoes de Escrita:

- Seguir o padrao estrutural ja estabelecido em customers e scooters
- Expor os endpoints GET /api/v1/service-orders (listagem com filtros por estado, NIF e numero de serie), POST, GET /:id, PATCH /:id, PATCH /:id/status, PATCH /:id/diagnosis, PATCH /:id/budget, POST /:id/budget/approval, GET /:id/history
- Aplicar requireRole(['manager','mechanic']) no inicio de cada handler e exigir role=manager na transicao para delivered
- Persistir a criacao da ordem em transacao Prisma unica, incluindo ServiceOrderHistoryEntry com action='created'
- Validar transicoes contra validTransitions e devolver appError(409, 'invalid_status_transition') em violacao
- Manter o handler fino: regras de negocio so em service.ts, acesso a Prisma so em repository.ts

Restricoes: Sem regras de negocio no handler. Sem acesso a Prisma fora do repository. Sem esquemas Zod proprios; usar os de packages/contracts.

Análise Crítica. A análise crítica deste *prompt*, retrospectivamente conduzida pela equipa, identifica três pontos fortes e dois pontos fracos. Como pontos fortes, a indicação explícita do padrão estrutural a seguir (index.ts, service.ts, repository.ts) com referência a módulos já consolidados (customers, scooters) permitiu ao modelo replicar quase sem desvios o estilo da equipa, a enumeração exhaustiva dos *endpoints* com método HTTP e caminho eliminou ambiguidade quanto ao contrato a expor, e a referência direta aos esquemas *Zod* já existentes em packages/contracts forçou aderência aos contratos partilhados em vez de o modelo gerar os seus próprios. Como pontos fracos, a descrição da máquina de estados foi resumida demais (*a transição para delivered exige role=manager*), pelo que o modelo a implementou inicialmente como *warning* de *log* em vez de 403 forbidden, exigindo iteração

corretiva, e os cenários de erro previstos no formato uniforme da API foram referidos apenas para a transição de estado, deixando outros *endpoints* com mensagens de erro inconsistentes que tiveram de ser uniformizadas em iteração subsequente.

P-41: Modelo de Revisão de Código *Propósito:* propor um modelo de revisão de código em camadas que combine gating automático em CI, revisão humana mínima e revisão assistida por LLM.

Contexto: Equipa de cinco estudantes a desenvolver em ramos paralelos sobre o mesmo monorepo, com integração contínua a executar lint, typecheck, testes e build em cada pull request.

Objetivo: Propor um modelo de revisão de código em camadas que combine gating automático em CI, revisão humana mínima e revisão assistida por LLM.

Instruções de Escrita:

- Indicar critérios explícitos de bloqueio de merge por tipo de falha
- Fixar o número mínimo de revisores humanos por pull request, justificado pela dimensão da equipa
- Definir o papel do LLM como revisor complementar (nunca substituto)
- Indicar quando faz sentido solicitar revisão assistida por LLM

Restrições: Sem exigência de dois revisores humanos. Sem usar o LLM como aprovador único.

Análise Crítica. A proposta inicial do LLM aplicou uma política de revisão obrigatória por dois pares humanos, padrão em equipas de produção. Esta política foi ajustada para um único revisor humano, dada a dimensão da equipa e o volume de *pull requests*, dado que a exigência de dois revisores teria criado um estrangulamento incompatível com o ritmo de desenvolvimento de três semanas. A contribuição mais útil do modelo foi uma escala explícita de bloqueio. Falhas de *lint* e *typecheck* são bloqueantes sem exceção. Falha em testes é bloqueante mas pode ser desbloqueada por consenso da equipa quando o teste em causa for explicitamente reconhecido como inválido. Falha de *build* é bloqueante sem exceção. Esta clareza evitou debates recorrentes sobre o estatuto de cada falha. A reflexão da equipa sobre este modelo foi particularmente cuidada, dado que constitui o mecanismo central de garantia de qualidade num cenário em que a quase totalidade do código foi produzida pelo LLM. A questão de como assegurar, de forma sustentada, que o resultado satisfaz os requisitos não é trivial, e a resposta consistiu na articulação de três camadas complementares de validação. Em primeiro lugar, o *gating* automático em CI elimina classes inteiras de problemas mecânicos antes de qualquer revisão humana, libertando o revisor para se concentrar em aspetos semânticos. Em segundo lugar, a revisão humana, ainda que por um único par, ataca precisamente o tipo de erro a que o LLM é mais permeável, designadamente a coerência com regras de negócio que o modelo conhece superficialmente. Em terceiro lugar, a revisão assistida por LLM é

utilizada como complemento, nunca como substituto da revisão humana, em particular para detetar duplicações entre módulos e padrões de inconsistência transversais. A montante destas camadas, a granularidade reduzida da implementação descrita na Secção 3.3 já contribui para diminuir o número de erros silenciosos suscetíveis de passar despercebidos. Esta articulação não garante ausência de defeitos, mas, na avaliação da equipa, ofereceu uma confiança razoável de que o código produzido respeita o desenho aprovado, sem impor uma sobrecarga incompatível com o calendário.

P-42: Detecção de *Code Smells* e *Refactoring* *Propósito:* identificar os code smells mais frequentes na base de código e propor refactoring concreto para cada smell.

Contexto: Onze módulos do backend já implementados em `apps/api/src/modules`, com necessidade de revisão transversal a identificar problemas que escapam a revisão local de pull request.

Objetivo: Identificar os code smells mais frequentes na base de código e propor refactoring concreto para cada smell.

Instruções de Escrita:

- Aplicar a taxonomia de Fowler (long method, duplicated code, primitive obsession, feature envy, data clump, shotgun surgery, entre outros)
- Para cada smell indicar módulo afetado e proposta de refactoring com exemplo antes/depois
- Privilegiar smells que comprometam a modificabilidade ou a coerência entre módulos

Restrições: Sem smells puramente estilísticos. Sem abstrações prematuras que apresentem mais custo do que benefício a esta dimensão.

Análise Crítica. Antes de abordar o resultado da interação, importa enquadrar o conceito utilizado. Os *code smells* são indícios de fragilidade no código que, sem constituírem erros funcionais, prejudicam a sua manutenção, evolução ou legibilidade. Foram catalogados por Fowler em [5], e incluem padrões como duplicação de código, métodos demasiado longos, abstrações inadequadas ou acoplamentos excessivos entre componentes. A sua deteção não é trivial, dado que muitos passam despercebidos numa leitura linear e só se tornam evidentes a quem mantém em mente a estrutura global do sistema, o que torna o LLM um auxiliar particularmente útil neste tipo de análise. O LLM identificou oito categorias de *smell* distintas. A equipa concordou com cinco e contestou três. A sugestão de extrair os literais de configuração financeira para constantes (*magic numbers*) foi rejeitada, dado que estes valores são alterados pela aplicação em *runtime* via *configuration* e não constituem constantes de código. A observação de *primitive obsession* sobre o NIF foi rejeitada por colidir com a decisão arquitetural de usar o NIF como chave primária natural, documentada no Capítulo 2. A recomendação

de extrair um *repository abstrato base* para encapsular operações comuns foi adiada, por se considerar que a abstração prematura apresentaria mais custo do que benefício à dimensão atual do sistema. As restantes cinco sugestões deram origem a *refactorings* efetivos. A execução dos *refactorings* seguiu uma abordagem iterativa e assistida pelo modelo. Para cada *smell* aceite, a equipa isolou o segmento de código afetado, pediu ao LLM a transformação pretendida com indicação explícita do padrão alvo, por exemplo extrair um bloco repetido para uma função utilitária partilhada ou separar uma rotina longa em métodos privados coesos, executou a *suíte* de testes antes e depois da alteração e confirmou que o comportamento observável se mantinha intacto. Esta forma de trabalhar revelou-se particularmente adequada à limpeza de código já existente. O LLM mostra-se eficaz em transformações estruturais de pequeno alcance, em que o objetivo é claro e a verificação pode ser delegada à *suíte* de testes, mas a equipa observou que perde eficácia quando o *refactoring* exige compreensão de invariantes não documentadas no código. Nesses casos a equipa preferiu conduzir as alterações manualmente e reservar o modelo para a revisão posterior.

P-43: Métricas de Desenvolvimento *Propósito:* extrair e organizar um conjunto de métricas de volume da base de código e estimar a proporção de código com origem direta no LLM.

Contexto: Base de código final do projeto (apps/api, apps/web, packages/contracts) e necessidade de sustentar uma análise crítica honesta do desenvolvimento assistido por LLM.

Objetivo: Extrair e organizar um conjunto de métricas de volume da base de código e estimar a proporção de código com origem direta no LLM.

Instruções de Escrita:

- Indicar linhas de código por pacote e por módulo, número de ficheiros e número de endpoints expostos
- Estimar a proporção LLM vs equipa em termos qualitativos, em alternativa a percentagens precisas
- Discutir as limitações intrínsecas deste tipo de medição

Restrições: Sem percentagens com falsa precisão. Sem inferir génese a partir apenas do histórico Git.

Análise Crítica. O LLM alertou desde logo para a fragilidade da medição automática da proporção *LLM vs humano*. O histórico de *Git* regista o autor do *commit* mas não a génese do código. O código gerado é frequentemente revisto, terminado e refinado pela equipa antes do *commit*. A fronteira entre *escrito* e *revisto* é difusa quando a revisão envolve reformulação extensa. A equipa adotou esta cautela e optou por estimativas qualitativas baseadas na memória dos autores, em alternativa a percentagens precisas que sugeririam falsa exatidão.

A.8 Fase de Verificação, Validação e Qualidade

P-44: Estratégia de Verificação e Validação *Propósito:* propor uma estratégia de V&V adequada, distinguindo verificação (sistema construído corretamente) de validação (sistema correto a ser construído).

Contexto: Sistema implementado no Capítulo 3 (monolito modular em TypeScript, 11 módulos do backend, frontend em React, PostgreSQL via Prisma, integração contínua sobre Node.js 20), horizonte temporal limitado e restrições legais não negociáveis (faturação sequencial, RGPD).

Objetivo: Propor uma estratégia de V&V adequada, distinguindo verificação (sistema construído corretamente) de validação (sistema correto a ser construído).

Instruções de Escrita:

- Indicar tipos de testes a adotar, níveis em que se aplicam, ferramentas e critérios de bloqueio em CI
- Discutir o papel do LLM como gerador de testes face ao esforço de revisão subsequente
- Estruturar a estratégia em camadas claras com responsabilidades disjuntas

Restrições: Sem testes ponta-a-ponta automáticos sobre o frontend. Sem ferramentas que introduzam fragilidade desproporcionada.

Análise Crítica. A proposta do LLM partiu da pirâmide clássica de testes, com testes unitários em maior número, testes de integração em quantidade intermédia e testes ponta-a-ponta em quantidade reduzida. A equipa refinou duas opções. Substituiu os testes ponta-a-ponta automáticos sobre o *frontend*, cuja instrumentação introduziria fragilidade e custo de manutenção desproporcionados, por uma camada de testes de sistema ao nível da API. Acrescentou ainda uma camada transversal dedicada aos atributos não funcionais da norma ISO/IEC 25010. A contribuição mais útil do modelo foi a distinção operacional entre verificação e validação, que estruturou a divisão das subsecções que se seguem. A experiência da equipa ao longo desta etapa fixou uma observação importante sobre o comportamento do LLM enquanto gerador de testes. O modelo revela-se particularmente eficaz na produção rápida de testes para comportamentos isolados e situações simples, em que a verificação se resume a confirmar valores de retorno ou estados imediatos. Esta competência diminui sensivelmente quando se trata de testar comportamentos encadeados, designadamente sequências de operações em que o resultado de um passo condiciona o seguinte ou em que regras de negócio só se manifestam pela interação entre módulos. Mesmo nos casos em que o LLM afirma estar a cobrir cenários encadeados, a inspeção atenta revela que essa cobertura é, em regra, superficial, com encadeamentos limitados a dois ou três passos e sem testar verdadeiramente as condições limite que tornam esses cenários relevantes. Os *edge cases* mais delicados, por dependerem de relações

que exigem leitura cuidadosa do código, foram tipicamente omitidos pelo modelo e introduzidos pela equipa em iterações de complemento.

P-45: Arquitetura de Testes de Verificação *Propósito:* propor uma arquitetura de testes automáticos de verificação em três camadas (unitária, integração, sistema).

Contexto: Sistema implementado no Capítulo 3 (Fastify+TypeScript, 11 módulos do backend, PostgreSQL via Prisma, esquemas Zod em `packages/contracts`) e estratégia de V&V definida em 4.1.

Objetivo: Propor uma arquitetura de testes automáticos de verificação em três camadas (unitária, integração, sistema).

Instruções de Escrita:

- Para cada camada indicar âmbito coberto, test runner adotado, forma de exercer a API (mocks vs. instância real), tratamento da persistência e reprodução de sessão autenticada
- Identificar invariantes de domínio que merecem teste explícito
- Listar os anti-padrões a evitar na geração por LLM (mocks de Prisma, requisitos fabricados, esquemas Zod paralelos)

Restrições: Sem mocks do Prisma. Sem requisitos não documentados no SRS. Sem dependências adicionais quando o test runner nativo for suficiente.

Análise Crítica. A resposta do LLM convergiu para o *test runner* nativo `node:test` executado via `tsx --test`, dispensando dependências adicionais como *Vitest* ou *Jest*, e propôs `app.inject()` do *Fastify* como cliente HTTP nos testes, eliminando a necessidade de abrir portos e reduzindo o tempo de execução face a alternativas como *supertest*. Recomendou a validação contra base de dados *PostgreSQL* real, sem *mocks* do *Prisma*, opção que cobre o comportamento efetivo do *schema*, das migrações e das restrições relacionais. A contribuição mais relevante foi a fixação dos anti-padrões, designadamente a proibição de *mocks* do *Prisma*, a proibição de fabricar requisitos não documentados no SRS, e a obrigação de importar esquemas *Zod* do pacote `@gengis-khan/contracts` em vez de declarar variantes paralelas nos testes.

P-46: Procedimento de Validação contra o SRS *Propósito:* desenhar um procedimento de validação que demonstre que o sistema implementado responde efetivamente aos requisitos elicitados, distinguindo validação automática de validação manual.

Contexto: SRS com 16 user stories (8 do gerente, 8 do mecânico) e 29 casos de uso da Subseção 1.4, sistema implementado e disponível em ambiente local.

Objetivo: Desenhar um procedimento de validacao que demonstre que o sistema implementado responde efectivamente aos requisitos elicitados, distinguindo validacao automatica de validacao manual.

Instrucoes de Escrita:

- Estruturar a validacao automatica como testes de aceitacao mapeados um-para-um a user story, executados em CI
- Estruturar a validacao manual como percurso de cada UC contra o sistema corrente em ambiente local
- Manter rastreabilidade explicita entre cada teste e o requisito que valida (ficheiro de rastreabilidade)
- Redigir os testes de aceitacao em linguagem proxima da do SRS, evitando jargao de implementacao

Restricoes: Sem mistura de validacao com verificacao. Sem dependencia de stakeholders externos no horizonte academico.

Análise Crítica. O LLM propôs a estruturação dos testes de aceitação como um ficheiro por papel de utilizador, com cada `it()` correspondente a uma *user story* singular. A equipa adotou esta estrutura e complementou-a com a manutenção de um ficheiro `docs/srs-traceability.md` no repositório, que documenta a correspondência entre requisitos funcionais, *user stories* e testes. A contribuição mais relevante foi a sugestão de redigir os testes de aceitação em linguagem deliberadamente orientada ao perfil de *stakeholder*, evitando jargão de implementação e privilegiando expressões próximas das que aparecem no SRS. Para a validação manual, o modelo propôs uma matriz de UC contra resultado observado, opção adotada pela equipa pela transparência que confere ao processo. A limitação mais relevante face à proposta do LLM foi a impossibilidade de conduzir uma sessão de validação com *stakeholders* reais no horizonte do calendário académico.

P-47: Avaliação da Qualidade da *Suite* de Testes *Propósito:* avaliar a qualidade da própria *suite* enquanto artefacto de V&V, identificando resultados sólidos e fragilidades.

Contexto: Suite de 157 testes implementados nas Subseccoes 4.2 e 4.3, resultados de execucao em integracao continua (taxa de aprovacao, cobertura, tempo) e ficheiro `docs/srs-traceability.md`.

Objetivo: Avaliar a qualidade da propria suite enquanto artefacto de V &V, identificando resultados solidos e fragilidades.

Instrucoes de Escrita:

- Aplicar dimensoes qualitativas e quantitativas: cobertura de codigo, cobertura de requisitos, equilibrio entre camadas, estabilidade, tempo de execucao, profundidade dos cenarios
- Distinguir cobertura de codigo (o que o teste atravessa) de cobertura de requisitos (o que o teste valida)

- Identificar lacunas significativas a sinalizar como prioridade de evolucao

Restricoes: Nao misturar a qualidade dos testes com a qualidade do sistema testado. Sem reproducao redundante de tabelas ja apresentadas.

Análise Crítica. A primeira proposta do LLM concentrou-se em dimensões quantitativas, designadamente cobertura agregada e contagem de testes por camada, reproduzindo o que já constava da Subsecção 4.2. Após reformulação do *prompt* para exigir dimensões qualitativas adicionais (equilíbrio entre camadas, profundidade dos cenários, mapeamento explícito a requisitos), o modelo convergiu para a grelha aqui apresentada. A contribuição mais relevante foi a distinção entre cobertura de código (o que o teste atravessa) e cobertura de requisitos (o que o teste valida), distinção que se mantém explícita no quadro avaliativo.

P-48: Geração da *Suite* de Testes por LLM *Propósito:* gerar o ficheiro <modulo>.test.ts cobrindo o fluxo normal e os erros de validacao principais de cada endpoint, com rastreabilidade explicita ao requisito do SRS.

Contexto: Template de teste estabelecido em apps/api/src/app.test.ts, esquemas Zod em packages/contracts/src/<modulo> e requisitos do SRS rastreaveis.

Objetivo: Gerar o ficheiro <modulo>.test.ts cobrindo o fluxo normal e os erros de validacao principais de cada endpoint, com rastreabilidade explicita ao requisito do SRS.

Instrucoes de Escrita:

- Seguir o template de apps/api/src/app.test.ts e o padrao de setup por beforeEach
- Importar os esquemas Zod do pacote de contratos partilhados (sem criar esquemas paralelos)
- Usar app.inject() do Fastify, PostgreSQL real e JWT assinado diretamente
- Identificar regras de dominio criticas que merecem teste explicito (transicoes de estado, controlo de stock, faturacao)
- Marcar cada it() com a referencia ao requisito que valida

Restricoes: Sem supertest. Sem mocks de Prisma. Sem requisitos fabricados nem campos omitidos do contrato. Sem cenarios apenas de fluxo normal quando o prompt enumera erros.

Análise Crítica. O LLM revelou competência elevada na geração de ficheiros de testes fiéis ao *template* estabelecido, integrando esquemas *Zod* do pacote de contratos partilhados, padrão de

setup por *beforeEach* e rastreabilidade explícita ao requisito. A pressão exercida pelo *prompt* sobre o uso de instância *PostgreSQL* real e a proibição de *mocks Prisma* foi respeitada em todos os módulos. As falhas observadas concentraram-se em dois padrões. Primeiro, omissão silenciosa de campos do contrato, ilustrada pelo caso do DTO de *PurchaseOrder* em que o atributo *status* foi inicialmente omitido pelo modelo, produzindo dois testes que passavam mas validavam um contrato incompleto. A divergência foi detetada por revisão humana antes da integração e corrigida por iteração. Segundo, propensão para escrever apenas o *fluxo normal* mesmo quando o *prompt* enumerava cenários de exceção, o que exigiu pedidos de complemento dirigidos para garantir cobertura efetiva dos casos de falha.

Síntese da Utilização de LLMs no Projeto

O processo documentado neste anexo contabiliza 48 *prompts* distintos utilizados ao longo das quatro etapas do projeto, designadamente Engenharia de Requisitos, Arquitetura e Design, Implementação e Verificação e Validação. A análise quantitativa e qualitativa desta utilização permite extrair algumas observações metodológicas relevantes:

- **Iteratividade:** vários artefactos (nomeadamente a tabela de requisitos e a especificação de *Use Cases*) exigiram múltiplas iterações de *prompt*, com refinamento progressivo das instruções em resposta a limitações detetadas nos *outputs* anteriores.
- **Estruturação crescente:** observa-se uma evolução da complexidade e especificidade dos *prompts* ao longo do projeto, desde instruções sucintas nas fases iniciais até especificações detalhadas com persona, estrutura obrigatória e referências bibliográficas explícitas nas fases mais avançadas.
- **Validação humana constante:** todos os artefactos produzidos foram sujeitos a revisão crítica pela equipa, com correção de imprecisões factuais, remoção de conteúdo não fundamentado e reformulação de passagens excessivamente genéricas.
- **Regra de Ouro:** em tarefas de extração estrutural (como a elaboração da tabela de requisitos), impôs-se explicitamente a proibição de o modelo presumir ou inventar informação, exigindo ancoragem estrita em fontes documentais.

Anexo B: Entrevista-001, Definição do Domínio (*Prompt* e Transcrição)

Prompt Utilizado

Contexto: Projeto da UC de LI4, sistema de gestao das "Oficinas Gengis Khan". Fase de definicao do dominio, anterior ao levantamento formal de requisitos.

Objetivo: Gerar a transcricao de uma entrevista tecnica entre um Analista de Sistemas e o Engenheiro da oficina, a utilizar como base documental para a "Definicao do Dominio" e o "SRS".

Instrucoes de Escrita:

- Cobrir quatro topicos: Registo e Identificacao, Ciclo de Vida da Reparacao, Gestao de Pecas e Custos, Output e Relatorios
- Manter registo natural de dialogo entre dois interlocutores tecnicos
- Privilegiar exemplos concretos sobre afirmacoes genericas

Restricoes: Forneca apenas a transcricao da entrevista, sem introducoes meta-textuais nem comentarios fora do dialogo.

[Numa segunda iteracao foi solicitada a reescrita do texto de contextualizacao com base no dominio extraido da entrevista gerada .]

Transcrição

Campo	Valor
Data	17 de fevereiro de 2026
Local	Oficinas Gengis Khan, Braga
Duração	1h 45min
Identificador	Entrevista-001

Participantes:

- [ES1] Engenheiro de *Software* 1 (Aluno da Universidade do Minho)
- [ES2] Engenheiro de *Software* 2 (Aluno da Universidade do Minho)
- [Gerente] António Silva, Gerente e Proprietário da Oficina
- [Mecânico] Ricardo Costa, Mecânico Chefe

PARTE 1: Introdução e Contexto (0:00 – 0:10)

[ES1]: Boa tarde a ambos. Agradeço desde já a disponibilidade para esta reunião. Como referimos por *email* estamos a desenvolver um sistema de gestão para oficinas de reparação de trotinetes no âmbito do nosso projeto académico na Universidade do Minho. O objetivo desta primeira conversa é entender em detalhe como funciona o vosso dia a dia: que processos seguem, que informação geram e gerem, quem faz o quê. Ainda não vamos falar do sistema em si; queremos primeiro perceber bem o vosso negócio.

[Gerente]: Boa tarde. Claro, faz sentido. Estejam à vontade para perguntar tudo o que precisarem.

[Mecânico]: Boa tarde. Estou cá para ajudar no que for preciso.

[ES2]: Perfeito. Então vamos começar pelo princípio. Quando um cliente chega à oficina com uma trotinete, o que acontece?

PARTE 2: O Cliente e o Equipamento (0:10 – 0:25)

[Mecânico]: Normalmente sou eu ou o meu colega Miguel que recebemos o cliente. Pedimos os dados básicos: nome completo, contacto telefónico, *email* e morada. Ah, e o NIF para a fatura.

[ES1]: E registam todos esses dados de forma estruturada atualmente?

[Gerente]: Tentamos. Temos um caderno de clientes, mas nem sempre está atualizado. Alguns clientes são recorrentes, outros aparecem uma única vez.

[ES2]: E que tipo de clientes recebem? São sempre particulares?

[Gerente]: Não, temos de dois tipos. A maioria são clientes individuais, particulares. Mas também temos alguns clientes empresariais, empresas de mobilidade partilhada, frotas de empresas. Esses funcionam de forma diferente: normalmente pagam a 30 dias, têm um volume maior de trotinetes e, às vezes, condições comerciais específicas.

[ES1]: Interessante. E os particulares pagam como?

[Gerente]: No momento em que vêm buscar a trotinete reparada. Pagam e levam.

[Mecânico]: Uma coisa importante é que, de vez em quando, alguns clientes querem mudar o número de telemóvel ou o *email*, e essa informação precisa poder ser atualizada com facilidade.

[ES1]: Certo, *email* e telemóvel devem ser campos editáveis.

[ES2]: E em relação à trotinete em si, que informação recolhem sobre o equipamento?

[Mecânico]: Anotamos a marca, o modelo e sempre o número de série. O número de série é fundamental porque é único para cada trotinete.

[ES1]: E esse número de série, onde é que normalmente se encontra numa trotinete?

[Mecânico]: Geralmente está gravado na parte inferior do *deck*, perto da roda traseira, ou às vezes na haste. Cada fabricante tem o seu sítio. É uma sequência alfanumérica, tipo “XM2021-4567890”.

[ES2]: E registam também a data de compra da trotinete ou a data do último serviço?

[Gerente]: A data de compra não, mas seria útil. Já a data do último serviço, apenas se for um cliente recorrente e tivermos essa informação acessível.

[ES1]: E há mais alguma informação que recolham sobre a trotinete neste momento inicial?

[Mecânico]: Sim, observações gerais sobre o estado do equipamento. Por exemplo, “muito oxidada”, “pneus gastos”, “*display* partido”. Isso ajuda-nos no diagnóstico inicial.

PARTE 3: A Ordem de Serviço e o Diagnóstico (0:25 – 0:40)

[ES2]: Muito bem. Depois de registarem o cliente e a trotinete, o que acontece a seguir?

[Mecânico]: Criamos uma ordem de serviço. Basicamente é um documento onde anotamos o que o cliente reporta como problema. Por exemplo: “trotinete não liga”, “travão traseiro não funciona”, “bateria descarrega muito rápido”.

[ES1]: E essa ordem de serviço tem algum número de identificação?

[Gerente]: Sim, usamos um número sequencial. Começámos este ano com OS-2026-001, OS-2026-002 e por aí fora. É para podermos referenciar rapidamente.

[ES2]: E que outras informação constam na ordem de serviço?

[Mecânico]: A data de entrada, obviamente. O problema reportado pelo cliente. E depois, quando faço o diagnóstico, adiciono as minhas observações técnicas.

[ES1]: Pode dar-nos um exemplo concreto de diagnóstico?

[Mecânico]: Claro. Imagine que o cliente diz “a trotinete não liga”. Eu faço o diagnóstico e descubro que o problema é um fusível queimado no controlador. Então escrevo algo como: “Diagnóstico: fusível F2 do controlador queimado. Causa provável: curto-circuito. Solução: substituir fusível e verificar cablagem.”

[ES2]: E esse diagnóstico é feito logo no momento em que recebem a trotinete?

[Mecânico]: Nem sempre. Se tivermos muitas trotinetes para reparar, o diagnóstico pode demorar um ou dois dias. Por isso é importante ter um estado associado à ordem de serviço.

[ES1]: Estados? Pode explicar melhor o que quer dizer com isso?

[Gerente]: Sim, é a fase em que a reparação se encontra. Temos usado: “Recebida”, “Em Diagnóstico”, “Aguarda Peças”, “Em Reparação”, “Concluída” e “Entregue ao Cliente”.

[Mecânico]: Às vezes também temos “Aguarda Aprovação do Cliente”, quando o orçamento é alto e precisamos que o cliente confirme se quer avançar com a reparação.

[ES2]: Portanto, sete estados possíveis. E cada ordem de serviço vai passando de estado em estado?

[Gerente]: Exatamente. Hoje fazemos isso no papel, anotando à mão quando muda. Mas nem sempre nos lembramos de registar a data e hora da mudança.

PARTE 4: O Processo de Reparação (0:40 – 0:55)

[ES1]: Agora vamos falar sobre a reparação propriamente dita. Como funciona o trabalho na bancada?

[Mecânico]: Cada coisa que fazemos na trotineta é uma intervenção separada. Por exemplo, se tenho de substituir um travão, isso é uma intervenção: “Substituição do travão traseiro mecânico, 45 minutos”. Se depois tiver de ajustar a tensão do cabo, é outra intervenção: “Ajuste de tensão do cabo do travão, 15 minutos”.

[ES2]: Então cada ordem de serviço pode ter múltiplas intervenções?

[Mecânico]: Sim, quase sempre tem. Raramente uma reparação envolve apenas uma coisa. Normalmente são várias pequenas tarefas.

[ES1]: E para cada intervenção, registam sempre o tempo que demorou?

[Mecânico]: Tentamos, mas nem sempre é rigoroso. Anotamos mais ou menos. Às vezes arredondo para cima ou para baixo.

[ES2]: E além do tempo, que outras informação associam a cada intervenção?

[Mecânico]: A descrição do trabalho realizado, as peças que usei e quem fez o trabalho. Aqui somos dois mecânicos, eu e o Miguel, por isso é importante saber quem fez o quê.

[ES1]: E como é que associam as peças usadas a uma intervenção específica?

[Mecânico]: Atualmente anoto num papel: “Intervenção X usadas 1 unidade de travão-TRAS-001”. Depois o gerente vai buscar essa informação para fazer a fatura.

PARTE 5: As Peças e o *Stock* (0:55 – 1:05)

[ES2]: Vamos então falar das peças. Que informação mantêm sobre cada peça?

[Gerente]: Cada peça tem uma referência interna que usamos, tipo “travão-TRAS-001”. Depois temos a descrição, por exemplo “Travão Mecânico Traseiro Universal”. O fornecedor de onde compramos, o preço de custo que pagamos e o preço de venda que cobramos ao cliente.

[ES1]: E como calculam o preço de venda?

[Gerente]: Normalmente aplicamos uma margem de 40% sobre o preço de custo. Mas algumas peças têm margens diferentes. As baterias, por exemplo, têm margem menor porque já são caras por si.

[ES2]: E em termos de *stock*, como funciona o controlo?

[Mecânico]: Temos a quantidade em *stock* atual. E depois definimos um *stock* mínimo para cada peça. Quando a quantidade desce abaixo do mínimo, sabemos que temos de encomendar.

[ES1]: E esse *stock* mínimo, quem é que o define?

[Gerente]: Sou eu, com base na experiência. Por exemplo, travões temos sempre *stock* mínimo de 10 unidades porque usamos muitos. Já *displays*, que são mais específicos por modelo, podemos ter *stock* mínimo de 2 ou 3.

[ES2]: E quantas referências de peças diferentes têm?

[Mecânico]: Umas 300 referências diferentes entre peças originais e genéricas.

PARTE 6: Os Fornecedores (1:05 – 1:10)

[ES1]: E de onde vêm essas peças? Trabalham com quantos fornecedores?

[Gerente]: Temos dois principais: um para peças genéricas e outro para peças originais das marcas principais. Ocasionalmente compramos a um terceiro quando há algo muito específico.

[ES2]: E como fazem as encomendas atualmente?

[Gerente]: Quando vejo que uma peça está abaixo do mínimo, telefono ou mando *email* ao fornecedor. Às vezes até uso WhatsApp. Mas não é nada organizado. Acontece perder-se a noção e só dar conta quando já precisamos da peça urgentemente.

[ES1]: E registam as encomendas de alguma forma?

[Gerente]: Devíamos, mas nem sempre acontece. Depois quando as peças chegam, às vezes não me lembro exatamente o que tinha pedido.

[ES2]: E que dados mantêm sobre cada fornecedor?

[Gerente]: O nome, *email* e telefone. E também as condições de pagamento: um deles é pagamento a 30 dias, o outro é pronto pagamento com 5% de desconto, e claro eles às vezes mudam tanto de *email* como de telemóvel, logo devemos também conseguir editar esses campos.

PARTE 7: A Faturação (1:10 – 1:20)

[ES1]: Vamos agora perceber como funciona a parte financeira. Como é que calculam o valor de uma reparação?

[Gerente]: Somamos o custo das peças vendidas mais a mão de obra. A mão de obra cobramos a 25 euros por hora.

[ES2]: Pode dar-nos um exemplo concreto?

[Gerente]: Claro. Se uma reparação demorou 2 horas e usou peças no valor de 50 euros ao preço de venda, o total seria $50 + (2 \times 25) = 100$ euros, mais IVA a 23%.

[ES1]: E quando é que emitem a fatura?

[Gerente]: Quando o cliente vem buscar a trotinete e a reparação está concluída. Nesse momento fazemos a fatura com todos os detalhes: peças utilizadas, horas de trabalho, subtotal, IVA e total.

[ES2]: E a fatura tem algum número de identificação?

[Gerente]: Sim, também sequencial. FAT-2026-001, FAT-2026-002 etc. É obrigatório por lei ter numeração sequencial sem quebras.

[ES1]: E incluem o detalhe das intervenções na fatura?

[Gerente]: Sim, discriminamos tudo. O cliente gosta de ver pelo que está a pagar. Colocamos cada peça com o seu preço e as horas de trabalho.

[ES2]: E quanto aos clientes empresariais que pagam a 30 dias, como controlam isso?

[Gerente]: É aí que temos mais dificuldade. Às vezes perco a noção de quais faturas estão pagas e quais não. Hoje faço isso tudo de cabeça ou num ficheiro Excel, mas não é fiável.

PARTE 8: A Comunicação com Clientes (1:20 – 1:25)

[ES1]: E como é que comunicam com os clientes durante o processo de reparação?

[Mecânico]: Normalmente telefono quando a reparação fica concluída. Mas nem sempre consigo contactar logo, deixo mensagem e às vezes demora. Outros clientes preferem SMS ou WhatsApp.

[Gerente]: É um processo muito manual e desperdiçamos tempo nisso. E quando precisamos de aprovação do orçamento é a mesma coisa, telefonar e esperar resposta...

[ES2]: E o cliente consegue saber o estado da reparação sem vos contactar?

[Gerente]: Não, de todo. Só se nos telefonar a perguntar.

PARTE 9: A Gestão e Tomadas de Decisão (1:25 – 1:35)

[ES1]: Para terminar esta primeira conversa, gostaríamos de perceber como é que toma decisões de gestão no dia a dia.

[Gerente]: Essa é uma boa pergunta. A verdade é que me falta informação para tomar decisões bem fundamentadas. Gostava de saber quanto faturamos num determinado mês e quais foram as peças mais usadas. Hoje, para ter essa informação, tenho de passar horas a cruzar dados de diferentes fontes.

[ES2]: E do ponto de vista operacional?

[Mecânico]: Seria útil saber quais são os problemas mais comuns. Por exemplo, se 70% das reparações envolvem baterias, isso diz-nos algo.

[Gerente]: E quantas ordens de serviço estão em cada estado. Se tivermos muitas em “Aguarda Peças”, sabemos que temos um problema de *stock*.

PARTE 10: Encerramento (1:35 – 1:40)

[ES2]: Muito bem, creio que temos uma boa compreensão de como funciona a oficina. Vamos agora trabalhar na documentação do domínio, organizar tudo o que nos disseram, identificar os conceitos principais, as relações entre eles e preparar um modelo de domínio.

[ES1]: Depois disso, gostaríamos de agendar uma segunda reunião convosco para discutir em concreto o que o sistema deve fazer, as funcionalidades, os requisitos técnicos, as prioridades. Assim já vamos com uma base sólida de conhecimento do vosso negócio.

[Gerente]: Faz todo o sentido. Quando pensam agendar essa segunda reunião?

[ES2]: Idealmente dentro de uma a duas semanas, depois de termos a contextualização e o modelo de domínio prontos. Contactamos por *email* para combinar a data.

[Gerente]: Perfeito. Ficamos a aguardar então.

[Mecânico]: Se entretanto se lembrarem de alguma coisa que queiram esclarecer, estejam à vontade para contactar.

[ES1]: Muito obrigado pela vossa disponibilidade e por toda a informação. Foi muito produtivo.

[Gerente]: De nada, estamos aqui para o que precisarem.

[Mecânico]: Boa sorte com o projeto!

[Fim da Transcrição]

Assinaturas:

_____ Engenheiro de *Software* 1

_____ Engenheiro de *Software* 2

_____ António Silva, Gerente

_____ Ricardo Costa, Mecânico Chefe

Anexo C: Entrevista-002, Elicitação de Requisitos (*Prompt* e Transcrição)

Prompt Utilizado

Contexto: Projeto das Oficinas Gengis Khan na UC de LI4, posterior a Entrevista-001 (definição do domínio), com documentação de domínio e modelo conceptual já produzidos, e necessidade de elicitação formal de requisitos junto do Gerente e do Mecânico Chefe.

Objetivo: Gerar a transcrição da segunda entrevista técnica, formalizando a sessão de elicitação de requisitos que servirá de base a especificação formal de RF e RNF.

Instruções de Escrita:

- Abrir com recapitulação e objectivos da sessão
- Cobrir requisitos para registo de clientes e equipamentos (incluindo unicidade de NIF e número de série)

- Cobrir requisitos para ordens de serviço (estados, permissões, visibilidade operacional)
- Cobrir requisitos para intervenções e cronómetro integrado
- Cobrir requisitos para gestão de stock e fornecedores
- Cobrir requisitos para faturação (numeração sequencial, IVA parametrizável, estados de pagamento)
- Cobrir requisitos para notificações automáticas e para relatórios
- Discutir priorização de funcionalidades (essenciais, importantes, desejáveis)
- Cobrir requisitos não funcionais (plataforma web, tablets, segurança, RGPD, backups) e permissões de utilizadores (RBAC)

Restrições: Sem decisões de implementação técnica. Sem requisitos não discutidos com os stakeholders durante a sessão.

Transcrição

Campo	Valor
Data	20 de fevereiro de 2026
Local	Oficinas Gengis Khan, Braga
Duração	1h 35min
Identificador	Entrevista-002

Participantes:

- [ES1] Engenheiro de *Software* 1 (Aluno da Universidade do Minho)
- [ES2] Engenheiro de *Software* 2 (Aluno da Universidade do Minho)
- [Gerente] António Silva, Gerente e Proprietário da Oficina
- [Mecânico] Ricardo Costa, Mecânico Chefe

PARTE 1: Recapitulação e Objetivos da Sessão (0:00 – 0:10)

[ES1]: Boa tarde novamente. Desde a nossa última conversa, trabalhamos na documentação do domínio, organizámos os conceitos que nos explicaram, identificámos as entidades principais (clientes, trotinetes, ordens de serviço, intervenções, peças, fornecedores, faturas) e preparámos um modelo de domínio que vos vamos mostrar oportunamente para validação.

[ES2]: O objetivo de hoje é diferente. Agora que já percebemos como funciona a oficina, queremos discutir convosco o que esperam que o sistema faça concretamente. Vamos falar

de funcionalidades, de como gostariam que as coisas funcionassem no sistema e de requisitos técnicos.

[Gerente]: Muito bem, estamos prontos.

[Mecânico]: Entretanto lembrei-me de mais umas coisas que podem ser úteis.

[ES1]: Ótimo, vamos ouvir tudo.

PARTE 2: Requisitos para o Registo de Clientes e Equipamentos (0:10 – 0:20)

[ES2]: Na primeira reunião percebemos que registam os dados do cliente e da trotinete no momento da receção. Para o sistema, precisamos de definir exatamente o que deve ser obrigatório e o que é opcional. Propomos: nome completo, NIF, telefone e *email* como obrigatórios; morada como opcional. Faz sentido?

[Gerente]: Sim, *email* é importante para as notificações. E o NIF é obrigatório por causa da faturação.

[ES1]: E para clientes empresariais, há dados adicionais?

[Gerente]: Sim, a denominação social da empresa e seria bom ter um campo para o limite de crédito e as condições de pagamento. Assim o sistema podia avisar-me quando um cliente empresarial está perto de atingir o limite.

[ES2]: Anotado. E para a trotinete, na última reunião mencionaram marca, modelo, número de série e observações. O número de série deve ser único no sistema, concordam?

[Mecânico]: Sim, absolutamente. É o que identifica cada trotinete de forma inequívoca.

[ES1]: E gostariam de poder consultar o histórico completo de uma trotinete através do número de série? Ou seja, ver todas as reparações anteriores?

[Mecânico]: Isso seria fantástico! Se um cliente volta com a mesma trotinete seis meses depois, eu consigo ver tudo o que foi feito antes. Hoje dependo da memória.

[Gerente]: E do lado do cliente também, ver o histórico completo: quantas vezes veio, quanto já gastou, se tem faturas por pagar.

PARTE 3: Requisitos para as Ordens de Serviço (0:20 – 0:35)

[ES2]: Na primeira reunião descreveram-nos sete estados para a ordem de serviço: Recebida, Em Diagnóstico, Aguarda Aprovação do Cliente, Aguarda Peças, Em Reparação, Concluída e Entregue ao Cliente. Para o sistema, como esperam que isto funcione?

[Gerente]: O ideal seria que cada mudança de estado fosse registada automaticamente com a data e hora. Assim sabíamos quanto tempo uma trotinete esteve em cada fase.

[ES1]: Isso é perfeitamente viável. E em termos de quem pode mudar o estado?

[Mecânico]: Os mecânicos deviam poder mudar para a maioria dos estados, de “Recebida” para “Em Diagnóstico”, de “Em Diagnóstico” para as fases seguintes. Mas talvez o estado “Entregue ao Cliente” devesse ser confirmado pelo gerente, porque implica pagamento.

[Gerente]: Sim, concordo. E o estado “Aguarda Aprovação do Cliente”, quando passamos para esse estado, o sistema deveria automaticamente enviar um *email* ao cliente com o orçamento detalhado, pedindo confirmação.

[ES2]: Estamos a entrar nas notificações, vamos abordar isso em detalhe já a seguir. Mais algum requisito sobre as ordens de serviço?

[Gerente]: Sim, gostava de poder ver rapidamente quantas ordens estão em cada estado. Tipo um painel resumo quando abro o sistema.

[Gerente]: Olha, e uma coisa que quero deixar clara desde já: não esperem que tudo tenha a mesma urgência. Há coisas que, se não estiverem no sistema desde o primeiro dia, a oficina simplesmente não funciona, como o registo das ordens de serviço e a gestão dos estados da reparação. Mas há outras que seriam muito úteis, como os relatórios analíticos ou as notificações automáticas, mas que, se não estiverem na primeira versão, conseguimos viver com isso por um tempo. Portanto, quando forem definir o que entra primeiro, tenham isso em conta: há requisitos críticos e há requisitos que podem esperar.

PARTE 4: Requisitos para Intervenções e Cronómetro (0:35 – 0:45)

[ES1]: Na primeira reunião, o Senhor Ricardo mencionou que nem sempre é rigoroso no registo do tempo de cada intervenção. Pensámos numa solução: um cronómetro integrado no sistema. O mecânico inicia o cronómetro quando começa a trabalhar numa intervenção e para quando termina. O que acham?

[Mecânico]: Isso seria excelente! Assim o tempo ficava registado automaticamente, sem eu ter de adivinhar. E se me interrompem a meio, por exemplo, para atender um cliente, posso pausar e retomar.

[ES2]: Exatamente, com funcionalidade de pausa. E cada intervenção ficaria associada ao mecânico que a realizou, à descrição do trabalho, ao tempo e às peças utilizadas.

[Mecânico]: Perfeito. E as peças, quando eu seleciono uma peça para associar a uma intervenção, o *stock* deveria diminuir automaticamente?

[ES1]: Sim, essa é a ideia. E se o *stock* dessa peça ficar abaixo do nível mínimo, o sistema gera um alerta.

[Gerente]: Isso resolve-nos um problema enorme. Hoje só damos conta que falta uma peça quando já a precisamos.

PARTE 5: Requisitos para Gestão de *Stock* e Fornecedores (0:45 – 0:55)

[ES2]: Ainda sobre o *stock*, na primeira reunião explicaram-nos como funciona o aprovisionamento. Para o sistema, que funcionalidades esperam?

[Gerente]: Primeiro, que me avise quando uma peça atinge o *stock* mínimo. Segundo, seria excelente se o sistema pudesse gerar automaticamente uma lista de peças a encomendar, todas as que estão abaixo do mínimo, e enviar essa lista diretamente para o fornecedor por *email*, como uma pré-encomenda.

[ES1]: E quando fazem uma encomenda, que informação deve ficar registada?

[Gerente]: A data da encomenda, as peças e quantidades pedidas, o fornecedor e a data de entrega.

[Mecânico]: E o estado da encomenda: “Solicitada” ou “Recebida”. Assim podemos facilmente distinguir quais encomendas ainda estão por chegar.

PARTE 6: Requisitos para Faturação (0:55 – 1:05)

[ES1]: Na primeira reunião explicaram-nos como calculam o valor da reparação. Para o sistema, esperam que a fatura seja gerada automaticamente?

[Gerente]: Sim. Quando a ordem de serviço passa para “Concluída”, o sistema já tem toda a informação: peças usadas com preços de venda, horas de mão de obra. Devia calcular tudo automaticamente.

[ES2]: E a taxa de mão de obra de 25 €/hora, disseram-nos que poderia mudar no futuro. Deve ser um parâmetro configurável?

[Gerente]: Sim, e o IVA também. Hoje é 23%, mas se mudar, quero poder alterar no sistema sem precisar de ajuda técnica.

[ES1]: E a fatura deve incluir o detalhe de cada peça e o tempo de mão de obra?

[Gerente]: Sim, tudo discriminado. E cada fatura com numeração sequencial sem quebras: FAT-2026-001, FAT-2026-002. É obrigatório por lei.

[ES2]: E quanto ao estado de pagamento? Na primeira reunião mencionaram que os clientes empresariais pagam a 30 dias.

[Gerente]: Exatamente. Cada fatura deve ter um estado: “Pendente”, “Paga”, “Atrasada”. E eu preciso de ver rapidamente quais faturas estão pendentes ou atrasadas, especialmente dos clientes empresariais.

PARTE 7: Requisitos para Notificações (1:05 – 1:10)

[ES1]: Vamos agora falar das notificações automáticas. Em que momentos gostariam que o sistema enviasse comunicações ao cliente?

[Gerente]: Primeiro, uma confirmação de receção quando aceitamos a trotinete, com o número da ordem de serviço e um prazo estimado.

[Mecânico]: Depois, quando passamos para “Aguarda Aprovação do Cliente”, enviar o orçamento detalhado por *email* e pedir confirmação.

[Gerente]: Quando a reparação fica concluída, avisar que a trotinete está pronta e indicar o valor total, para o cliente já vir preparado.

[ES2]: E mencionaram também alertas em caso de atraso.

[Gerente]: Sim, por exemplo, se uma trotinete está há mais de 10 dias em “Aguarda Peças”, enviar um *email* ao cliente a informar do atraso e a pedir desculpa.

[ES1]: E o conteúdo dessas notificações deve ser personalizável?

[Gerente]: Sim, com *templates* que possamos ajustar. E o canal principal seria *email*, mas se no futuro fosse possível integrar SMS, seria fantástico.

[Mecânico]: Isso pouparia-nos muito tempo. Hoje perdemos tempo a telefonar e a deixar mensagens.

PARTE 8: Requisitos para Relatórios (1:10 – 1:15)

[ES2]: Na primeira reunião falaram sobre a necessidade de informação para tomar decisões. Vamos definir que relatórios o sistema deve disponibilizar.

[Gerente]: Faturação por período mensal, trimestral, anual. Peças mais utilizadas. Custos em peças por período.

[ES1]: E do lado operacional?

[Mecânico]: Problemas mais comuns, um *ranking* dos tipos de avaria mais frequentes. Tempo médio de reparação, global e por tipo de problema.

[Gerente]: Distribuição de ordens de serviço por estado num dado momento. E desempenho por mecânico: número de intervenções, tempo médio, mas usado de forma construtiva.

[Mecânico]: Concordo com isso. Desde que seja para nos ajudar a melhorar.

PARTE 9: Priorização de Funcionalidades (1:15 – 1:20)

[ES2]: Antes de avançarmos para as notificações, gostaríamos de explorar um tema importante para o planeamento do desenvolvimento. Dado o conjunto de funcionalidades que identificámos até agora, considera que todas têm o mesmo nível de importância para o funcionamento da oficina?

[Gerente]: Não, de todo. Há coisas que são absolutamente indispensáveis desde o primeiro dia, sem elas o sistema não serve de nada. Por exemplo, criar ordens de serviço, registar intervenções e emitir faturas são o coração do negócio. Se isso não funcionar, de nada vale o resto.

[Mecânico]: Concordo. Para mim, o cronómetro e o registo automático de peças no *stock* são essenciais. Passo o dia todo a registar coisas, se não forem automáticas, vou perder mais tempo do que antes.

[ES1]: E há funcionalidades que considera importantes, mas que poderiam ser implementadas com mais calma?

[Gerente]: Sim. Os relatórios detalhados, por exemplo, são muito úteis para a gestão, mas consigo sobreviver sem eles durante uns meses. O mesmo para as notificações automáticas ao cliente. Seria ótimo ter desde o início, mas não é bloqueante.

[ES2]: E há funcionalidades que considera desejáveis, mas sem grande urgência?

[Gerente]: A faturação consolidada para clientes empresariais, por exemplo. Temos poucos clientes assim de momento, então pode esperar. E talvez a gestão de fornecedores mais avan-

çada, o histórico de encomendas e os alertas automáticos. É importante, mas não é o que me tira o sono agora.

[ES1]: Isso é muito útil para nós. Significa que ao definirmos os requisitos, devemos distinguir claramente entre o que é imprescindível, o que é importante mas não urgente, e o que é desejável a longo prazo.

[Gerente]: Exatamente. Não quero esperar um ano para ter um sistema funcional. Prefiro ter o essencial a funcionar bem dentro de três meses do que tudo mal feito.

PARTE 10: Requisitos Não Funcionais (1:20 – 1:25)

[ES1]: Para terminar, vamos falar de requisitos técnicos. Na primeira reunião mencionaram a preferência por um sistema *web*. Confirmam?

[Gerente]: Sim, *web*. Para poder aceder de qualquer lugar da oficina, de casa, de uma reunião com fornecedores.

[Mecânico]: E que funcione bem em *tablets*. Na bancada, um *tablet* é mais prático do que um computador.

[ES2]: Entendido, será uma aplicação *web* responsiva. E em termos de capacidade, na primeira reunião falaram em 150 ordens por mês atualmente, com crescimento para 500. O sistema será dimensionado para isso.

[ES1]: E quanto à segurança?

[Gerente]: Os dados dos clientes são confidenciais, tem de estar tudo protegido segundo o RGPD. E *backups* automáticos diários, pelo menos.

PARTE 11: Permissões de Utilizadores (1:25 – 1:30)

[ES2]: E em termos de utilizadores e permissões? Quantos utilizadores terão?

[Gerente]: Inicialmente três: eu, o Ricardo e o Miguel, o outro mecânico. Cada um com o seu *login*.

[ES1]: E diferentes níveis de acesso?

[Gerente]: Sim. Os mecânicos acedem a ordens de serviço, peças e registam intervenções. Mas relatórios financeiros, definição de preços e gestão de fornecedores, isso sou só eu.

[Mecânico]: Faz sentido. Eu não preciso de ver quanto o patrão ganha! (*ri*)

[Gerente]: (*ri*) É mais uma questão de organização e privacidade dos dados financeiros.

PARTE 12: Encerramento e Próximos Passos (1:30 – 1:35)

[ES2]: Muito bem, temos agora uma visão bastante completa do que o sistema deve fazer. Vamos compilar tudo num documento de especificação de requisitos formal, com requisitos funcionais, não funcionais, casos de uso e *user stories*.

[ES1]: Nas próximas semanas, começamos a trabalhar na arquitetura do sistema e nos primeiros protótipos de ecrã. Vamos partilhando convosco para validação.

[Gerente]: Quando é que pensam ter uma primeira versão funcional?

[ES2]: Segundo o plano do projeto, a implementação decorre entre março e abril. Mas antes disso vamos mostrar-vos protótipos e *mockups* para garantir que está a ir ao encontro das vossas necessidades.

[Mecânico]: E se entretanto nos lembrarmos de algo importante?

[ES1]: Podem contactar-nos por *email* ou telefone a qualquer momento. Isto é um processo iterativo, se surgirem novas ideias ou ajustes, incorporamos.

[Gerente]: Excelente. Ficamos a aguardar novidades então.

[ES2]: Muito obrigado mais uma vez pela disponibilidade. Tem sido muito produtivo trabalhar convosco.

[Gerente]: De nada, estamos aqui para o que precisarem.

[Mecânico]: Boa sorte com o projeto!

[Fim da Transcrição]

Assinaturas:

_____ Engenheiro de *Software* 1

_____ Engenheiro de *Software* 2

_____ António Silva, Gerente

_____ Ricardo Costa, Mecânico Chefe

Anexo D: Entrevista-003, Validação de Requisitos (*Prompt* e Transcrição)

Prompt Utilizado

Contexto: Equipa de LI4 reunida em Braga com o Gerente (Antonio Silva) e o Mecanico Chefe (Ricardo Costa) apos a primeira analise dos requisitos, com cinco ambiguidades a confrontar (Tabela 25) e necessidade de chegar a consenso para fechar a Baseline de requisitos.

Objetivo: Redigir a transcricao de uma reuniao simulada que confronta os stakeholders com as falhas logicas, omissoes ou contradicoes detectadas e regista a resolucao acordada para cada ambiguidade.

Instrucoes de Escrita:

- Cabecalho de Ata com Data, Local (Braga) e Participantes
- Introducao a explicar o conceito de edge cases e porque a linguagem natural gera ambiguidades
- Dialogo em discurso direto para cada ambiguidade da Tabela 25, com perfil do Antonio (preocupado com perdas financeiras, abandono e faturacao, tendendo a pedir processos burocraticos), perfil do Ricardo (preocupado com rapidez, sem disponibilidade para preencher campos com maos sujas) e Equipa LI4 com perguntas incisivas
- No final de cada ponto a Equipa LI4 resume a regra de negocio final acordada
- Encerramento com validacao formal e autorizacao do cliente para avanco para desenho/arquitetura
- PT-PT rigoroso (ecra, tablet, orcamento, faturar)
- Tom realista e profissional, com maturidade em Engenharia de Software

Restricoes: Sem ambiguidades nao listadas na Tabela 25. Sem perfis fora dos descritos.

Transcrição

Campo	Valor
Data	23 de março de 2026
Local	Oficinas Gengis Khan, Braga
Duração	1h 45min
Identificador	Entrevista-003, Sessão de Validação

Participantes:

- [ES1] Engenheiro de *Software* 1 (Equipa LI4, Universidade do Minho)
- [ES2] Engenheiro de *Software* 2 (Equipa LI4, Universidade do Minho)
- [Gerente] António Silva, Gerente e Proprietário da Oficina
- [Mecânico] Ricardo Costa, Mecânico Chefe

Introdução e Propósito da Sessão (0:00 – 0:10)

[ES1]: Bom dia, António e Ricardo. Obrigado pela vossa disponibilidade nesta fase crítica do projeto. Depois das duas entrevistas anteriores e do levantamento inicial de requisitos, procedemos a uma análise crítica do conjunto recolhido e identificámos cinco situações que precisamos de clarificar convosco antes de avançar para a fase de arquitetura.

[ES2]: Para vos contextualizar, o que encontrámos são o que em Engenharia de Software se designa por *edge cases*, casos limite em que a linguagem natural utilizada no levantamento admite múltiplas interpretações técnicas. Os computadores exigem regras exatas, e basta uma destas ambiguidades ficar por resolver para que a implementação siga um caminho que não corresponde ao que pretendem.

[Gerente]: Compreendo. Digam lá o que há para esclarecer, estamos à disposição.

[Mecânico]: Vamos a isso.

[ES1]: Temos cinco pontos para discutir, todos derivados da análise aos requisitos que resultaram das nossas conversas anteriores. Em cada ponto vamos propor uma resolução para fecharmos connosco, para que no final da reunião tenhamos um conjunto de requisitos consolidado, estável e sem ambiguidades.

PARTE 1: Identificação Única do Cliente (REQ-004) (0:10 – 0:30)

[ES2]: O primeiro ponto prende-se com o histórico do cliente. O requisito REQ-004 estabelece que o sistema deve permitir consultar o histórico completo de um cliente, quantas vezes veio à oficina, quanto já gastou e se tem faturas por pagar. Contudo, não especifica qual o campo que identifica um cliente de forma única no sistema.

[Gerente]: E isso é importante porquê? Não basta pesquisar pelo nome?

[ES1]: Essa é exatamente a questão. Imagine o seguinte cenário: o senhor António regista uma vez um cliente como “José Silva” e noutra ocasião o mesmo cliente é registado como “José M. Silva” ou “Jose Silva” sem acento. Para o sistema, são três pessoas diferentes. O histórico fica partido em três perfis e nunca conseguiremos dar-lhe uma visão agregada.

[Gerente]: Percebo. E se o cliente empresarial tiver limite de crédito atribuído, este problema torna-se grave: o mesmo cliente pode ultrapassar o limite em cada perfil sem que o sistema o detete.

[Mecânico]: E pelo telefone também não serve, já nos aconteceu clientes mudarem de número. O mesmo vale para o *email*.

[ES2]: Exatamente. Precisamos de um identificador que seja estável, único e obrigatório no ato de registo. A nossa proposta é utilizar o NIF como identificador primário, dado que já é obrigatório no vosso fluxo por causa da faturação.

[Gerente]: Faz todo o sentido. O NIF é único por lei, cada pessoa ou empresa só tem um. E já o pedimos sempre na receção.

[ES1]: E o sistema deve bloquear qualquer tentativa de registar um novo cliente com um NIF que já exista?

[Gerente]: Sim, absolutamente. Se já existir, deve aparecer o registo antigo e permitir atualizar os dados, por exemplo o telefone ou *email*, mas nunca criar um novo.

[Mecânico]: Concordo. Evita-nos também aquela situação incómoda de estarmos a pedir os dados todos a um cliente que já cá veio três vezes.

[ES2]: Perfeito. Fica então assente que a regra de sistema é: **o NIF é o identificador primário e universal do cliente; o registo duplicado por NIF é proibido por construção; qualquer tentativa de registo de NIF já existente apresenta o registo existente ao operador para possível atualização de dados.**

[Gerente]: Acordado.

PARTE 2: Criação da Ordem de Serviço e Estado Inicial (REQ-008 e REQ-011) (0:30 – 0:50)

[ES1]: Segundo ponto. Durante a análise detetámos que os requisitos REQ-008 e REQ-011 estão intimamente relacionados mas apresentados como requisitos separados, o que cria ambiguidade. O REQ-008 diz que o sistema deve permitir criar uma ordem de serviço com numeração sequencial e data de entrada. O REQ-011 diz que o sistema deve criar a ordem no estado inicial “Recebida”. Isto descreve o mesmo momento operacional, a criação da OS, mas divide-o em dois requisitos distintos.

[Gerente]: E isso é um problema?

[ES2]: Potencialmente sim. Se tratarmos estes dois requisitos como separados, a implementação poderia considerar que a atribuição do estado “Recebida” é uma ação posterior e opcional. Já aconteceu em projetos que uma OS fosse criada sem estado e ficasse num limbo informático, sem aparecer nas listagens por estado.

[Mecânico]: Isso seria mau. Se eu crio a OS e ela não aparece em lado nenhum, perdemos a trotinete no sistema.

[ES1]: A nossa proposta é fundir os dois requisitos num só: a criação da ordem de serviço deve ser uma operação atômica que atribui simultaneamente o número sequencial, regista a data de entrada e define automaticamente o estado inicial como “Recebida”. Não há como uma OS existir sem estes três atributos em simultâneo.

[Gerente]: Isso é o que faz sentido no dia a dia. Quando abrimos uma ficha, é logo tudo ao mesmo tempo.

[Mecânico]: E os outros campos, como o diagnóstico, ficam para depois?

[ES2]: Exatamente, esse é precisamente o ponto seguinte que queremos clarificar. Mas para já, fica assente que **a criação de uma OS é uma operação única que inclui: numeração sequencial automática, registo da data de entrada, atribuição automática do estado “Recebida”, e associação obrigatória ao cliente e à trotinete.** Confirmam?

[Gerente]: Confirmamos.

[Mecânico]: Confirmamos.

PARTE 3: Momento de Preenchimento do Diagnóstico (REQ-009) (0:50 – 1:10)

[ES1]: Terceiro ponto, na sequência do anterior. O requisito REQ-009 diz que o sistema deve permitir registar, na ordem de serviço, o problema reportado pelo cliente e o diagnóstico técnico efetuado. A questão é: em que momento é que estes campos são preenchidos?

[Mecânico]: Bem, o problema reportado pelo cliente é logo, no ato da receção. Mas o diagnóstico técnico, esse nem sempre é feito no mesmo momento.

[ES2]: Exatamente, e é essa a ambiguidade. Se interpretarmos o requisito de forma estrita, o diagnóstico seria obrigatório para criar a OS, o que criaria um gargalo operacional.

[Gerente]: Isso seria um problema grave. Às vezes chegam cinco clientes ao balcão em meia hora, não vamos pedir ao Ricardo que faça o diagnóstico de cada trotinete antes de sequer aceitar a próxima.

[Mecânico]: E há muitas vezes em que o cliente entrega a trotinete de manhã e eu só consigo olhar para ela à tarde. O diagnóstico pode demorar um dia ou dois se estivermos com muito trabalho.

[ES1]: Fica então claro que o diagnóstico, para ser útil, não pode ser obrigatório no momento de criação da OS. A nossa proposta é que a criação da OS exija apenas um conjunto mínimo de campos obrigatórios: identificação do cliente, identificação da trotinete e data de entrada.

Todos os outros campos, incluindo o diagnóstico técnico, as intervenções realizadas, o orçamento e a data de conclusão, são adicionados progressivamente ao longo do ciclo de vida da OS.

[Gerente]: Isso reflete exatamente como trabalhamos hoje.

[Mecânico]: E o problema reportado pelo cliente?

[ES2]: Boa questão. Propomos que o problema reportado seja incluído nos campos obrigatórios da criação, já que é essa a razão pela qual o cliente trouxe a trotinete. Sem isso, a OS não faz sentido. O diagnóstico técnico, esse sim, é adicionado depois pelo mecânico.

[Mecânico]: Concordo.

[Gerente]: Acordado. **Campos obrigatórios na criação da OS: cliente, trotinete, data de entrada e problema reportado. Diagnóstico técnico e todos os outros campos são preenchidos progressivamente à medida que a reparação avança.**

PARTE 4: Redundância entre REQ-023 e REQ-020 (1:10 – 1:25)

[ES2]: Quarto ponto. Detetámos uma sobreposição entre dois requisitos da área de *stock*. O REQ-020 define o catálogo de peças com referência, descrição, fornecedor, preços e *stock* atual. O REQ-023 diz que o sistema deve permitir consultar a disponibilidade atual das peças para apoiar a reparação. Repare que o REQ-020 já inclui o *stock* atual como atributo visível do catálogo.

[Gerente]: E portanto o REQ-023 está a dizer a mesma coisa com outras palavras?

[ES1]: Exatamente. Se interpretarmos estes dois requisitos como independentes, podemos acabar com duas funcionalidades distintas para aceder à mesma informação, uma na área do catálogo de peças e outra na área da reparação. Isso gera inconsistência na interface e duplicação da lógica de negócio.

[Mecânico]: Mas eu preciso de ver o *stock* quando estou a fazer uma intervenção, não posso ir ao catálogo todas as vezes.

[ES2]: Sim, e essa necessidade fica coberta de outra forma. Quando associamos uma peça a uma intervenção, o sistema já mostra o *stock* disponível e, conforme acordado no REQ-021, bloqueia a associação se o *stock* for zero. A consulta ao *stock* está integrada no próprio ato de associação, não precisa de um ecrã separado.

[Mecânico]: Isso é muito melhor, confesso. Não quero andar a alternar entre ecrãs.

[Gerente]: Então o REQ-023 desaparece?

[ES1]: Sim, é considerado redundante e removido da versão consolidada. **O *stock* atual é um atributo do catálogo (REQ-020) e a verificação de disponibilidade no contexto de uma intervenção é garantida pelo comportamento do REQ-021.**

[Gerente]: Acordado.

PARTE 5: Âmbito do Controlo de Faturas Pendentes (REQ-033) (1:25 – 1:40)

[ES1]: Último ponto. O requisito REQ-033 fala da consulta rápida de faturas pendentes e atrasadas, mas não especifica se esta funcionalidade se aplica a todos os clientes ou apenas a um subconjunto.

[Gerente]: Ah, mas isso é só para os clientes empresariais. Os particulares pagam na hora em que vêm buscar a trotinete, nunca têm faturas pendentes.

[ES2]: Exatamente o ponto. Se aplicarmos o requisito indiscriminadamente, o sistema vai gerar alertas e listagens de “faturas pendentes” para clientes particulares que, por definição, nunca têm dívidas estruturais. Isso é ruído operacional desnecessário.

[Gerente]: Sim, os particulares só podem ter uma fatura pendente se, por algum motivo excecional, não pagarem na hora, o que praticamente nunca acontece. E mesmo nesse caso, seria resolvido imediatamente.

[Mecânico]: Já as empresas são outra história, com os prazos de 30 dias, temos lá sempre algumas faturas em aberto ao mesmo tempo.

[ES1]: Propomos então que o requisito seja reformulado para explicitar que a gestão de faturas pendentes, o controlo de atrasos e a emissão de alertas de pagamento são mecanismos exclusivamente aplicáveis a clientes empresariais com prazo de pagamento acordado, tipicamente 30 dias.

[Gerente]: Sim, é assim que faz sentido.

[ES2]: **Fica então assente: a consulta e a gestão de faturas pendentes ou atrasadas aplicam-se exclusivamente a clientes empresariais com prazo de pagamento acordado. Para clientes particulares, o pagamento é registado como imediato no ato da entrega e não gera estados de “pendente” ou “atrasado”.**

[Gerente]: Acordado.

Encerramento e Validação Formal (1:40 – 1:45)

[ES1]: Para terminar, gostaríamos de formalizar que, com as cinco clarificações acordadas hoje, a nossa base de requisitos está fechada. Isto significa que podemos avançar com segurança para a fase de arquitetura e modelação comportamental, sabendo que os pressupostos estão alinhados com as vossas expectativas operacionais.

[Gerente]: Da nossa parte, as dúvidas ficam todas sanadas. Confirmo que os requisitos refletem como pretendemos que o sistema funcione.

[Mecânico]: Concordo. E acho que estas conversas de revisão são importantes, ajudam-nos a perceber detalhes que não tínhamos equacionado.

[ES2]: Ficamos agradecidos pela vossa disponibilidade. As resoluções aqui acordadas serão incorporadas na especificação formal dos requisitos e refletidas nos diagramas de *Use Cases* e UML que iremos produzir a seguir.

[ES1]: Obrigado pela reunião. Até breve.

[Gerente]: De nada, estamos aqui para o que precisarem.

[Mecânico]: Boa sorte para a próxima fase!

[Fim da Transcrição]

Assinaturas:

_____ Engenheiro de *Software* 1

_____ Engenheiro de *Software* 2

_____ António Silva, Gerente

_____ Ricardo Costa, Mecânico Chefe

Anexo E: Transcrição de Documentos Físicos da Oficina

Prompt Utilizado

Contexto: Projeto das Oficinas Gengis Khan (LI4 - UMinho), com a definicao do sistema, duas entrevistas e user stories ja produzidas, e necessidade de simular documentos fisicos pre-existentes a transicao digital.

Objetivo: Simular as paginas do caderno operacional da oficina e documentos fisicos relacionados (ordens de servico, faturas, registos de stock) cobrindo cinco dias operacionais.

Instrucoes de Escrita:

- Antepor um texto introdutorio formal a explicar que o conteudo resulta da transcricao do caderno e documentos fisicos
- Simular a escrita do gerente ou do mecanico chefe consoante a accao, identificando o autor de cada registo
- Representar os documentos em texto puro, sem formatacoes especificas
- Maximizar cobertura de situacoes representativas ao longo dos cinco dias
- PT-PT

Restricoes: Reflectir apenas o funcionamento antes do sistema de gestao, sem melhorias desejadas. Sem inserir requisitos novos.

Metadados do Documento

Campo	Valor
Documento	Transcrição de Registos Físicos de Operação
Período Abrangido	10 a 14 de fevereiro de 2026
Finalidade	Suporte ao levantamento de requisitos do sistema de gestão digital
Elaborado por	Equipa Técnica, Oficina Gengis Khan
Data de Transcrição	Fevereiro de 2026
Classificação	Interno / Confidencial

Introdução

O presente documento constitui uma transcrição literal do conteúdo dos registos físicos utilizados pela *Oficinas Gengis Khan* no âmbito da sua operação diária, previamente à implementação do sistema de gestão digital. A transcrição abrange o período de 10 a 14 de fevereiro de 2026 e inclui os seguintes tipos de documentos: caderno de operações diárias, folhas de ordens de serviço, faturas emitidas aos clientes, caderno de *stock* com movimentações de peças, e anotações

de encomendas efetuadas a fornecedores. Esta transcrição serve de suporte ao levantamento de requisitos do sistema a desenvolver, permitindo uma compreensão detalhada dos processos operacionais e das informação atualmente registada pela oficina de forma manual.

E.1 Caderno de Operações

Segunda-feira, 10 de fevereiro de 2026 Período da Manhã

Entrada de equipamento, cliente novo (Ricardo)

“Cliente novo, Sr. Paulo Henriques. Trotinete Xiaomi M365 Pro, número de série XM365P-2023-44521. Queixa-se que não acelera bem, perde força a meio. Telefone 916 234 567, email paulo.henriques@mail.pt. Abri OS-2026-031.”

Diagnóstico OS-2026-031 (Ricardo)

“Problema no cabo do acelerador, está danificado. Telefonei ao cliente com orçamento: peça 18 € mais mão de obra, total à volta de 35 € com IVA. Aprovou.”

Período da Tarde

Reparação OS-2026-031 concluída (Ricardo)

“Substituí cabo do acelerador, demorei 25 minutos. Gastei 1 cabo acelerador Xiaomi ref. CABO-ACEL-XIA. Fui ver ao stock e só temos mais 2, o mínimo devia ser 5. Apontei para encomendar. Cliente veio buscar ao fim do dia.”

Faturação FAT-2026-067 (António)

“Fatura emitida para Sr. Paulo Henriques. Pagou em dinheiro, 35,06 €.”

Terça-feira, 11 de fevereiro de 2026 Período da Manhã

Entrada de equipamento, cliente empresa (Ricardo)

“Entrou trotinete de empresa, MobiUrban Lda. Contacto: Dr. Rui Fonseca, telefone 253 789 456, email rui.fonseca@mobiurban.pt. Trotinete Segway Ninebot Max, série SEG-NM-2024-78823. Queixa: barulho estranho na roda de trás. Abri OS-2026-032. Esta empresa tem conta conosco, paga a 30 dias.”

Diagnóstico OS-2026-032 (Ricardo)

“Rolamento da roda traseira gasto. Temos de trocar, mas não há em stock. Mandeí email ao Dr. Rui com orçamento: 28 € a peça mais uma hora de trabalho, fica perto de 75 €.”

Período da Tarde

Continuação OS-2026-029, Ninebot ES2 do Sr. Tiago Almeida

(Ricardo)

“Trotinete ficou cá na sexta-feira. Problema era no travão traseiro. Troquei pastilhas e fiz afinação. Demorei 40 minutos. Peças utilizadas: 2× PAST-TRAV-ES2.”

“Dr. Rui telefonou, aprova orçamento da OS-2026-032.”

Encomenda a fornecedor SparkParts

(Ricardo)

“Não temos rolamentos Segway nem cabos acelerador Xiaomi. Mandeí email à SparkParts:”

- 5× CABO-ACEL-XIA, cabo acelerador Xiaomi
- 3× ROL-TRAS-SEG, rolamento traseiro Segway

Quarta-feira, 12 de fevereiro de 2026 Período da Manhã

Levantamento de equipamento, OS-2026-029

(António)

“Sr. Tiago Almeida veio buscar a trotinete. Emitida fatura FAT-2026-068. Pagou com cartão.”

Entrada de equipamento, nova cliente

(Ricardo)

“Chegou a Sra. Mariana Costa, cliente nova. Xiaomi Essential, série XME-2024-12098. Pneu da frente furado. Telefone 934 567 123. Abri OS-2026-033. Como era rápido, a senhora esperou. Troquei a câmara de ar, 20 minutos, peça CAM-AR-DIANT-XIA. Levou logo.”

Faturação FAT-2026-069

(António)

“Fatura emitida para Sra. Mariana Costa. Pagou em dinheiro.”

Período da Tarde

Reparação OS-2026-030, Xiaomi Pro 2 do Sr. Fernando Reis

(Ricardo)

“Trotinete está cá há uma semana. Bateria avariada. Trabalho demorado, necessário desmontar quase toda a trotinete. Demorei 1 hora e 20 minutos. Peça BAT-XIA-PRO2. Fui ver o stock: só ficou 1 bateria Pro 2, mínimo são 2, necessário encomendar. Telefonei ao Sr. Fernando, pode vir buscar quando quiser.”

E.2 Ordens de Serviço

Campo	Informação
Número OS	OS-2026-031
Data de Entrada	10/02/2026
Cliente	Paulo Henriques
NIF	234 567 891
Telefone	916 234 567
Email	paulo.henriques@mail.pt
Tipo de Cliente	Particular
Equipamento	Xiaomi M365 Pro
Número de Série	XM365P-2023-44521
Problema Reportado	Não acelera bem, perde força a meio do percurso
Diagnóstico	Cabo do acelerador danificado, necessita substituição
Orçamento Apresentado	≈ 35,00 €
Aprovação do Cliente	Sim
Intervenção Realizada	Substituição do cabo acelerador
Tempo de Intervenção	25 minutos
Peças Utilizadas	1× CABO-ACEL-XIA
Mecânico Responsável	Ricardo
Data de Conclusão	10/02/2026
Data de Entrega	10/02/2026
Fatura Associada	FAT-2026-067

Ordem de Serviço OS-2026-031

Campo	Informação
Número OS	OS-2026-032
Data de Entrada	11/02/2026
Cliente	MobiUrban Lda
NIF	509 876 543
Contacto	Dr. Rui Fonseca
Telefone	253 789 456
<i>Email</i>	rui.fonseca@mobiurban.pt
Tipo de Cliente	Empresa, crédito 30 dias
Equipamento	Segway Ninebot Max
Número de Série	SEG-NM-2024-78823
Problema Reportado	Barulho estranho na roda traseira
Diagnóstico	Rolamento da roda traseira gasto
Orçamento Apresentado	≈ 75,00 €
Aprovação do Cliente	Sim, 11/02 por telefone
Estado	A aguardar peças (rolamento encomendado)

Ordem de Serviço OS-2026-032

Campo	Informação
Número OS	OS-2026-033
Data de Entrada	12/02/2026
Cliente	Mariana Costa
NIF	276 543 210
Telefone	934 567 123
Tipo de Cliente	Particular
Equipamento	Xiaomi Essential
Número de Série	XME-2024-12098
Problema Reportado	Pneu dianteiro furado
Diagnóstico	Câmara de ar furada
Intervenção Realizada	Substituição da câmara de ar dianteira
Tempo de Intervenção	20 minutos
Peças Utilizadas	1× CAM-AR-DIANT-XIA
Mecânico Responsável	Ricardo
Data de Conclusão	12/02/2026
Data de Entrega	12/02/2026
Fatura Associada	FAT-2026-069

Ordem de Serviço OS-2026-033

E.3 Faturas Emitidas

Campo	Informação
Emitente	Oficina Gengis Khan
Morada	Rua das Trotinetes, 123, 4700-000 Braga
NIF Emitente	515 678 901
N.º Fatura	FAT-2026-067
Data	10/02/2026
Cliente	Paulo Henriques
NIF Cliente	234 567 891
OS Associada	OS-2026-031

Fatura **FAT-2026-067** Descrição dos Serviços:

Descrição	Ref.	Qtd.	Valor
Cabo acelerador Xiaomi	CABO-ACEL-XIA	1 un.	18,00 €
Mão de obra (25 min.)			10,42 €
Subtotal			28,42 €
IVA 23 %			6,54 €
Total			34,96 €

Forma de Pagamento: Dinheiro | Estado: Pago | Assinatura: António Silva

Campo	Informação
Emitente	Oficina Gengis Khan
Morada	Rua das Trotinetes, 123, 4700-000 Braga
NIF Emitente	515 678 901
N.º Fatura	FAT-2026-068
Data	12/02/2026
Cliente	Tiago Almeida
NIF Cliente	198 765 432
OS Associada	OS-2026-029

Fatura **FAT-2026-068** Descrição dos Serviços:

Descrição	Ref.	Qtd.	Valor
Pastilhas travão Ninebot ES2	PAST-TRAV-ES2	2 un.	24,00 €
Mão de obra (40 min.)			16,67 €
Subtotal			40,67 €
IVA 23 %			9,35 €
Total			50,02 €

Forma de Pagamento: Multibanco | Estado: Pago | Assinatura: António Silva

Campo	Informação
Emitente	Oficina Gengis Khan
Morada	Rua das Trotinetes, 123, 4700-000 Braga
NIF Emitente	515 678 901
N.º Fatura	FAT-2026-069
Data	12/02/2026
Cliente	Mariana Costa
NIF Cliente	276 543 210
OS Associada	OS-2026-033

Fatura FAT-2026-069 Descrição dos Serviços:

Descrição	Ref.	Qtd.	Valor
Câmara ar dianteira Xiaomi	CAM-AR-DIANT-XIA	1 un.	8,00 €
Mão de obra (20 min.)			8,33 €
Subtotal			16,33 €
IVA 23 %			3,76 €
Total			20,09 €

Forma de Pagamento: Dinheiro | Estado: Pago | Assinatura: António Silva

E.4 Anotações de Encomendas

Campo	Informação
Data	11/02/2026
Fornecedor	SparkParts
Email	sparks.parts@email.pt
Método de Envio	Email
Prazo Estimado	3 a 5 dias úteis
Registado por	Ricardo

Encomenda de 11/02/2026 Peças Encomendadas:

Referência	Descrição	Quantidade
CABO-ACEL-XIA	Cabo acelerador Xiaomi	5 un.
ROL-TRAS-SEG	Rolamento traseiro Segway	3 un.

E.5 Caderno de *Stock* (Excerto)

Nota: os itens assinalados como abaixo do mínimo requerem encomenda imediata.

Campo	Informação
Referência	CABO-ACEL-XIA
Descrição	Cabo acelerador Xiaomi
Fornecedor	SparkParts
Preço de Custo	9,50 €
Preço de Venda	18,00 €
Stock Mínimo	5 un.

Referência: CABO-ACEL-XIA, Cabo Acelerador Xiaomi Movimentos de *Stock*:

Data	Tipo	Referência	Var.	Saldo	Responsável
03/02/2026	Entrada (encomenda)		+10	7	António
10/02/2026	Saída (OS)	OS-2026-031	-1	2	Ricardo

ATENÇÃO: *stock* abaixo do mínimo definido (mín. 5 un. / atual 2 un.)

Campo	Informação
Referência	BAT-XIA-PRO2
Descrição	Bateria Xiaomi Pro 2
Fornecedor	ElecTrot
Preço de Custo	52,00 €
Preço de Venda	89,00 €
Stock Mínimo	2 un.

Referência: BAT-XIA-PRO2, Bateria Xiaomi Pro 2 Movimentos de *Stock*:

Data	Tipo	Referência	Var.	Saldo	Responsável
20/01/2026	Entrada (encomenda)		+3	3	António
05/02/2026	Saída (OS)	OS-2026-025	−1	2	Ricardo
12/02/2026	Saída (OS)	OS-2026-030	−1	1	Ricardo

ATENÇÃO: *stock* abaixo do mínimo definido (mín. 2 un. / atual 1 un.)

Documento elaborado pela Oficina Gengis Khan, uso interno. Rua das Trotinetes, 123, 4700-000 Braga, NIF 515 678 901.

Anexo F: Registo de Decisões Arquiteturais (ADRs)

ADR-0001. Organização em *Monorepo* com Contratos Partilhados

Campo	Valor
Identificador	ADR-0001
Título	Organização em <i>monorepo</i> com contratos partilhados
Estado	Aceite
Data	Etapa 2, Conceção e <i>Design</i> (março de 2026)
Decisores	Equipa de projeto (5 elementos)
Decisão relacionada com	Subsecções 2.1.3 e 2.1.4

Contexto. O sistema a desenvolver no âmbito das Oficinas Gengis Khan é constituído por duas aplicações distintas mas estreitamente acopladas, designadamente um *backend* em Node.js que expõe uma API REST e uma aplicação *frontend* do tipo *Single Page Application*. Ambas partilham conceitos de domínio (clientes, trotinetes, ordens de serviço, faturas) e

contratos de dados (formas de *request* e *response*, enumerações de estados, invariantes de validação). Num cenário de desenvolvimento convencional, cada alteração a um contrato exigiria modificações sincronizadas em dois repositórios separados, com risco considerável de divergência silenciosa entre camadas e de quebras em tempo de execução dificilmente detetáveis por verificação estática. Acresce que a equipa tem prazos académicos curtos e experiência limitada em coordenação multirrepositório, o que eleva o custo de incidentes desta natureza.

Decisão. A equipa adotou uma organização em *monorepo* único, gerido através de *pnpm workspaces* e orquestrado por Turborepo. A estrutura do repositório compreende as componentes seguintes.

- `apps/api`: aplicação *backend* em Fastify e TypeScript, estruturada como monólito modular;
- `apps/web`: aplicação *frontend* em React, Vite e TypeScript, empacotada como SPA;
- `packages/contracts`: pacote de contratos partilhados, contendo esquemas Zod, enumerações de domínio e tipos derivados por inferência;
- `packages/tooling-typescript` e `packages/tooling-eslint`: configurações transversais de linguagem e estilo;
- `supabase/`: configuração da instância local de PostgreSQL para desenvolvimento;
- `docs/`: documentação arquitetural, especificação OpenAPI e manuais de configuração.

O Prisma assume o papel de única autoridade sobre o *schema* da base de dados e suas migrações. O Supabase CLI é utilizado exclusivamente em ambiente de desenvolvimento local, mantendo-se a produção independente de serviços da plataforma.

Alternativas Ponderadas.

- **Dois repositórios Git separados** (um por aplicação). Rejeitada: impossibilita a partilha literal de código entre camadas, exige mecanismos de versionamento independente do pacote de contratos e introduz *overhead* de coordenação em refactorizações *cross-cutting*;
- **Monorepo sem orquestrador** (apenas *pnpm workspaces*, sem Turborepo). Rejeitada: ausência de *cache* partilhado e de declaração explícita de dependências entre tarefas penalizaria o tempo de *feedback* em alterações incrementais;
- **Monorepo com alternativas ao Zod** (geração de tipos a partir de OpenAPI, *trRPC*). Rejeitadas em decisão posterior: ver Subsecção 2.1.4.

Consequências Positivas.

- Os contratos de *request* e *response* residem num único pacote partilhado, eliminando a possibilidade de divergência silenciosa entre *frontend* e *backend*;
- A validação em *runtime* no *backend* e a verificação estática de tipos no *frontend* partilham a mesma fonte, reduzindo a duplicação de esforço;
- O repositório mantém uma única linguagem (TypeScript) em ambas as camadas da aplicação, favorecendo mobilidade de competências na equipa;
- A produção permanece portátil, sem dependência dos serviços da plataforma Supabase, facilitando uma eventual transição para outro provedor.

Consequências Negativas.

- A configuração inicial do *workspace* exige um investimento técnico maior do que a inicialização de dois projetos independentes;
- Exige disciplina contínua na definição de fronteiras entre pacotes, sob pena de acoplamento estrutural excessivo e perda de separação efetiva de responsabilidades;
- O tempo de arranque do *pipeline* de integração contínua aumenta com o número de pacotes, mitigado parcialmente pelo *cache* de Turborepo.

ADR-0002. Padrão Arquitetural, Monólito Modular em Três Camadas

Campo	Valor
Identificador	ADR-0002
Título	Padrão arquitetural: monólito modular em três camadas
Estado	Aceite
Data	Etapa 2, Conceção e <i>Design</i> (março de 2026)
Decisores	Equipa de projeto (5 elementos)
Decisão relacionada com	Subsecção 2.1.1

Contexto. Na transição da engenharia de requisitos para o *design*, era necessário fixar o padrão arquitetural do sistema. As 44 RF e 5 RNF do SRS, conjugadas com as restrições do projeto (equipa de cinco estudantes, horizonte de catorze semanas, faturação sequencial sob Decreto-Lei 28/2019, conformidade RGPD), impunham uma escolha entre três famílias de soluções amplamente documentadas, designadamente monólito em camadas clássico, monólito modular com fronteiras explícitas por domínio, e microserviços com fronteiras de processo.

Decisão. A equipa adotou o **monólito modular em três camadas** (apresentação, lógica de negócio, dados), com separação estrita entre módulos funcionais ao nível do código mas mantendo um único processo de *deployment* e uma única fronteira de rede. Os limites de módulo são definidos por domínio funcional (*auth, customers, scooters, service-orders, interventions, inventory, suppliers-procurement, billing, notifications, configuration, reports*) desde o início, preparando o sistema para uma eventual extração futura de microserviços caso o crescimento o justifique.

Alternativas Ponderadas. A Tabela 5.1 sumariza a análise comparativa entre as três alternativas consideradas, segundo seis critérios.

Tabela 5.1: Análise comparativa de padrões arquiteturais candidatos

Critério	Monólito em camadas	Monólito modular (adotado)	Microserviços
Complexidade de desenvolvimento	Baixa	Moderada	Elevada
Complexidade operacional	Baixa (1 processo, 1 <i>pipeline</i>)	Baixa (1 processo, módulos isolados)	Elevada (N processos, orquestração)
Encapsulamento de responsabilidades	Reduzido (partilha de modelos entre funções)	Elevado (fronteiras modulares explícitas)	Máximo (fronteiras de processo)
Adequação ao volume operacional esperado	Adequada	Adequada	Sobredimensionada
Adequação ao calendário académico	Adequada	Adequada	Inadequada
Risco de acoplamento accidental	Elevado	Contido por convenção	Baixo

O **monólito em camadas clássico** foi descartado pelo elevado risco de acoplamento accidental, dado que a partilha de modelos entre funções tende a degradar a coesão modular ao longo do ciclo de vida do produto. Os **microserviços** foram descartados por desadequação ao calendário académico e ao volume operacional projetado, dado que o custo de coordenação inerente à operação de múltiplos serviços com *pipelines* de *deploy* independentes não tem contrapartida em benefício mensurável à dimensão do projeto.

Consequências Positivas.

- As fronteiras modulares explícitas sustentam o trabalho da equipa: cada elemento assume a responsabilidade por um módulo coeso, com interfaces de integração delimitadas, reduzindo o acoplamento entre ramos de desenvolvimento paralelos e facilitando a revisão por pares;
- O processo de *deployment* unificado e o *pipeline* único de integração contínua mantêm a complexidade operacional baixa, adequada à dimensão da equipa e ao calendário do projeto;

- A separação interna por módulos prepara o sistema para uma eventual extração futura para microserviços, caso o volume operacional ou as exigências de escalabilidade venham a justificá-la.

Consequências Negativas.

- A disciplina das fronteiras modulares depende da convenção da equipa, não de uma fronteira de processo: a sua erosão ao longo do tempo é um risco identificado e mitigado por revisão de código sistemática;
- Mantém-se a impossibilidade de escalar módulos individualmente, que só seria atingida com a transição posterior para microserviços.

Anexo G: Catálogo Completo de *Endpoints* do Subsistema Operacional

O presente anexo reúne o catálogo completo dos *endpoints* HTTP expostos pela API do sistema, conforme implementados pelo *backend Fastify*. As Tabelas 2.8 a 2.11 do corpo do relatório (Subsecção 2.5.2) apresentam apenas excertos representativos. As tabelas que se seguem apresentam, por subsistema, a totalidade dos *endpoints* do contrato.

Os caminhos são apresentados sem o prefixo `/api/v1/` por economia de espaço, ainda que este se aplique sem exceção. A coluna *Papéis* adota M para manager, T para mechanic e *aut.* para qualquer utilizador autenticado.

G.1. Subsistema Operacional

Tabela 5.2: Catálogo completo de *endpoints* do subsistema operacional

Método	Caminho	Objetivo	Papéis
<i>Autenticação</i>			
POST	/auth/login	Estabelece sessão e emite <i>cookie</i>	público
POST	/auth/logout	Termina a sessão atual	aut.
GET	/auth/session	Consulta a sessão atual	aut.
GET	/auth/me	Devolve o perfil do utilizador autenticado	aut.
<i>Clientes</i>			
GET	/customers	Lista clientes (filtros opcionais)	M, T
POST	/customers	Regista novo cliente	M, T
GET	/customers/:nif	Consulta cliente por NIF	M, T
PATCH	/customers/:nif	Atualiza dados editáveis do cliente	M, T

(continua na página seguinte)

Tabela 5.2 (continuação)

Método	Caminho	Objetivo	Papéis
GET	/customers/:nif/history	Histórico de ordens do cliente	M, T
<i>Trotinetes</i>			
GET	/scooters	Lista trotinetes registadas	M, T
POST	/scooters	Regista nova trotinete	M, T
GET	/scooters/:serialNumber	Consulta trotinete por número de série	M, T
PATCH	/scooters/:serialNumber	Atualiza dados da trotinete	M, T
GET	/scooters/:serialNumber/repairs	Histórico de reparações da trotinete	M, T
<i>Ordens de Serviço</i>			
GET	/service-orders	Lista ordens de serviço (filtros opcionais)	M, T
POST	/service-orders	Cria nova ordem de serviço	M, T
GET	/service-orders/summary	Resumo agregado por estado	M, T
GET	/service-orders/:id	Consulta ordem de serviço	M, T
PATCH	/service-orders/:id	Atualiza problema reportado e prazo previsto	M, T
PATCH	/service-orders/:id/status	Transição de estado da ordem	M, T
PATCH	/service-orders/:id/diagnosis	Regista diagnóstico técnico	M, T
PATCH	/service-orders/:id/budget	Regista orçamento detalhado	M, T
POST	/service-orders/:id/budget/approval	Regista decisão do cliente sobre o orçamento	M, T
GET	/service-orders/:id/history	Histórico de transições e ações da ordem	M, T
<i>Intervenções</i>			
GET	/service-orders/:id/interventions	Lista intervenções da ordem	M, T
POST	/service-orders/:id/interventions	Cria intervenção sobre a ordem	M, T
GET	/interventions/:id	Consulta intervenção	M, T
PATCH	/interventions/:id	Atualiza descrição, mecânico e notas	M, T
PATCH	/interventions/:id/timer	Transição genérica do cronómetro	M, T
POST	/interventions/:id/timer/start	Inicia o cronómetro	M, T
POST	/interventions/:id/timer/pause	Suspende o cronómetro	M, T
POST	/interventions/:id/timer/stop	Encerra definitivamente o cronómetro	M, T
GET	/interventions/:id/parts	Lista peças aplicadas na intervenção	M, T
POST	/interventions/:id/parts	Associa peça à intervenção (consome stock)	M, T

G.2. Subsistema de Aprovisionamento e Inventário

Tabela 5.3: Catálogo completo de *endpoints* do subsistema de aprovisionamento e inventário

Método	Caminho	Objetivo	Papéis
<i>Catálogo de Peças</i>			
GET	/parts	Lista peças (filtros opcionais)	M, T
POST	/parts	Regista nova peça no catálogo	M, T
GET	/parts/low-stock	Lista peças abaixo do <i>stock</i> mínimo	M, T
GET	/parts/:partReference	Consulta peça por referência interna	M, T
PATCH	/parts/:partReference	Atualiza dados editáveis da peça	M, T
<i>Movimentos de Stock</i>			
GET	/stock-movements	Lista movimentos de <i>stock</i>	M, T
POST	/parts/:partReference/stock-adjustment	Regista ajuste manual de <i>stock</i>	M, T
<i>Fornecedores</i>			
GET	/suppliers	Lista fornecedores	M, T
POST	/suppliers	Regista novo fornecedor	M, T
GET	/suppliers/:supplierId	Consulta fornecedor	M, T
PATCH	/suppliers/:supplierId	Atualiza dados do fornecedor	M, T
<i>Encomendas a Fornecedores</i>			
GET	/purchase-orders	Lista encomendas a fornecedores	M, T
POST	/purchase-orders	Cria encomenda manual	M, T
POST	/purchase-orders/generate-from-low-stock	Cria encomendas a partir de peças com <i>stock</i> mínimo	M, T
GET	/purchase-orders/:purchaseOrderId	Consulta encomenda	M, T
PATCH	/purchase-orders/:purchaseOrderId	Atualiza encomenda	M, T
PATCH	/purchase-orders/:purchaseOrderId/receive	Marca encomenda como recebida (atualiza <i>stock</i>)	M, T

G.3. Subsistema de Faturação, Notificações e Configuração

Tabela 5.4: Catálogo completo de *endpoints* do subsistema de faturação, notificações e configuração

Método	Caminho	Objetivo	Papéis
<i>Faturação</i>			
GET	/invoices	Lista faturas emitidas	M
POST	/invoices	Cria fatura sem cálculo automático	M
POST	/invoices/issue	Emite fatura sobre ordem de serviço concluída	M
GET	/invoices/pending-business	Lista faturas pendentes ou atrasadas	M
GET	/invoices/:id	Consulta fatura	M
GET	/invoices/:id/pdf	Devolve fatura em formato PDF	M
POST	/invoices/:id/payments	Regista pagamento sobre a fatura	M
PATCH	/invoices/:id/payment	Atualiza estado de pagamento	M
<i>Notificações</i>			
GET	/notifications	Lista notificações já emitidas	M, T
POST	/notifications	Cria registo de notificação sem envio	M, T
POST	/notifications/send	Envia notificação ao cliente por <i>email</i>	M, T
GET	/notification-templates	Lista <i>templates</i> de notificação configurados	M, T
PATCH	/notification-templates/:templateId	Atualiza <i>template</i> de notificação	M, T
<i>Configuração Financeira</i>			
GET	/config/financial-parameters	Consulta taxa horária de mão de obra e taxa de IVA	M
PATCH	/config/financial-parameters	Atualiza taxa horária de mão de obra e taxa de IVA	M

G.4. Reportes Transversais

Tabela 5.5: Catálogo completo de *endpoints* de relatório transversal

Método	Caminho	Objetivo	Papéis
GET	/reports/operations	Indicadores operacionais agregados	M
GET	/reports/billing	Indicadores de faturação por intervalo	M
GET	/reports/costs	Indicadores de custo de mão de obra e peças	M
GET	/reports/parts-usage	Análise de consumo de peças por intervalo	M
GET	/reports/common-failures	Identificação de avarias mais frequentes	M
GET	/reports/repair-time	Análise de tempo médio de reparação	M
GET	/reports/mechanic-productivity	Indicadores de produtividade por mecânico	M

Anexo H: Catálogo Completo de Requisitos Levantados e Consolidados

O presente anexo reproduz a totalidade das tabelas de requisitos produzidas durante a fase de elicitação e consolidação. A Subsecção 1.3 do corpo do relatório apresenta apenas excertos representativos das três tabelas, remetendo para este anexo a versão integral.

H.1. Requisitos Levantados (Listagem Bruta)

A Tabela 5.6 contém a listagem integral dos 47 requisitos identificados durante o processo iterativo de elicitação, antes da categorização por tipo e da fusão de duplicados. Esta listagem corresponde à versão imediatamente posterior às entrevistas e à análise documental, e representa a fronteira entre o levantamento e a especificação formal.

Tabela 5.6: Requisitos levantados não estruturados

Nº	Data	Descrição	Vista de Uso	Fonte	Analista
REQ-001	20/02/2026	O sistema deve permitir registar clientes com nome completo, NIF, telefone e <i>email</i> obrigatórios, e morada opcional.	Atendimento	António Silva / Entrevista-002	Francisco Martins

continua na próxima página

Tabela 5.6 – continuação

Nº	Data	Descrição	Vista de Uso	Fonte	Analista
REQ-002	17/02/2026	O sistema deve distinguir clientes particulares de clientes empresariais.	Atendimento	António Silva / Entrevista-001	Nuno Rebelo
REQ-003	20/02/2026	O sistema deve permitir registar, para clientes empresariais, a denominação social, o limite de crédito e as condições de pagamento.	Atendimento	António Silva / Entrevista-002	Marco Sèvegrand
REQ-004	20/02/2026	O sistema deve permitir consultar o histórico do cliente, incluindo quantas vezes recorreu à oficina, quanto já gastou e se tem faturas por pagar.	Atendimento	António Silva / Entrevista-002	Lucas Robertson
REQ-005	20/02/2026	O sistema deve permitir que os campos de <i>email</i> e telemóvel de qualquer cliente sejam editáveis após o registo.	Atendimento	Ricardo Costa / Entrevista-001	Nuno Rebelo
REQ-006	17/02/2026	O sistema deve permitir registar cada trotinete com marca, modelo, número de série único e observações sobre o estado geral do equipamento.	Atendimento	Ricardo Costa / Entrevista-001	Marco Ferreira
REQ-007	20/02/2026	O sistema deve permitir consultar o histórico completo de reparações de uma trotinete através do número de série.	Atendimento	Ricardo Costa / Entrevista-002	Francisco Martins
REQ-008	17/02/2026	O sistema deve permitir criar uma ordem de serviço com numeração sequencial e data de entrada.	Gestão de OS	António Silva / Entrevista-001	Nuno Rebelo
REQ-009	17/02/2026	O sistema deve permitir registar, na ordem de serviço, o problema reportado pelo cliente e o diagnóstico técnico efetuado.	Gestão de OS	Ricardo Costa / Entrevista-001	Marco Sèvegrand
REQ-010	11/02/2026	O sistema deve permitir registar, na ordem de serviço, o orçamento apresentado e a aprovação do cliente.	Gestão de OS	Ricardo Costa / Registos Oficina, OS-2026-032	Lucas Robertson
REQ-011	20/02/2026	O sistema deve criar a ordem de serviço no estado inicial "Recebida".	Gestão de OS	Ricardo Costa / Entrevista-002	Marco Ferreira
REQ-012	17/02/2026	O sistema deve suportar os estados da OS: "Recebida", "Em Diagnóstico", "Aguarda Aprovação do Cliente", "Aguarda Peças", "Em Reparação", "Concluída" e "Entregue ao Cliente".	Gestão de OS	António Silva e Ricardo Costa / Entrevista-001	Francisco Martins

continua na próxima página

Tabela 5.6 – continuação

Nº	Data	Descrição	Vista de Uso	Fonte	Analista
REQ-013	20/02/2026	O sistema deve registar automaticamente a data e hora de cada mudança de estado da ordem de serviço e manter o respetivo histórico.	Gestão de OS	António Silva / Entrevista-002	Nuno Rebelo
REQ-014	20/02/2026	O sistema deve permitir que os mecânicos alterem os estados operacionais da ordem de serviço e que o estado “Entregue ao Cliente” seja confirmado pelo gerente.	Gestão de OS	Ricardo Costa e António Silva / Entrevista-002	Marco Sèvegrand
REQ-015	20/02/2026	O sistema deve apresentar um resumo com a quantidade de ordens de serviço em cada estado.	Gestão de OS	António Silva / Entrevista-002	Lucas Robertson
REQ-016	17/02/2026	O sistema deve permitir associar múltiplas intervenções a uma mesma ordem de serviço.	Reparação	Ricardo Costa / Entrevista-001	Marco Ferreira
REQ-017	17/02/2026	O sistema deve permitir registar, em cada intervenção, a descrição do trabalho realizado, o mecânico responsável, o tempo gasto e as peças utilizadas.	Reparação	Ricardo Costa / Entrevista-001	Francisco Martins
REQ-018	20/02/2026	O sistema deve disponibilizar um cronómetro integrado, com pausa e retoma, para medir o tempo de cada intervenção.	Reparação	Ricardo Costa / Entrevista-002	Nuno Rebelo
REQ-019	10/02/2026	O sistema deve registar, na ordem de serviço, a data de conclusão, a data de entrega e a fatura associada.	Reparação	Ricardo Costa / Registos Oficina, OS-2026-031	Marco Sèvegrand
REQ-020	17/02/2026	O sistema deve manter um catálogo de peças com referência interna, descrição, fornecedor, preço de custo, preço de venda, <i>stock</i> atual e <i>stock</i> mínimo.	Gestão de Stock	António Silva e Ricardo Costa / Entrevista-001	Lucas Robertson
REQ-021	20/02/2026	O sistema deve permitir associar peças a uma intervenção e diminuir automaticamente o <i>stock</i> dessas peças.	Gestão de Stock	Ricardo Costa / Entrevista-002	Marco Ferreira
REQ-022	10/02/2026	O sistema deve registar movimentos de <i>stock</i> com data, tipo de movimento, origem, variação, saldo e responsável.	Gestão de Stock	Oficina Gengis Khan / Caderno de Stock	Francisco Martins

continua na próxima página

Tabela 5.6 – continuação

Nº	Data	Descrição	Vista de Uso	Fonte	Analista
REQ-023	17/02/2026	O sistema deve permitir consultar a disponibilidade atual das peças para apoiar a reparação.	Gestão de Stock	Ricardo Costa / Entrevista-001	Nuno Rebelo
REQ-024	17/02/2026	O sistema deve manter, para cada fornecedor, o nome, <i>email</i> , telefone e as condições de pagamento.	Fornecedores	António Silva / Entrevista-001	Marco Sèvegrand
REQ-025	17/02/2026	O sistema deve permitir que os campos de <i>email</i> e telemóvel de qualquer fornecedor sejam editáveis após o registo.	Fornecedores	António Silva / Entrevista-001	Marco Ferreira
REQ-026	17/02/2026	O sistema deve calcular o valor da reparação com base no preço de venda das peças, no tempo de mão de obra e no IVA aplicável.	Faturação	António Silva / Entrevista-001	Lucas Robertson
REQ-027	20/02/2026	O sistema deve gerar automaticamente a fatura a partir da ordem de serviço concluída.	Faturação	António Silva / Entrevista-002	Marco Ferreira
REQ-028	17/02/2026	O sistema deve apresentar na fatura o detalhe das peças utilizadas e das horas de trabalho.	Faturação	António Silva / Entrevista-001	Francisco Martins
REQ-029	20/02/2026	O sistema deve calcular automaticamente subtotal, IVA e total da fatura.	Faturação	António Silva / Entrevista-002	Nuno Rebelo
REQ-030	17/02/2026	O sistema deve emitir faturas com numeração sequencial contínua, sem quebras.	Faturação	António Silva / Entrevista-001	Marco Sèvegrand
REQ-031	20/02/2026	O sistema deve permitir configurar a taxa horária da mão de obra e a taxa de IVA.	Faturação	António Silva / Entrevista-002	Lucas Robertson
REQ-032	12/02/2026	O sistema deve registar, na fatura, a forma de pagamento e o estado de pagamento.	Faturação	Oficina Gengis Khan / Faturas FAT-2026-068 e 069	Marco Ferreira
REQ-033	20/02/2026	O sistema deve permitir consultar rapidamente as faturas pendentes e atrasadas, em especial as de clientes empresariais.	Faturação	António Silva / Entrevista-002	Francisco Martins
REQ-034	20/02/2026	O sistema deve ser uma aplicação <i>web</i> acessível na oficina e remotamente.	Plataforma	António Silva / Entrevista-002	Nuno Rebelo
REQ-035	20/02/2026	O sistema deve funcionar adequadamente em <i>tablets</i> .	Plataforma	Ricardo Costa / Entrevista-002	Marco Sèvegrand
REQ-036	20/02/2026	O sistema deve suportar utilizadores com <i>login</i> individual.	Segurança	António Silva / Entrevista-002	Lucas Robertson

continua na próxima página

Tabela 5.6 – continuação

Nº	Data	Descrição	Vista de Uso	Fonte	Analista
REQ-037	20/02/2026	O sistema deve dar acesso total ao gerente.	Segurança	António Silva / Entrevista-002	Marco Ferreira
REQ-038	20/02/2026	O sistema deve permitir que os mecânicos criem e atualizem OS, registem diagnósticos e intervenções, associem peças e consultem <i>stock</i> e histórico técnico.	Segurança	António Silva / Entrevista-002	Marco Ferreira
REQ-039	20/02/2026	O sistema deve proteger os dados pessoais dos clientes em conformidade com o RGPD.	Segurança	António Silva / Entrevista-002	Francisco Martins
REQ-040	20/02/2026	O sistema deve executar <i>backups</i> automáticos diários.	Segurança	António Silva / Entrevista-002	Nuno Rebelo
REQ-041	20/02/2026	O sistema deve suportar o volume atual de cerca de 150 ordens de serviço por mês e o crescimento previsto para 500 por mês.	Plataforma	António Silva / Entrevista-002	Marco Sèvegrand
REQ-042	20/02/2026	O sistema deve enviar ao cliente uma confirmação de receção quando a trotinete é aceite, incluindo o número da ordem de serviço e um prazo estimado.	Notificações	António Silva / Entrevista-002	Francisco Martins
REQ-043	20/02/2026	O sistema deve enviar ao cliente o orçamento detalhado por <i>email</i> quando a ordem de serviço passa para o estado “Aguarda Aprovação do Cliente”, solicitando confirmação.	Notificações	António Silva e Ricardo Costa / Entrevista-002	Nuno Rebelo
REQ-044	20/02/2026	O sistema deve enviar ao cliente uma notificação quando a reparação estiver concluída, informando que a trotinete está pronta e indicando o valor total.	Notificações	António Silva / Entrevista-002	Marco Sèvegrand
REQ-045	20/02/2026	O sistema deve enviar ao cliente um <i>email</i> de atraso quando uma trotinete permanecer mais de 10 dias no estado “Aguarda Peças”.	Notificações	António Silva / Entrevista-002	Lucas Robertson
REQ-046	20/02/2026	O sistema deve permitir personalizar o conteúdo das notificações através de <i>templates</i> ajustáveis.	Notificações	António Silva / Entrevista-002	Marco Ferreira

continua na próxima página

Tabela 5.6 – continuação

Nº	Data	Descrição	Vista de Uso	Fonte	Analista
REQ-047	20/02/2026	O sistema deve usar o <i>email</i> como canal principal para as notificações automáticas ao cliente.	Notificações	António Silva / Entrevista-002	Francisco Martins

continua na próxima página

Tabela 5.7 – continuação

Nº	Data	Descrição	Vista de Uso	Fonte	Analista
H.2. Requisitos Funcionais Consolidados Após a consolidação descrita na Subsecção 1.3, com remoção de duplicados e renumeração subsequente, foram fixados 44 requisitos funcionais. A Tabela 5.7 apresenta a sua totalidade. Tabela 5.7: Requisitos funcionais consolidados					
Nº	Data	Descrição	Vista de Uso	Fonte	Analista
RF-01	2026-02-20	O sistema deve permitir registar clientes com nome completo, NIF (usado como identificador), telefone e <i>email</i> obrigatórios, e morada opcional.	Atendimento	António Silva / Entrevista-002	Francisco Martins
RF-02	2026-02-17	O sistema deve distinguir clientes particulares de clientes empresariais.	Atendimento	António Silva / Entrevista-001	Nuno Rebelo
RF-03	2026-02-20	O sistema deve gerar um alerta automático quando um cliente empresarial estiver perto de atingir ou ultrapassar o seu limite de crédito.	Atendimento	António Silva / Entrevista-002	Nuno Rebelo
RF-04	2026-02-20	O sistema deve permitir registar, para clientes empresariais, a denominação social, o limite de crédito e as condições de pagamento.	Atendimento	António Silva / Entrevista-002	Marco Sè-vegrand
RF-05	2026-02-20	O sistema deve permitir consultar o histórico do cliente através do NIF, incluindo quantas vezes recorreu à oficina, quanto já gastou e se tem faturas por pagar.	Atendimento	António Silva / Entrevista-002	Lucas Robertson
RF-06	2026-02-20	O sistema deve permitir que os campos de <i>email</i> e telemóvel de qualquer cliente sejam editáveis após o registo.	Atendimento	Ricardo Costa / Entrevista-001	Nuno Rebelo
RF-07	2026-02-17	O sistema deve permitir registar cada trotinete com marca, modelo, número de série único, observações sobre o estado geral do equipamento e cliente associado.	Atendimento	Ricardo Costa / Entrevista-001	Marco Ferreira
				continua na próxima página	

Tabela 5.7 – continuação

Nº	Data	Descrição	Vista de Uso	Fonte	Analista
RF-08	2026-02-17	O sistema deve permitir criar uma ordem de serviço com numeração sequencial, data de entrada, estado inicial “Recebida”, cliente e trotinete associados e problema reportado.	Gestão de OS	António Silva / Entrevista-001	Nuno Rebelo
RF-09	2026-02-17	O sistema deve permitir registar, na ordem de serviço, o diagnóstico técnico efetuado, após a ordem ter sido criada.	Gestão de OS	Ricardo Costa / Entrevista-001	Marco Sèvegrand
RF-10	2026-02-11	O sistema deve permitir registar, na ordem de serviço, o orçamento apresentado e a aprovação do cliente.	Gestão de OS	Ricardo Costa / Registos Oficina, OS-2026-032	Lucas Robertson
RF-11	2026-02-17	O sistema deve suportar os estados da OS: “Recebida”, “Em Diagnóstico”, “Aguarda Aprovação do Cliente”, “Aguarda Peças”, “Em Reparação”, “Concluída” e “Entregue ao Cliente”.	Gestão de OS	António Silva e Ricardo Costa / Entrevista-001	Francisco Martins
RF-12	2026-02-20	O sistema deve registar automaticamente a data e hora de cada mudança de estado da ordem de serviço e manter o respetivo histórico.	Gestão de OS	António Silva / Entrevista-002	Nuno Rebelo
RF-13	2026-02-20	O sistema deve permitir que os mecânicos alterem os estados operacionais das OS e que o estado “Entregue ao Cliente” seja confirmado pelo gerente.	Segurança	Ricardo Costa e António Silva / Entrevista-002	Marco Sèvegrand
RF-14	2026-02-20	O sistema deve apresentar um resumo com a quantidade de ordens de serviço em cada estado.	Gestão de OS	António Silva / Entrevista-002	Lucas Robertson
RF-15	2026-02-17	O sistema deve permitir associar múltiplas intervenções a uma mesma ordem de serviço.	Reparação	Ricardo Costa / Entrevista-001	Marco Ferreira
RF-16	2026-02-17	O sistema deve permitir registar, em cada intervenção, a descrição do trabalho realizado, o mecânico responsável, o tempo gasto e as peças utilizadas.	Reparação	Ricardo Costa / Entrevista-001	Francisco Martins
RF-17	2026-02-20	O sistema deve disponibilizar um cronómetro integrado, com pausa e retoma, para medir o tempo de cada intervenção.	Reparação	Ricardo Costa / Entrevista-002	Nuno Rebelo

continua na próxima página

Tabela 5.7 – continuação

Nº	Data	Descrição	Vista de Uso	Fonte	Analista
RF-18	2026-02-10	O sistema deve registar, na ordem de serviço, a data de conclusão, a data de entrega e a fatura associada.	Reparação	Ricardo Costa / Registos Oficina, OS-2026-031	Marco Sè-vegrand
RF-19	2026-02-17	O sistema deve manter um catálogo de peças com referência interna, descrição, fornecedor, preço de custo, preço de venda, <i>stock</i> atual e <i>stock</i> mínimo.	Gestão de Stock	António Silva e Ricardo Costa / Entrevista-001	Lucas Robertson
RF-20	2026-02-17	O sistema deve permitir adicionar peças do catálogo e editar descrição, fornecedor, preço de custo, preço de venda e <i>stock</i> mínimo.	Gestão de Stock	António Silva e Ricardo Costa / Entrevista-001	Marco Ferreira
RF-21	2026-02-20	O sistema deve permitir associar peças a uma intervenção e diminuir automaticamente o <i>stock</i> dessas peças.	Gestão de Stock	Ricardo Costa / Entrevista-001	Marco Ferreira
RF-22	2026-02-10	O sistema deve registar movimentos de <i>stock</i> com referência de peça, data, tipo de movimento, origem, variação, saldo e responsável.	Gestão de Stock	Oficina Gengis Khan / Caderno de Stock	Francisco Martins
RF-23	2026-02-10	O sistema deve permitir consultar a disponibilidade atual de peças, apresentando um alerta visual sempre que o inventário dessa peça desça ou iguale o nível mínimo predefinido.	Gestão de Stock	User Stories / US-G06	Nuno Rebelo
RF-24	2026-02-10	O sistema deve gerar automaticamente uma lista (pré-encomenda) com as peças abaixo do <i>stock</i> mínimo e enviá-la diretamente ao fornecedor correspondente por <i>email</i> .	Fornecedores	António Silva / Entrevista-002	Nuno Rebelo
RF-25	2026-02-10	O sistema deve permitir registar novas encomendas, alterar o seu estado ("Solicitada" ou "Recebida") e manter um histórico por fornecedor que inclua a data de criação, peças, quantidades e data de entrega.	Fornecedores	Ricardo Costa / Entrevista-002	Marco Sè-vegrand
RF-26	2026-02-17	O sistema deve permitir registar fornecedores com nome, <i>email</i> , telefone e as condições de pagamento.	Fornecedores	António Silva / Entrevista-001	Marco Sè-vegrand
				continua na próxima página	

Tabela 5.7 – continuação

Nº	Data	Descrição	Vista de Uso	Fonte	Analista
RF-27	2026-02-17	O sistema deve permitir que os campos de <i>email</i> , telemóvel e condições de pagamento de qualquer fornecedor sejam editáveis após o registo.	Fornecedores	António Silva / Entrevista-001	Marco Ferreira
RF-28	2026-02-17	O sistema deve calcular o valor da reparação com base no preço de venda das peças, no tempo de mão de obra e no IVA aplicável.	Faturação	António Silva / Entrevista-001	Lucas Robertson
RF-29	2026-02-20	O sistema deve gerar automaticamente a fatura a partir da ordem de serviço concluída.	Faturação	António Silva / Entrevista-002	Marco Ferreira
RF-30	2026-02-20	O sistema deve calcular automaticamente subtotal, IVA e total da fatura.	Faturação	António Silva / Entrevista-002	Nuno Rebelo
RF-31	2026-02-17	O sistema deve emitir faturas com numeração sequencial contínua, sem quebras.	Faturação	António Silva / Entrevista-001	Marco Sèvegrand
RF-32	2026-02-20	O sistema deve permitir configurar a taxa horária da mão de obra e a taxa de IVA.	Faturação	António Silva / Entrevista-002	Lucas Robertson
RF-33	2026-02-12	O sistema deve registar, na fatura, a forma de pagamento e o estado de pagamento.	Faturação	Oficina Gengis Khan / Faturas FAT-2026-068 e 069	Marco Ferreira
RF-34	2026-02-20	O sistema deve permitir consultar rapidamente as faturas pendentes e atrasadas de clientes empresariais.	Faturação	António Silva / Entrevista-002	Francisco Martins
RF-35	2026-02-20	O sistema deve disponibilizar relatórios filtráveis por período (mensal, trimestral, anual) sobre faturação, custos, peças mais utilizadas, problemas/avarias mais comuns, tempo médio de reparação e indicadores de produtividade por mecânico.	Relatórios	António Silva e Ricardo Costa / Entrevista-002	Lucas Robertson
RF-36	2026-02-20	O sistema deve suportar utilizadores com <i>login</i> individual.	Segurança	António Silva / Entrevista-002	Lucas Robertson
RF-37	2026-02-20	O sistema deve dar acesso total ao gerente.	Segurança	António Silva / Entrevista-002	Marco Ferreira
RF-38	2026-02-20	O sistema deve permitir que os mecânicos criem OS, registem diagnósticos e intervenções, associem peças e consultem <i>stock</i> e histórico técnico.	Segurança	António Silva / Entrevista-002	Marco Ferreira
				continua na próxima página	

Tabela 5.7 – continuação

Nº	Data	Descrição	Vista de Uso	Fonte	Analista
RF-39	2026-02-20	O sistema deve enviar ao cliente uma confirmação de receção quando a trotinete é aceite, incluindo o número da ordem de serviço e um prazo estimado.	Notificações	António Silva / Entrevista-002	Francisco Martins
RF-40	2026-02-20	O sistema deve enviar ao cliente o orçamento detalhado por <i>email</i> quando a ordem de serviço passa para o estado “Aguarda Aprovação do Cliente”, solicitando confirmação.	Notificações	António Silva e Ricardo Costa / Entrevista-002	Nuno Rebelo
RF-41	2026-02-20	O sistema deve enviar ao cliente uma notificação quando a reparação estiver concluída, informando que a trotinete está pronta e indicando o valor total.	Notificações	António Silva / Entrevista-002	Marco Sèvegrand
RF-42	2026-02-20	O sistema deve enviar ao cliente um <i>email</i> de atraso quando uma trotinete permanecer mais de 10 dias no estado “Aguarda Peças”, informando do atraso e apresentando um pedido de desculpa.	Notificações	António Silva / Entrevista-002	Lucas Robertson
RF-43	2026-02-20	O sistema deve permitir personalizar o conteúdo das notificações através de <i>templates</i> ajustáveis.	Notificações	António Silva / Entrevista-002	Marco Ferreira
RF-44	2026-02-20	O sistema deve usar o <i>email</i> como canal principal para as notificações automáticas ao cliente.	Notificações	António Silva / Entrevista-002	Francisco Martins

H.3. Requisitos Não Funcionais Consolidados

A Tabela 5.8 apresenta os cinco requisitos não funcionais consolidados, incidentes sobre três dimensões (plataforma e acessibilidade, segurança e conformidade legal, escalabilidade).

Tabela 5.8: Requisitos não funcionais consolidados

Nº	Data	Descrição	Vista de Uso	Fonte	Analista
RNF-01	2026-02-20	O sistema deve ser uma aplicação <i>web</i> acessível na oficina e remotamente.	Plataforma	António Silva / Entrevista-002	Nuno Rebelo
RNF-02	2026-02-20	O sistema deve funcionar adequadamente em <i>tablets</i> .	Plataforma	Ricardo Costa / Entrevista-002	Marco Sèvegrand

Tabela 5.8 – continuação

RNF-03	2026-02-20	O sistema deve proteger os dados pessoais dos clientes em conformidade com o RGPD.	Segurança	António Silva / Entrevista-002	Francisco Martins
RNF-04	2026-02-20	O sistema deve executar <i>backups</i> automáticos diários.	Segurança	António Silva / Entrevista-002	Nuno Rebelo
RNF-05	2026-02-20	O sistema deve suportar o volume atual de cerca de 150 ordens de serviço por mês e o crescimento previsto para 500 por mês.	Plataforma	António Silva / Entrevista-002	Marco Sèvegrand

Anexo I: Catálogo Completo de *User Stories*

O presente anexo reproduz a totalidade das 16 *user stories* produzidas durante a fase de levantamento e consolidação de necessidades, organizadas pelos dois papéis operacionais primários do sistema, designadamente Gerente e Mecânico. A Subsecção 1.3.1 do corpo do relatório apresenta duas histórias representativas, uma por papel, remetendo para o presente anexo a versão integral.

I.1. *User Stories*: Gerente (António Silva)

US-G01: Registo e manutenção de clientes

Como Gerente, eu pretendo registar e manter clientes (particulares e empresas), para que a faturação e as condições comerciais sejam aplicadas corretamente.

Critérios de Aceitação:

- Deve ser possível indicar se o cliente é *Particular* ou *Empresa* no registo.
- Para empresas, deve ser possível registar condições comerciais (prazos, limites, contacto).
- O sistema deve evitar duplicações por NIF.

US-G02: Acompanhamento de faturas pendentes e controlo de crédito

Como Gerente, eu pretendo consultar faturas pendentes e atrasadas por cliente (sobretudo empresas), para que possa gerir risco de tesouraria e efetuar cobranças atempadas.

Critérios de Aceitação:

- Deve existir listagem filtrável por estado de pagamento (pendente/pago/atrasado).
- Deve ser visível a data de vencimento e dias de atraso.
- Deve ser possível registar pagamentos com modalidade.

US-G03: Emissão de fatura com numeração sequencial

Como Gerente, eu pretendo emitir faturas automaticamente a partir de ordens de serviço concluídas, para que a faturação seja mais rápida, sem erros de cálculo e conforme a numeração sequencial obrigatória.

Critérios de Aceitação:

- Só deve ser possível faturar uma OS no estado *Concluída*.
- A numeração deve ser automática, sequencial e sem quebras (FAT-AAAA-NNN).
- O sistema deve calcular subtotal, IVA e total de forma automática.

US-G04: Parametrização de taxas e regras fiscais

Como Gerente, eu pretendo configurar a taxa horária e a taxa de IVA, para que o sistema reflita as condições comerciais e fiscais em vigor.

Critérios de Aceitação:

- Apenas o perfil de gerente pode alterar parâmetros.
- Alterações devem ficar registadas para auditoria.
- Faturas emitidas não devem ser alteradas retroativamente.

US-G05: Visibilidade operacional por estado de OS

Como Gerente, eu pretendo visualizar quantas OS estão em cada estado e há quanto tempo, para que possa identificar bloqueios (por exemplo, “Aguarda Peças”) e ajustar prioridades.

Critérios de Aceitação:

- O sistema deve apresentar contagens por estado.
- Deve ser possível aceder à lista filtrada ao selecionar um estado.
- OS paradas há mais de 10 dias devem ser destacadas.

US-G06: Alertas de *stock* mínimo e decisão de reposição

Como Gerente, eu pretendo receber alertas quando o *stock* atinge o mínimo, para que possa encomendar antecipadamente e evitar ruturas que atrasem reparações.

Critérios de Aceitação:

- Deve existir alerta visual persistente para $stock \leq$ mínimo.
- O alerta deve identificar peça, *stock* atual e mínimo.
- Deve existir ligação rápida para criar encomenda.

US-G07: Criação e envio de encomendas a fornecedores

Como Gerente, eu pretendo criar e enviar encomendas com referência e quantidades de peças,

para que a comunicação com fornecedores seja estruturada e rastreável.

Critérios de Aceitação:

- Deve ser possível selecionar fornecedor e adicionar várias peças.
- Deve existir registo de estado da encomenda (solicitada/recebida).
- Deve ser possível registar receção e atualizar *stock*.

US-G08: Relatórios para suporte à decisão

Como Gerente, eu pretendo consultar relatórios de faturação, peças mais usadas e produtividade, para que possa tomar decisões com base em dados e não em estimativas.

Critérios de Aceitação:

- Relatórios devem ser filtráveis por período temporal.
- Devem incluir métricas agregadas (totais e *rankings*).
- A consulta deve ser restrita ao perfil gerente.

I.2. User Stories: Mecânico (Ricardo Costa / Miguel Abrantes)

US-M01: Receção de equipamento e abertura de OS

Como Mecânico, eu pretendo registar a entrada de uma trotinete e abrir uma OS rapidamente, para que a receção do cliente não seja atrasada por burocracia.

Critérios de Aceitação:

- Deve ser possível identificar cliente existente por NIF/contacto.
- O número de série deve ser obrigatório e validado como único.
- A OS deve ser criada com numeração automática e estado inicial *Recebida*.

US-M02: Registo de diagnóstico técnico

Como Mecânico, eu pretendo registar o diagnóstico técnico na OS, para que fique documentada a causa provável e a solução proposta.

Critérios de Aceitação:

- O diagnóstico deve permitir texto livre.
- Deve ficar associado ao mecânico responsável.
- Deve ficar disponível no histórico do equipamento.

US-M03: Atualização de estado da OS com registo temporal

Como Mecânico, eu pretendo alterar o estado da OS conforme o trabalho progride, para que

exista rastreabilidade do processo e visibilidade interna.

CrITÉRIOS de Aceitação:

- Cada mudança de estado deve registar data/hora e autor.
- O sistema deve suportar os sete estados definidos.
- Deve existir histórico completo de transições.

US-M04: Registo de intervenções por OS

Como Mecânico, eu pretendo registar várias intervenções numa OS, para que o trabalho realizado fique detalhado e faturável.

CrITÉRIOS de Aceitação:

- Deve ser possível adicionar múltiplas intervenções por OS.
- Cada intervenção deve ter descrição, responsável e tempo.
- O conjunto de intervenções deve ser refletido na faturação.

US-M05: Cronómetro para registo de mão de obra

Como Mecânico, eu pretendo usar um cronómetro para medir o tempo de intervenção, para que a mão de obra não dependa de estimativas manuais.

CrITÉRIOS de Aceitação:

- Deve existir cronómetro com iniciar/pausar/parar.
- O tempo final deve ser registado automaticamente na intervenção.
- Deve ser possível corrigir com justificação (se aplicável).

US-M06: Associação de peças utilizadas e atualização de *stock*

Como Mecânico, eu pretendo associar peças usadas a uma intervenção, para que o *stock* seja baixado automaticamente e a faturação seja correta.

CrITÉRIOS de Aceitação:

- Deve ser possível pesquisar peças por referência/descrição.
- O sistema deve bloquear a utilização de peça com *stock* zero.
- A baixa de *stock* deve ocorrer no momento da associação.

US-M07: Consulta do histórico por número de série

Como Mecânico, eu pretendo consultar o histórico de reparações de uma trotinete pelo número de série, para que possa detetar padrões de avaria e acelerar diagnósticos.

CrITÉRIOS de Aceitação:

- Deve listar OS anteriores, diagnósticos e intervenções.

- Deve apresentar peças substituídas em cada reparação.
- Deve ser acessível a partir da ficha da trotinete e da OS.

US-M08: Identificação rápida de bloqueios por falta de peças

Como Mecânico, eu pretendo saber antecipadamente se existem peças disponíveis para uma intervenção, para que evite iniciar reparações que vão ficar paradas por rutura de *stock*.

Critérios de Aceitação:

- Deve ser visível o *stock* disponível no momento de seleção da peça.
- Deve existir indicação clara quando uma OS está em *Aguarda Peças*.
- Deve ser possível sinalizar necessidade de encomenda ao gerente.

Anexo J: Catálogo Completo dos Casos de Uso

O presente anexo reproduz a especificação integral dos 29 casos de uso identificados durante a fase de modelação comportamental, agrupados nas seis áreas funcionais do sistema. A Subsecção 1.4 do corpo do relatório apresenta apenas dois casos de uso representativos, remetendo para o presente anexo a especificação completa.

J.1. Gestão de Clientes e Equipamentos

UC-01: Registar Cliente

Atores	Gerente
Descrição	Um gerente regista no sistema um novo cliente, particular ou empresarial.
Requisitos	RF-01, RF-02, RF-04
Pré-condição	Gerente está autenticado.
Pós-condição	Novo cliente fica registado no sistema com todos os seus dados associados.
Fluxo Normal	1. Gerente insere nome completo, NIF, contacto telefónico, <i>email</i> e morada. 2. Gerente classifica o cliente como “particular”. 3. Sistema valida o NIF inserido. 4. Sistema regista o cliente.

Fluxo Alternativo	FA1 [Gerente classifica o cliente como “empresa”] (passo 2): 2.1 Gerente insere denominação social, limite de crédito e condições de pagamento. 2.2 Regressa a 3.
Fluxo de Exceção	FE1 [NIF não é válido] (passo 3): 3.1 Sistema informa o problema com o NIF. 3.2 Sistema cancela o registo de cliente.

UC-02: Pesquisar Cliente

Atores	Gerente
Descrição	Um gerente procura um cliente no sistema.
Requisitos	RF-05, RF-06
Pré-condição	Gerente está autenticado.
Pós-condição	O cliente alvo é encontrado.
Fluxo Normal	1. Gerente faz a pesquisa por nome, telefone, <i>email</i> ou NIF. 2. Sistema apresenta os resultados correspondentes. 3. Gerente seleciona o cliente pretendido.
Fluxo Alternativo	<i>(não aplicável)</i>
Fluxo de Exceção	FE1 [Nenhum cliente encontrado] (passo 2): 2.1 Sistema informa que não existe nenhum cliente correspondente ao critério de pesquisa.

UC-03: Consultar Histórico de Cliente

Atores	Gerente
Descrição	Um gerente consulta o histórico de um cliente existente no sistema.
Requisitos	RF-03, RF-05
Pré-condição	Gerente está autenticado.
Pós-condição	O histórico do cliente é exibido.
Fluxo Normal	1. « <i>include</i> » Pesquisar Cliente (UC-02). 2. Sistema verifica que o cliente é do tipo particular. 3. Sistema apresenta o histórico do cliente: número de visitas à oficina, total gasto e faturas por pagar.

Fluxo Alternativo	FA1 [Cliente é do tipo empresarial e está perto ou acima do limite de crédito] (passo 2): 2.1 Sistema apresenta alerta visual de limite de crédito, indicando o limite definido e o valor em dívida. 2.2 Regressa a 3.
Fluxo de Exceção	<i>(não aplicável)</i>

UC-04: Editar Dados de Contacto de Cliente

Atores	Gerente
Descrição	Um gerente atualiza o <i>email</i> e/ou o telemóvel de um cliente já registado.
Requisitos	RF-06
Pré-condição	Gerente está autenticado.
Pós-condição	Dados de contacto do cliente ficam atualizados no sistema.
Fluxo Normal	1. « <i>include</i> » Pesquisar Cliente (UC-02). 2. Gerente edita o <i>email</i> e/ou o telemóvel. 3. Sistema guarda as alterações.
Fluxo Alternativo	<i>(não aplicável)</i>
Fluxo de Exceção	<i>(não aplicável)</i>

UC-05: Registrar Trotinete

Atores	Gerente
Descrição	Um gerente regista uma trotinete no sistema.
Requisitos	RF-07
Pré-condição	Gerente está autenticado.
Pós-condição	Trotinete fica registada no sistema com número de série único e associada a um cliente.
Fluxo Normal	1. Gerente insere NIF do cliente proprietário da trotinete. 2. Sistema valida o NIF. 3. Gerente insere marca, modelo, número de série e observações sobre o estado geral. 4. Sistema valida a unicidade do número de série. 5. Sistema regista a trotinete, associando-a ao cliente indicado.
Fluxo Alternativo	<i>(não aplicável)</i>

Fluxo de Exceção	FE1 [NIF inválido ou inexistente] (passo 2): 2.1 Sistema informa o problema com o NIF. 2.2 Sistema cancela o registo da trotinete. FE2 [Número de série já existe no sistema] (passo 4): 4.1 Sistema informa que o número de série já está registado. 4.2 Sistema cancela o registo da trotinete.
-------------------------	--

UC-06: Consultar Histórico de Reparações de Trotinete

Atores	Gerente, Mecânico
Descrição	Um funcionário consulta o histórico completo de reparações de uma trotinete através do número de série.
Requisitos	RF-08
Pré-condição	Funcionário está autenticado.
Pós-condição	Histórico de reparações da trotinete é exibido.
Fluxo Normal	1. Funcionário insere o número de série da trotinete. 2. Sistema procura a trotinete pelo número de série. 3. Sistema apresenta o histórico completo de reparações associadas àquele número de série.
Fluxo Alternativo	<i>(não aplicável)</i>
Fluxo de Exceção	FE1 [Número de série não encontrado] (passo 2): 2.1 Sistema informa que não existe nenhuma trotinete com esse número de série.

J.2. Ciclo de Vida das Ordens de Serviço

UC-07: Criar Ordem de Serviço

Atores	Gerente, Mecânico
Descrição	Um funcionário cria uma nova ordem de serviço para uma trotinete entregue por um cliente.
Requisitos	RF-09, RF-12, RF-13, RF-41
Pré-condição	Funcionário está autenticado. Cliente está registado no sistema. Trotinete está registada no sistema e associada ao cliente em questão.
Pós-condição	OS é criada com numeração sequencial, estado "Recebida", data de entrada e problema reportado. Cliente é notificado por <i>email</i> .

Fluxo Normal	1. Gerente insere o NIF do cliente e o número de série da trotinete. 2. Sistema valida o cliente e a trotinete. 3. Sistema cria a OS com numeração sequencial e estado inicial "Recebida". 4. Sistema regista automaticamente a data e hora da criação. 5. Funcionário insere o problema reportado pelo cliente. 6. Funcionário insere o prazo estimado de reparação. 7. Sistema envia ao cliente uma confirmação de receção por <i>email</i> com o número da OS e o prazo estimado.
Fluxo Alternativo	<i>(não aplicável)</i>
Fluxo de Exceção	FE1 [Cliente ou trotinete não encontrados] (passo 2): 2.1 Sistema informa que o cliente ou a trotinete não existe no sistema. 2.2 Sistema cancela a criação da OS.

UC-08: Registar Diagnóstico da OS

Atores	Mecânico, Gerente
Descrição	Um funcionário regista o diagnóstico técnico numa ordem de serviço.
Requisitos	RF-10, RF-12, RF-13
Pré-condição	Funcionário está autenticado. Existem OS com estado "Recebida".
Pós-condição	Diagnóstico é registado na OS. Estado é atualizado para "Em Diagnóstico". Data e hora da transição são registadas.
Fluxo Normal	1. Funcionário seleciona a OS. 2. Funcionário altera o estado da OS para "Em Diagnóstico". 3. Sistema regista automaticamente a data e hora da mudança de estado. 4. Funcionário insere o diagnóstico técnico. 5. Sistema guarda o diagnóstico na OS.
Fluxo Alternativo	<i>(não aplicável)</i>
Fluxo de Exceção	<i>(não aplicável)</i>

UC-09: Registar Orçamento e Aprovação de Cliente

Atores	Mecânico, Gerente
---------------	-------------------

Descrição	Um funcionário registra o orçamento apresentado e a resposta do cliente numa ordem de serviço.
Requisitos	RF-11, RF-12, RF-13, RF-42
Pré-condição	Gerente está autenticado. OS existe com diagnóstico registrado.
Pós-condição	Orçamento e aprovação do cliente ficam registrados na OS. Estado é atualizado. Cliente é notificado por <i>email</i> .
Fluxo Normal	1. Funcionário seleciona a OS. 2. Funcionário insere o valor e a descrição do orçamento previsto. 3. Sistema altera o estado da OS para "Aguarda Aprovação do Cliente". 4. Sistema registra automaticamente a data e hora da mudança de estado. 5. Sistema envia ao cliente o orçamento detalhado por <i>email</i>, solicitando confirmação. 6. Funcionário registra a aprovação do cliente. 7. Sistema guarda a aprovação na OS.
Fluxo Alternativo	FA1 [Cliente recusa o orçamento] (passo 6): 6.1 Funcionário registra a recusa do cliente na OS. 6.2 Sistema altera o estado da OS para "Concluída". 6.3 Sistema registra automaticamente a data e hora da mudança de estado.
Fluxo de Exceção	(<i>não aplicável</i>)

UC-10: Registrar Intervenção

Atores	Mecânico, Gerente
Descrição	Um funcionário registra uma intervenção numa ordem de serviço, incluindo o trabalho realizado, o tempo gasto e as peças utilizadas.
Requisitos	RF-12, RF-13, RF-16, RF-17, RF-18, RF-22, RF-23
Pré-condição	Funcionário está autenticado. Existem OS com estado "Em Reparação" ou "Aguarda Aprovação" com aprovação registrada.
Pós-condição	Intervenção é registrada na OS com descrição, mecânico, tempo e peças. <i>Stock</i> das peças é diminuído automaticamente. Movimento de <i>stock</i> é registrado.

Fluxo Normal	1. Funcionário seleciona uma OS no estado “Em Reparação”. 2. Funcionário cria uma nova intervenção. 3. Funcionário insere a descrição do trabalho a realizar. 4. Funcionário inicia o cronómetro integrado para medir o tempo de intervenção, podendo pausá-lo e retomá-lo. 5. Funcionário associa as peças utilizadas na intervenção. 6. Sistema diminui automaticamente o <i>stock</i> de cada peça associada. 7. Sistema regista cada movimento de <i>stock</i> com a referência da peça, data atual, tipo, origem (número da OS), variação, saldo e responsável. 8. Utilizador para o cronómetro e confirma a intervenção. 9. Sistema guarda a intervenção na OS com o mecânico responsável, tempo gasto e peças utilizadas.
Fluxo Alternativo	FA1 [Funcionário seleciona uma OS no estado “Aguarda Aprovação” com aprovação do cliente registada] (passo 1): 1 a.1 Sistema altera o estado da OS para “Em Reparação”. 1 a.2 Sistema regista automaticamente a data e hora da mudança de estado. 1 a.3 Regressa a 2. FA2 [Funcionário seleciona uma OS no estado “Aguarda Peças”] (passo 1): 1 b.1 Sistema verifica a disponibilidade de todas as peças associadas à intervenção. 1 b.2 Regressa a 6.
Fluxo de Exceção	FE1 [Não existem peças suficientes em <i>stock</i>] (passo 6): 6.1 Sistema avisa quais peças estão em falta e em que quantidades. 6.2 Sistema altera o estado da OS para “Aguarda Peças”. 6.3 Sistema regista automaticamente a data e hora da mudança de estado. FE2 [Não existem peças suficientes em <i>stock</i>] (passo 1.b.1): 1.b.1.1 Sistema avisa quais peças ainda estão em falta e em que quantidades.

UC-11: Concluir OS

Atores	Gerente, Mecânico
Descrição	Um funcionário conclui uma ordem de serviço.

Requisitos	RF-12, RF-13, RF-19, RF-29, RF-43
Pré-condição	Funcionário está autenticado. Existem OS com estado “Em Reparação”.
Pós-condição	Estado da OS é atualizado para “Concluída”. Cliente é notificado por <i>email</i> .
Fluxo Normal	1. Funcionário seleciona a OS. 2. Funcionário altera o estado da OS para “Concluída”. 3. Sistema regista automaticamente a data e hora da mudança de estado (data de conclusão). 4. Sistema calcula automaticamente o valor da reparação com base no preço de venda das peças, no tempo de mão de obra (aplicando a taxa horária configurada) e no IVA aplicável. 5. Sistema notifica o cliente por <i>email</i>, informando que a reparação está concluída e indicando o valor total.
Fluxo Alternativo	<i>(não aplicável)</i>
Fluxo de Exceção	<i>(não aplicável)</i>

UC-12: Confirmar Entrega ao Cliente

Atores	Gerente
Descrição	O gerente confirma a entrega da trotinete ao cliente, concluindo o ciclo de vida da OS.
Requisitos	RF-12, RF-13, RF-14, RF-19
Pré-condição	Gerente está autenticado. Existem OS com estado “Concluída” e fatura associada.
Pós-condição	Estado da OS é atualizado para “Entregue ao Cliente”. Data de entrega é registada na OS.
Fluxo Normal	1. Gerente seleciona a OS. 2. Gerente confirma a entrega da trotinete ao cliente. 3. Sistema altera o estado da OS para “Entregue ao Cliente”. 4. Sistema regista automaticamente a data e hora da mudança de estado (data de entrega).
Fluxo Alternativo	<i>(não aplicável)</i>
Fluxo de Exceção	<i>(não aplicável)</i>

UC-13: Consultar Resumo de Estados de OS

Atores	Gerente
Descrição	Um gerente consulta um resumo com a quantidade de ordens de serviço em cada estado.
Requisitos	RF-15
Pré-condição	Gerente está autenticado.
Pós-condição	Resumo de OS por estado é exibido.
Fluxo Normal	1. Gerente acede ao painel de resumo de ordens de serviço. 2. Sistema apresenta a quantidade de OS em cada estado ("Recebida", "Em Diagnóstico", "Aguarda Aprovação do Cliente", "Aguarda Peças", "Em Reparação", "Concluída", "Entregue ao Cliente").
Fluxo Alternativo	<i>(não aplicável)</i>
Fluxo de Exceção	<i>(não aplicável)</i>

J.3. Gestão de Peças e *Stock*

UC-14: Adicionar Peça ao Catálogo

Atores	Gerente
Descrição	Um gerente adiciona uma peça ao catálogo do sistema.
Requisitos	RF-20, RF-21
Pré-condição	Gerente está autenticado.
Pós-condição	Peça é registada no catálogo com todos os seus atributos.
Fluxo Normal	1. Gerente insere referência da nova peça. 2. Sistema valida a unicidade da referência. 3. Gerente insere nome do fornecedor da nova peça. 4. Sistema encontra o fornecedor. 5. Gerente insere descrição, preço de custo, preço de venda, <i>stock</i> atual e <i>stock</i> mínimo. 6. Sistema regista a nova peça no catálogo.
Fluxo Alternativo	<i>(não aplicável)</i>
Fluxo de Exceção	FE1 [Referência já existe] (passo 2): 2.1 Sistema alerta que a referência já existe. FE2 [Fornecedor não existe] (passo 4): 4.1 Sistema alerta que o fornecedor não existe.

UC-15: Editar Peça do Catálogo

Atores	Gerente
Descrição	Um gerente edita os dados de uma peça do catálogo do sistema.
Requisitos	RF-20, RF-21
Pré-condição	Gerente está autenticado.
Pós-condição	Alterações à peça ficam registradas.
Fluxo Normal	1. Gerente insere referência da peça. 2. Sistema encontra a peça. 3. Gerente edita, à escolha, descrição, fornecedor, preço de custo, preço de venda e <i>stock</i> mínimo. 4. Sistema guarda as alterações.
Fluxo Alternativo	<i>(não aplicável)</i>
Fluxo de Exceção	FE1 [Referência não existe] (passo 2): 2.1 Sistema alerta que a referência não existe.

UC-16: Consultar *Stock*

Atores	Gerente, Mecânico
Descrição	Um funcionário consulta o <i>stock</i> atual de peças.
Requisitos	RF-24
Pré-condição	Funcionário está autenticado.
Pós-condição	Disponibilidade de peças é exibida.
Fluxo Normal	1. Sistema lista todas as peças em <i>stock</i> e a respetiva informação. 2. Sistema não deteta peças abaixo do nível mínimo.
Fluxo Alternativo	FA1 [Há peças abaixo do nível mínimo] (passo 2): 2.1 Sistema lista todas as peças cujo <i>stock</i> atual é igual ou inferior ao <i>stock</i> mínimo.
Fluxo de Exceção	<i>(não aplicável)</i>

J.4. Gestão de Fornecedores e Encomendas

UC-17: Registar Fornecedor

Atores	Gerente
Descrição	Um gerente regista um fornecedor no sistema.
Requisitos	RF-27
Pré-condição	Gerente está autenticado.
Pós-condição	Fornecedor é registado.
Fluxo Normal	1. Gerente insere o nome do fornecedor. 2. Sistema valida a unicidade do nome. 3. Gerente insere <i>email</i>, telemóvel e condições de pagamento. 4. Sistema regista fornecedor.
Fluxo Alternativo	<i>(não aplicável)</i>
Fluxo de Exceção	FE1 [Nome de fornecedor já existe] (passo 2): 2.1 Sistema alerta que o nome já existe.

UC-18: Editar Fornecedor

Atores	Gerente
Descrição	Um gerente edita os dados de um fornecedor.
Requisitos	RF-28
Pré-condição	Gerente está autenticado.
Pós-condição	Dados do fornecedor são atualizados.
Fluxo Normal	1. Gerente insere o nome do fornecedor. 2. Sistema encontra o fornecedor. 3. Gerente edita, à escolha, <i>email</i>, telemóvel e condições de pagamento. 4. Sistema guarda as alterações.
Fluxo Alternativo	<i>(não aplicável)</i>
Fluxo de Exceção	FE1 [Nome de fornecedor não existe] (passo 2): 2.1 Sistema alerta que o nome não existe.

UC-19: Fazer Encomenda

Atores	Gerente
---------------	---------

Descrição	Um gerente cria uma encomenda de peças a um fornecedor.
Requisitos	RF-25, RF-26
Pré-condição	Gerente está autenticado.
Pós-condição	Encomenda é registada no sistema com data de criação, peças e quantidades.
Fluxo Normal	1. Gerente cria uma nova encomenda. 2. Gerente seleciona o fornecedor. 3. Gerente seleciona manualmente as peças e indica as quantidades. 4. Sistema valida que todas as peças provêm do mesmo fornecedor. 5. Sistema regista a encomenda com estado “Solicitada” e data de criação. 6. Sistema envia a encomenda ao fornecedor via <i>email</i>.
Fluxo Alternativo	FA1 [Gerente gera pré-encomenda automática] (passo 3): 3.1 Sistema adiciona à encomenda todas as peças do fornecedor indicado cuja quantidade disponível é igual ou inferior ao nível mínimo. 3.2 Gerente indica as quantidades de cada peça. 3.3 Gerente escolhe não adicionar mais peças. 3.4 Regressa a 5. FA2 [Gerente escolhe adicionar mais peças] (passo 3.3): 3.1 Regressa a 3.
Fluxo de Exceção	(<i>não aplicável</i>)

UC-20: Receber Encomenda

Atores	Gerente
Descrição	Um gerente indica a receção de uma encomenda.
Requisitos	RF-23, RF-26
Pré-condição	Gerente está autenticado. Existem encomendas no estado “Solicitada”.
Pós-condição	Encomenda fica no estado “Recebida” com data de entrega. <i>Stock</i> de peças é atualizado.

Fluxo Normal	1. Gerente seleciona uma encomenda no estado “Solicitada”. 2. Gerente indica que a encomenda foi recebida. 3. Sistema regista automaticamente a data de entrega. 4. Sistema regista cada movimento de <i>stock</i> com a referência de cada peça, data atual, tipo, origem (número da encomenda), variação, saldo e responsável.
Fluxo Alternativo	<i>(não aplicável)</i>
Fluxo de Exceção	<i>(não aplicável)</i>

UC-21: Consultar Histórico de Encomendas

Atores	Gerente
Descrição	Um gerente consulta o histórico de encomendas feitas a um fornecedor.
Requisitos	RF-26
Pré-condição	Gerente está autenticado.
Pós-condição	Histórico de encomendas é exibido.
Fluxo Normal	1. Gerente seleciona um fornecedor. 2. Sistema exhibe histórico de encomendas, com data de criação, peças, quantidades e data de entrega (se aplicável).
Fluxo Alternativo	<i>(não aplicável)</i>
Fluxo de Exceção	<i>(não aplicável)</i>

U.5. Faturação e Pagamento

UC-22: Gerar Fatura

Atores	Gerente
Descrição	Um gerente gera a fatura de uma ordem de serviço concluída.
Requisitos	RF-19, RF-29, RF-30, RF-31, RF-32, RF-33
Pré-condição	Gerente está autenticado. Existem OS com estado “Concluída” e intervenções registadas, sem fatura associada.
Pós-condição	Fatura é gerada com numeração sequencial e fica registada na OS.
Fluxo Normal	1. Gerente seleciona a OS concluída. 2. Sistema calcula automaticamente o valor da reparação com base no preço de venda das peças, no tempo de mão de obra (aplicando a taxa horária configurada) e no IVA aplicável. 3. Sistema gera automaticamente a fatura com numeração sequencial contínua, detalhando as peças utilizadas e as horas de trabalho, bem como subtotal, IVA e total calculados automaticamente. 4. Sistema associa à OS a fatura gerada.
Fluxo Alternativo	<i>(não aplicável)</i>
Fluxo de Exceção	<i>(não aplicável)</i>

UC-23: Registrar Pagamento

Atores	Gerente
Descrição	Um gerente regista a forma e o estado de pagamento de uma fatura.
Requisitos	RF-35
Pré-condição	Gerente está autenticado.
Pós-condição	Forma e estado de pagamento são registados na fatura.
Fluxo Normal	1. Gerente seleciona uma fatura não paga. 2. Gerente regista a forma de pagamento e confirma o pagamento. 3. Sistema atualiza o estado de pagamento da fatura.
Fluxo Alternativo	<i>(não aplicável)</i>
Fluxo de Exceção	<i>(não aplicável)</i>

UC-24: Consultar Faturas Pendentes e Atrasadas

Atores	Gerente
Descrição	Um gerente consulta as faturas pendentes e atrasadas de clientes empresariais.
Requisitos	RF-36
Pré-condição	Gerente está autenticado.
Pós-condição	Faturas pendentes e atrasadas são exibidas.
Fluxo Normal	1. Sistema exibe faturas pendentes e o cliente devedor. 2. Sistema destaca as faturas em atraso, considerando as condições de pagamento associadas ao cliente.
Fluxo Alternativo	<i>(não aplicável)</i>
Fluxo de Exceção	<i>(não aplicável)</i>

UC-25: Configurar Parâmetros de Faturação

Atores	Gerente
Descrição	Um gerente configura a taxa horária da mão de obra e a taxa de IVA aplicável.
Requisitos	RF-34
Pré-condição	Gerente está autenticado.
Pós-condição	Parâmetros de faturação são atualizados no sistema.
Fluxo Normal	1. Gerente insere a taxa horária da mão de obra e/ou a taxa de IVA. 2. Sistema guarda as alterações.
Fluxo Alternativo	<i>(não aplicável)</i>
Fluxo de Exceção	<i>(não aplicável)</i>

U.6. Relatórios e Configuração

UC-26: Consultar Relatórios

Atores	Gerente
Descrição	Um gerente consulta relatórios de gestão filtráveis por período.
Requisitos	RF-37
Pré-condição	Gerente está autenticado.
Pós-condição	Relatório filtrado é gerado e apresentado.
Fluxo Normal	1. Gerente seleciona o tipo de relatório: faturação, custos, peças mais utilizadas, problemas/avarias mais comuns, tempo médio de reparação ou indicadores de produtividade por mecânico. 2. Gerente define o período de análise (mensal, trimestral ou anual). 3. Sistema gera e apresenta o relatório filtrado.
Fluxo Alternativo	(não aplicável)
Fluxo de Exceção	(não aplicável)

UC-27: Editar *Templates* de Notificação

Atores	Gerente
Descrição	Um gerente personaliza o conteúdo dos <i>templates</i> das notificações automáticas enviadas aos clientes.
Requisitos	RF-45
Pré-condição	Gerente está autenticado.
Pós-condição	<i>Templates</i> de notificação são atualizados e guardados no sistema.
Fluxo Normal	1. Gerente seleciona o <i>template</i> a personalizar (confirmação de receção, envio de orçamento, conclusão de reparação, atraso de entrega). 2. Gerente edita o conteúdo do <i>template</i>. 3. Sistema guarda o <i>template</i> atualizado.
Fluxo Alternativo	(não aplicável)
Fluxo de Exceção	(não aplicável)

J.7. Segurança e Notificações Assíncronas

UC-28: Autenticar

Atores	Gerente, Mecânico
Descrição	Um funcionário autentica-se no sistema com as suas credenciais individuais.
Requisitos	RF-38, RF-39
Pré-condição	True
Pós-condição	Funcionário fica autenticado com permissões correspondentes ao seu perfil.
Fluxo Normal	1. Funcionário insere as suas credenciais (<i>login</i> individual). 2. Sistema valida as credenciais. 3. Sistema autentica o funcionário.
Fluxo Alternativo	(<i>não aplicável</i>)
Fluxo de Exceção	FE1 [Credenciais inválidas] (passo 2): 2.1 Sistema informa que as credenciais são inválidas. 2.2 Sistema cancela a autenticação.

UC-29: Notificar Cliente sobre Falta de Peças

Atores	Relógio do Sistema
Descrição	O sistema notifica um cliente sobre uma reparação que está há mais de 10 dias à espera de peças.
Requisitos	RF-44
Pré-condição	Existe uma OS no estado “Aguarda Peças” há mais de 10 dias.
Pós-condição	O cliente é notificado.
Fluxo Normal	1. Sistema deteta OS em “Aguarda Peças” há mais de 10 dias. 2. Sistema envia <i>email</i> ao cliente.
Fluxo Alternativo	(<i>não aplicável</i>)
Fluxo de Exceção	(<i>não aplicável</i>)

Anexo K: Composição Detalhada da *Stack* Tecnológica

O presente anexo reproduz a composição detalhada da *stack* tecnológica adotada para o sistema, com a justificação principal de cada escolha. A Subsecção 2.1.2 do corpo do relatório apresenta apenas um excerto com as decisões estruturantes; a tabela completa é aqui reproduzida.

Tabela 5.38: Composição detalhada da *stack* tecnológica adotada

Camada / Papel	Tecnologia	Justificação principal
<i>Runtime</i> do <i>backend</i>	<i>Node.js</i> ≥ 20	Modelo de I/O não bloqueante adequado a APIs REST; LTS estável; alinhamento com o ecossistema <i>TypeScript</i> do <i>frontend</i> .
Linguagem partilhada (<i>backend</i> + <i>frontend</i>)	<i>TypeScript</i> ≥ 5.8	Tipagem estática forte; partilha literal de tipos entre camadas; prevenção de classes inteiras de erros em compilação.
<i>Framework web</i> do <i>backend</i>	<i>Fastify 4</i>	Desempenho superior ao <i>Express</i> ; sistema de <i>plugins</i> bem definido; suporte nativo a validação de esquemas.
<i>Framework</i> do <i>frontend</i>	<i>React 18</i>	Ecossistema maduro; vasto conjunto de bibliotecas complementares; correspondência com competências adquiridas em UCs anteriores.
<i>Bundler</i> e servidor de desenvolvimento	<i>Vite 6</i>	Arranque instantâneo por ESM nativo; <i>Hot Module Replacement</i> eficiente; configuração mínima.
Estilização	<i>Tailwind CSS 3</i>	Consistência visual por construção; eliminação de CSS morto; velocidade de iteração em protótipos.
Validação de esquemas	<i>Zod 3</i>	Esquemas executáveis em tempo de execução com inferência estática de tipos; partilháveis entre <i>frontend</i> e <i>backend</i> .
Base de dados	<i>PostgreSQL 16</i>	Conformidade ACID estrita (condição da faturação sequencial); tipos ricos (<i>numeric</i> , <i>jsonb</i>); maturidade.
ORM e migrações	<i>Prisma 5</i>	<i>Schema</i> declarativo com geração de cliente fortemente tipado; autoridade única sobre migrações.
Autenticação	JWT em <i>cookie HttpOnly</i> + <i>bcryptjs</i>	Sessões <i>stateless</i> adequadas a APIs REST; mitigação de XSS pela não exposição do <i>token</i> ao <i>JavaScript</i> cliente.
Bibliotecas auxiliares	<i>TanStack Query</i> , <i>React Hook Form</i> , <i>PDFKit</i> , <i>Node-mailer</i> , <i>Pino</i>	Conjunto especializado para <i>cache</i> de servidor no <i>frontend</i> , formulários, emissão de PDFs, envio SMTP e <i>logging</i> estruturado.